© 2026 Sînică Alboaie. Această primă iterație a fost donată ScriptaHub (scriptahub.com) de Sînică Alboaie. Așteptăm feedback și contribuții din partea publicului. Este permisă folosirea educativă, necomercială.

O introducere intuitivă în Machine Learning

Cu exemple din viața de zi cu zi și de la biroul unui inginer

Cuprins

Nr.	Titlu
	Prefață
1	**Schimbarea de gândire**
1.1	Rețeta și ucenicul
1.2	Răspunsurile de pe spatele paginii
1.3	Sortând dulapul pe întuneric
1.4	Examenul pe care nu ai voie să-l vezi
2	**Trusa de unelte clasică**
2.1	O riglă, câteva arcuri și un variator
2.2	Spune-mi cine îți sunt vecinii
2.3	Douăzeci de întrebări
2.4	Înțelepciunea mulțimii și a editorilor
2.5	Rafturile pe lângă care am trecut
3	**Deep learning**
3.1	Neuronul căruia i s-a promis prea mult
3.2	Coborând muntele prin ceață
3.3	Ce a dezvăluit creierul pisicii
3.4	Telefonul fără fir
4	**Din ce e făcut un model**
4.1	Nu e un creier
4.2	Transformând lumea în numere
4.3	Totul e un tabel
4.4	Ce contează și cine decide
5	**Cum se construiesc și cum rulează modelele**
5.1	Cutia de Lego
5.2	Cele două vieți ale unui model
5.3	Un alt fel de memorie
5.4	Cât e destul
6	**Cealaltă mașină**
6.1	O mie de modele ale unei căni de cafea
6.2	Mașina de predicții
7	**Limbaj și sens**

Nr.	Titlu
7.1	O hartă în care sensul are o adresă
7.2	Ședința detectivilor
7.3	Jocul ghicitului cuvântului următor
7.4	Examenul cu cartea deschisă
7.5	Viața pe o tablă
7.6	De ce ajunge atât de departe ghicitul cuvântului următor
8	**Imagini generate, sisteme construite**
8.1	Învățând să desenezi
8.2	Planuri de arhitectură
8.3	Date cu o formă proprie
8.4	Ce ascunde instalația
9	**Feluri de a învăța**
9.1	Un câine, un labirint și o barcă ce n-a vrut să termine cursa
9.2	Multele feluri de a preda
10	**Cum să dai greș cu succes**
10.1	Tocilarul și leneșul
10.2	Calul care știa să numere
10.3	Detectorul de 99,9% și alarma de fum
10.4	Oglinda strâmbă
10.5	Cât de sigur e?
10.6	Deschiderea cutiei
11	**Modele în lumea reală**
11.1	Cocoșul și răsăritul
11.2	Cine îi notează pe examinatori
11.3	Abțibilduri, otravă și scrisori cu instrucțiuni
11.4	Ce își amintește modelul și al cui este
11.5	Viața lungă de după lansare
11.6	Marginea hărții
A	**Matematica din spatele fiecărui subcapitol**
B	**Dicționarul metaforelor**
C	**Mai departe**

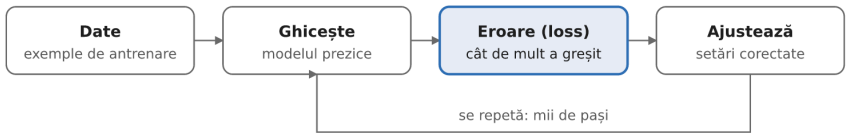
Prefață

Imaginează-ți o ingineră care petrece o săptămână întreagă reglând cache-ul unui serviciu foarte solicitat. Alege un time-to-live, așteaptă să se așeze graficele din dashboard, se uită la hit rate, mută puțin numărul și așteaptă din nou. Până vineri, hit rate-ul e excelent, iar ea e epuizată. La o cafea, cineva o întreabă dacă a făcut vreodată machine learning. Râde: nu, niciodată, așa ceva e pentru oameni cu doctorat.

Se înșală, iar restul acestei cărți e, într-un fel, explicația de ce.

Ce face ea toată săptămâna e să ghicească, să măsoare cât de greșit a ghicit, să ajusteze și să ghicească din nou. Bucla asta e întregul motor. Aproape fiecare model din carte, de la o dreaptă trasată printr-un nor de puncte până la modelele de limbaj care scriu cod alături de tine, rulează aceleași patru etape. De la un subcapitol la altul se schimbă mai ales ce anume se ajustează, cum se măsoară greșeala și cât de inteligent e aleasă ajustarea. Puținele metode care ocolesc bucla, cum ar fi căutarea vecinilor din Capitolul 2, sunt și ele interesante tocmai prin ce lasă deoparte.

Orice model învață în aceeași buclă



Diferența dintre săptămâna ei și un sistem de machine learning ține de scară și de răbdare. Ea rotește un singur buton, din intuiție, de câteva zeci de ori. Un model rotește de la câteva până la miliarde de butoane deodată, adesea de mii sau milioane de ori, și decide în ce parte să răsucească fiecare buton prin calcul, nu din instinct. Nu le încearcă pe rând, ceea ce ar dura o veșnicie cu un miliard de butoane: o singură trecere înapoi prin calcule îi spune fiecărui buton, deodată, în ce parte e coborârea și cât de abruptă e, cum explică Capitolul 3. Diferența de scară e uriașă și produce un comportament care pare magie. Dar nu e o diferență de natură. Dacă înțelegi săptămâna ei, poți înțelege mașina.

Pentru cine e cartea

Cartea e scrisă pentru oricine e curios să înțeleagă cu adevărat cum funcționează machine learning, nu doar să-i cunoască vocabularul. Dacă îți place să afli cum merg lucrurile pe dinăuntru și ai răbdarea unui student bun, ai tot ce-ți trebuie. O vor găsi utilă studenții care vor să intre în domeniu, inginerii și programatorii care folosesc deja aceste sisteme sau urmează să le construiască, cercetătorii din alte discipline care se apropie de machine learning, dar și product managerii, team lead-urii și investitorii care trebuie să cântărească ce promite o echipă, un furnizor sau un startup.

Nu trebuie să știi nimic despre machine learning ca să începi. Nu ai nevoie de mai multă matematică decât ai învățat la școală: în textul principal aproape că nu există formule. Acolo unde o formulă ar ajuta, ea așteaptă în Anexa A, explicată în cuvinte, iar când ajungi la ea ar trebui să sune ca un rezumat compact al unui lucru pe care îl înțelegi deja. Multe exemple vin din ingineria software, pentru că acolo lucrează cei mai mulți cititori, dar fiecare idee e introdusă mai întâi printr-un lucru din viața obișnuită: o

bucătărie, un examen, un dulap pe întuneric, un munte în ceață. Dacă n-ai scris niciodată cod, poți urmări cartea sărind peste paranteza ocazională de programare; nimic important nu se ascunde doar acolo.

Despre termeni

Termenii tehnici rămân în engleză, pentru că așa îi vei întâlni în documentație, în articole și în ședințe, iar o traducere ad-hoc ar suna strâmb. La prima apariție, fiecare termen e însoțit de o traducere aproximativă în română și de intuiția din spatele lui. De exemplu, **loss** („pierdere”) e un număr care spune cât de mult a greșit modelul: cu cât e mai mare, cu atât a „pierdut” mai mult față de răspunsul corect.

Cum e construită cartea

Sunt unsprezece capitole, fiecare un mic arc de subcapitole înrudite. **Capitolul 1** pune bazele noului mod de a gândi: învățarea din exemple în loc de scrierea regulilor și măsurarea onestă a acestei învățări. **Capitolul 2** e trusa de unelte clasică, metodele care fac încă cea mai mare parte a muncii pe datele din business. **Capitolul 3** e deep learning: ce este o rețea neuronală și cum învață. **Capitolele 4 și 5** deschid capota. Capitolul 4 arată ce este fizic un model și cum sunt transformate datele într-o formă pe care o poate folosi; Capitolul 5, cum se îmbină piesele lui, cât costă să-l antrenezi și să-l rulezi, plus un drum pe care domeniul nu l-a ales. Acestea două sunt cele mai solicitante capitole din carte; ia-le puțin mai încet. **Capitolul 6** e un interludiu despre cealaltă mașină care învață, creierul, și despre ce spun azi unele teorii importante din neuroștiință că face el. **Capitolele 7 și 8** sunt lumea modernă: embeddings, attention, modele de limbaj, generatoare de imagini și arhitecturile construite din ele. **Capitolul 9** e despre multele feluri în care e învățat un model, iar **Capitolul 10** despre multele feluri în care un model bine învățat tot dă greș. **Capitolul 11** urmărește modelele în lumea reală, unde se întâlnesc cu experimente, benchmark-uri, atacatori, legi și cu trecerea timpului, și se încheie la marginea hărții, cu întrebările încă deschise despre încotro se îndreaptă domeniul.

Capitolele 2, 4, 5, 7, 8 și 10 se încheie fiecare cu un subcapitol sau două care merg mai adânc, pentru întrebările pe care le pun cel mai des cititorii curioși după ce ideea principală s-a așezat: pe lângă ce metode clasice a trecut turul trusei de unelte, pe ce se sprijină un model și cine decide, de câte date, de ce mărime și de câtă putere de calcul are nevoie, cât poate ține în minte deodată, de ce ajunge atât de departe ghicitul cuvântului următor, ce feluri de date nu vin ca un tabel, ce rutine se ascund în antrenarea modelelor mari, cât de sigur e un model pe el și ce se poate citi din interiorul lui. Mai multe dintre ele recunosc și ce au simplificat subcapitolele principale, acolo unde un răspuns onest era prea lung ca să fie dat pe loc. Merg puțin mai departe decât restul capitolului; la o primă lectură grăbită le poți parcurge în fugă, ca să te întorci la ele mai târziu.

Fiecare subcapitol se sprijină pe cele dinainte, iar cele mai multe se încheie cu o întrebare la care răspunde următorul. Când o întrebare ridicată într-un capitol e reluată mai târziu în carte, ultimul paragraf al capitolului, unul scurt numit „Mai departe”, spune unde; nu trebuie niciodată să sari înainte ca să urmărești argumentul.

Cum s-o citești

Prima dată, citește-o în ordine; fiecare subcapitol ia între zece și douăzeci de minute. Dacă ai mai puțin timp, merg bine câteva trasee scurte. Ca să înțelegi repede uneltele de AI de azi: Capitolul 1, subcapitolele 3.1 și 3.2, apoi Capitolele 7 și 10. Dacă ești manager sau investitor: Capitolele 1, 7 și 10; apoi, pentru costuri și pentru felul în care sunt învățate modelele, subcapitolele 5.2, 5.4 și 9.2; iar din Capitolul 11, subcapitolele despre cum citești clasamentele de benchmark-uri, despre confidențialitate, proprietate și lege și despre încotro s-ar putea îndrepta domeniul. Dacă ești inginer și urmează să construiești ceva: toată, cu atenție specială la Capitolele 4, 5, 10 și 11 și la subcapitolele 8.4 și 10.5. Dacă ești cercetător venit din alt domeniu: Capitolele 3–6, Capitolul 9 și subcapitolele 7.6 și 10.6.

Anexa B e un dicționar care pune fiecare imagine din carte lângă termenul tehnic pe care îl reprezintă. Ține-o aproape când începi să citești documentație și articole; e puntea dintre limbajul acestei cărți și cel al domeniului. Anexa C îți sugerează încotro s-o iei după ultima pagină: cursuri, cărți, unelte de încercat și câteva prime proiecte, într-o ordine rezonabilă.

Un avertisment înainte să începem. Fiecare imagine din carte minte puțin. Un model nu e un ucenic, o mulțime de la un târg de țară sau o cameră plină de detectivi, iar oriunde analogia încetează să mai fie adevărată, îți voi spune. Acelea sunt adesea momentele cele mai interesante, pentru că locul în care imaginea se rupe e de obicei locul în care mașina își surprinde constructorii.

Capitolul 1 — Schimbarea de gândire

1.1 Rețeta și ucenicul

Iată un mic exercițiu de programare. Scrie o funcție care primește o fotografie și întoarce `true` dacă în ea apare o pisică.

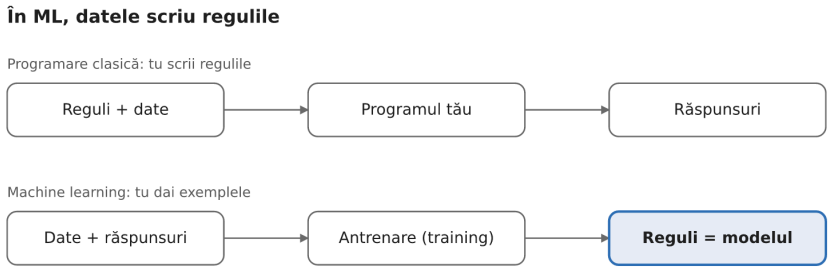
Începi încrezător. Pisicile au urechi ascuțite, așa că cauți două triunghiuri deasupra unei forme rotunjite. Apoi îți amintești de pisicile fotografiate din spate, de cele făcute covrig, de cele ascunse sub plapumă, din care se vede doar coada. Adaugi o regulă pentru mustăți și descoperi că și o perie de păr are mustăți, cel puțin din punctul de vedere al regulilor tale. Adaugi o regulă pentru blană, iar un papuc pufos începe să treacă drept pisică. La sfârșitul zilei ai sute de linii de condiții și o funcție care tot greșește mult prea des.

În acest timp, orice copil de trei ani face asta perfect. Întreabă-l cum și nu îți poate spune. Cunoașterea e reală; doar că nu există sub formă de reguli. Și ăsta e zidul de care se lovește programarea clasică.

Programarea clasică e arta de a scrie rețete. Fișa bunicii spune două sute de grame de făină, trei ouă, patruzeci de minute la 180 de grade, iar dacă o urmezi, obții prăjitura. Orice program ai scris vreodată e o rețetă de felul acesta: pași precișI, stabiliți dinainte de cineva care a înțeles problema. Funcționează minunat atâta timp cât lumea arată așa cum se așteaptă rețeta. Se strică cel mai rău într-o anumită situație, una foarte des întâlnită: când nu poți pune în cuvinte ce vrei.

Așa că imaginează-ți o altă bucătărie. Nu există nicio fișă cu rețeta, doar un ucenic. Îl lași să guste o sută de prăjituri reușite și o sută ratate, fiecare cu eticheta ei. La început, judecățile ucenicului sunt la întâmplare. Dar după fiecare prăjitură îi spui dacă a avut dreptate, iar încet-încet își formează un simț interior pentru proporțiile bune. Cere-i să le scrie și nu poate. Dă-i prăjitura numărul 201, pe care n-a gustat-o niciodată, și o judecă totuși corect.

Ucenicul acesta e machine learning („învățare automată”: calculatorul învață din exemple, nu din instrucțiuni). Săgețile s-au inversat: în loc să-i dai calculatorului reguli și date și să primești răspunsuri, îi dai datele împreună cu răspunsurile, iar el îți dă regulile.



Istoria filtrului de spam arată de ce a contat atât de mult această inversare. Primele filtre erau fișe de rețetă: blochează e-mailurile care conțin „FREE MONEY”, blochează expeditorii de pe o listă neagră, blochează orice are cinci semne de exclamare. Spammerii s-au adaptat în câteva zile. „FR3E M0NEY” trecea nestingherit, în fiecare dimineață apăreau domenii noi, iar fiecare regulă scrisă de apărători

pornea o nouă rundă dintr-o cursă a înarmărilor pe care n-o puteau câștiga. În 2002, programatorul Paul Graham a publicat un eseu intitulat *A Plan for Spam*, care a ajutat la răspândirea unei alte idei: nu mai scrie reguli, lasă filtrul să învețe din mesajele pe care fiecare utilizator le marcase deja ca spam sau nu. Filtrul urma să descopere singur ce cuvinte, linkuri și tipare trădează un spam. Când spammerii își schimbau tactica, nimeni nu mai rescria nimic; filtrul învăța pur și simplu din exemplele noi. Filtrele mici de felul acesta își puteau actualiza numărătorile de cuvinte chiar în clipa în care utilizatorul apăsa „spam", pentru că antrenarea lor era puțin mai mult decât o numărătoare; modelele mai mari sunt de obicei reantrenate periodic, cu cele mai noi exemple adăugate. Oricum ar fi, oamenii schimbă datele, nu codul. Cursa înarmărilor nu s-a terminat, dar apărătorii aveau în sfârșit o armă care ținea pasul.

S-ar putea să fii chiar acum într-o situație asemănătoare. Imaginează-ți o echipă de suport care primește sute de tichete pe zi și trebuie să trimită fiecare tichet către Billing, Infrastructure sau Mobile. Cineva scrie reguli pe cuvinte-cheie, iar „invoice" merge la Billing. Merge pentru cazurile evidente. Apoi un client scrie „Mi s-au luat banii de două ori după ce a crăpat aplicația", care nu pomenește nici de facturi, nici de telefoane, iar regulile ridică din umeri. Însă în sistemul de ticketing stau tichetele strânse în ani de zile, pe care oamenii le-au direcționat deja corect. Fiecare dintre ele e o prăjitură cu etichetă. Ucenicul le așteaptă.

Din ce e făcut acest ucenic? Pe dedesubt, un model tot o funcție e, la fel ca cele pe care le scrii tu. Diferența e că are spații goale în el: numere numite **parameters** („parametri") sau **weights** („ponderi", adică cât de mult cântărește fiecare lucru în decizie), ale căror valori nu sunt scrise de nimeni de mână. **Training** („antrenarea") e căutarea acelor valori care fac funcția să se potrivească exemplelor. Când antrenarea se termină, „regula" învățată de ucenic nu e un `if` pe care l-ai putea citi într-un code review. E o listă lungă de numere: câteva zeci pentru un model mic, miliarde pentru unul mare. Oamenii numesc modelele cutii negre. Cutia nu e magică; e scrisă într-o limbă a numerelor, nu într-o limbă a condițiilor, o limbă pe care oamenii o citesc foarte greu.

Asta nu înseamnă că rețeta a murit. Validarea unui IBAN, calculul TVA-ului, verificarea că o dată e în trecut: aici regulile sunt cunoscute, exacte și adesea scrise în lege. Un ucenic ar fi mai lent, mai greu de auditat și, din când în când, greșit cu toată încrederea. Dacă poți scrie regula în douăzeci de linii și se schimbă rar, scrie regula. Machine learning își merită costul considerabil (în date, în monitorizare, în a trăi cu incertitudinea) doar când regulile sunt prea multe, prea subtile sau se schimbă prea repede.

Mai e un lucru despre ucenic, și el va bântui restul cărții. Un ucenic om înțelege *de ce* zahărul face prăjitura dulce. Un model ca acesta nu are o asemenea înțelegere; observă doar ce merge împreună. Să zicem că toate prăjiturile reușite din fotografiile tale au stat, întâmplător, pe o farfurie albă, iar toate cele ratate pe una albastră. Modelul va învăța că „farfurie albă înseamnă prăjitură bună", va lua notă maximă pe exemplele tale și va fi foarte mulțumit de el însuși. Niciun scor pe exemple care împărtășesc aceeași întâmplare nu va dezvălui asta. Afli adevărul testând modelul pe prăjituri la care farfuria și calitatea nu se potrivesc, o prăjitură bună pe o farfurie albastră, sau întrebând modelul pe ce intrări se sprijină cel mai mult.

Ucenicul învață exact ce îl învață exemplele, nimic mai mult și nimic mai puțin. Ceea ce înseamnă că cea mai importantă întrebare din machine learning nu e ce algoritm să folosești. E de unde vin exemplele și cine a scris răspunsurile pe ele.

1.2 Răspunsurile de pe spatele paginii

Undeva în compania ta există o coloană de date la care nu se gândește nimeni și care ar putea fi cel mai valoros lucru pe care îl aveți.

Nu stă într-un data warehouse sofisticat. E câmpul `resolved_by_team` de pe tichetele vechi de suport. E `duration_seconds` de la fiecare build de CI (continuous integration, „integrare continuă”: robotul care compilează și testează codul la fiecare modificare) de la începutul proiectului. E căsuța în care un inginer, după ce a investigat un test picat, l-a marcat drept „flaky” („capricios”, un test care pică uneori fără motiv real) sau „bug real”. Fiecare dintre ele e un răspuns, scris de cineva care știa, atașat unei întrebări. Adună destule și ai exact ce îi trebuie ucenicului: prăjituri cu etichete.

Învățarea din întrebări împerecheate cu răspunsurile lor se numește **supervised learning** („învățare supervizată”, adică sub ochiul unui profesor care știe răspunsurile), iar versiunea ei de zi cu zi e ceva ce ai făcut la școală. Rezolvi o problemă din culegere și te uiți la sfârșitul culegerii să verifici. Răspunsul era B; tu ai zis A. Nimeni nu-ți explică de ce. Primești doar verdictul, așa că îți ajustezi puțin gândirea și mergi mai departe. După câteva mii de probleme, îți rămâne ceva în cap pe care nu l-ai putea pune complet în cuvinte, dar care rezolvă corect următoarea problemă.

Oricine a învățat germană cu cartonașe știe cât de ciudat și de frumos e sentimentul acesta. Repeți sute de substantive cu articolul pe spate și într-o zi observi că pur și simplu *știi* că un cuvânt terminat în *-ung* ia *die*. Niciun profesor nu ți-a enunțat regula. S-a condensat din exemple, cum se formează bruma pe un geam. Iată toată promisiunea învățării supervizate, într-un singur moment mărunt: regula n-a fost dată niciodată, doar exemplele și verdictele.

Întoarce-te acum la coloana `duration_seconds`. Fiecare build din istoricul CI vine cu împrejurările lui: câte fișiere s-au schimbat, ce module au atins, câte teste au rulat, dacă cache-ul era cald. În vocabularul domeniului, aceste împrejurări sunt **features** („trăsături”: proprietățile măsurabile ale fiecărui build, la care modelul are voie să se uite). Durata reală e **label**-ul („eticheta”: răspunsul de pe spatele paginii). Dă-i unui model câteva mii de build-uri vechi și învață să estimeze durata următorului build înainte ca acesta să înceapă, ceea ce e chiar util când planifici mașinile de build sau când vrei să avertizezi un developer că schimbarea lui va ține CI-ul ocupat patruzeci de minute. Alte trăsături ar putea fi vârsta unui client, culoarea unui pixel sau dacă un cuvânt apare sau nu într-un text. Așezarea trăsăturilor într-un tabel, cu un rând pentru fiecare exemplu și o coloană pentru fiecare trăsătură, e doar un aranjament comod, firesc pentru înregistrări de business ca aceste build-uri; pentru fotografii, text și sunet, a căror materie primă sunt pixelii, cuvintele și eșantioanele, povestea e alta, iar un capitol ulterior o spune.

Căsuța pentru testele flaky funcționează la fel, doar că răspunsul e o categorie, nu un număr. Modelul vede istoricul testului, mesajul de eroare, mașina pe care a rulat și ora din zi și învață să sugereze „probabil flaky” înainte să se uite cineva. Prezicerea unui număr se numește **regression** („regresie”); prezicerea unei categorii se numește **classification** („clasificare”). Împreună, acoperă cea mai mare parte din ce face învățarea supervizată.

Mai e un ingredient, ușor de trecut cu vederea pentru că sună atât de banal. După fiecare încercare, ceva trebuie să spună, printr-un singur număr, cât de greșit a fost răspunsul. Numărul acela e **loss**-ul („pierderea”), iar felul în care îl definești modelează discret tot ce învață modelul. Să zicem că prezici durata livrărilor. Dacă doar aduni minutele de eroare, o întârziere de o oră contează cât zece întârzieri de câte șase minute. Clienții nu simt așa: o livrare întârziată cu o oră aduce un telefon furios, în timp ce

mai multe livrări întârziate puțin trec neobservate. Așa că alegerea obișnuită e să ridici erorile la pătrat, iar atunci o greșeală de șaizeci de minute cântărește mult mai mult decât zece greșeli de câte șase. Loss-ul îl aleg oamenii, nu algoritmul, și el spune ce anume contează pentru tine. Pătratele sunt populare din acest motiv și pentru că dau o vale netedă, ușor de coborât; dacă fiecare minut de întârziere costă cu adevărat la fel, poți aduna pur și simplu minutele, iar modelul învață să țintească livrarea tipică, în loc să fie tras spre rarele dezastre.

Pentru categorii, loss-ul obișnuit are și el o personalitate. E blând cu un model care a fost nesigur și a greșit, și necruțător cu unul care a fost sigur pe el și a greșit. Știi asta de la prognoza meteo. O meteorologă care a spus „50% șanse de ploaie” într-o zi ploioasă nu s-a făcut de râs. Una care a spus „99% soare” în aceeași zi și-a pierdut credibilitatea pentru o lună. Loss-ul obișnuit se uită doar la probabilitatea pe care modelul a dat-o lucrului care s-a întâmplat de fapt și îi ia logaritmul: să-i dai adevărului 50% costă puțin, să-i dai 1% costă mult, iar să-i dai 0% costă infinit, motiv pentru care un clasificator bine antrenat nu spune aproape niciodată „imposibil”.

Până aici povestea pare ordonată: aduni răspunsuri, definești un loss, lași modelul să învețe. Aici însă povestea ia o întorsătură.

Imaginează-ți o echipă sub presiune. Un test pică vineri după-amiaza, release-ul e blocat și cineva trebuie să facă merge înainte de weekend. Testul e marcat „flaky”, build-ul e rulat din nou, toată lumea pleacă acasă. Se întâmplă din nou vinerea următoare, și în cea de după. Peste câteva luni, cineva antrenează un model pe căsuța de teste flaky. Modelul învață, fidel și corect, că testele care pică vineri după-amiaza sunt flaky. Modelul n-a greșit. A învățat exact ce era scris pe spatele paginii.

Asta e adevărul incomod din inima învățării supervizate: un model poate fi doar atât de bun cât răspunsurile lui, iar răspunsurile reale sunt scrise de oameni obosiți, de scripturi cu bug-uri și de procese vechi pe care nu le-a pus nimeni la îndoială. Răspunsurile din culegere sunt verificate de un editor; ale tale, de obicei, nu. De aceea, în proiectele reale, strângerea și curățarea etichetelor costă adesea mai mult decât construirea modelului propriu-zis, iar cei mai buni ingineri de machine learning petrec surprinzător de mult timp citind exemple individuale în loc să regleze algoritmi. Modelul nu poate observa singur etichetele greșite, pentru că etichetele sunt singura lui idee despre adevăr. Există totuși un truc util: antrenezi un model, apoi te uiți la exemplele la care contrazice, sigur pe el, eticheta; multe se dovedesc a fi căsuțele bifate vineri după-amiaza, iar lista aceasta scurtă e un loc mult mai bun de unde să înceapă un om să citească decât tot muntele.

Ceea ce ridică o întrebare firească. Dacă nu ai deloc răspunsuri? Nicio etichetă, nicio căsuță, doar un munte de date pe care nu le-a sortat nimeni vreodată. Poate o mașină să învețe ceva din așa ceva?

1.3 Sortând dulapul pe întuneric

Se ia curentul într-o seară de iarnă, exact când ai răsturnat pe pat un coș de rufe curate. N-ai lanternă și nici răbdare să aștepți, așa că începi să sortezi pe pipăite. Lucrurile groase, cu mâneci lungi, într-o grămadă. Cele subțiri și scurte în alta. Cele țepene, cu nasturi, într-a treia, cele mici și moi într-a patra. Când revine lumina, grămezile au perfect sens: pulovere, tricouri, cămăși, șosete. Nimeni nu ți-a spus că există aceste categorii. Le-ai găsit pentru că hainele chiar erau diferite, iar lucrurile asemănătoare au ajuns împreună.

Asta e răspunsul la întrebarea de la sfârșitul subcapitolului trecut. Da, o mașină poate învăța fără răspunsuri, cam așa cum ai sortat tu rufele. Învățarea fără etichete se numește **unsupervised learning** („învățare nesupervizată”: fără profesor, fără răspunsuri), iar forma ei cea mai des întâlnită, găsirea grupurilor de lucruri asemănătoare, se numește **clustering** („grupare”, de la *cluster*, ciorchine).

O vei aprecia cel mai mult la trei dimineața. A sunat o alertă, iar sistemul de loguri arată patruzeci de mii de stack traces („urme de stivă”, raportul complet al unei erori) din ultima oră. Nimeni nu poate citi patruzeci de mii de stack traces. Dar cele mai multe sunt aceleași câteva probleme repetate, decorate cu timestamp-uri, ID-uri de utilizator și numere de request diferite. Grupează urmele după asemănare și zidul de text se reduce la o listă scurtă: eroarea asta, de douăsprezece mii de ori; aceea, de nouă mii; una nouă și ciudată, de șaptesprezece ori, începând de la 02:41. Citești lista, nu zidul, iar cea nouă și ciudată e probabil incidentul tău. Ca să ajungi aici, mai întâi se scoate zgomotul: timestamp-urile, ID-urile de utilizator și numerele de request sunt înlocuite cu niște marcaje generice, astfel încât două urme ale aceluiași bug devin un text aproape identic. Apoi fiecare urmă e transformată în numere, de exemplu numărând ce cuvinte și ce nume de funcții conține, iar urmele asemănătoare sunt cele ale căror liste de numere sunt apropiate.

Cum decide un calculator ce merge împreună? Metoda clasică, **K-Means** („K medii”: găsește K centre, fiecare fiind media punctelor din grupul lui), funcționează cum ar alege un lanț de pizzerii unde să-și pună punctele de livrare într-un oraș nou. Decizi că îți permiți, să zicem, cinci puncte și le arunci pe hartă la întâmplare. Fiecare casă e alocată celui mai apropiat punct. Apoi fiecare punct se mută în mijlocul caselor pe care le deservește. Acum unele case sunt mai aproape de alt punct, așa că se mută, iar punctele se deplasează din nou. Repetă, și după câteva runde nu se mai schimbă nimic: punctele stau în centrele cartierelor naturale. Observă cine a ales numărul cinci. Tu, nu algoritmul. Ca să-l alegi bine, de obicei încerci mai multe valori și urmărești cât de mult se strâng grupurile cu fiecare punct de livrare în plus: primele ajută mult, apoi câștigurile se aplatizează, iar cotul acelei curbe e o alegere rezonabilă. Judecata finală rămâne a ta: deschizi grupurile și te întrebi dacă înseamnă ceva. Există și metode care găsesc singure numărul de grupuri.

Aceeași idee devine un sistem de avertizare timpurie. Fiecare server dintr-o flotă raportează încărcarea CPU (central processing unit, procesorul), memoria, latența, rata de erori și traficul. Serverele sănătoase ajung în câteva grupuri strânse, ca niște case în jurul punctelor de pizza. Un server care stă departe de toate grupurile se poartă altfel decât oricare altul, chiar dacă nicio metrică n-a depășit încă pragul de alertă. Găsirea unor astfel de excepții se numește **anomaly detection** („detectarea anomaliilor”) și e unul dintre cele mai practice lucruri pe care le face învățarea nesupervizată. Unde începe „departe” decizi tu, de obicei puțin dincolo de locul în care au stat aproape toate serverele sănătoase în ultima lună: trage linia prea aproape și ajungi la oboseala de alerte, prea departe și serverul care o ia razna e prins târziu.

Grupurile nu sunt singura structură ascunsă care merită găsită. Ține o cană de cafea în fața unei lanterne și uită-te la umbra ei pe perete. Din lateral vezi toarta și ai recunoaște-o oriunde drept cană. De sus, e un cerc simplu, care ar putea fi un pahar, o doză sau o monedă. Dacă ai avea voie să păstrezi o singură umbră, ai roti cana până când umbra ar spune cel mai mult.

Fișele tehnice au aceeași proprietate. Un laptop vine cu treizeci de numere: nuclee, frecvență, RAM, memorie grafică, preț, greutate, diagonală, autonomie și altele. Dar numerele acestea nu sunt independente. Mașinile cu mai multe nuclee au de obicei mai mult RAM, un GPU (graphics processing unit, cipul plăcii video) mai bun și un preț mai mare; cele mai ușoare au de obicei ecrane mai mici și

autonomie mai mare. Cele treizeci de numere spun, în mare, două povești: „cât de puternic” și „cât de portabil”. **Principal component analysis** („analiza componentelor principale”), pe scurt PCA, găsește automat astfel de direcții-rezumat și te lasă să descrii fiecare laptop cu două numere în loc de treizeci, pierzând surprinzător de puțin. E versiunea matematică a alegerii celei mai grăitoare umbre. Caută direcția în care laptopurile diferă cel mai mult, apoi următoarea direcție cu cea mai mare variație, perpendiculară pe prima, ca să nu repete ce spune deja aceea, și tot așa. Să păstrezi două direcții înseamnă să renunți la diferențele mai mici din toate celelalte, ceea ce e în regulă dacă erau mai ales zgomot și dureros dacă acolo trăia o proprietate rară, dar importantă. Inginerii o folosesc ca să strângă un dashboard cu două sute de metrici pe serviciu într-un singur grafic bidimensional, unde serviciul care o ia razna față de frații lui sare imediat în ochi.

Toate acestea sunt utile. Acum vine partea care ar trebui să te facă puțin bănuitor.

În 1976, sonda Viking a NASA a fotografiat pe Marte un deal care semăna fără dubiu cu o față umană privind în sus din deșert. Ani de zile a alimentat teorii despre civilizații antice. Mai târziu, imagini mai clare au arătat un platou erodat oarecare. Fața nu existase niciodată; creierul uman e pur și simplu construit să găsească fețe, în nori, în prize, în fața unei mașini. Algoritmii de clustering au aceeași boală. Dă-i lui K-Means zgomot pur și cere-i cinci grupuri, și îți dă cinci grupuri, fiecare cu un centru elegant, de fiecare dată. Nu răspunde niciodată „nu e nimic aici”.

Mai e o capcană, mai discretă. Când ai sortat rufele, tu ai ales să sortezi după grosime. Un algoritm sortează după ce se întâmplă să măsoare noțiunea lui de distanță. Grupează angajații după salariul în euro și anii de experiență, iar o diferență de 1.000 de euro înghite o diferență de zece ani, doar pentru că numerele sunt mai mari. „Grupurile naturale” pe care le descoperi sunt atunci simple benzi salariale. De aceea datele sunt de obicei rescalate înainte de clustering, ca fiecare trăsătură să vorbească la un volum asemănător.

Așadar, învățarea nesupervizată îți dă ipoteze, nu concluzii. Spune: lucrurile astea par să țină împreună, acela pare ciudat. Dacă asta înseamnă ceva e decizia ta, iar singurul mod sigur de a o lua e să deschizi fiecare grup și să te uiți la exemple reale.

Ceea ce ne aduce la întrebarea cu care trebuie să se confrunte, până la urmă, orice metodă din carte, supervizată sau nu. Cum știi, cinstit, că un model funcționează? Echipele au sărbătorit prea devreme mai des decât ai crede.

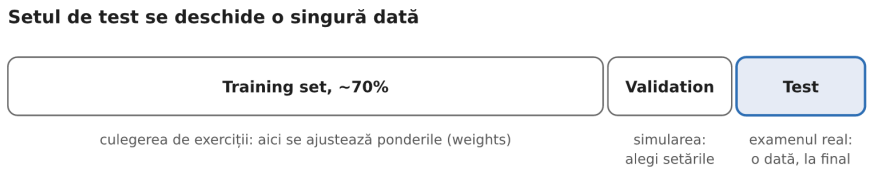
1.4 Examenul pe care nu ai voie să-l vezi

Imaginează-ți o echipă care a petrecut două luni construind un router de tichete. Demo-ul merge minunat: îi dau o mie de tichete și trimite 98% dintre ele la echipa potrivită. Se taie tort. La o lună după lansare, un lider de suport trimite discret un spreadsheet. În producție, routerul nimerește cam de șapte ori din zece. Nimeni nu găsește vreun bug. Codul e curat, modelul se încarcă bine, datele de intrare arată normal.

Ce a mers prost e aproape jenant de simplu și se întâmplă tot timpul. Cele o mie de tichete din demo fuseseră tichete din care modelul învățase. Echipa îi măsurase memoria, nu priceperea. E ca și cum ai judeca un candidat la interviu punându-i exact întrebările pe care i le-ai dat să le învețe cu o seară înainte.

Gândește-te cum funcționează examenul pentru permisul auto, pentru că rezolvă problema destul de elegant. Înveți dintr-o culegere de chestionare, pe care o poți parcurge de câte ori vrei, cu răspunsuri cu tot. Pe parcurs dai simulări, care îți spun dacă metoda ta de învățat funcționează și ce capitole mai trebuie reluate. Apoi vine examenul real, o singură dată, la final. Dacă cineva ți-ar fi strecurat dinainte întrebările reale, nota ar fi cât se poate de reală, dar n-ar arăta deloc dacă știi să conduci.

Machine learning copiază exact această structură. **Training set**-ul („setul de antrenare”) e culegerea: pe el se potrivesc ponderile modelului. **Validation set**-ul („setul de validare”) e simularea: îl folosești ca să compari abordări și să alegi setări. **Test set**-ul („setul de test”) e examenul real și înseamnă ceva doar dacă rămâne sigilat până la final.



Plicul sigilat e mai greu de ținut sigilat decât pare. Să zicem că evaluezi pe setul de test, observi că scorul e cam mic, modifici ceva, evaluezi din nou, modifici din nou, de douăzeci de ori, și la final raportezi cel mai bun număr. Fiecare privire aruncată pe furiș a lăsat să scape puțină informație, iar după douăzeci de priviri ai reglat modelul pe acel examen anume. Orice inginer cunoaște versiunea de CI a acestui lucru: rulezi din nou un pipeline capricios până iese verde și declari build-ul sănătos. Rularea verde chiar s-a întâmplat. Doar că îți spune mult mai puțin decât pare.

Ajută să ții separate în minte două feluri de numere. Parametrii, spațiile goale din funcție pe care le completează antrenarea, sunt ce învață modelul singur din culegere: cunoștințele din capul elevului. **Hyperparameters** („hiperparametrii”, parametrii de deasupra parametrilor) sunt alegeri despre cum are loc învățarea: cât de mare e modelul, cât de repede învață, cât timp se antrenează. În imaginea permisului auto, ei sunt câte ore pe zi înveți și ce culegere cumperi. Modelul își stabilește parametrii; tu stabilești hiperparametrii, și îi stabilești urmărind simulările. Și urmărirea simulărilor lasă să scape informație, exact în același fel, doar că e permisă: după multe runde de reglaj, scorul de validare ajunge să fie ușor măgulitor, și tocmai de aceea există setul de test, singurul număr după care nimeni n-a avut voie să regleze nimic.

Când datele sunt puține, să păstrezi deoparte o simulare întreagă pare risipă, și există un truc elegant pentru asta. Împarte datele în cinci părți și antrenează de cinci ori, folosind de fiecare dată altă parte drept simulare și celelalte patru drept culegere. Fiecare exemplu e folosit de patru ori la exerciții și o dată la simulare, iar media celor cinci scoruri e un verdict mai stabil decât oricare singur. Asta e **cross-validation** („validare încrucișată”: părțile își schimbă pe rând rolurile).

Mai e o lecție ascunsă în povestea routerului de tichete și ea privește ce i se dă elevului să învețe. Un model vede doar trăsăturile pe care i le dai. Dă-i un timestamp brut precum 2026-03-14T08:47:12Z și majoritatea modelelor pot face foarte puțin cu el; e doar un număr mare care tot crește. Dă-i același moment ca „ora din zi: 8”, „ziua săptămânii: sâmbătă” și „zi de sărbătoare: nu”, și dintr-odată poate

învăța că dimineața de sâmbătă e liniștită, iar cea de luni e haos. Transformarea datelor brute în intrări care scot în evidență ce contează se numește **feature engineering** („ingineria trăsăturilor”), iar pe datele obișnuite de business contează adesea mai mult decât alegerea algoritmului.

Acum răsturnarea pentru care niciun examen auto nu te pregătește. Examenul de condus e tras din același regulament an de an. Datele tale, nu. Clienții își schimbă obiceiurile, prețurile se mișcă, apare un concurent, argoul se schimbă, o pandemie rearanjează totul în două săptămâni. Un model care a strălucit pe datele de anul trecut poate să-și piardă discret avantajul anul acesta fără să se schimbe nicio linie de cod. Alunecarea lentă a datelor din spatele acestei erodări se numește **data drift** („deriva datelor”: lumea alunecă încet de sub model) și e motivul pentru care modelele din producție au nevoie de monitorizare ca orice alt serviciu, cu alerte când intrările sau acuratețea lor încep să rătăcească. Când răspunsurile adevărate sosesc abia după câteva săptămâni, urmărești intrările, care sosesc imediat: dacă tichetele din săptămâna asta sunt mai lungi, vin din țări noi sau pomenesc cuvinte pe care modelul le-a văzut rar, lumea s-a mișcat înainte ca vreun număr de acurateţe să poată spune asta. Remediul obișnuit e lipsit de strălucire: reantrenarea pe date recente, periodic sau ori de câte ori sună alarmele.

Schimbă și felul în care ar trebui să sigilezi plicul. Pentru orice se desfășoară în timp, nu alege niciodată setul de test la întâmplare. Antrenează pe trecut și testează pe perioada cea mai recentă, așa cum va fi folosit modelul de fapt. Un amestec aleatoriu îl lasă pe model să învețe din marțea viitoare în timp ce e testat pe lunea trecută, iar un model care a văzut viitorul va părea întotdeauna genial.

Ajunge despre cum judeci un elev. E timpul să cunoaștem câțiva elevi adevărați, începând cu cel mai modest dintre toți: o riglă dreaptă, o mână de cuie și câteva arcuri.

Mai departe. Dacă te-ai întrebat cum e prins un model care trișează, ca acela cu farfuria albă, subcapitolul 10.2 răspunde, iar dacă te-ai întrebat câte exemple etichetate sunt de ajuns, răspunde subcapitolul 5.4. Dacă poate cineva citi ce a învăţat un model din lunga lui listă de numere e întrebarea subcapitolului 10.6. Iar ce înseamnă să ții un model util atunci când lumea se îndepărtează de datele lui e povestea subcapitolului 11.5.

Capitolul 2 — Trusa de unelte clasică

2.1 O riglă, câteva arcuri și un variator

Ia o scândură și bate câte un cui pentru fiecare apartament vândut anul trecut în cartierul tău: de la stânga la dreapta e suprafața, de jos în sus e prețul. Acum ai un nor de cuie care urcă, aproximativ, pe măsură ce mergi spre dreapta. Leagă de fiecare cui un arc scurt, îndreptat drept în sus sau în jos, și prinde toate arcurile de o riglă lungă și rigidă. Apoi dă-i drumul.

Unde se oprește rigla?

Se bălăngăne, arcurile trag de sus și de jos, și se așază în singura poziție în care energia totală din arcuri e cât se poate de mică. Poziția aceea e chiar dreapta găsită de **linear regression** („regresie liniară”: cea mai bună linie dreaptă printr-un nor de puncte), cel mai vechi și încă unul dintre cele mai utile modele din carte. N-a trebuit să calculezi nimic. Au făcut-o arcurile. Un calculator n-are arcuri, iar pentru o linie dreaptă nici nu-i trebuie: o formulă dă poziția de repaus direct, dintr-un singur calcul. Pentru modelele care n-au o astfel de formulă, calculatorul imită pas cu pas rigla care se bălăngăne, și asta e coborârea pe munte din capitolul următor.

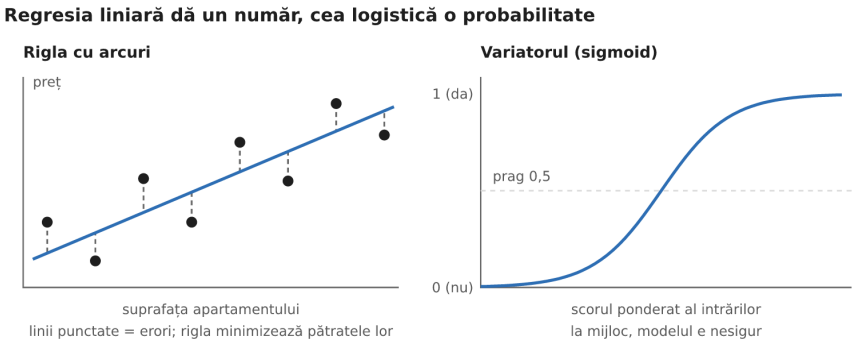
De ce arcuri și nu sfori? Pentru că energia dintr-un arc întins crește cu *pătratul* întinderii. Trage un arc de două ori mai departe și va înmagazina de patru ori mai multă energie. Exact asta minimizează regresia liniară: suma pătratelor distanțelor verticale dintre puncte și dreaptă. Și explică ceva ce poate ai văzut mergând prost în practică. Adaugă un penthouse vândut de zece ori mai scump decât orice altceva de pe stradă, iar arcul lui e atât de întins încât trage toată rigla spre el. Celelalte predicții se înrăutățesc, în ansamblu, ca să facă loc unui singur punct extrem, un **outlier** („valoare aberantă”, punctul care stă departe de toate celelalte). Un singur cui poate îndoi linia pentru toată lumea.

Acest mic model fizic apare mereu în inginerie. Pune pe un grafic latența unui endpoint de API (application programming interface, „interfața prin care programele vorbesc între ele”) în funcție de mărimea payload-ului cererii și de obicei obții un nor care urcă. Lasă arcurile să se așeze și rigla îți dă o propoziție pe care o poate folosi un product manager: fiecare megabyte în plus costă cam atâtea milisecunde, peste o întârziere fixă. Panta și înălțimea de pornire sunt singurele două numere învățate de model, iar spre deosebire de aproape orice alt model din carte, le poți citi și te poți certa cu ele. Cu multe intrări, să citești ponderile ca pe un clasament al importanței cere mai multă grijă decât pare, după cum va arăta un capitol ulterior.

Dar unele întrebări nu pot primi răspuns de la o riglă, pentru că răspunsul nu e un număr. Va eșua acest deployment? E plata asta o fraudă? Va renunța clientul acesta? Întreabă o linie dreaptă și îți va prezice voioasă 1,7 sau minus 0,3, care nu înseamnă nimic ca răspunsuri la o întrebare cu da sau nu.

Soluția e fermecătoare. Mai întâi calculezi același fel de scor ponderat, adunând dovezile. Apoi treci scorul printr-un variator de lumină. Când scorul e foarte mic, lumina e stinsă: zero, nu. Când e foarte mare, lumina e aprinsă complet: unu, da. Între ele, lumina crește treptat, iar intensitatea ei o citești ca pe o probabilitate. O valoare de 0,83 înseamnă „destul de probabil da”. Curba în formă de S a variatorului se numește **sigmoid** („sigmoidă”, de la litera grecească sigma, echivalentul lui S, pentru că curba are forma unui S), iar întregul aranjament, scor plus variator, e **logistic regression** („regresie logistică”), care, în ciuda numelui, e o metodă de clasificare. O precauție despre imagine: luminozitatea unui

variator e pur și simplu poziția butonului, dar 0,83 al modelului e o probabilitate onestă doar dacă antrenarea l-a făcut așa, iar multe modele se dovedesc mai sigure pe ele decât le justifică istoricul; verificarea asta se numește calibrare.



Imaginează-ți-o aplicată pe istoricul deployment-urilor tale. Fiecare deployment trecut vine cu câteva fapte: câte linii s-au schimbat, câte servicii au fost atinse, dacă s-a schimbat schema bazei de date, la ce oră a fost lansat. Fiecare e etichetat după cum a urmat sau nu un rollback („revenirea la versiunea anterioară”). Modelul învață cât de mult ar trebui să împingă scorul fiecare fapt, iar fiecare deployment nou primește un risc între 0 și 1. Ai putea cere un al doilea reviewer peste 0,3 și ai putea refuza deployment-urile de vineri seara peste 0,6. Observă că pragurile nu sunt decizia modelului; sunt ale tale și spun cât risc poate înghiți echipa ta. Tema asta va reveni cu putere.

Sub ambele modele stă aceeași structură: fiecare intrare e înmulțită cu o **weight** („pondere”) care spune cât contează, rezultatele se adună, iar o constantă numită **bias** („deplasare”) mută totul în sus sau în jos. Regresia liniară se oprește aici. Cea logistică adaugă variatorul. Iar locul în care variatorul arată exact 0,5, linia dintre da și nu, se numește **decision boundary** („granița de decizie”). Ține minte ideea, pentru că în capitolul următor vei descoperi că regresia logistică e, discret, un singur neuron artificial.

Până aici, totul e ordonat. Acum rigla își arată limitele, iar prima limită e chiar în numele ei: e dreaptă.

Orice inginer care a făcut un load test („test de încărcare”) pe un serviciu a văzut o relație care nu e dreaptă. La 20% CPU, latența e bună. La 40%, tot bună. La 60%, bună. La 85%, cererile încep să stea la coadă și latența o ia brusc în sus, ca o crosă de hochei. O riglă dreaptă prin aceste puncte face ceva teribil: subestimează latența exact în zona care te interesează cel mai mult, aproape de saturație. Adesea, remediul nu e un algoritm mai sofisticat, ci intrări mai bune. Dă-i modelului încărcarea la pătrat sau logaritmul unei mărimi, iar o linie dreaptă în noile intrări devine o curbă în cele vechi. Ce intrări suplimentare să-i dai afli mai ales desenând datele și cunoscând domeniul: cozile explodează aproape de saturație, iar mărimile și timpii de așteptare tind să se întindă pe multe ordine de mărime. Apoi încerci candidații pe simulare; capitolul următor e povestea modelelor care învață singure astfel de curbe.

A doua limită e mai subtilă și nu ține de fapt de riglă. Vânzările de înghețată și înecurile cresc și scad împreună, frumos, pe o linie dreaptă. Înghețata nu provoacă înecuri; vara le provoacă pe amândouă. În datele de inginerie, aceeași iluzie e peste tot. Serviciile cu cele mai multe teste au adesea cele mai multe rapoarte de bug-uri, nu pentru că testele creează bug-uri, ci pentru că acelea sunt serviciile cele mai mari, mai aglomerate și mai critice. O pondere mare înseamnă că două lucruri se mișcă împreună. Nu

înseamnă niciodată, singură, că unul îl cauzează pe celălalt: un model potrivit pe date istorice poate doar să raporteze ce a mers împreună, iar ca să deosebești o cauză de un simplu tovarăș de drum îți trebuie un experiment, cum sunt A/B testele care împart traficul la întâmplare, subiect pe care îl reia ultimul capitol.

Rigla face o presupunere puternică despre lume: că e, mai mult sau mai puțin, o linie dreaptă. Dar dacă ai refuza să presupui orice și ai întreba pur și simplu vecinii?

2.2 Spune-mi cine îți sunt vecinii

Tocmai te-ai mutat într-un oraș nou și ai găsit un apartament care îți place. Prețul cerut pare mare, dar n-ai idee ce e normal aici. Nu construiești un model de prețuri. Cauți cele cinci apartamente cele mai asemănătoare care s-au vândut recent, pe aceeași stradă, cu suprafață și etaj asemănătoare, și faci media prețurilor lor. Dacă prețul cerut e mult peste medie, negociezi.

Fără să-ți dai seama, ai rulat **k-nearest neighbors** („cei mai apropiați k vecini”), cu k egal cu cinci. E poate cel mai omenesc algoritm din carte, pentru că așa judecăm aproape orice lucru nou. Un restaurant e probabil bun dacă cei trei prieteni cu gusturile cele mai apropiate de ale tale l-au adorat. Un film merită probabil văzut dacă oamenii cărora le-au plăcut aceleași filme ca ție l-au notat bine. Multe casete „clienții ca tine au mai cumpărat” de pe site-urile de cumpărături sunt ideea asta rulată la scară industrială. Sistemele reale de recomandare adaugă destul de multe peste ea.

Lucrează discret și în uneltele de inginerie. Imaginează-ți un bug tracker care primește un raport nou: „Butonul de login nu face nimic pe Safari”. Înainte să-l trieze cineva, unealta îi caută cei mai apropiați vecini printre problemele existente și găsește „Click-ul de sign-in nu răspunde în browserul de pe macOS”, deschis săptămâna trecută, deja alocat, deja pe jumătate rezolvat. Cuvintele aproape că nu se suprapun, așa că unealta nu se poate mulțumi să numere cuvintele comune; ea măsoară cât de apropiate sunt sensurile, printr-un truc pe care îl explică un capitol ulterior, iar duplicatul e prins înainte ca doi ingineri să piardă o după-amiază pe aceeași problemă. Sau imaginează-ți o unealtă de incidente care, în clipa în care sună o alertă, îți arată cele cinci incidente trecute cele mai asemănătoare, cu linkuri spre felul în care a fost rezolvat fiecare. Adesea, soluția stă într-unul dintre acele postmortem-uri.

Ce face k-NN neobișnuit e că nu învață nimic dinainte. Nu există o fază de antrenare în care se potrivesc ponderi. Datele, păstrate așa cum sunt, *sunt* modelul. Toată munca se face când sosește o întrebare: măsori distanța de la cazul nou la fiecare caz păstrat, iei cele mai apropiate k și le lași să voteze o categorie sau le faci media numerelor. E cel mai leneș algoritm imaginabil, iar lenea asta vine cu un dar. Fiecare predicție poate fi explicată arătând spre exemple reale: credem că apartamentul acesta valorează atât din cauza acestor cinci vânzări. Lenea aceasta are un preț la scară mare: să măsori distanța la o sută de milioane de cazuri păstrate pentru fiecare întrebare e prea lent, așa că sistemele reale construiesc dinainte un index, cam ca secțiunile unei biblioteci, care sare direct în cartierul potrivit și verifică doar câteva mii de candidați. Din când în când întorc al doilea cel mai apropiat vecin în locul celui mai apropiat, un compromis pe care îl acceptă aproape toată lumea; acestea sunt bazele de date vectoriale care reapar mai târziu în carte.

Singura setare pe care trebuie s-o alegi e k, iar ea se poartă ca un buton între nervozitate și ceață. Cu k egal cu unu, un singur vecin ciudat, un tichet etichetat greșit sau o vânzare între rude, decide totul. Cu k egal cu cinci sute, faci media pe jumătate de oraș și toate apartamentele arată la fel. Undeva la mijloc e o valoare rezonabilă, iar o găsești ca de obicei: încercând câteva pe simulare.

Până aici, vecinii au fost înțelepți. Acum se întâmplă ceva ciudat și e unul dintre faptele cu adevărat contraintuitive ale domeniului.

Pe o hartă a străzilor, „aproape” e evident. Două dimensiuni și ochiul tău face treaba. Dar să zicem că descrii fiecare om nu prin două numere, ci prin cinci sute: înălțime, vârstă, venit, câte cărți are, câte ore doarme, culoarea preferată codificată ca număr și tot așa. Ia oricare doi oameni. La unele dintre cele cinci sute de trăsături sunt obligatoriu foarte diferiți. Adună toate aceste diferențe și fiecare ajunge cam la fel de departe de oricare altul. Vecinul tău „cel mai apropiat” e abia mai aproape decât cel mai îndepărtat. Într-un spațiu cu multe dimensiuni, toată lumea e străină.

Asta e **curse of dimensionality** („blestemul dimensionalității”) și e motivul pentru care k-NN merge cel mai bine cu un număr modest de features cu sens sau după ce ceva a comprimat multe features brute în câteva informative. Și, la fel ca clustering-ul, k-NN cade în capcana scalei: dacă trăsăturile nu sunt rescalate, cea cu numerele cele mai mari domină orice distanță.

Să întrebi vecinii merge minunat când știi ce face două lucruri să fie asemănătoare. Când nu știi, când ai zeci de features și doar câteva contează, ai nevoie de altă strategie. În loc să compari totul deodată, ai vrea să pui întrebările potrivite, în ordinea potrivită. Exact așa se câștigă jocul celor douăzeci de întrebări.

2.3 Douăzeci de întrebări

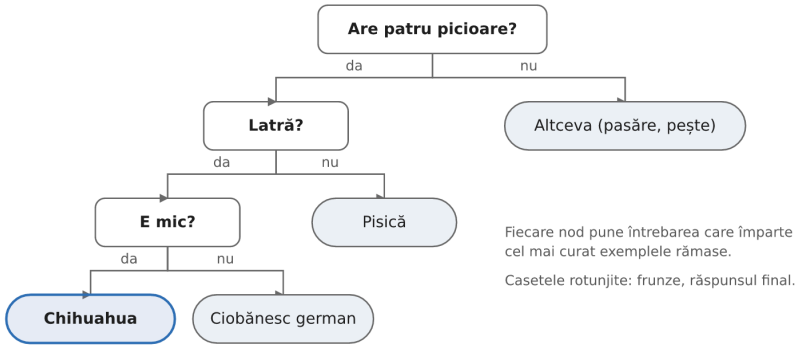
Release-ul pleacă în două ore, iar ceva din ultimele două sute de commit-uri a stricat pagina de checkout. Nimeni nu știe care commit. Să le testezi pe rând ar dura toată noaptea.

Apoi cineva tastează `git bisect` și starea din cameră se schimbă. Bisect scoate commit-ul de la mijloc și pune o singură întrebare: e bug-ul aici? Oricare ar fi răspunsul, jumătate dintre suspecți au dispărut. Încă o întrebare, încă o jumătate. După opt întrebări, două sute de commit-uri s-au redus la unul, iar vinovatul e pe ecran înainte să se răcească cafeaua.

Ce face bisect atât de satisfăcător nu e viteza calculatorului. E calitatea întrebărilor. Fiecare e aleasă ca să elimine cât mai multă incertitudine, oricare ar fi răspunsul.

Joci jocul ăsta din copilărie. La douăzeci de întrebări, un începător deschide cu „E un Chihuahua?” și nu află aproape nimic din inevitabilul „nu”. Un jucător bun deschide cu „Are patru picioare?”, pentru că oricare răspuns taie lumea în două. Apoi „Latră?”, apoi „E mic?”. După trei întrebări, candidații s-au redus la o mână.

Trei întrebări da/nu duc la Chihuahua



Un **decision tree** („arbore de decizie”) e acest joc, jucat de o mașină care a studiat mii de runde trecute. Ai văzut de multe ori versiunea desenată de mână: lista de triaj de la urgențe, schema de diagnostic a mecanicului, scriptul „ați încercat să-l reporniți?” de la help desk. Cineva cu experiență a scris acele scheme din ani de cazuri. Un arbore de decizie le scrie automat, din date.

Ca să facă asta, are nevoie de un mod de a deosebi o întrebare bună de una proastă, iar răspunsul vine dintr-un loc neașteptat: sertarul cu șosete. Un sertar doar cu șosete negre e perfect ordonat; bagi mâna fără să te uiți și știi ce vei scoate. Un sertar cu jumătate șosete negre și jumătate albe e cât se poate de dezordonat; fiecare apucare e cap sau pajură. **Entropy** („entropia”: măsura amestecăturii) și **Gini index** („indicele Gini”) sunt două moduri de a pune un număr pe această dezordine. O întrebare bună transformă un sertar dezordonat în două mai ordonate. Cât de multă ordine aduce se numește **information gain** („câștig de informație”), iar la fiecare pas arborele încearcă pur și simplu fiecare trăsătură și fiecare prag posibil („mai mult de 400 de linii schimbate?”, „suprafața peste 54 de metri pătrați?”) și păstrează întrebarea cu cel mai mare câștig. Apoi repetă procesul în fiecare sertar nou. Adună cât de multă ordine a adus fiecare trăsătură în toate întrebările din arbore și obții un clasament al trăsăturilor: o primă privire utilă asupra celor care au contat, deși nu una imparțială, din motive pe care le explică un capitol ulterior.

Imaginează-ți ce produce asta pe propriul tău istoric. Dă-i unui arbore fiecare pull request („cerere de includere a unei schimbări în cod”) din ultimii doi ani, fiecare etichetat după cum a fost sau nu anulat în decurs de o săptămână. S-ar putea întoarce cu un runbook pe care nu l-a scris nimeni vreodată: atinge migrații de bază de date? Dacă da, e mai mare de câteva sute de linii? Dacă da, a fost revizuit de o singură persoană? Atunci rata de anulare e mare. Fiecare drum de la vârful arborelui la o frunză se citește ca o regulă pe care ar putea-o enunța un inginer senior, și exact de aceea arborii sunt iubiți acolo unde deciziile trebuie explicate, cum ar fi aprobarea creditelor și verificările de conformitate. Au și maniere plăcute: lucrează la fel de bine cu numere și cu categorii și nu le pasă că o trăsătură e în euro și alta în ani. Un arbore întreabă doar dacă o valoare e peste sau sub un prag, iar rescalarea unei trăsături mută pragul odată cu valorile, fără să schimbe care exemple cad de o parte și care de cealaltă, așa că pe el nu-l atinge capcana scalei care i-a prins pe vecini.

Sună ideal. Dar arborele are două defecte de caracter și sunt serioase.

Primul e nerăbdarea. Un jucător priceput gândește câteva întrebări în avans; uneori o primă întrebare puțin mai slabă pregătește o a doua genială. Un arbore nu face niciodată asta. La fiecare pas își ia cea mai bună întrebare disponibilă chiar acum și nu mai revine asupra ei. E **greedy** („lăcom": ia mereu câștigul imediat), în sensul tehnic.

Al doilea defect e mai grav: un arbore nu știe când să se oprească din întrebat. Lăsat în pace, tot împarte până când fiecare sertar e perfect ordonat, ceea ce pe date reale înseamnă adesea doar unul sau câteva exemple per frunză. Ajunge să întrebe lucruri precum „Are 54,3 metri pătrați, e la etajul trei și are ușă verde?". Întrebarea asta separă un apartament de toate celelalte din datele de antrenare și e inutilă pentru orice apartament pe care arborele nu l-a văzut. Arborele a încetat să învețe tipare și a început să memoreze. Remediul obișnuit e să fixezi limite dinainte: nu mai mult de, să zicem, șase întrebări în adâncime, niciun sertar cu mai puțin de douăzeci de exemple sau nicio întrebare care aduce mai puțină ordine decât un minim. Sunt hiperparametri, aleși urmărind simularea.

Mai rău, memorarea e instabilă. Schimbă câteva rânduri din datele de antrenare și prima întrebare se poate schimba, iar odată cu ea tot ce e dedesubt. Doi analiști care antrenează arbori pe aproape aceleași date pot obține arbori care nu seamănă deloc. Un singur arbore e, cu alte cuvinte, un expert strălucit dar imprevizibil: lizibil, rapid și mult prea impresionat de cazurile anume pe care le-a văzut întâmplător.

Există un remediu și vine dintr-o observație făcută la un târg de țară acum mai bine de un secol. Dacă, în loc de un singur expert imprevizibil, ai întreba o mulțime întreagă?

2.4 Înțelepciunea mulțimii și a editorilor

În toamna lui 1906, statisticianul Francis Galton, pe atunci trecut de optzeci de ani, se plimba printr-un târg de animale din Plymouth și a dat peste un concurs. Pentru o taxă mică, vizitatorii puteau ghici greutatea unui bou după ce avea să fie tăiat și preparat. Au participat vreo opt sute de oameni: măcelari și fermieri care se pricepeau la vite și destui funcționari și familii care nu se pricepeau. Galton, care avea o părere proastă despre judecata omului obișnuit, a strâns apoi biletele, curios să vadă cât de bine se descurcase mulțimea.

Estimările individuale erau împrăștiate în toate direcțiile. Dar când a luat valoarea de mijloc a tuturor, ea a ieșit la aproximativ un procent distanță de greutatea reală. Mulțimea, ca mulțime, avusese aproape perfect dreptate. Galton a publicat rezultatul anul următor în revista Nature, sub titlul *Vox Populi*.

De ce a mers? Pentru că eroarea fiecăruia arăta într-o altă direcție. Unul a estimat prea mult pentru că boul părea mare de unde stătea el; alta prea puțin pentru că se gândea la o rasă mai mică. Greșelile independente, adunate, se anulează în mare parte. Ce supraviețuiește e partea pe care toată lumea a văzut-o corect.

Revino acum la arborele de decizie imprevizibil din subcapitolul trecut, expertul strălucit care memorează fiecare ciudățenie a cazurilor văzute. Un **random forest** („pădure aleatoare": mulți arbori, fiecare crescut puțin altfel) construiește o mulțime Galton din astfel de experți, iar partea inteligentă e cum îi face independenți. Fiecare arbore primește o copie ușor diferită a datelor de antrenare, extrasă la întâmplare astfel încât unele rânduri apar de mai multe ori, iar altele lipsesc; asta se numește **bootstrap sample** („eșantion bootstrap", după expresia englezească a te ridica trăgând de propriile șireturi: din datele pe care le ai, fabrici variații noi). Și la fiecare întrebare pe care o pune, fiecare arbore are voie să ia în calcul doar câteva trăsături alese la întâmplare. Fiecare expert e pe jumătate orb altfel, așa că

fiecare face alte greșeli. Când pădurea votează sau face media, ciudățeniile individuale se anulează, exact ca estimările de la târg. Antrenarea mai multor modele pe eșantioane bootstrap și combinarea lor se numește **bagging** (de la *bootstrap aggregating*, „agregare de eșantioane bootstrap”); alegerea aleatoare a trăsăturilor e ce o transformă într-o pădure.

Dar târgul are și o versiune mai întunecată, care explică designul. Imaginează-ți că, înainte să scrie cineva vreun bilet, un fermier gălăgios și-a anunțat estimarea în fața tuturor. Jumătate dintre oameni s-ar fi ancorat în ea. Erorile lor n-ar mai fi fost independente; ar fi înclinat toate în aceeași direcție, iar media n-ar mai fi anulat nimic. Mulțimea ar fi devenit un cor care cântă nota greșită a unui singur om.

Arborii se confruntă exact cu acest pericol. Dacă o trăsătură din datele tale e foarte puternică, fiecare arbore va vrea să întrebe mai întâi despre ea și vor face cu toții aceleași greșeli. Ascunderea unui subset aleator de trăsături de fiecare arbore, la fiecare pas, e felul pădurii de a ține fermierul gălăgios afară din cameră.

Cunoști deja principiul din code review. Un al doilea reviewer cu exact același background, aceleași obiceiuri și aceleași puncte oarbe ca tine prinde puțin din ce n-ai fi prins tu. Un reviewer care se gândește la securitate în timp ce tu te gândești la performanță prinde mult mai mult, tocmai pentru că punctele voastre oarbe diferă. Diversitatea nu e un bonus drăguț. E inima mecanismului.

Mai există un mod de a transforma experți slabi într-unul puternic și seamănă mai puțin cu o mulțime și mai mult cu o editură.

Un autor scrie o primă ciornă brută. O editoare o citește și repară doar problemele de structură; nu rescrie cartea. Un redactor repară apoi doar frazele stângace lăsate de editoare. Un corector prinde greșelile de tipar rămase. Nimeni nu reface toată munca. Fiecare se specializează în ce au greșit cei dinainte, iar cartea care iese e mult mai bună decât ar fi putut-o face oricare dintre ei singur.

Gradient boosting („boosting” înseamnă „amplificare”: fiecare model nou îmbunătățește ce au lăsat în urmă cele dinainte) funcționează așa. Primul arbore mic face o estimare grosieră, să zicem a duratei unei livrări. Al doilea arbore nu e rugat deloc să prezică duratele livrărilor. E rugat să prezică *erorile* primului arbore: unde a fost prea optimist, unde prea pesimist. Al treilea prezice ce e încă greșit după primii doi și tot așa, timp de sute de runde. Fiecare arbore în parte e grosolan, nu mai adânc de câteva întrebări, însă lanțul de corecturi devine remarcabil de precis.

Dacă firma ta dă scoruri de fraudă plăților, prezice ce clienți vor renunța sau estimează timpi de livrare dintr-un tabel cu câteva zeci de coloane, sunt șanse mari ca un model de boosting să facă treaba, dintr-una dintre bibliotecile cunoscute precum XGBoost, LightGBM sau CatBoost. Pe acest fel de date tabelare, ele sunt de obicei modelele de bătut și bat adesea rețelele neuronale. Motivul e că un tabel vine de obicei cu features deja făcute: câteva zeci, fiecare cu sens de sine stătător, în care semnalul e purtat de praguri nete precum „mai mult de trei autentificări eșuate”. Rețelele își merită costul construind features din pixeli și cuvinte brute; într-un tabel le rămâne puțin de construit, în timp ce arborii se descurcă firesc cu unități amestecate și praguri nete.

Cele două metode repară slăbiciuni opuse, iar domeniul are nume pentru amândouă. Un random forest ia experți detaliați care reacționează exagerat la datele anume pe care le-au văzut și îi calmează prin medie; reduce **variance** („varianța”: cât de mult sar rezultatele de la un set de date la altul). Boosting-ul ia experți simpli, sistematic prea grosolani, și îi ascute prin corecturi; reduce **bias**-ul, aici în sensul de eroare sistematică, greșeala care înclină mereu în aceeași direcție (nu constanta adăugată de riglă; un

capitol ulterior lămurește multele sensuri ale cuvântului). În practică, o pădure e o alegere implicită robustă care iartă setările neglijente, în timp ce boosting-ul câștigă de obicei la precizie, dar are nevoie de supraveghere, mai ales ca să știi când să te oprești din adăugat arbori înainte ca editorii să înceapă să corecteze zgomotul. La o pădure, mai mulți arbori nu duc la overfitting, memorarea care l-a stricat pe arborele singur: media mulțimii pur și simplu nu se mai schimbă după câteva sute, iar singurul cost e timpul. La boosting, fiecare arbore nou urmărește ce e încă greșit, iar după o vreme asta e mai ales zgomot, așa că te oprești când scorul pe simulare nu se mai îmbunătățește.

Și o limită se aplică amândurora. Dacă datele sunt strâmbe pentru toată lumea, dacă boul a fost cântărit pe un cântar stricat, nicio mulțime și niciun lanț de editori nu le vor îndrepta.

Rigla, vecinii, arborii și mulțimile nu sunt însă toată trusa, iar câteva dintre celelalte unelte merită măcar o privire rapidă înainte să mergem mai departe.

2.5 Rafturile pe lângă care am trecut

Intri într-un magazin mare de bricolaj după o cutie de șuruburi și mergi drept pe singurul culoar de care ai nevoie, pe lângă alte patruzeci, cu rafturi pline de unelte pe care nu le-ai folosit niciodată. Turul trusei clasice de unelte din acest capitol a fost o plimbare de felul acesta. A trecut pe trei culoare, rigla, vecinii și arborii, plus mulțimile construite din arbori, pentru că aceste idei duc greul pe datele din business. Dar deschide documentația oricărei biblioteci de machine learning și lista de metode se întinde pe pagini întregi. Câteva dintre acele nume poartă idei pentru care subcapitolele de până acum n-au avut loc.

Începe cu granița. Granița de decizie a variatorului stătea pur și simplu acolo unde o lăsase antrenarea. Imaginează-ți în schimb două sate de o parte și de alta a unei câmpii largi și o comisie însărcinată să construiască între ele un drum care să nu aparțină niciunuia. Multe drumuri ar despărți casele. Comisia îl alege pe cel care lasă cea mai lată fâșie de pământ gol de ambele părți, cu gândul că un drum lipit de gardurile unui sat va greși la prima casă nouă construită la marginea lui. Aceasta e **support vector machine** („mașina cu vectori-suport”: alege granița sprijinită pe cazurile cele mai apropiate de ea), pe scurt SVM: dintre toate granițele care despart două clase, o alege pe cea cu cea mai largă **margin** („marjă”, fâșia goală de o parte și de alta a graniței). Doar casele cele mai apropiate de drum decid pe unde trece; ele sunt vectorii-suport care dau numele metodei, iar orice casă mai din spate s-ar putea muta fără să schimbe nimic. Datele reale se despart rareori curat, așa că versiunea practică lasă câteva case să stea pe partea greșită a drumului, cu prețul unei penalizări.

A doua idee e cea ingenioasă. Unele sate nu pot fi despărțite de niciun drum drept, pentru că unul e un inel în jurul celuilalt. Răspunsul SVM-ului, **kernel trick** („trucul nucleului”: judeci asemănarea ca într-un spațiu mai bogat, fără să-l construiești), e să judece terenul într-un spațiu mai bogat, în care un drum drept chiar merge. Adaugă distanța fiecărei case față de centru ca o coordonată în plus, iar satul din interior devine o vale și cel din exterior un deal, ușor de despărțit printr-o tăietură plată. Trucul e că SVM-ul nu construiește niciodată acel spațiu mai bogat. Tot ce-i trebuie e cât de asemănătoare e fiecare pereche de case, iar un **kernel** („nucleu”, fără legătură cu nucleele rețelelor pentru imagini din capitolul următor) e o formulă care calculează asemănarea pe care perechea ar avea-o în spațiul mai bogat, chiar și într-unul infinit de mare, fără să ajungă vreodată acolo. Pe la sfârșitul anilor ’90 și în anii 2000, SVM-urile au fost printre cei mai puternici clasificatori de uz general și rămân o alegere rezonabilă pentru seturi mici de date cu multe trăsături, de pildă câteva sute de pacienți descriși prin mii de

măsurători. Imaginea se rupe în două locuri. Drumul cel mai lat trăiește într-un spațiu inventat de kernel, pe care nimeni nu-l poate desena sau străbate. Iar compararea fiecărei perechi de case devine scumpă când sunt milioane, ceea ce explică în parte de ce arborii cu boosting și rețelele au preluat conducerea pe datele mari.

Al doilea culoar e despre convingeri. O doctoriță vede un pacient care tușește. Înainte de orice analiză, are deja o idee despre cât de răspândită e luna asta fiecare boală: răcelile sunt peste tot, pneumonia e mai rară. Această bănuială de pornire e **prior**-ul („a priori": ce credeai înainte de dovezi). Apoi vin dovezile, o febră, un hârâit în plămâni, iar pentru fiecare se întreabă cât de probabil ar fi simptomul sub fiecare boală și își mută bănuiala în consecință. Ce crede la final e **posterior**-ul („a posteriori": ce crezi după dovezi). Regula care spune exact cum se face mutarea e **Bayes' rule** („regula lui Bayes"), dintr-un eseu al pastorului Thomas Bayes publicat în 1763, după moartea lui, și e felul cinstit de a combina ce credeai cu ce tocmai ai văzut. Cea mai utilă lecție a ei pentru ingineri e tocmai cea pe care oamenii o greșesc cel mai des. Un test exact în 99% din cazuri, pentru o boală pe care o are un om din zece mii, va produce totuși mai ales alarme false, pentru că prior-ul era atât de mic. Aceeași aritmetică revine ori de câte ori un model caută ceva rar și explică de ce acuratețea singură spune atât de puțin despre asemenea modele.

Acum fă-o pe doctoriță leneșă. Să zicem că se preface că fiecare simptom n-are legătură cu celelalte, așa că o febră nu-i spune nimic despre dacă să se aștepte la tuse. Atunci fiecare simptom devine o înmulțire separată a șanselor, iar calculul devine banal. Scurtătura aceasta e **naive Bayes** („Bayes naiv": presupune că fiecare indiciu e independent de celelalte), iar „naiv" e cuvântul cinstit, pentru că presupunerea e falsă aproape peste tot: „gratis" și „bani" nu sunt independente într-un e-mail. Totuși, metode de acest fel au alimentat multe dintre filtrele de spam care învățau, cele care au înlocuit regulile scrise de mână de la începutul cărții, iar naive Bayes e încă un prim model surprinzător de bun pentru text. Ordonează bine e-mailurile, deși probabilitățile pe care le calculează sunt mult exagerate, pentru că, numărând de două ori dovezi înrudite, își umflă încrederea fără să schimbe de obicei direcția în care înclină. Aici imaginea se rupe într-o direcție utilă. O doctoriță care ar ignora felul în care simptomele vin împreună ar fi o doctoriță proastă, iar metoda care face exact asta funcționează oricum. O presupunere greșită poate fi inofensivă pentru ordonare și dăunătoare pentru încredere, iar asta e o lecție despre orice simplificare, inclusiv despre cele ale acestei cărți.

O rudă de pe același culoar merită o propoziție, pentru că o vom reîntâlni mai târziu în carte. Un **Gaussian process** („proces gaussian", după curba în formă de clopot a lui Gauss) prezice un număr împreună cu cât de nesigur e. Gândește-te la o sârmă flexibilă prinsă cu câte un cui în fiecare punct cunoscut: întinsă lângă cuie, liberă să se legene în golurile lungi dintre ele, așa că nesiguranța crește oriunde datele sunt rare. Devine lent pe seturi mari de date, dar e excelent pe seturi mici și scumpe, de aceea e folosit la alegerea hiperparametrilor; își are locul și printre uneltele prin care un model spune cât de sigur e. Imaginea sârmei ascunde singura alegere reală: cât de rigidă e sârma, adică până unde ajunge influența unui cui, o stabilește analistul sau se reglează din date, iar o rigiditate prost aleasă produce între cuie aberații spuse cu toată încrederea.

Al treilea culoar plătește o datorie veche. K-Means avea nevoie să alegi tu numărul de grupuri și găsea cinci grupuri în zgomot pur. Imaginează-ți în schimb un festival de muzică văzut de la o dronă. Nimeni nu-ți spune câte grupuri de prieteni sunt. Un grup e pur și simplu oriunde oamenii stau aproape unii de alții, se întinde cât timp mulțimea rămâne deasă, iar cei câțiva care rătăcesc între scene nu aparțin niciunui grup. **Density-based clustering** („gruparea după densitate": un grup e oriunde punctele stau înghesuite) funcționează așa, iar DBSCAN e exemplul clasic: găsește atâtea grupuri câte sugerează

densitatea, de orice formă, și îi etichetează pe rătăciți drept **noise** („zgomot”). Asta răspunde pe jumătate problemei feței de pe Marte. Dată fiind o împrăștiere la întâmplare, o metodă bazată pe densitate va raporta adesea o singură pată mare sau mai ales zgomot, nu cinci grupuri ordonate, deși tot tu alegi ce înseamnă „aproape”, iar o alegere neatentă poate scoate grupuri din nimic ca orice altă metodă. O a doua familie, **hierarchical clustering** („gruparea ierarhică”), construiește în schimb un arbore genealogic. Unește cele mai asemănătoare două elemente, apoi următoarele două, până când totul e un singur grup, iar tu decizi abia apoi unde tai arborele, în trei ramuri sau în treizeci.

Pe același culoar se vând și imaginile care apar pe orice slide despre embeddings, listele lungi de numere prin care capitolele următoare așază cuvinte și imagini pe o hartă a sensului: un nor de puncte colorate în două dimensiuni, cu insule ordonate de elemente asemănătoare. Sunt făcute de obicei cu metode numite **t-SNE** și **UMAP**, care turtesc sute de dimensiuni pe o pagină încercând să țină fiecare punct lângă vecinii lui cei mai apropiați. Gândește-te la harta lumii de pe peretele unei clase, pe care Groenlanda pare cam cât Africa. Orice hartă plată a unei planete rotunde deformează ceva, iar hărțile acestea păstrează vecinătățile cu prețul a aproape tot restul. Mărimea unei insule și distanța dintre două insule înseamnă adesea foarte puțin, iar setări diferite pot rupe un grup real în mai multe sau pot lipi două laolaltă. Tratează insulele ca pe niște indicii care merită deschise, niciodată ca pe niște descoperiri.

Ultimul culoar stă acolo unde se întâlnesc datele etichetate și cele neetichetate. Adesea ai câteva sute de tichete etichetate și o sută de mii neetichetate. **Semi-supervised learning** („învățarea semi-supervizată”: puține răspunsuri, multe exemple fără răspuns) le folosește pe amândouă: antrenezi pe puținele etichete, lași modelul să eticheteze restul acolo unde e foarte sigur, reantrenezi pe grămada mai mare și repeți, ca o angajată nouă care învață din cazurile marcate de mentorul ei și apoi din propriile judecăți de care e cea mai sigură. Riscul e limpede: greșelile făcute cu încredere se întorc în antrenare drept adevăr, iar metoda merge cel mai bine când clasele formează grupuri bine despărțite. **Weak supervision** („supervizarea slabă”: etichete scrise de reguli imperfecte, nu de oameni caz cu caz) vine din partea cealaltă. În loc să eticheteze exemplele unul câte unul, experții scriu reguli rapide și imperfecte, printre ele și vechile reguli pe cuvinte-cheie ale filtrelor de spam, iar un model învață din locurile în care regulile cad de acord și din cele în care se contrazic. Regulile erau prea grosolane ca să fie produsul; sunt destul de bune ca să fie profesorul. Împreună cu active learning și învățarea auto-supervizată, pe care le vom întâlni mai târziu în carte, ele umplu spațiul dintre o cheie de răspunsuri completă și niciuna.

Clasele rare, precum puținele fraude dintre milioane de plăți cinstite, au și ele tratamente standard. Poți schimba ponderile din loss, ceea ce se numește **reweighting** („reponderare”), ca fiecare fraudă ratată să coste cât sute de alarme false, sau poți face **resampling** („reeșantionare”) pe datele de antrenare, arătându-i modelului cazurile rare mai des sau pe cele comune mai rar. Ambele îl fac pe model să fie atent la cazurile rare. Ambele îi deformează însă și simțul despre cât de comun e lucrul rar, așa că scorurile lui trebuie recalibrate, adică verificate și corectate după cât de des se întâmplă lucrurile în realitate, înainte ca cineva să le citească drept probabilități.

Fiecare metodă din acest capitol a presupus în tăcere că datele sunt un tabel de rânduri independente, care pot fi amestecate liber. Multe date valoroase nu sunt așa. Vin ordonate în timp, sub formă de rețea sau ca o listă a cărei ordine e tot rostul, iar asemenea date își primesc subcapitolul lor mai târziu, după ce vom avea la îndemână mașinăria deep learning. Spre mașinăria aceasta mergem acum. Tot ce am văzut până aici a depins de cineva care alege trăsături bune pentru model; următorul capitol e despre modele care învață să și le construiască singure. Povestea lui începe cu un titlu de ziar din 1958 și cu o promisiune care s-a dovedit cu mult prematură.

Mai departe. Dacă te-ai întrebat cum alegi praguri ca acelea pentru riscul unui deployment, subcapitolul 10.3 răspunde, iar dacă te-ai întrebat dacă acel 0,83 al unui model merită crezut ca probabilitate, răspunde subcapitolul 10.5. Cum deosebești o cauză de ceva care doar se mișcă odată cu ea e subiectul subcapitolului 11.1. Iar dacă te-ai întrebat ce faci cu date care vin ordonate în timp, ca rețea sau ca listă ordonată, subcapitolul 8.3 răspunde.

Capitolul 3 — Deep learning

3.1 Neuronul căruia i s-a promis prea mult

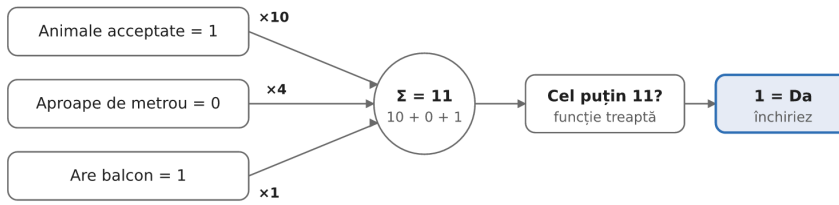
În iulie 1958, ziarele relatau despre o demonstrație a Marinei Statelor Unite. Un tânăr psiholog pe nume Frank Rosenblatt construise o mașină pe care o numea **perceptron** (de la *perception*, „percepție”: o mașină care percepe), iar după puțin exercițiu învățase să deosebească cartonașele marcate în stânga de cele marcate în dreapta. Relatările erau înflăcărate. The New York Times scria că Marina se aștepta ca asemenea mașini să poată într-o zi să meargă, să vorbească, să vadă, să scrie și chiar să fie conștiente de propria existență.

Merită să știi ce făcea mașina de fapt, pentru că ideea din spatele ei e modestă și, totodată, sămânța a tot ce înseamnă deep learning („învățare profundă”, adică cu multe straturi) azi. Și aproape sigur ai reinventat-o chiar tu.

Să zicem că ești în căutarea unui apartament și vrei să decizi repede, da sau nu. Verifici trei lucruri: sunt acceptate animalele, e aproape de metrou, are balcon? Fiecare răspuns e 1 pentru da și 0 pentru nu. Nu toate criteriile contează la fel. Ai un câine, deci animalele primesc ponderea 10. Metroul primește 4. Balconul, un bonus plăcut, primește 1. Și îți fixezi un prag: închiriezi doar dacă apartamentul strânge cel puțin 11 puncte.

Un apartament aproape de metrou și cu balcon obține 4 + 1 = 5. Sub prag: nu. Unul care acceptă animale și are balcon obține 10 + 1 = 11. Trece pragul: da.

Animalele și balconul ating exact pragul de 11



Pragul e bias-ul deghizat: scazi 11 din sumă și neuronul se activează când rezultatul e 0 sau mai mult.

Asta e tot ce e un perceptron: cântărești dovezile, le aduni și compari cu un prag. Dacă ai fost vreodată de gardă (on call), poate ai scris unul fără să-i știi numele. Multe echipe îl trezesc pe inginerul de gardă cu o regulă de felul acesta: adaugă 5 puncte dacă rata de erori e peste normal, 3 dacă cele mai lente cereri sunt mai lente ca de obicei, 8 dacă clienții au deschis tichete în ultimele zece minute, și dă alarma când totalul ajunge la 10. Contribuția lui Rosenblatt n-a fost suma în sine. A fost o procedură care îi permitea mașinii să-și găsească singură ponderile și pragul, din exemple, în loc să fie stabilite într-o ședință.

Timp de un deceniu, entuziasmul a ținut. Apoi, în 1969, Marvin Minsky și Seymour Papert au publicat o carte numită *Perceptrons*, care explica, cu precizie matematică, ce nu va putea face niciodată un singur perceptron. Cel mai faimos exemplu e jenant de mic.

Imaginează-ți o bază de date principală și replica ei. Vrei o alarmă când cele două *nu sunt de acord*: exact una dintre ele raportează o scriere reușită. Dacă reușesc amândouă, e în regulă. Dacă eșuează amândouă, e cu totul altă alarmă. Încearcă să construiești lista de bifat. Orice ponderi care se declanșează pentru „doar principala” și pentru „doar replica” se vor declanșa și pentru „amândouă”, sau altfel pentru „niciuna”. Nu există nicio setare care să meargă, nu pentru că n-ai fi destul de deștept, ci pentru că o singură sumă ponderată cu un prag poate împărți lumea doar cu o linie dreaptă, iar „exact una din două” nu poate fi separat cu o singură linie dreaptă. Aceasta e **problema XOR** (exclusive or, „sau exclusiv”: una sau alta, dar nu amândouă), iar dezamăgirea de după carte a ajutat la împingerea cercetării în rețele neuronale într-o iarnă lungă.

Partea frustrantă e că ieșirea se vedea încă de atunci. Folosește mai mult de un neuron. Lasă-l pe unul să învețe să detecteze „principala da, replica nu”. Lasă-l pe altul să detecteze „principala nu, replica da”. Apoi lasă-l pe al treilea să se activeze dacă oricare dintre primii doi se activează. Trei liste de bifat simple, așezate pe două straturi, rezolvă ce o singură listă n-ar putea niciodată. Câți neuroni și câte straturi îi trebuie unei probleme reale e un hiperparametru, ales încercând câteva mărimi pe simulare. În principiu, un singur strat ascuns cu destui neuroni poate aproxima aproape orice relație, dar în practică mai multe straturi fac aceeași treabă cu mult mai puțini neuroni, pentru că straturile de mai târziu își construiesc judecățile din judecățile celor dinainte. Pune neuronii așa, cu un strat de intrare, unul sau mai multe **hidden layers** („straturi ascunse”, pentru că nu le vezi nici la intrare, nici la ieșire) și un strat de ieșire, și ai un **multi-layer perceptron** („perceptron multistrat”), pe scurt MLP.

Două lucruri despre o astfel de rețea sunt ușor de înțeles greșit și amândouă contează.

Întâi, nimeni nu le dă neuronilor ascunși sarcinile. Nu există un neuron programat să judece „cartierul” sau „actele”. Neuronii ascunși își descoperă singuri sub-criteriile în timpul antrenării. Gândește-te la o recrutoare cu experiență care, după ce a citit zece mii de CV-uri, și-a format intuiții pe care nu le-a numit niciodată: un anumit fel de pauză în carieră, un anumit fel de a descrie un proiect. Neuronii ascunși sunt așa. Își inventează judecăți intermediare utile, iar adesea acele judecăți nu au nume în nicio limbă omenească. Dacă nu ajung toți la aceeași judecată, e pentru că pornesc de la ponderi aleatorii diferite: dacă toți neuronii ascunși ar porni identici, fiecare ar primi exact aceeași vină și ar rămâne identic pentru totdeauna, iar puțină întâmplare la început e ce îi lasă să alunece spre specialități diferite.

Apoi, între straturi trebuie să existe o îndoitură. Suma fiecărui neuron trece printr-o **activation function** („funcție de activare”: decide cât de tare „se aprinde” neuronul) înainte să meargă mai departe. Fără ea, straturile se prăbușesc unul în altul. Gândește-te la schimbul din euro în dolari și apoi din dolari în lire: oricâte conversii ai înlănțui, rezultatul e un singur curs fix de la euro la lire. O sută de straturi de sume ponderate simple sunt, matematic, identice cu un singur strat. O mică îndoitură la fiecare pas, cum ar fi **ReLU** (rectified linear unit, „unitate liniară rectificată”: lasă să treacă valorile pozitive și transformă valorile negative în zero), e ce permite unui teanc de neuroni simpli să deseneze granițe complicate. Reușește asta așa cum poți desena o curbă din multe segmente drepte scurte: fiecare neuron contribuie cu o îndoitură în locul lui, iar sute de îndoituri, adunate și suprapuse, pot urmări aproape orice formă. Fiecare neuron are și un bias, constanta pe care am întâlnit-o la riglă, iar ea se dovedește a fi pur și simplu pragul mutat de cealaltă parte a comparației: în loc de „scor cel puțin 11”, neuronul calculează „scor minus 11” și întreabă dacă rezultatul e pozitiv. Un singur neuron a cărui îndoitură e variatorul, adică sigmoida, calculează exact ce calculează regresia logistică: cântărește dovezile, adaugă bias-ul și trece suma prin curba în S. Clasificatorul din capitolul trecut a fost tot timpul o rețea cu un singur neuron.

Așadar, de ce a durat până în anii ’80 ca rețelele multistrat să decoleze? Pentru că să știi că un strat ascuns *ar putea* rezolva problema nu e același lucru cu a ști cum să-i găsești ponderile. Cu un singur neuron e ușor să vezi care pondere a fost greșită. Cu trei straturi, când răspunsul final e greșit, care dintre sutele de ponderi din interior e de vină, și cât de mult?

Întrebarea a așteptat ani de zile un răspuns bun. Răspunsul s-a dovedit a implica un munte, o ceață groasă și un mod foarte atent de a da vina înapoi.

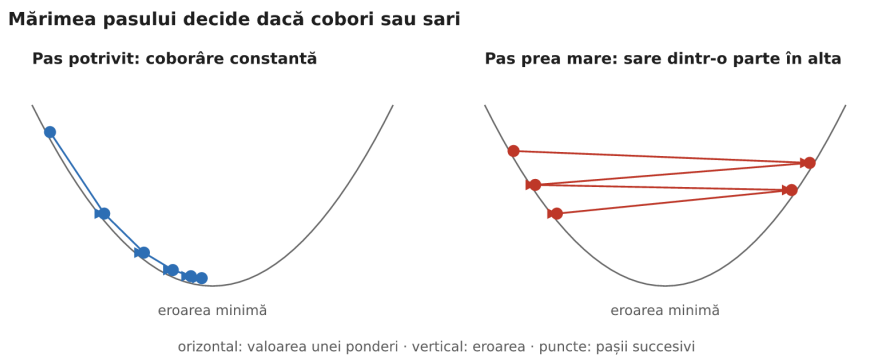
3.2 Coborând muntele prin ceață

Ești undeva pe un munte, noaptea, într-o ceață atât de deasă încât nu-ți vezi nici bocancii. Trebuie să cobori în vale. N-ai hartă și nici lumină. Ce faci?

Pipăi terenul. Prin tălpi simți încotro coboară panta și cât de abrupt. Așa că faci un pas în direcția în care coboară cel mai tare. Apoi te oprești, pipăi din nou și mai faci un pas. Pas cu pas, fără să vezi vreodată valea, cobori.

Așa învață practic orice rețea neuronală modernă, iar odată ce ai văzut asta, e greu să n-o mai vezi. Muntele e eroarea modelului. Fiecare poziție de pe el corespunde unei anumite setări a tuturor ponderilor, iar altitudinea în acel punct e cât de mult greșește modelul cu acele ponderi. Antrenarea înseamnă să mergi la vale. Drumul începe dintr-un punct la întâmplare: ponderile pornesc ca numere mici, aleatorii. Două antrenări pornite din puncte diferite ajung de obicei în văi diferite, de adâncimi asemănătoare, și acesta e unul dintre motivele pentru care două antrenări ale aceleiași rețele dau rareori ponderi identice. Panta de sub bocanci, măsurată în toate direcțiile deodată, se numește **gradient** („pantă”, de la latinescul *gradus*, treaptă), iar metoda, pipăi panta și fă un pas la vale, e **gradient descent** („coborâre pe gradient”).

Lungimea pasului se dovedește extrem de importantă și are un nume: **learning rate** („rata de învățare”). Fă pași minusculi și la răsăritul soarelui vei fi încă pe drum. Fă pași uriași și se întâmplă ceva ciudat. Sari peste fundul văii, aterizezi pe panta opusă, simți că terenul coboară înapoi pe unde ai venit, sari din nou și te bălăngăni dintr-o parte în alta fără să te așezi vreodată.



Dacă săriturile acestea îți par cunoscute, poate le-ai văzut în producție. Imaginează-ți un autoscaler (sistemul care adaugă sau scoate automat servere după încărcare) configurat să reacționeze tare la orice schimbare. Traficul crește puțin; adaugă zece instanțe. Încărcarea pe instanță scade brusc; scoate zece.

Încărcarea explodează; le adaugă la loc. Clusterul oscilează toată noaptea, iar omul de gardă află pe pielea lui cum se simte un learning rate prea mare. Remediul, în ambele lumi, e același: pași mai mici, mai calmi. Lungimea pasului se alege parțial prin încercări pe o antrenare scurtă, iar într-o antrenare serioasă rareori rămâne aceeași de la primul pas până la ultimul; felul în care e variată e una dintre rutinele pe care le desface un capitol ulterior.

Asta e mersul. Acum partea grea, întrebarea lăsată în suspensie de subcapitolul anterior. Într-o rețea cu multe straturi, de unde știi încotro coboară panta pentru fiecare dintre miile de ponderi ascunse adânc în interior?

Imaginează-ți un atelier de mobilă cu trei posturi în linie: debitare, asamblare și vopsire. Un client returnează un dulap strâmb. Inspectorul nu dă vina pe toți în mod egal și nu începe de la început. Pornește de la capăt și merge înapoi. Vopsitorul a primit o ușă deja strâmbă, deci doar o mică parte din vină e a lui. Asamblorul a lipit-o strâmb pentru că panoul a sosit cu doi milimetri mai scurt, așa că cea mai mare parte a vinei merge mai înapoi, la debitare. Când inspectorul ajunge la începutul liniei, fiecare post știe care e partea lui din defect și își ajustează procedura exact atât.

Asta e **backpropagation** („propagarea înapoi”, a erorii). Eroarea e măsurată la ieșire și trimisă înapoi prin rețea, strat cu strat, iar la fiecare strat regula din analiza matematică pentru efecte înlănțuite, chain rule („regula lanțului”), împarte vina între ponderi după cât de mult a influențat fiecare rezultatul. În 1986, David Rumelhart, Geoffrey Hinton și Ronald Williams au publicat un articol scurt în *Nature* care arăta cât de bine funcționa metoda pentru rețelele multistrat, iar iarna lungă a început să se dezghețe.

Dacă lucrezi cu microservicii, ai deja intuiția. Când o pagină se încarcă încet, distributed tracing („trasarea distribuită”: urmărirea unei cereri prin toate serviciile pe care le atinge) îți arată unde s-a dus timpul: patruzeci de milisecunde în frontend, trei sute în API, din care două sute optzeci petrecute așteptând o singură interogare în baza de date. Nu optimizezi frontend-ul; repari interogarea. Backpropagation e distributed tracing pentru greșeli în loc de milisecunde. Îi spune fiecărei ponderi, din milioane, cât din eroarea finală a trecut prin ea.

Uită-te acum cum învață lista noastră de bifat pentru apartamente, cu toate piesele la locul lor. Ponderile ei pornesc la întâmplare: balcon 10, metrou 1. Un client e trimis într-un apartament cu un balcon imens, departe de orice stație de metrou, și se întoarce furios. Recenzia proastă e eroarea. Lista nu e ruptă într-un acces de nervi. Backpropagation calculează cât a contribuit fiecare pondere: balconul a împins scorul mult în sus, deci ponderea lui scade puțin; metroul abia a contat, deci ponderea lui crește puțin. Fiecare corectură e vina înmulțită cu learning rate-ul. După zeci de mii de clienți, lista se așază undeva pe la balcon 2, metrou 9, și nimeni n-a trebuit să-i spună vreodată.

În practică, modelul nu îi consultă pe toți clienții înainte de un pas. Să calculezi panta exactă pe milioane de exemple ar face fiecare pas insuportabil de lent. În schimb se uită la o mână mică, aleasă la întâmplare, un **mini-batch** („un mic lot”) de poate câteva zeci sau câteva sute de exemple, estimează panta din ele și face pasul. E logica unui sondaj de opinie: o mie de oameni bine aleși îți spun cam ce cred milioane. Estimarea e zgomotoasă, deci coborârea se bălăngăne, și de aceea metoda se numește **stochastic** gradient descent („coborâre stohastică”, adică cu o doză de întâmplare), SGD pe scurt. O trecere completă prin toate datele de antrenare se numește **epoch** („epocă”). Antrenarea modernă mai adaugă de obicei o rafinare: optimizatori precum **Adam** își amintesc pantele recente și adaptează mărimea pasului pentru fiecare pondere, ca o drumeață cu experiență care își lungește pasul pe teren stabil și îl scurtează pe grohotiș.

Mai e o ultimă surpriză pe muntele ăsta și răstoarnă o teamă care i-a frământat pe cercetători ani de zile. Un munte real are două direcții orizontale. Peisajul erorii unei rețele are câte o direcție pentru fiecare pondere, milioane. Lumea se temea că drumețul va rămâne prins într-o mică adâncitură la jumătatea drumului, un **local minimum** („minim local": un punct jos doar față de vecinătatea lui), fără să poată ieși. În spații atât de vaste, se pare, gropile care îl prind pe drumeț mult deasupra fundului văii sunt rare. Cele mai multe locuri plate sunt șei: coboară într-o direcție pe care n-ai încercat-o încă, iar tremurul mini-batch-urilor tinde s-o găsească. Drumețul care se rătăcește în practică e de obicei cel cu pasul greșit. Drumețul nu știe niciodată dacă a ajuns în punctul cel mai de jos al întregului munte, dar asta contează mai puțin decât pare: în practică te oprești când eroarea pe simulare nu se mai îmbunătățește, adesea înainte ca eroarea de antrenare să se oprească din scădere, iar oprirea timpurie, cum va arăta un capitol ulterior, e un avantaj, nu un compromis.

Cu backpropagation, rețelele puteau în sfârșit să învețe. Dar îndreaptă una spre o fotografie și apare o problemă nouă. O fotografie modestă are un milion de pixeli; dacă fiecare neuron din primul strat are nevoie de propria pondere pentru fiecare pixel, rețeaua se îneacă în ponderi înainte să învețe ceva. Ieșirea a venit, în parte, dintr-un experiment cu o pisică, la sfârșitul anilor '50.

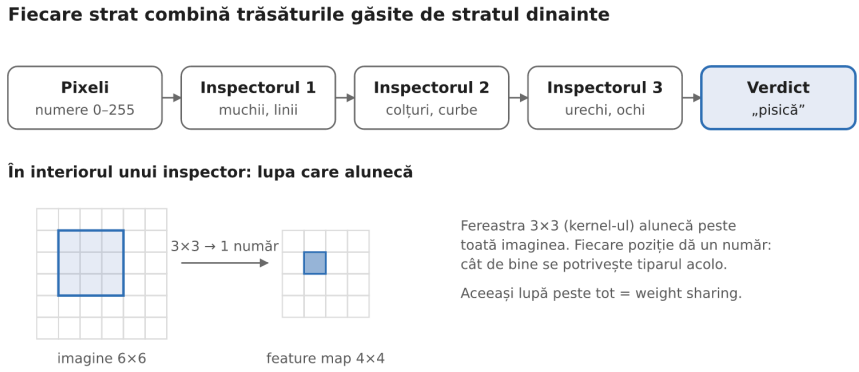
3.3 Ce a dezvăluit creierul pisicii

În 1958, doi tineri cercetători în neuroștiințe de la Johns Hopkins, David Hubel și Torsten Wiesel, înregistrau activitatea unor neuroni individuali din cortexul vizual al unei pisici anesteziate. Planul era să proiecteze pete de lumină pe un ecran în fața pisicii și să vadă ce pete fac un neuron să se activeze. Ore în șir, nu s-a întâmplat mare lucru. Apoi, în timp ce unul dintre ei introducea o lamelă de sticlă în proiector, marginea ei abia vizibilă a plimbat peste ecran o linie subțire de umbră, iar neuronul a început să descarce în rafale, pârâind în difuzor ca o mitralieră.

Neuronului nu-i păsa de pete. Îi păsa de *muchii*, la un anumit unghi, într-o anumită zonă a câmpului vizual. În anii următori, Hubel și Wiesel au cartografiat o întreagă ierarhie: celule simple care reacționează la muchii, celule mai complexe care reacționează la combinații de muchii și etape mai departe care reacționează la tipare tot mai bogate. Lucrarea le-a adus un Premiu Nobel în 1981 și a sădit o idee pe care informaticienii aveau s-o transforme într-una dintre cele mai reușite idei din machine learning.

Iată ideea, ca poveste. O echipă de inspectori examinează o fotografie, fiecare cu o lupă mică. Prima inspectoare nu are idee ce e o pisică. Caută doar schimbări bruște de luminozitate și notează unde sunt linii. Al doilea inspector nu vede deloc fotografia, doar notițele primei, și marchează locurile unde două linii se întâlnesc într-un unghi. Al treilea, citind acele notițe, observă că două unghiuri ascuțite alăturate arată ca o pereche de urechi. Ultimul pune cap la cap urechi, ochi și mustăți și spune: pisică.

Nimeni din acest lanț nu vede pisica întreagă. E asamblată din detalii locale care devin tot mai abstracte la fiecare nivel, așa cum citești un cuvânt din litere și literele din trăsături de condei.



Lupa e ceva ce ai folosit tu însuți. Filtrele de blur, sharpen și edge-detect din orice editor foto sunt mici grile de numere, de obicei trei pe trei, plimbate peste imagine. La fiecare poziție, grila e înmulțită cu pixelii de sub ea și rezultatele sunt adunate într-un singur pixel nou. O grilă de detectare a muchiilor dă un număr mare oriunde luminozitatea se schimbă brusc și aproape nimic pe zonele netede. Plimbarea unei grile mici peste o imagine e o **convolution** („convoluție”, de la latinescul *convolvere*, a înfășura, a răsuci împreună), grila însăși e un **kernel** („nucleu”) sau **filter** („filtru”), iar o **convolutional neural network** („rețea neuronală convoluțională”), pe scurt CNN, e un teanc de straturi care fac exact asta, cu o răsturnare decisivă: nimeni nu proiectează grilele. Numerele lor sunt învățate prin backpropagation, iar rețeaua inventează orice detectori o ajută să-și reducă eroarea. Unii ajung să semene remarcabil cu detectorii de muchii găsiți de Hubel și Wiesel la pisică. Mulți n-au niciun nume.

Asta dizolvă și problema care încheia subcapitolul trecut, milionul de ponderi per neuron. Gândește-te cum funcționează un linter. Nu ține o regulă separată pentru linia 1, alta pentru linia 2, alta pentru linia 3 din fișierul tău. Are o singură regulă și o aplică pe fiecare linie. Un CNN face la fel: un singur detector mic, cu mâna lui de ponderi, e aplicat în fiecare poziție din imagine, pentru că o muchie e o muchie oriunde ar apărea. Asta e **weight sharing** („partajarea ponderilor”: aceleași ponderi refolosite peste tot) și reduce numărul de ponderi cu ordine de mărime. Toleranța la mutarea unui obiect în altă parte a imaginii vine astfel în mare parte gratuit; toleranța la rotiri și la schimbări de mărime, nu, iar rețeaua le învață doar văzând exemple rotite și redimensionate, motiv pentru care fotografiile de antrenare sunt adesea rotite, decupate și mărite intenționat, în timp ce condensarea notițelor, descrisă imediat mai jos, adaugă o mică toleranță la deplasări mărunte.

Inspectorii de mai sus văd mai mult din imagine pentru că, între straturi, notițele sunt condensate. Un truc des întâlnit păstrează doar valoarea cea mai puternică din fiecare bloc de doi pe doi, ceea ce se numește **pooling** („comasare”). Fiecare strat rezumă astfel o zonă mai mare din fotografia originală decât cel dinainte, până când inspectorii din vârf se uită, practic, la întreaga imagine prin notițele lor.

Ideea a avut o incubație lungă. La sfârșitul anilor ’80 și în anii ’90, Yann LeCun și colegii lui au antrenat rețele convoluționale să citească cifre scrise de mână, iar versiuni ale sistemului lor au fost folosite ca să citească sumele de pe cecuri bancare. Apoi, în 2012, un CNN mult mai adânc, antrenat pe plăci video și cunoscut ulterior ca AlexNet, a intrat în competiția de recunoaștere a imaginilor

ImageNet (o colecție publică de peste un milion de fotografii etichetate) și a câștigat la o diferență care a uluit domeniul. În câțiva ani, CNN-urile citeau imagini medicale, inspectau plăci electronice pe liniile de producție, sortau colete în depozite și comparau capturi de ecran în testele de regresie vizuală.

Și apoi inspectorii au început să arate un obicei tulburător. Se agață de orice reduce eroarea, inclusiv de lucruri pe care nu le-a intenționat nimeni. Într-o demonstrație cunoscută, niște cercetători au antrenat intenționat un clasificator pe fotografii în care lupii apăreau mereu pe zăpadă, iar câinii husky niciodată, apoi au examinat pe ce se baza. Învățase, în mare, să detecteze zăpada. Proiectele reale pot cădea în aceeași capcană din greșeală: imaginează-ți un detector de defecte care învață în tăcere lumina singurei stații de inspecție unde au fost fotografiate întâmplător cele mai multe piese defecte. Cercetătorii au mai descoperit că CNN-urile antrenate pe colecții obișnuite de fotografii se bazează pe textură mult mai mult decât oamenii. Arată-i unui om un contur de pisică umplut cu piele de elefant și va spune pisică; o astfel de rețea e adesea mai impresionată de piele. Apărarea e să rupi coincidența din date: fotografiezi piese defecte la fiecare stație, cauți câini husky pe zăpadă și lupi pe iarbă și testezi intenționat pe astfel de cazuri nepotrivite înainte să ai încredere în model. Rețeaua tinde să ia cel mai ușor indiciu care funcționează; treaba ta e să te asiguri că indiciul ușor e cel corect.

Inspectorii au fost construiți pentru imagini, unde totul vine deodată, într-o grilă ordonată. Limbajul e altfel. Cuvintele vin unul după altul, propozițiile n-au o lungime fixă, iar sensul unui cuvânt de la sfârșit poate depinde de ceva spus chiar la început. Ca să citească o propoziție, o rețea are nevoie de un fel de memorie. Primele încercări de a-i da una semănau mult cu un joc de petrecere pentru copii.

3.4 Telefonul fără fir

La o petrecere de ziua unui copil, doisprezece copii stau în șir. Primul îi șoptește o propoziție celui de-al doilea, care îi șoptește ce a auzit celui de-al treilea, și tot așa până la capătul șirului. Ultimul copil se ridică și anunță mesajul. „Bunica a cumpărat o bicicletă roșie” a devenit „Bunicul a prins un pește mort”, iar toată lumea izbucnește în chicoteli.

Primele încercări serioase de a face rețelele neuronale să citească limbaj funcționau foarte asemănător cu acest joc și i-au moștenit slăbiciunea.

Ideea e firească. O propoziție vine câte un cuvânt pe rând, deci lasă rețeaua să citească câte un cuvânt pe rând și să ducă un rezumat de la un cuvânt la următorul. La fiecare pas, combină cuvântul nou cu rezumatul de până atunci și produce un rezumat actualizat. Rezumatul acesta se numește **hidden state** („starea ascunsă”), iar o rețea construită așa e o **recurrent neural network** („rețea neuronală recurentă”, pentru că aceeași operație revine la fiecare cuvânt), pe scurt RNN.

Dacă ai scris vreodată un `reduce` sau un `fold`, îi cunoști deja forma. Parcurgi o listă în ordine și la fiecare pas combini elementul curent cu un acumulator ca să obții acumulatorul nou. Un RNN e exact asta, cu două răsturnări: acumulatorul e o listă de numere de dimensiune fixă, iar funcția care îl combină cu fiecare cuvânt nou e o mică rețea neuronală ale cărei ponderi sunt învățate. Oricât de lungă ar fi propoziția, acumulatorul nu crește niciodată. Această dimensiune fixă e și frumusețea designului, și blestemul lui. În practică, acumulatorul are de obicei de la câteva sute până la vreo două mii de numere, alese o dată de proiectant: mult loc pentru o propoziție și foarte puțin pentru un roman întreg. Imaginea telefonului fără fir e înșelătoare într-un singur punct, cauza pierderii: copiii aud greșit din întâmplare, pe când rețeaua decide, după reguli învățate, ce păstrează și ce lasă să cadă, iar necazul e doar că un rezumat de dimensiune fixă trebuie să lase ceva să cadă.

Ca să simți blestemul, imaginează-ți că citești un roman polițist după o regulă ciudată: ai voie să păstrezi o singură fișă de notițe, iar după fiecare pagină trebuie s-o actualizezi. Până la capitolul douăzeci, fișa conține un rezumat excelent al scenei curente. Dar culoarea paltonului purtat de criminal, pomenită o singură dată în capitolul doi, a fost tăiată de mult ca să facă loc pentru ceva mai recent. Când detectivul dezvăluie în sfârșit că paltonul era verde, n-ai idee de ce contează.

Mai e un motiv, mai adânc, pentru uitare și ține de felul în care învață rețeaua. Antrenarea trebuie să trimită vina înapoi, iar într-un RNN asta înseamnă înapoi prin fiecare cuvânt al propoziției. La fiecare pas înapoi, vina e înmulțită cu un număr, iar dacă acel număr e de obicei puțin sub unu, semnalul se micșorează ca o dobândă compusă care merge invers. Înmulțește cu 0,9 de cincizeci de ori și rămâne cam o jumătate de procent. (Înmulțește în schimb cu 1,1 și vina crește de peste o sută de ori, necazul opus, numit exploding gradient („gradientul care explodează”), care se dovedește mai ușor de îmblânzit.) Primele cuvinte nu primesc aproape nicio corectură, așa că rețeaua abia dacă învață să lege lucruri aflate departe unul de altul. Aceasta e problema **vanishing gradient** („gradientul care se stinge”).

În 1997, Sepp Hochreiter și Jürgen Schmidhuber au propus un acumulator mai inteligent, numit **LSTM** (long short-term memory, „memorie de scurtă durată care ține mult”). În loc de o singură fișă rescrisă la fiecare pagină, imaginează-ți că fiecare cititor primește un mic carnet cu operații explicite: un comutator care decide ce se șterge (o rafinare adăugată câțiva ani mai târziu), unul care decide ce se scrie și unul care decide ce se citește chiar acum. Aceste comutatoare învățate se numesc **gates** („porți”). În termeni de programare, e diferența dintre o singură variabilă suprascrisă la fiecare iterație și un mic struct în care un câmp poate fi lăsat în mod deliberat neatins timp de o sută de iterații. Culoarea paltonului poate supraviețui până la capitolul douăzeci. Nimeni nu le spune comutatoarelor ce să șteargă sau ce să păstreze: fiecare e un mic neuron cu un variator, care se uită la cuvântul curent și la carnet și dă un număr între zero și unu, cât să șteargă, să scrie sau să citească, iar ponderile lui sunt învățate prin backpropagation, ca orice altceva, așa că rețeaua descoperă singură ce fel de detalii merită de obicei păstrate.

LSTM-urile au fost un triumf. Timp de aproximativ un deceniu, ele și rudele lor au alimentat recunoașterea vocală, predicțiile tastaturii de pe telefon și traducerea automată; când Google și-a mutat serviciul de traducere pe rețele neuronale, în 2016, în centrul lui era un sistem bazat pe LSTM. Ideile recurente apar încă oriunde datele chiar vin câte o bucată pe rând, cum ar fi fluxurile de la senzori și telemetria în timp real, iar o familie mai nouă, numită state-space models („modele în spațiul stărilor”; Mamba e un exemplu cunoscut), reia ideea rezumatului în mers cu o matematică mai bună.

Dar două probleme au refuzat să dispară. Carnetul a ajutat, dar mesajele tot călătoreau din mână în mână și, pe texte foarte lungi, tot se estompau. Iar structura din mână în mână era lentă: cuvântul cincizeci nu poate fi procesat până nu e gata cuvântul patruzeci și nouă, așa cum nu poți paraleliza un `reduce` în care fiecare pas depinde de rezultatul precedent. Plăcile video sunt construite să facă mii de lucruri deodată, iar un RNN le pune să stea la coadă.

Merită să te oprești o clipă asupra rostului șoptitului. Fiecare copil avea nevoie să știe mesajul de până atunci doar pentru că nu-l putea auzi direct pe primul copil. Așa că în 2017 un grup de cercetători de la Google a pus o întrebare care, privind înapoi, pare aproape naivă. Ce-ar fi dacă n-ar mai trebui să șoptească nimeni? Ce-ar fi dacă fiecare cuvânt dintr-o propoziție s-ar putea uita pur și simplu la toate celelalte, deodată? Articolul lor avea un titlu încrezător, *Attention Is All You Need* („Atenția e tot ce-ți

trebuie”), și a schimbat domeniul. Înainte să-l urmăm, însă, e momentul să ne oprim și să ne uităm la ce am construit de fapt în ultimele patru subcapitole. I-am tot spus rețea neuronală, iar un număr surprinzător de mare de oameni, inclusiv ingineri, cred că asta înseamnă că am construit un mic creier.

Mai departe. Dacă te-ai întrebat ce înseamnă de fapt ca fiecare cuvânt să se uite la toate celelalte, subcapitolul 7.2 desface ideea. De ce oprirea timpurie ajută în loc să strice se explică în subcapitolul 10.1, iar cum află cercetătorii că o rețea s-a uitat la zăpadă în loc de lup e subiectul subcapitolului 10.6. Restul echipamentului drumeției, de la momentum și warmup la ponderile de pornire alese cu grijă și la plafonul pus gradienților care explodează, e în subcapitolul 8.4.

Capitolul 4 — Din ce e făcut un model

4.1 Nu e un creier

Imaginează-ți că îi explici ce lucrezi unei prietene neurolog. Spui că echipa ta antrenează o rețea neuronală să depisteze suduri defecte. Ea se luminează și te întreabă dacă rețeaua are ceva ca un cortex vizual, dacă obosește, dacă poate uita. Pentru o clipă nu știi ce să răspunzi, pentru că răspunsul cinstit e că rețeaua ta are în comun cu creierul pacienților ei cam cât are un avion de hârtie cu un porumbel. Amândoi zboară. Asemănarea se oprește acolo.

Ca să vezi cât de puțin au în comun, urmărește o secundă din viața fiecăruia.

Începe cu un singur neuron din creierul pacientului, undeva în zona cortexului care se ocupă de atingere. E o celulă, cu un corp cât grosimea unui fir de păr și un hățiș de ramuri subțiri care ajung la mii de alte celule. Fiecare ramură se termină într-o sinapsă, un spațiu microscopic în care semnalul e purtat de substanțe chimice, nu de fire. În această secundă, la sinapse sosesc impulsuri din toate direcțiile: câteva sute de la vecinul acesta, o duzină de la acela, o rafală de la al treilea. Unele împing celula spre declanșare; altele o țin pe loc. Înăuntru, sarcina se adună ca apa în spatele unui baraj. De trei sau patru ori în secunda aceea, sarcina trece de un prag și celula se declanșează: un impuls electric scurt, numit spike, care aleargă pe singura ei fibră de ieșire spre toate celulele cu care e legată. Apoi sarcina se prăbușește și barajul începe să se umple din nou. Nimic din aceste impulsuri nu e un număr. Celulele următoare primesc un ritm: câte impulsuri, cât de apropiate, în ce tipar în timp. Iar în tăcere, sinapsele înseși se schimbă. Una folosită intens se întărește puțin; chimia ia minute și ore, iar o parte din ea va fi consolidată la noapte, în somn. Toată celula, făcând toate acestea, consumă cam atâta energie cât un fir de praf care cade.

Acum aceeași secundă pentru un neuron din detectorul tău de suduri. E un rând de numere într-un tabel. Sosește un vector de intrări, poate câteva sute de valori. Fiecare e înmulțită cu ponderea ei, produsele se adună, se adaugă un bias, iar totalul trece printr-o îndoitură, ReLU, care transformă rezultatele negative în zero. Iese un singur număr. Asta e toată secunda, și de fapt a durat câteva miliardimi din ea. Fără ritm, fără sincronizare, fără chimie; nimic din celulă nu s-a schimbat, pentru că ponderile se schimbă doar la antrenare, iar asta nu e antrenare. Va face exact același lucru, cu exact aceleași numere, data viitoare când va sosi aceeași intrare, și data de după, la nesfârșit. Cel puțin în principiu; o mică excepție ne așteaptă spre finalul cărții. Când un chatbot răspunde de fiecare dată altfel la aceeași întrebare, însă, întâmplarea e adăugată în cea mai mare parte după aceea, intenționat: rețeaua produce de fiecare dată practic aceeași listă de probabilități, iar un pas separat dă cu zarul ca să aleagă un cuvânt din ea.

O singură idee a supraviețuit: cântărește intrările, apoi activează-te sau nu

	Neuron biologic	Neuron artificial
Semnal	impulsuri electrice, cu ritm	un număr pe pas
Intrări	mii de sinapse	de la câteva la câteva mii de ponderi
Învățare	chimie, creștere, somn	ajustarea numerelor pe gradient
Viteză	cel mult câteva sute de impulsuri/s	miliarde de operații/s
Energie	cam 20 W pentru tot creierul	kilowați pentru un model mare
Număr	cam 86 de miliarde de neuroni	de la mii la milioane de unități

Comun: multe intrări ponderate → un prag → o ieșire care alimentează alte unități

Atunci de ce numele? În 1943, un neurofiziolog, Warren McCulloch, și un tânăr logician, Walter Pitts, au scris un articol în care argumentau că o celulă nervoasă poate fi privită ca o mică unitate logică: primește semnale de la multe celule, le adună și, dacă totalul trece de un prag, transmite mai departe celulelor din aval. Imaginea aceasta, multe intrări, un prag, o ieșire, e exact lista de bifat pentru apartamente din capitolul anterior. Rosenblatt a împrumutat-o pentru perceptron cincisprezece ani mai târziu, iar cuvântul „neural” a rămas lipit de domeniu de atunci. E ideea centrală care a trecut cu adevărat dincolo: o unitate care cântărește multe intrări, decide dacă să transmită ceva și e legată de multe altele ca ea. Tot restul din secunda pe care tocmai ai urmărit-o, ritmul, chimia, creșterea lentă, cei douăzeci de wați pentru tot creierul, a rămas pe dinafară.

Până și cuvântul „strat” (layer) și-a schimbat pe drum sensul. Cortexul tău vizual chiar are straturi, iar ierarhia lui Hubel și Wiesel a fost o inspirație reală pentru rețelele convoluționale. Dar cablajul creierului e un hățiș dens de bucle, cu semnale care circulă înapoi și lateral la fel de mult ca înainte. Teancul curat, de la stânga la dreapta, al unei rețele arată ca o schemă de creier dintr-un manual, nu ca un creier.

De ce ar conta toate acestea pentru cineva care vrea doar să folosească bine aceste unelte? Pentru că metafora modelează discret așteptările, iar așteptările sunt greșite în ambele direcții.

Într-o direcție, comparația cu creierul face rețelele să pară mai capabile decât sunt. Oamenii presupun că o rețea „înțelege” imaginile așa cum le înțeleg ei, că „își amintește” o conversație sau că „știe” când nu e sigură. Detectorul de suduri nu înțelege sudurile. A învățat o funcție foarte bună de la tipare de pixeli la un număr și poate fi păcălit de o schimbare de lumină pe care niciun inspector uman n-ar observa-o măcar. Și adesea nu știe ce nu știe, iar aici e una dintre rădăcinile halucinațiilor pe care le vom întâlni mai târziu.

În cealaltă direcție, comparația face rețelele să pară mai misterioase decât sunt. Ingineri care n-ar ezita niciodată să depaneze un compilator tratează uneori o rețea antrenată ca pe un oracol, ceva pe care îl poți doar tatona din afară. E o greșeală. O rețea e o funcție făcută din aritmetică, iar fiecare pas al ei poate fi afișat, desenat și inspectat. Următoarele subcapitole fac exact asta.

Ajută să înlocuiești imaginea creierului cu una mai modestă. Gândește-te la mixerul de sunet de la un concert, panoul lung cu sute de cursoare, fiecare modelând un pic din ce iese din boxe. O rețea neuronală e acel panou cu un milion de cursoare, sau un miliard. Antrenarea e procesul lung de a găsi pozițiile cursoarelor care fac ieșirea să se potrivească exemplelor. Mixerul se rupe într-un loc important:

pe un mixer adevărat fiecare cursor are o etichetă și face un singur lucru, pe când într-o rețea niciun cursor nu înseamnă ceva de unul singur, iar ce fac împreună mii de cursoare e încă greu de descifrat, atât de greu încât citirea unei rețele a devenit un domeniu de cercetare în sine. Imaginea e mai puțin palpitantă decât un creier într-o cutie și mult mai utilă. Îți spune că rețeaua poate produce doar ce îi permite aritmetica ei, că tot ce știe stă în pozițiile cursoarelor și că, dacă poți citi cursoarele, nimic dinăuntru nu e, în principiu, de neatins.

Și totuși. Avionul de hârtie chiar zboară. Aceeași aritmetică modestă, scalată și aranjată bine, recunoaște fețe, traduce limbi și scrie cod, lucruri despre care acum douăzeci de ani se spunea că ar necesita mai întâi înțelegerea creierului. Poate cel mai cinstit mod de a o spune e acesta: n-am copiat creierul. Am împrumutat o singură idee de la el și apoi ne-am văzut de drum, și încă suntem puțin surprinși de unde ne-a dus drumul acela.

Cursoarele ridică însă o întrebare practică pe care metafora creierului o ascundea. Un cursor mișcă un număr. Așa că, înainte ca orice să poată fi dat acestei mașini, o fotografie, o propoziție, un client, o melodie, trebuie să devină numere. Asta se dovedește a fi un meșteșug în sine, iar multe dintre eșecurile tăcute din machine learning se întâmplă acolo, înainte să fie implicat vreun model.

4.2 Transformând lumea în numere

Imaginează-ți o ingineră în prima săptămână la un loc de muncă nou, care primește un folder și o propoziție: „Fă-ne ceva care să prezică ce tichete de suport vor escalada.” Folderul conține trei ani de tichete, exportate din patru sisteme diferite. Deschide primul fișier așteptându-se la un tabel ordonat și găsește ce găsește până la urmă orice inginer: un haos.

Textul tichetelor e scris liber, în engleză, cu greșeli de tipar, stack traces lipite și câte un paragraf furios scris numai cu majuscule. Datele calendaristice sunt în trei formate. Coloana „prioritate” spune P1 într-un sistem, Critical în altul și 1 în al treilea. Țara clientului e când un cod, când un nume, când goală. Unele rânduri sunt duplicate. Unele au timpul de rezolvare completat cu −1. Iar singurul lucru de care are nevoie, o etichetă care să spună dacă tichetul a escaladat, stă într-un alt fișier, legat printr-un ID care nu se potrivește mereu.

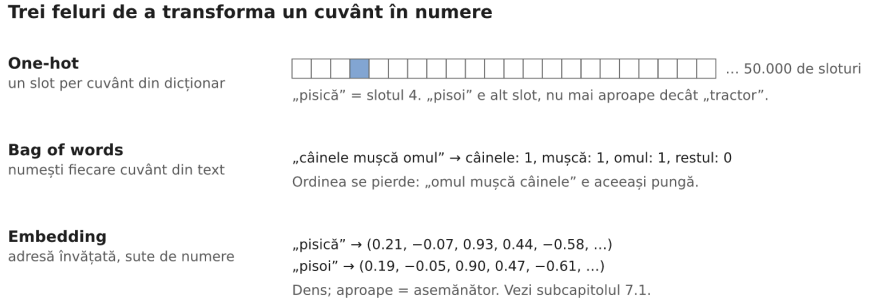
Nimic din folder nu e un număr pe care un model îl poate folosi. Subcapitolul acesta e despre meșteșugul de a-l face să fie, și e partea din machine learning pe care niciun demo n-o arată și căreia cei mai mulți practicieni îi dedică o mare parte din timp. Sondajele printre data scientists ajung iar și iar la aceeași concluzie: pregătirea datelor le ia mai mult din săptămână decât modelarea, adesea mult mai mult.

Începe cu cea mai ușoară trăsătură și urmărește cum se dovedește că nu e ușoară. Timpul de rezolvare, în ore. E deja un număr, deci poate intra direct, nu? Uită-te la valori: cele mai multe tichete se rezolvă între două și cincizeci de ore, câteva iau șase sute, iar câteva spun −1. Acel −1 e felul cuiva de a scrie „necunoscut”, iar dacă îl lasă așa, modelul va învăța că un tichet rezolvat în minus o oră e un fel foarte special de tichet. Valorile lipsă (missing values) trebuie găsite și trebuie luată o decizie pentru ele: eliminate, completate cu o valoare tipică sau marcate cu un indicator separat „lipsea”, ca modelul să poată învăța că golul însuși înseamnă ceva. Apoi vin tichetele de șase sute de ore. Pe o scală dreaptă înghit tot restul, și am văzut deja ce face un singur cui îndepărtat cu o riglă prinsă în arcuri. O mișcare

obișnuită e să iei logaritmul: două ore devin aproximativ 0,7, cincizeci devin aproximativ 3,9, șase sute aproximativ 6,4. Distanțele reflectă acum felul în care oamenii chiar resimt așteptarea, unde diferența dintre două ore și zece pare mai mare decât cea dintre cinci sute și cinci sute opt.

Acum o trăsătură care e un cuvânt: țara. Un model nu poate înmulți „Germania” cu o pondere. Scurtătura tentantă e să numerotezi țările: Germania 1, Franța 2, Japonia 3. E greșit într-un fel subtil: îi spune modelului că Japonia e de trei ori Germania și că Franța stă la jumătatea drumului între ele. Codificarea cinstită e să dai fiecărei țări propriul comutator, o intrare nouă care e 1 pentru acea țară și 0 pentru toate celelalte. Asta e **one-hot encoding** („codificare cu un singur bit aprins”: dintr-un șir de comutatoare, doar unul e pornit), și e exact ce face un formular cu variante multiple cu rândul lui de căsuțe. Costul e lățimea: două sute de țări devin două sute de intrări. O țară care n-a apărut niciodată la antrenare nu are propriul comutator, așa că toate comutatoarele arată zero, iar modelul o tratează ca pe „niciuna dintre țările pe care le cunosc”; o pregătire atentă a datelor adaugă pentru acest caz un comutator explicit „altele” și strânge în el, încă de la antrenare, valorile foarte rare, ca modelul să fi învățat deja ceva rezonabil de făcut cu el. Pentru o trăsătură cu câteva mii de valori posibile, ID-uri de produse sau de utilizatori, rândul de căsuțe devine absurd, iar acolo preiau controlul adresele învățate numite embeddings („încorporări”: fiecare lucru e așezat ca un punct într-un spațiu de numere): fiecare ID primește o listă scurtă de numere pe care antrenarea le ajustează, astfel încât ID-urile care se comportă la fel ajung cu numere asemănătoare.

Textul tichetelor e cazul greu și are o istorie care merită știută, pentru că cele trei feluri de a-l trata sunt trei trepte ale unei scări pe care a urcat-o tot domeniul.



Prima treaptă e să tratezi un document ca pe un **bag of words** („pungă de cuvinte”: arunci cuvintele într-o pungă și le numeri, fără să-ți pese de ordine): numeri de câte ori apare fiecare cuvânt din dicționar, iar acea listă de numărători devine reprezentarea numerică a documentului. Ordinea se pierde, „câinele mușcă omul” și „omul mușcă câinele” devin identice, iar cuvinte ca „și” sau „de” domină fiecare numărătoare. O rafinare mică numită **TF-IDF** (term frequency–inverse document frequency, „frecvența termenului înmulțită cu raritatea lui în toate documentele”) rezolvă a doua problemă, ponderând fiecare cuvânt după cât de rar e în toate documentele, astfel încât „și” nu mai contează aproape deloc, iar „kernel panic” contează mult. Oricât de grosolană, punga de cuvinte a alimentat motoare de căutare și filtre de spam timp de decenii și e încă un prim model puternic pentru tichete. Un tichet care conține „taxat de două ori”, „rambursare” și „avocat” va escalada foarte probabil, și nu e nevoie de nicio înțelegere a gramaticii ca să vezi asta.

A doua treaptă e să păstrezi ordinea și să tai textul în tokeni („jetoane”: bucăți de cuvinte), astfel încât modelele de secvențe să-l poată citi în ordine, bucată cu bucată. A treia treaptă e să transformi fiecare token într-un embedding, o adresă densă în care cuvintele asemănătoare sunt apropiate, și de aici încep modelele moderne de limbaj. Inginera noastră, în prima ei săptămână, ar face bine să înceapă pe prima treaptă. Ia o după-amiază, e ușor de explicat șefului și îi spune dacă textul conține vreun semnal înainte să petreacă o lună pe a treia.

Mai există un fel de trăsătură, cea pe care trebuie s-o inventezi. Routerul de tichete a arătat deja cazul cel mai simplu, un timestamp brut transformat în ora din zi, ziua săptămânii și zi de sărbătoare, iar același fel de a gândi merge mult mai departe. Nivelul abonamentului clientului, adus dintr-o altă bază de date. Câte tichete a deschis clientul în ultima lună. Dacă textul tichetului pomenește pe nume un concurent. Niciuna dintre aceste trăsături nu există în folder; le creează ea, gândindu-se la ce s-ar uita un manager de suport cu experiență. E tot feature engineering, iar pe o problemă tabelară ca aceasta un model mediocru cu trăsăturile potrivite bate de obicei un model strălucit cu trăsăturile greșite.

De aici apare o întrebare care împarte domeniul. Mai face cineva asta de mână? Pentru imagini și limbaj, în mare parte nu: rețelele adânci își învață singure features, și ăsta e tot rostul inspectorilor convoluționali și al embeddings. Nimeni nu-i spune unui model modern de vedere să caute muchii. Pentru datele de business, tabelele cu câteva zeci de coloane pe care le au de fapt cele mai multe companii, feature engineering-ul de mână e cât se poate de viu, iar inginerii care cunosc domeniul îl fac cel mai bine. Rezumatul cinstit e că deep learning a mutat meșteșugul de la inventarea de features la selectarea atentă a exemplelor, fără să-l elimine.

Mai e un ultim pas înainte ca tabelul să întâlnească un model, mic și ușor de uitat. Trăsăturile vin în unități foarte diferite: ore, numărători, un comutator 0-sau-1, un logaritm de ore. Amintește-ți salariile în euro puse lângă anii de experiență: numerele mari le îneacă pe cele mici. Așa că fiecare trăsătură e **scaled** („scalată”, adusă la o scară comună), de obicei scăzând media și împărțind la împrăștiere, astfel încât fiecare trăsătură vorbește cam la același volum. Modelele de tip arbore nu se sinchisesc; aproape toate celelalte, da.

Și iată capcana în care cad până și oamenii atenți. De unde vin mediile și împrăștierile pentru scalare? Dacă le calculează pe tot folderul, cu setul de test cu tot, atunci setul de test s-a scurs discret în antrenare. Fiecare număr pe care pregătirea îl învață din date, media folosită la scalare, valoarea tipică folosită la completarea golurilor, dicționarul de cuvinte, trebuie învățat doar din setul de antrenare și apoi doar aplicat restului. Disciplina practică e să pui fiecare pas de pregătire într-un singur obiect de tip pipeline („conductă”: o înlănțuire de pași rulată mereu la fel), ajustat o singură dată, pe datele de antrenare, și apoi aplicat neschimbat peste tot. Tratează-l ca pe un pas de build: determinist, versionat și rulat la fel în notebook și în producție.

Inginera noastră își încheie prima săptămână fără niciun model. Are un tabel: un rând per tichet, vreo patruzeci de coloane, fiecare valoare un număr pe o scară rezonabilă, o etichetă curată și un pipeline care poate produce același tabel din tichetele de luna viitoare. Șeful ei e puțin dezamăgit. N-ar trebui să fie. Ea a făcut partea pe care cele mai multe proiecte o greșesc, iar modelul care urmează va fi bun sau prost în mare parte din cauza ei.

Ceea ce lasă întrebarea evidentă: ce face de fapt modelul cu acest tabel? Răspunsul, surprinzător, e că nu lucrează niciodată cu altceva *decât* cu tabele.

4.3 Totul e un tabel

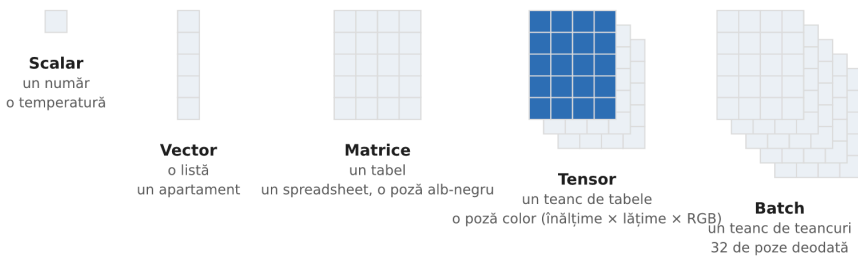
O colegă îți trimite un fișier numit `weld_detector.safetensors`. Are patru sute de megabyți. Nu conține cod, nici documentație, nici README. „Asta e modelul”, spune ea. „Tot ce știe e acolo.”

E o afirmație ciudată și merită testată. Așa că deschide-l. Nu cu un framework, ci cu un script care îl citește brut, așa cum ai trage cu ochiul în orice fișier binar necunoscut.

Ce găsești e aproape dezamăgitor. Fișierul e o listă de blocuri cu nume, iar fiecare bloc e același fel de lucru: o grilă de numere zecimale, cu o etichetă și o formă. `layer1.weight`, forma (32, 3, 3, 3), 864 de numere. `layer1.bias`, forma (32,), 32 de numere. `layer2.weight`, forma (64, 32, 3, 3), 18.432 de numere. Și tot așa, vreo douăzeci de blocuri, până la `classifier.weight` și `classifier.bias`. Adună toate numerele din toate blocurile și obții cam o sută de milioane. Acesta e modelul: o sută de milioane de zecimale, sortate în grile etichetate. Fără reguli, fără cod, fără nimic în formă de pisică. Cursoarele de la începutul capitolului, scrise pe hârtie.

Grilele au un nume, **tensor** (de la latinescul *tendere*, a întinde: un obiect care se întinde în mai multe direcții), care sună mult mai pompos decât e. Construiește-l din lucruri pe care le știi deja. Un singur număr e un **scalar**: temperatura de azi. O listă de numere e un **vector**: cele patruzeci de trăsături ale unui tichet din subcapitolul trecut, într-o ordine fixă. Un tabel de numere e o **matrix** („matrice”): întregul spreadsheet sau o fotografie alb-negru, în care fiecare celulă ține luminozitatea unui pixel. Mai adaugă direcții și ai un tensor. O fotografie color are trei: înălțimea, lățimea și cele trei canale de culoare. Un video are patru: timpul, înălțimea, lățimea, culoarea. Un teanc de treizeci și două de fotografii date modelului împreună are tot patru: care fotografie, înălțimea, lățimea, culoarea.

Scalar, vector, matrice, tensor: aceeași idee, cu mai multe axe



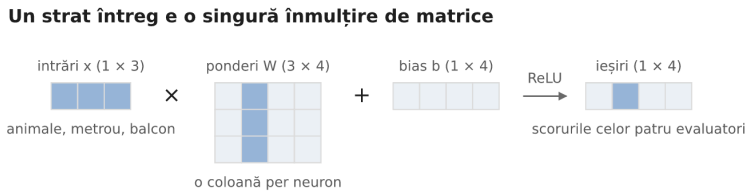
Fiecare pas adaugă o axă. Shape-ul unui tensor e lista lungimilor axelor lui,
de exemplu (32, 224, 224, 3): 32 de poze, fiecare de 224×224 pixeli, fiecare pixel cu 3 culori.

Shape-ul („forma”) e doar lista lungimilor fiecărei direcții, iar odată ce poți citi formele ca pe niște propoziții, jumătate din misterul codului de deep learning se evaporă. `layer1.weight` cu forma (32, 3, 3, 3) se citește: treizeci și două de filtre, fiecare uitându-se la trei canale de culoare, fiecare de trei pixeli pe trei pixeli. E o cutie cu treizeci și două dintre lupele convoluționale, iar acum știi că sunt de trei pe trei. Multe dintre erorile pe care le vei întâlni în acest domeniu sunt nepotriviri de formă, momentul în care încerci să dai un (32, 224, 224, 3) unui lucru care aștepta (32, 3, 224, 224). E unul dintre cele mai vechi bug-uri din programare, argumente în ordinea greșită, într-o lume în care compilatorul nu-l poate prinde în locul tău.

Până aici fișierul e doar stocare. Întrebarea interesantă e cum o sută de milioane de numere în repaus devin un verdict despre o sudură. Ca s-o vezi, micșorează problema până o poți rezolva de mână.

Întoarce-te la lista de bifat pentru apartamente: trei intrări, trei ponderi, le aduni, compari cu un prag. Scrie cele trei răspunsuri ca un rând scurt, (1, 0, 1), iar cele trei ponderi ca o coloană scurtă, (10, 4, 1). Înmulțește fiecare pereche și adună: 10 plus 0 plus 1 fac 11. Mica operație, rând contra coloană, înmulțire și adunare, se numește **dot product** („produs scalar”, scris cu un punct între cei doi vectori) și e un neuron care își face toată treaba.

Acum să zicem că patru evaluatori se uită la același apartament, fiecare cu propriile ponderi: începutul unui strat ascuns. Așază cele patru coloane de ponderi una lângă alta și ai o grilă, trei rânduri pe patru coloane. Înmulțește rândul de intrare cu acea grilă și ies patru numere deodată, câte unul per evaluator. Asta e **matrix multiplication** („înmulțire de matrice”), și iată propoziția pentru care există subcapitolul acesta: un strat întreg dintr-o rețea e exact una dintre acestea, urmată de adunarea bias-urilor și de îndoirea fiecărui rezultat cu funcția de activare. `layer1.weight` e o astfel de grilă. Forward pass-ul („trecerea înainte”) e intrarea înmulțită printr-o grilă după alta, strat după strat, până când ultima grilă produce un singur număr: defect sau nu.



Coloana evidențiată e un neuron: ponderile lui întâlnesc intrările, iar suma cade într-o celulă.
Patru neuroni = patru coloane. Un batch de 32 de apartamente face din x un tabel 32 × 3; restul nu se schimbă.

Fiecare celulă e independentă de celelalte: exact pentru asta e construit un GPU cu mii de nuclee mici.

Imaginea își arată valoarea în momentul în care ai mai mult de un apartament. Să zicem că sosesc treizeci și două deodată. Nu le parcurgi într-o buclă. Pui cele treizeci și două de rânduri de intrare într-un tabel, treizeci și două de rânduri pe trei coloane, și îl înmulțești cu exact aceeași grilă de trei pe patru. Iese un tabel de treizeci și două pe patru: fiecare apartament, fiecare evaluator, într-o singură operație. De aceea codul de deep learning nu conține aproape niciodată o buclă `for` peste exemple. Bucla e pliată în înmulțire, iar din acea pliere vine viteza.

Ceea ce explică, în sfârșit, plăcile video. De ce sunt cipurile făcute pentru jocuri video motoarele inteligenței artificiale moderne? Uită-te la rezultatul de treizeci și două pe patru. Fiecare celulă din el e un produs scalar care poate fi calculat fără să știi nimic despre vreo altă celulă. Un CPU e o mână de nuclee foarte deștepte, construite să ruleze repede un singur lucru complicat. Un GPU are mii de nuclee simple, construite să facă același lucru simplu pe mii de pixeli deodată. Înmulțirea de matrice e mii de produse scalare identice care nu depind unele de altele. Parcă ar fi fost croită pentru GPU-uri, cu decenii înainte să se gândească cineva la asta. Când a sosit rezultatul ImageNet din 2012, autorii lui antrenaseră pe două plăci video obișnuite, într-un dormitor, iar o bună parte din deceniul de deep learning de atunci încoace a fost povestea unor grile mai mari pe mai multe plăci. Azi, o singură placă

video de centru de date are zeci de gigabyți de memorie, uneori mult peste o sută, și face de ordinul unui milion de miliarde de înmulțiri simple pe secundă la precizie redusă; cele mai mari antrenări leagă între ele zeci de mii de astfel de plăci, timp de luni.

Așadar colega ta avea dreptate, iar acum poți vedea exact de ce. Încarcă grilele în memorie. Transformă o fotografie în tensorul ei de pixeli. Înmulțește-o prin grila unu, îndoaie; grila doi, îndoaie; și tot așa până la capăt. Iese un singur număr. Nu se întâmplă nimic altceva. Metafora creierului poate fi lăsată deoparte; modelul e un spreadsheet care a învățat, iar rularea lui e aritmetică pe care ai putea-o face, în principiu, cu creionul, dacă ai avea câțiva ani liberi.

O ultimă privire la fișier, pentru că și mărimea lui spune o poveste. O sută de milioane de numere, patru sute de megabyți: asta înseamnă patru octeți per număr, formatul obișnuit în virgulă mobilă pe 32 de biți. Treci la un model de limbaj cu șapte miliarde de numere și același format are nevoie de douăzeci și opt de gigabyți, mai multă memorie decât au cele mai multe plăci video. Așa că inginerii stochează exact aceleași numere cu mai puțină precizie, pe șaisprezece biți, pe opt, uneori pe patru. Asta e **quantization** („cuantizare”: rotunjirea numerelor la mai puține trepte posibile) și e o mare parte din motivul pentru care un model care avea nevoie de un server anul trecut poate rula pe un laptop anul acesta. Gândește-te la ea ca la un JPEG pentru ponderi. Se pierde puțină fidelitate, se câștigă mult spațiu, iar de cele mai multe ori imaginea arată încă bine. Detectorul de suduri, cuantizat pe opt biți, ar avea o sută de megabyți și, foarte probabil, ar fi la fel de bun. Merge pentru că cele mai multe ponderi sunt numere mici, strânse în jurul lui zero, iar fiecare ieșire adună mii de produse, așa că micile erori de rotunjire se anulează în mare parte, ca estimările de la târgul lui Galton. Mergi prea departe, la patru biți sau mai puțin fără grijă, și calitatea chiar scade, mai ales la întrebările grele și la modelele mai mici, și de aceea modelele cuantizate sunt testate din nou înainte de lansare.

Ai văzut materialul. Dar fișierul avea vreo douăzeci de grile, cu forme și nume diferite, iar cineva a decis că vor fi exact acestea, în exact această ordine. Cutia de piese din care au ales și problemele care au forțat fiecare piesă să existe sunt subiectul capitolului următor. Mai întâi, încă o întrebare despre materialul însuși: din tot ce primește, cum decide un model ce contează?

4.4 Ce contează și cine decide

O companie scoate la concurs un post de inginer și primește două sute de CV-uri. Patru oameni ajută la triere, și fiecare decide altfel ce contează. Recrutoarea junior lucrează după o fișă de punctaj primită de la HR: trei puncte pentru fiecare an de experiență, cinci pentru o diplomă relevantă, două pentru fiecare limbaj de programare de pe listă, aceeași fișă pentru fiecare CV. A doua urmează o schemă logică: a livrat candidatul un sistem în producție? Dacă da, a lucrat într-o echipă de peste zece oameni? Fiecare răspuns trimite CV-ul pe o ramură sau pe alta. A treia, o recrutoare cu douăzeci de ani de meserie, citește un CV și pur și simplu are o senzație despre el, clădită din mii de observații mărunte pe care nu le-a numit niciodată. Iar a patra face ceva ce celelalte nu fac: lasă fiecare CV să-i spună unde să se uite. Pe unul, o pauză de doi ani îi atrage atenția și caută rândul care o explică. Pe altul, pauza nu contează, pentru că lista de proiecte spune tot. Ce citește atent se schimbă de la un CV la altul.

Cele patru corespund principalelor moduri în care un model decide ce contează și merită luate pe rând, pentru că „ce contează pentru model?” e una dintre întrebările cele mai des puse și cel mai vag lămurite din domeniu. Primele trei țin de modelele întâlnite până acum: rigla, arborii și rețelele adânci. A patra

ține de un mecanism numit attention („atenție”), din inima modelelor de limbaj de azi, și va trebui să-și aștepte rândul până când cartea ajunge la limbaj.

Să începem chiar cu cuvântul. Un feature („trăsătură”) a însemnat până acum în carte două lucruri ușor diferite și va însemna curând un al treilea. La început era o proprietate a fiecărui exemplu pe care i-o dai modelului, scrisă adesea ca o coloană într-un tabel: liniile modificate, ora din zi, țara ca șir de comutatoare. Apoi neuronii ascunși își inventau propriile trăsături, judecăți intermediare ca intuițiile nenumite ale recrutoarei cu experiență. Iar când cartea va ajunge la limbaj, sensul va locui în direcții pe o hartă, amestecuri de multe coordonate, și de obicei la asta se gândesc cercetătorii când vorbesc despre trăsăturile din interiorul unei rețele mari. Un pixel, în schimb, e doar o intrare: trăsăturile unei imagini, o muchie, o ureche, o bucată de blană, sunt ce construiește rețeaua din mulți pixeli. Un cuvânt stă la mijloc. Sosește ca simplu număr de ordine într-un vocabular și devine ceva cu sens abia după ce își primește embedding-ul, adresa lui învățată, și e citit împreună cu cuvintele din jur.

Acum fișa de punctaj. Un model liniar, rigla din trusa de unelte, decide ce contează o singură dată, în timpul antrenării, alegând câte o pondere pentru fiecare trăsătură, și apoi aplică aceleași ponderi fiecărui caz. De aici tentația de a citi ponderile ca importanță: cea mai mare pondere, cea mai importantă trăsătură. Merge doar cu grijă. O pondere se măsoară pe unitatea intrării ei, așa că o pondere pe megabyte și una pe header de cerere nu pot fi comparate până când ambele trăsături n-au fost aduse la o scară comună. Iar când două trăsături spun aproape același lucru, de pildă liniile adăugate și liniile modificate, modelul poate împărți meritul între ele aproape arbitrar: o pondere mare pe una și una negativă pe cealaltă pot da cam aceleași predicții ca două ponderi modeste. Reantrenează pe date puțin diferite și împărțirea se poate schimba, în timp ce predicțiile abia se mișcă. Fișa de punctaj se poate citi, dar ce citești e una dintre multele fișe care ar fi mers cam la fel de bine.

Schema logică decide ce contează prin testul sertarului cu șosete: la fiecare pas câștigă trăsătura a cărei întrebare face cea mai multă ordine în sertare. Clasamentul care adună ordinea făcută de fiecare trăsătură, cel pe care majoritatea bibliotecilor îl afișează implicit, e o primă privire utilă, dar părtinitoare. O trăsătură cu mii de valori distincte, un ID de utilizator sau un timestamp exact, oferă mii de praguri posibile, iar din simplul număr de încercări unele vor face ordine întâmplătoare în sertarele de antrenare. Asemenea trăsături tind să pară mai importante decât sunt.

Există un test mai onest, și merge pentru orice model. Ia setul de validare și, pentru o singură trăsătură, amestecă valorile ei între exemple, astfel încât fiecare tichet poartă acum țara altui tichet, iar restul rămâne pe loc. Apoi măsoară din nou scorul. Dacă scade brusc, modelul s-a sprijinit pe trăsătura aceea; dacă abia se mișcă, modelul se descurcă și fără ea. Metoda se numește **permutation importance** („importanța prin permutare”: cât pierde modelul când valorile unei trăsături sunt amestecate la întâmplare), a fost introdusă de statisticianul Leo Breiman în lucrările lui despre random forests, iar marele ei merit e că îl întreabă pe model, nu pe analist. Are și ea un punct orb, și sunt tot cei doi gemeni. Amestecă „liniile adăugate” cât timp „liniile modificate” rămân intacte, iar modelul se sprijină pe celălalt geamăn, scorul abia scade, iar ambele trăsături pot părea neimportante, deși împreună duc predicția.

O propoziție despre toate aceste măsuri trebuie spusă direct, pentru că e cea mai frecventă greșeală de interpretare din domeniu. Importanța înseamnă că modelul a folosit trăsătura. Nu înseamnă niciodată, singură, că acea trăsătură cauzează rezultatul. Cu datele potrivite, un model ar putea clasa vânzările de înghețată drept un excelent predictor al înecurilor.

Recrutoarea cu experiență e rețeaua multistrat. Nimeni nu le spune neuronilor ei ascunși ce să caute; backpropagation îl transformă pe fiecare în judecata intermediară care ajută la scăderea loss-ului, oricare ar fi ea. Asta înseamnă și că nu există un singur loc spre care să arăți: verdictul ei asupra unui caz trece prin sute de intuiții, fiecare amestecând multe intrări. Poți întreba totuși din afară ce intrări au contat, prin amestecare sau cu uneltele făcute pentru a privi în interiorul rețelelor, care vin spre finalul cărții, dar răspunsul e acum rezumatul a ceva mult mai încâlcit.

Rămâne întrebarea practică pe care orice echipă o pune până la urmă: să-i dau modelului pur și simplu toate trăsăturile pe care le găsesc? Mai multe trăsături nu sunt gratis. Fiecare e încă o dimensiune în care vecinii cei mai apropiați se îndepărtează unii de alții, încă o ocazie de a se agăța de o coincidență, farfuria albă sau zăpada din spatele lupilor, și încă o posibilă scurgere. Un remediu numit regularizare, care îi cere modelului un mic preț pentru complexitate, atenuează costul, iar una dintre variantele lui împinge ponderile trăsăturilor inutile exact la zero, eliminându-le discret; multe date îl atenuează și mai mult. Dar punctul de plecare rezonabil e cel pe care o urmează un recrutor bun: include ce ai motive să crezi că ar putea conta, verifică pe ce s-a sprijinit de fapt modelul și fii bănuitor când o trăsătură pe care o credeai minoră se dovedește dominantă.

Aici comisia de recrutare încetează să fie o imagine bună. O recrutoare îți poate spune de ce a respins un CV, iar motivele ei țin de ce cere cu adevărat postul. Noțiunea de „a conta” a unui model e doar o asociere statistică cu loss-ul lui, pe datele pe care s-a întâmplat să le vadă. Schimbă datele, angajează în alt oraș sau în alt deceniu, și ce contează pentru model se schimbă odată cu ele, fără ca modelul să observe și fără ca cineva să fie anunțat.

Mai departe. Dacă te-ai întrebat de ce un chatbot răspunde de fiecare dată altfel la aceeași întrebare, subcapitolul 7.3 explică zarul și setarea numită temperatură. Cum își primesc cuvintele adresele învățate, embeddings, e subiectul subcapitolului 7.1, iar felul în care citește a patra recrutoare, lăsând fiecare caz să-i spună unde să se uite, e mecanismul de attention din subcapitolul 7.2, iar subcapitolul 7.5 revine la ea ca să arate ce poate și ce nu poate vedea.

Capitolul 5 — Cum se construiesc și cum rulează modelele

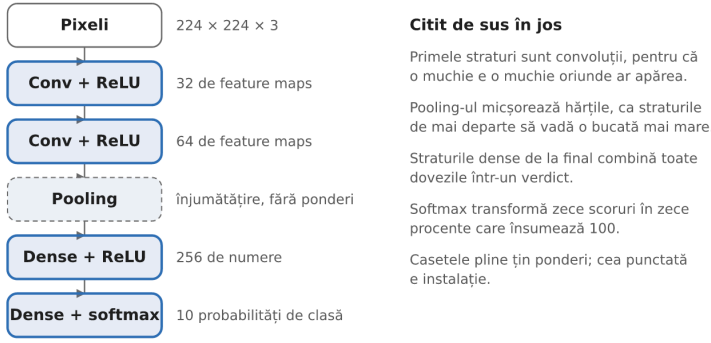
5.1 Cutia de Lego

Să zicem că decizi să construiești de la zero un clasificator de imagini, înarmat doar cu cele două idei din subcapitolele trecute: un strat e o grilă de ponderi, iar un teanc de straturi cu îndoituri între ele poate învăța aproape orice. Zece mii de fotografii cu cifre scrise de mână, etichetate. Cât de greu poate fi?

Primul tău design e cel evident. Aplatizezi fiecare fotografie într-un rând lung de pixeli, 784 pentru o imagine mică de 28 pe 28, și dai rândul unui dense layer („strat dens”: fiecare intrare legată de fiecare neuron) cu câteva sute de neuroni, apoi încă unuia, apoi zece ieșiri pentru cele zece cifre. Îl antrenezi. Merge, mai mult sau mai puțin: cam 97% corect. Încurajator. Apoi încerci fotografii reale, de 224 pe 224, color, și designul se prăbușește. Doar primul strat ar avea nevoie de 150.000 de intrări înmulțite cu câteva sute de neuroni, zeci de milioane de ponderi, doar ca să se uite la pixeli. Mai rău, un „7” mutat cu doi pixeli la dreapta ajunge pe intrări complet diferite, iar modelul trebuie să-l învețe din nou de la zero. N-are idee că un șapte e un șapte oriunde ar sta.

Tocmai ai descoperit de ce există convoluțiile. Înlocuiește primele straturi dense cu lupele inspectorilor din povestea creierului pisicii, fiecare o grilă mică de ponderi folosită la fel în toate pozițiile, cu pasul de pooling între ele, care micșorează hărțile. Numărul de ponderi scade de o mie de ori. Șaptele mutat e recunoscut pentru că lupa care l-a găsit în stânga îl găsește și în dreapta. Rețeaua ta are acum forma pe care o aveau cititoarele de cifre ale lui LeCun în anii ’90 și citește fotografii reale.

Piese îmbinate: un mic clasificator de imagini



Încurajat, o faci mai adâncă. Dacă patru straturi sunt bune, sigur douăzeci sunt mai bune? Le pui unul peste altul și antrenezi. Loss-ul nu scade. Stă pe loc. Încerci treizeci de straturi și devine *mai rău* decât versiunea cu patru, chiar pe datele de antrenare, ceea ce ar trebui să fie imposibil: o rețea mai adâncă ar putea măcar s-o copieze pe cea mică și să nu facă nimic în straturile în plus. Ceva din antrenarea însăși eșuează. Acesta e zidul de care s-a lovit deep learning-ul pe la 2014, iar două invenții l-au dărâmat.

Prima a venit din urmărirea numerelor. Când un semnal trece prin multe straturi, scara lui alunecă: după câteva straturi, toate valorile sunt minuscule, după încă câteva, totul e uriaș, iar gradient descent se descurcă prost cu ambele. Remediul a fost aproape jenant de practic. După fiecare strat, readuci numerele într-un interval standard, așa cum un termostat readuce camera la douăzeci de grade orice vreme ar fi afară. Asta e **normalization** („normalizarea”), introdusă în 2015 sub numele de batch normalization, iar cu ea, împreună cu o inițializare atentă a ponderilor, rețelele adânci au devenit mult mai ușor de antrenat. Modelele de limbaj folosesc un termostat puțin diferit, câte unul pentru fiecare token în loc de unul pentru fiecare batch.

A doua a venit din întrebarea de ce o rețea mai adâncă nu poate măcar s-o copieze pe una mai mică. În teorie poate, dar să învețe să lase un semnal să treacă printr-un strat *neschimbat* e surprinzător de greu pentru un teanc de ponderi care pornesc la întâmplare. Atunci de ce s-o pui să învețe asta? Dă-i fiecărui strat un ocol. În loc ca un strat să-și înlocuiască intrarea cu ieșirea, el își *adaugă* ieșirea la intrare, iar suma merge mai departe. Stratul trebuie să învețe doar ce să schimbe; dacă n-are nimic de adăugat, zero e un lucru ușor de învățat, iar semnalul trece neatins. Aceasta e **residual connection** („conexiunea reziduală”: rețeaua învață doar restul, diferența), iar arhitectura construită pe ea, ResNet (residual network), a câștigat competiția ImageNet din 2015 cu 152 de straturi, o adâncime mult peste orice funcționase cu un an înainte. Inginerii pot recunoaște tiparul dintr-o altă lume: un lanț de middleware în care fiecare pas poate modifica o cerere sau o poate lăsa să treacă neatinsă, în loc de unul în care fiecare pas trebuie să reconstruiască toată cererea de la zero. Și a mai fost un bonus. Backpropagation, vina care merge înapoi de la ieșire, avea acum o bandă expres: putea curge direct pe ocoluri, fără să fie diluată de fiecare strat din cale.

Rețeaua ta de treizeci de straturi se antrenează acum minunat. Obține 99,8% pe fotografiile de antrenare. Pe fotografii noi, abia 91%. Ai construit un tocilar: a memorat fotografiile de antrenare în loc să învețe să citească cifre. Unul dintre remedii, în formă de strat, e destul de ciudat ca să rămână în minte. În timpul antrenării, oprești la întâmplare o parte dintre neuronii fiecărui strat, alții la fiecare pas. La folosirea reală, nu oprești nimic. Rețeaua nu se mai poate baza pe niciun neuron anume că va fi mereu acolo, așa că învață redundant, mai mulți neuroni împărțind fiecare sarcină. Acesta e **dropout**-ul („abandonul”, de la neuronii care ies temporar din joc) și e versiunea neuronală a unui manager care trimite la întâmplare oameni acasă în timpul repetițiilor, ca echipa să învețe să se acopere reciproc. Scorul pe fotografii noi urcă la 96%.

Fă un pas înapoi și uită-te la ce ai construit, pentru că toată cutia e acum pe masă.



Există două feluri de piese. Primul fel face munca și ține ponderi învățate, iar fiecare piesă întruchipează un pariu despre date. **Dense layer**-ul nu pariază pe nimic; leagă totul de tot, ceea ce îl face general și scump. Convolution-ul pariază că lucrurile apropiate țin împreună și că un tipar înseamnă același lucru oriunde ar apărea, pariul potrivit pentru imagini, sunet și orice stă pe o grilă. **Recurrent layer**-ul pariază că lucrurile vin în ordine și că trecutul contează. **Attention layer**-ul („stratul de atenție”), inima modelelor de limbaj de azi, pariază că orice din intrare poate conta pentru orice altceva și lasă intrarea însăși să decidă ce. Când pariul unui strat se potrivește cu datele, el învață din mult mai puține exemple, pentru că nu trebuie să descopere ce presupune deja. Când pariul e greșit, nicio cantitate de date nu-l salvează complet.

Al doilea fel e instalația: piese cu puține sau deloc ponderi, care există ca să țină primul fel antrenabil. Normalizarea, termostatul. Conexiunea reziduală, ocolul. Dropout-ul, zilele libere de la repetiții. Activation function-ul, îndoitura dintre straturi, atât de subțire încât abia contează ca piesă și atât de esențială încât nu o lași niciodată deoparte. Iar la intrare, pentru orice vine ca nume și nu ca număr, **embedding layer**-ul („stratul de embedding”): un tabel de căutare cu un rând pentru fiecare token, produs sau utilizator posibil, ale cărui rânduri sunt învățate. E felul în care începe un model de limbaj. Spre deosebire de Lego-ul adevărat, însă, piesele nu sunt gata când le îmbini: fiecare își capătă forma abia în antrenare, alături de vecinele ei, așa că un strat scos dintr-un model antrenat nu poate fi pur și simplu prins în altul fără antrenare suplimentară.

Acum dezvăluirea care face diagramele de arhitectură lizibile. Din afară, grădina zoologică de nume de modele, ResNet, U-Net, BERT, GPT, pare nesfârșită. Dinăuntru, aproape toate sunt aceleași vreo zece piese, aranjate în ordini diferite. Iar aranjamentul urmează un tipar pe care tocmai l-ai redescoperit singur: muncitori specializați, plini de presupuneri, aproape de intrare, unde datele își mai au structura brută, și muncitori generali, fără presupuneri, aproape de ieșire, unde structura a fost deja redusă la dovezi. Un model de vorbire începe cu convoluții peste sunet și se termină cu straturi dense. Un Transformer începe cu un embedding și se termină cu un strat dens și un **softmax** („maximul blând”: piesa de instalație care transformă un rând de scoruri în procente care însumează o sută) peste vocabular. Softmax-ul face fiecare scor pozitiv ridicând un număr fix la puterea scorului, apoi împarte fiecare rezultat la totalul lor; din cauza puterilor, un scor cu doar câteva puncte peste celelalte ia aproape toată probabilitatea, motiv pentru care un model pare adesea mai hotărât decât arată scorurile lui brute. Mijloacele diferă. Capetele sunt aproape mereu aceleași. Cât despre cine proiectează un mijloc nou, de cele mai multe ori nimeni nu o face de la zero: echipele iau un plan care și-a dovedit deja valoarea, schimbă câte o piesă pe rând și păstrează schimbarea doar dacă scorul pe simulare se îmbunătățește; există și căutări automate care încearcă mii de aranjamente, dar costul lor face ca cea mai mare parte a muncii practice să fie refolosire atentă.

Dacă vrei un model mental confortabil, gândește-te la un pipeline de date. Fiecare strat e o etapă care primește un tensor de o formă și emite un tensor de altă formă. Convoluțiile fac hărțile mai mici și mai adânci, pooling-ul le înjumătățește, straturile dense schimbă lungimea unui vector. Arhitectura e o semnătură de tip, formă la intrare, formă la ieșire, iar cele mai multe discuții reale de design sunt despre ce forme curg pe unde. Primul lucru de făcut cu orice model necunoscut e să afișezi forma după fiecare etapă.

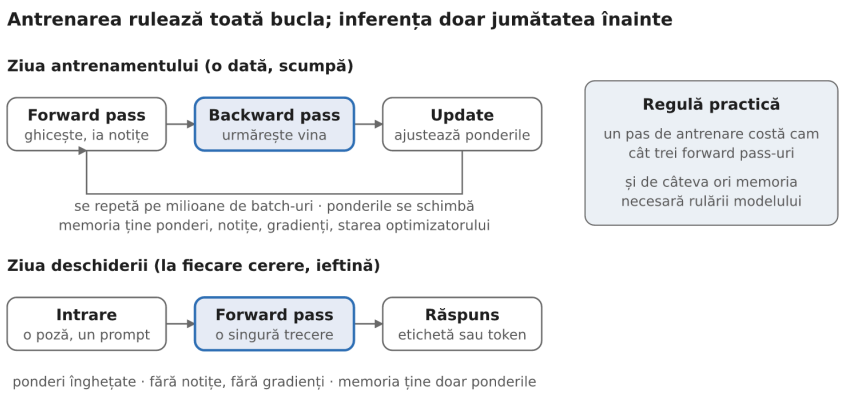
Ai acum o rețea care funcționează și o idee despre rostul fiecărei părți. Ce nu ai încă e o senzație a costului, și mai ales de ce după-amiaza petrecută antrenând-o va părea un fleac, în anii următori, pe lângă costul simplei ei rulări.

5.2 Cele două vieți ale unui model

Un restaurant are două zile complet diferite. E anul dinainte de deschidere: bucătarul gustă sute de variante ale fiecărui fel, ajustează sarea, schimbă furnizorul, rescrie meniul de trei ori, arzând bani fără niciun client la orizont. Și e ziua deschiderii, și fiecare zi de după: rețetele sunt fixe, bucătăria le execută, iar singura întrebare e câte farfurii pot ieși pe oră și cu ce cost.

Un model are exact aceste două vieți, iar aproape tot ce e confuz în economia machine learning-ului vine din amestecarea lor.

Prima viață e training-ul („antrenarea”). E bucla din prefață, rulată serios: forward pass, măsori eroarea, backward pass („trecerea înapoi”), ajustezi fiecare pondere, următorul batch. Se întâmplă o dată, sau o dată per versiune. E anul dinainte de deschidere. A doua viață e **inference** („inferența”, adică a trage concluzii): modelul e gata, ponderile lui sunt înghețate, iar tu îi dai intrări și culegi răspunsuri. Intră o fotografie, iese „pisică”. Intră un prompt, iese un token. E ziua deschiderii, repetată la fiecare cerere, cât timp modelul e în serviciu. Aici imaginea restaurantului se rupe într-un fel instructiv: o bucătărie adevărată învață din fiecare reclamație din ziua deschiderii, pe când un model nu învață nimic de la al milionulea client, iar ce greșește va continua să greșească până când cineva antrenează o versiune nouă.



Ce e exact diferit între cele două? Parcurge un singur pas de antrenare și numără ce trebuie să țină mașina în memorie. Forward pass-ul trece intrarea prin fiecare strat, dar trebuie să și *ia notițe*: rezultatul intermediar de la fiecare strat, pentru că backward pass-ul va avea nevoie de ele ca să urmărească vina, exact cum inspectorul din atelierul de mobilă avea nevoie să vadă munca fiecărui post. Apoi backward pass-ul calculează un gradient pentru fiecare pondere, adică încă o copie de mărimea modelului. Apoi optimizatorul, dacă e Adam, ține pentru fiecare pondere o memorie a gradienților recenți, adică încă două copii. Adună totul, iar antrenarea unui model are nevoie de câteva ori mai multă memorie decât simpla păstrare a ponderilor și de aproximativ trei ori calculul unui forward pass, la fiecare pas, timp de milioane de pași.

Inferența aruncă aproape toate acestea. Fără notițe, pentru că nu va fi urmărită nicio vină înapoi. Fără gradienți, pentru că nu se va schimba nimic. Fără starea optimizatorului. Ponderile stau în memorie, intrarea se înmulțește prin ele și iese răspunsul. E doar forward pass-ul, o dată. De aceea un model care

a avut nevoie de un cluster de plăci video ca să fie antrenat poate rula adesea pe hardware mult mai modest, uneori chiar pe un telefon; și de aceea rularea unui model e ieftină per cerere, dar poate deveni totuși costul dominant, pentru că ziua deschiderii se întâmplă de un milion de ori.

Cele două vieți au și bug-uri diferite, iar recunoașterea vieții în care te afli economisește ore. Problemele de antrenare sunt despre *dacă modelul învață*: loss-ul nu scade, sau scade și apoi explodează, sau scade pe setul de antrenare în timp ce crește pe cel de validare. Acelea sunt probleme de learning rate, probleme de date, tocilarul care memorează și leneșul care nu învață destul. Problemele de inferență sunt despre *viteză, memorie și consecvență*: răspunsul durează prea mult, modelul nu încape pe dispozitiv sau aceeași intrare dă răspunsuri puțin diferite în producție față de notebook. Ultima are o cauză clasică. Dropout-ul oprește semnale în timpul antrenării și trebuie oprit el însuși la inferență; uită linia care trece modelul în modul de evaluare și răspunsurile din producție se clatină la întâmplare.

Undeva între cele două vieți stă un truc pe care inginerii îl întâlnesc mereu: **batching**-ul („gruparea în loturi”). La o brutărie, încălzirea cuptorului costă la fel fie că coci o pâine, fie douăzeci, așa că îl umpli. O placă video e la fel. Adesea, partea scumpă a inferenței e mutarea ponderilor din memorie în procesor, iar costul acesta se plătește o dată per forward pass, oricâte intrări ar merge împreună. Așa că serverele de inferență așteaptă câteva milisecunde ca să adune cereri, le pun într-un singur tensor de batch și le trec prin model împreună. Asta costă atât de puțin pentru că un batch de treizeci și două e doar un tabel mai înalt în aceeași înmulțire. Compromisul e între latență și debit (throughput), iar oricine a reglat o coadă de mesaje va recunoaște butonul.

Modelele de limbaj adaugă o răsucire zilei deschiderii care merită înțeleasă, pentru că explică facturile. Un clasificator rulează un forward pass per cerere. Un model de limbaj rulează un forward pass *pentru fiecare token pe care îl generează*, pentru că fiecare token nou e dat înapoi ca intrare pentru următorul. Un răspuns de cinci sute de cuvinte înseamnă câteva sute de forward pass-uri, unul după altul, iar modelul nu-l poate începe pe al doilea până nu l-a terminat pe primul. De aceea răspunsurile lungi costă mai mult decât cele scurte, de aceea tokenii de ieșire sunt de obicei mai scumpi decât cei de intrare și de aceea atâta efort de inginerie merge în cache-uirea rezultatelor intermediare de la tokenii anteriori, ca să nu fie recalculate de fiecare dată.

Ca să faci totul concret, urmărește o singură cerere până la factură. Să zicem că o companie rulează un asistent de suport pe un model de limbaj găzduit care taxează, în cifre rotunde, un dolar pe milion de tokeni de intrare și patru dolari pe milion de tokeni de ieșire. Întrebarea unui client, plus instrucțiunile și cele trei pasaje găsite pentru asistent, ajung la două mii de tokeni. Răspunsul are trei sute de tokeni. Intrarea costă două mii de milionimi de dolar, o cincime de cent. Ieșirea costă o mie două sute de milionimi, puțin peste o zecime de cent. O conversație, cam o treime de cent. Pare nimic până când asistentul gestionează patruzeci de mii de conversații pe zi, moment în care ajunge la vreo o sută treizeci de dolari pe zi și cam cincizeci de mii pe an, iar departamentul financiar începe să întrebe de ce tokenii de ieșire costă de patru ori mai mult decât cei de intrare. Mare parte din răspuns e bucla token cu token de mai sus: tokenii de ieșire se produc unul câte unul, fiecare cu forward pass-ul lui, în timp ce tokenii de intrare sunt procesați toți împreună, într-un singur batch. E diferența dintre a servi un fel deja pus în farfurie și a găti fiecare lingură la comandă. Aceeași aritmetică explică de ce inginerii echipei își petrec timpul cu prompturi mai scurte, cache-uri și modele mai mici pentru întrebările ușoare.

Așadar: antrenarea e anul scump de gustat și ajustat; inferența e ziua ieftină și repetată de servit, care pe durata vieții unui model folosit pe scară largă ajunge adesea să coste mai mult; iar fișierul modelului care trece dintr-o viață în alta nu e altceva decât teancul de grile din capitolul anterior, cu fiecare număr în sfârșit liniștit. Între cele două vieți mai e una, mai scurtă, în care un model terminat se întoarce la școală câteva săptămâni pentru o sarcină mai îngustă. Aceea e fine-tuning-ul („reglajul fin”), care va primi mai târziu un subcapitol al lui.

Înainte să părăsim sala mașinilor, un ocol. Toate rețelele neuronale de până acum din carte au stocat cunoștințele în același fel: ca ponderi învățate încet, prin gradient descent. Există o idee mai veche și mai ciudată despre cum ar putea ține minte o mașină, una care învață într-un singur pas și tratează un gând ca pe un șir foarte lung, aproape aleatoriu, de numere. A influențat discret felul în care oamenii gândesc despre embeddings și merită un subcapitol al ei.

5.3 Un alt fel de memorie

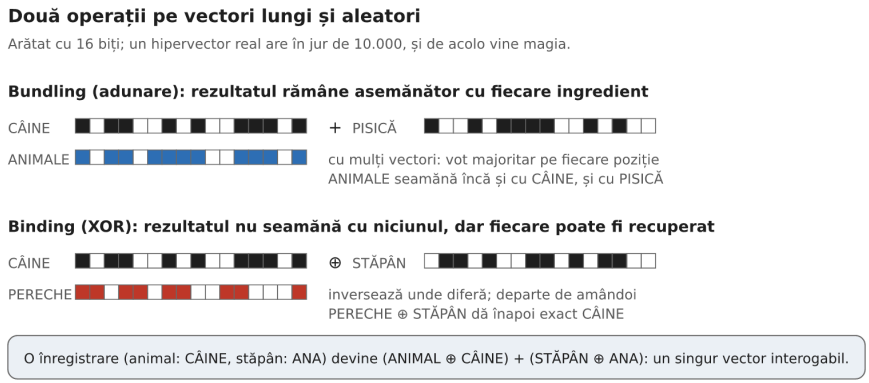
Ia două monede corecte și aruncă fiecare de zece mii de ori, notând rezultatele ca două șiruri lungi de cap și pajură. Acum compară șirurile poziție cu poziție. În câte poziții se potrivesc?

În jur de cinci mii, cu o sută în plus sau în minus. Așa arată, pur și simplu, întâmplarea. Și iată lucrul care face posibilă o întreagă ramură a informaticii: e practic imposibil, în zece mii de aruncări, să obții din întâmplare două șiruri care se potrivesc în șase mii de poziții. Șansele sunt astronomic de mici. Așa că în lumea șirurilor de zece mii de biți, „aleatoriu” înseamnă „cam pe jumătate asemănător”, iar orice e vizibil mai asemănător de atât nu poate fi o coincidență. Trebuie să însemne ceva.

Acesta e punctul de plecare al **hyperdimensional computing** („calcul hiperdimensional”: calcul cu vectori de mii de dimensiuni), numit și vector symbolic architectures („arhitecturi simbolice vectoriale”), pe scurt VSA, o idee dezvoltată de-a lungul mai multor decenii de cercetători precum Tony Plate și Pentti Kanerva. E o abordare care nu învață deloc prin gradient descent, iar a o vedea alături de rețelele neuronale clarifică ce face de fapt fiecare.

Ideea începe cu o atribuire îndrăzneață. Fiecare concept primește un șir aleatoriu de zece mii de biți, tras o dată și neschimbat niciodată. CÂINE e un șir aleatoriu. PISICĂ e altul. STĂPÂN, ANA, LUNI, fiecare își primește pe al lui. Datorită faptului cu monedele, toate aceste șiruri sunt aproape la fel de departe unul de altul, cam pe jumătate asemănătoare, ceea ce e felul vectorial de a spune „fără legătură”. Încă nu s-a învățat nimic. Șirurile sunt doar nume, destul de lungi încât să nu poată fi confundate vreodată.

Apoi vin două operații, iar împreună sunt toată gramatica.



Prima e **bundling** („înmănunchere”): aduni vectorii, făcând un vot majoritar pe fiecare poziție. Înmănunchezi CÂINE și PISICĂ și obții un șir nou, să-i zicem ANIMALE, vizibil mai mult decât pe jumătate asemănător cu CÂINE și vizibil mai mult decât pe jumătate asemănător cu PISICĂ. Rezultatul seamănă cu ingredientele lui. Poți înmănunchea o duzină de lucruri într-un singur vector și să-l întrebi mai târziu „CÂINE e aici?”, primind un da sigur prin măsurarea asemănării. Bundling-ul e o mulțime, stocată ca un singur șir. Fiecare element nou adaugă puțin zgomot peste toate celelalte: cu zece mii de biți, o mulțime de la câteva zeci până la vreo două sute de elemente poate fi încă interogată sigur, iar dincolo de asta ecourile membrilor încep să se scufunde în zgomot.

A doua e **binding** („legarea”): combini doi vectori cu XOR, care pune 1 acolo unde biții diferă și 0 acolo unde coincid; altfel spus, întoarce biții unui vector acolo unde celălalt are 1. Leagă CÂINE de STĂPÂN și obții un șir care e cam pe jumătate asemănător cu CÂINE și pe jumătate cu STĂPÂN, adică nu seamănă cu niciunul. E un nume nou, fără legătură. Iar XOR are o proprietate minunată: leagă rezultatul din nou de STĂPÂN și iese CÂINE, exact, pentru că al doilea XOR întoarce la loc exact biții pe care i-a întors primul. Binding-ul lipește două lucruri într-o pereche, iar lipiciul e reversibil.

Pune-le pe amândouă împreună și poți stoca o întreagă înregistrare într-un singur vector. Un rând de bază de date precum *animal: CÂINE, stăpân: ANA, zi: LUNI* devine (ANIMAL legat de CÂINE) înmănuncheat cu (STĂPÂN legat de ANA) înmănuncheat cu (ZI legată de LUNI). Un singur șir de zece mii de biți. Ca să întrebi „cine e stăpânul?”, legi înregistrarea de STĂPÂN; partea ANA iese clar, iar celelalte părți devin zgomot pe care îl spală simpla lungime a vectorului. E un key-value store în care întregul tabel, cheile și căutarea sunt toate același fel de obiect: un șir lung de biți.

Dacă ai folosit vreodată un Bloom filter (o structură care răspunde rapid „există X în mulțime?”, cu o mică șansă de eroare), asta îți va părea cunoscut. Și un Bloom filter stochează o mulțime într-un șir de biți de mărime fixă, și el răspunde rapid la întrebări de apartenență și și el își plătește compactitatea cu o mică șansă de confuzie. Calculul hiperdimensional e ideea aceasta crescută mare: nu doar „există X în mulțime?”, ci „cu ce e X în pereche?”, cu structura datelor trăind în aritmetică.

Acum, de ce i-ar păsa unui inginer care lucrează cu rețele neuronale?

Pentru că comparația lămurește. Într-o rețea neuronală, cunoștințele trăiesc în ponderi, iar ca să ajungă acolo e nevoie de milioane de pași de gradient. În calculul hiperdimensional, cunoștințele trăiesc în vectorii înșiși, iar ca să ajungă acolo e nevoie de un bundle sau un bind. Nu există buclă de antrenare. A învăța un fapt nou e o operație, iar a-l uita e tot o operație. Asta e aproape de felul în care se poartă o

bază de date și foarte departe de felul în care se poartă o rețea, unde adăugarea unui singur fapt nou înseamnă încă o rundă de antrenare și unde a-l împiedica pe un model să uite o abilitate veche în timp ce învață una nouă e o durere de cap cunoscută, cu numele puțin dramatic de *catastrophic forgetting* („uitare catastrofală”).

Abordarea explică ceva și despre embeddings, unde lucrurile asemănătoare primesc vectori apropiați. Faptul cu monedele îți spune de ce poate funcționa așa ceva în sute de dimensiuni. Într-un spațiu cu multe dimensiuni e o cantitate enormă de loc: poți așeza milioane de concepte astfel încât fiecare să fie departe de toate celelalte și să ai totuși o noțiune clară de „mai aproape decât întâmplarea”. Intuiția din puține dimensiuni nu mai funcționează aici. În trei dimensiuni nu poți înghesui multe puncte toate departe unele de altele. În zece mii, poți înghesui mai multe decât vei avea vreodată nevoie, iar abundența aceasta de loc e cea pe care se sprijină discret și embeddings, și hipervectorii. E faptul care, în povestea vecinilor celor mai apropiați, făcea din toată lumea un străin, văzut din partea cealaltă. Punctele aleatorii din multe dimensiuni sunt toate cam la fel de departe unele de altele. Pentru o căutare de vecini pe features brute, unde „aproape” trebuia să însemne ceva, asta strică totul; pentru calculul hiperdimensional, ale cărui nume sunt menite să nu aibă legătură între ele, e exact ce trebuie. Embeddings învățate fac apoi munca suplimentară de a apropia lucrurile înrudite.

Există un preț și el e motivul pentru care calculul hiperdimensional a rămas mai mult în laboratoare de cercetare și hardware specializat decât în IDE-ul tău (mediul de dezvoltare, editorul de cod). Numele aleatorii nu știu nimic. CÂINE și PISICĂ pornesc la fel de străine una de alta ca CÂINE și SPREADSHEET, iar orice asemănare dintre ele trebuie construită de mână sau prin bundling. O rețea neuronală, în schimb, *descoperă* că câinii și pisicile țin împreună din milioane de exemple și descoperă structuri pe care nu s-a gândit nimeni să le codifice. Calculul hiperdimensional e robust la zgomot, rapid, transparent și ieftin de rulat pe dispozitive mici, de aceea atrage interes pentru senzori și cipuri cu consum mic și de aceea unii cercetători îl văd ca pe un model al felului în care ar putea funcționa memoria într-un creier. Dar nu învață din date brute cum învață rețelele.

E tentant să le vezi ca rivale și mai util să le vezi ca complementare. Rețelele sunt excelente la transformarea intrărilor brute și dezordonate, pixeli, sunet, text, în vectori cu sens. Metodele hiperdimensionale sunt excelente la compunerea vectorilor în structuri, înregistrări, secvențe, relații și la interogarea lor fără nicio antrenare în plus. Mai multe grupuri de cercetare experimentează exact cu această predare de ștafetă: o rețea ca strat de percepție care produce vectori, iar aritmetica hipervectorilor ca strat de memorie și raționament care îi stochează și îi combină.

Și mai e un motiv să ții minte subcapitolul acesta pentru mai târziu. Mecanismul de attention, căutarea în dicționar „cu aproximație” din inima modelelor mari de limbaj de azi, e și el un mod de a stoca și regăsi perechi cheie-valoare sub formă de vectori și de a amesteca multe dintre ele într-una singură. A venit din altă direcție și e învățat, nu atribuit, dar când vei vedea întrebări potrivite cu chei ca să scoată valori, s-ar putea să te gândești la o înregistrare legată de STĂPÂN și la ANA ieșind din zgomot.

Turul sălii mașinilor e aproape gata. Știm din ce e făcută o rețea, cum se îmbină piesele ei, ce se întâmplă în anul antrenării și în ziua deschiderii și am văzut un drum pe care domeniul nu l-a ales. Rămâne o întrebare practică, pe care o pune orice proiect înainte să înceapă: câte date, cât de mare un model și câți bani sunt de ajuns?

5.4 Cât e destul

Oricine s-a apucat de chitară la vârsta adultă cunoaşte forma primului an. În prima lună, progresul e spectaculos: de la un acord bâjbâit la o melodie întreagă. Pe la şase luni, câştigurile sunt încă reale, dar mai mici, şi fiecare costă mai multe seri. În al doilea an, încă o sută de ore din aceleaşi exerciţii abia te mai urnesc, iar ce te-ar ajuta e cu totul altceva: un profesor care vede obiceiul care te ţine pe loc sau un instrument mai bun, care ţine acordajul. Exersatul contează în continuare. Doar că nu mai e piedica principală.

Modelele învaţă pe acelaşi fel de curbă, iar întrebările pe care oamenii le pun cel mai des despre un proiect (de câte exemple avem nevoie, ne-ar ajuta un model mai mare, cât o să coste toate astea) sunt de fapt întrebări despre locul de pe curbă în care se află.

Începe cu unealta cea mai practică, pentru că răspunde la prima întrebare pentru datele tale, nu pentru ale altcuiva. Antrenează modelul pe 10% din datele de antrenare, apoi pe 20%, apoi pe 50%, apoi pe toate, şi măsoară de fiecare dată eroarea pe acelaşi set de validare. Pune cele patru puncte pe un grafic. Asta e o **learning curve** („curbă de învăţare”: cum scade eroarea pe măsură ce modelul primeşte mai multe date), iar forma ei e un verdict. Dacă eroarea de validare încă scade abrupt la 100%, mai multe exemple vor ajuta foarte probabil, iar etichetarea unui nou lot e bani bine cheltuiţi. Dacă s-a aplatizat, mai multe date de acelaşi fel vor aduce puţin; eşti chitaristul din anul al doilea. Combin-o cu un obicei simplu, să citeşti mereu eroarea de antrenare lângă cea de validare, şi de obicei îţi dai seama de ce manetă să tragi. O diferenţă mare între cele două, care se îngustează pe măsură ce cresc datele, arată tot tocilarul, flămând de exemple, iar mai multe date vor ajuta. Două erori mari şi apropiate arată un model prea simplu pentru sarcină, pe care mai multe exemple nu-l vor vindeca, dar un model mai expresiv sau trăsături mai bune ar putea. Două erori mici şi o curbă plată înseamnă că eşti aproape de tot ce îţi pot spune datele acestea, iar următorul câştig va veni din etichete mai curate sau dintr-o întrebare mai bine pusă.

Oamenii tot vor un număr şi e corect să dăm ordine de mărime aproximative, atâta timp cât sunt citite ca puncte de plecare pentru experimentul de mai sus, nu ca promisiuni. Un clasificator simplu pe date tabelare curate, cu câteva trăsături informative, e adesea util cu câteva sute până la câteva mii de rânduri etichetate pe clasă, iar clasa mai rară dictează de obicei necesarul: o mie de fraude contează mai mult decât un milion de plăţi cinstite. Un model de imagini construit prin transfer learning, pornind de la o reţea antrenată deja pe milioane de alte fotografii, se poate descurca bine cu câteva sute până la câteva mii de fotografii. Ca să înveţi un model de limbaj un format sau un ton, prin fine-tuning sau cu LoRA, o metodă uşoară care antrenează doar un mic adaos la modelul îngheţat, e nevoie adesea de sute până la câteva mii de exemple alese cu grijă, nu de milioane; ca să-i dai cunoştinţe noi, e de obicei mai bine să foloseşti regăsirea, adică să-l laşi să caute informaţia în documentele tale chiar în timp ce răspunde. Iar pre-antrenarea de la zero a unui model mare de limbaj e alt univers, cu text de antrenare măsurat în mii de miliarde de tokeni.

Două avertismente însoţesc fiecare dintre aceste numere. Calitatea bate cantitatea, cu mult: etichetele „flaky” de vineri după-amiaza nu devin mai bune când sunt mai multe; doar predau mai apăsat lecţia greşită. Iar exemplele trebuie să acopere cazurile pe care modelul le va întâlni. Aici imaginea chitarei se rupe. Pentru un muzician, o oră de game seamănă mult cu alta. Pentru un model, o mie de exemple care sunt aproape copii unele ale altora îl învaţă cam cât unul singur, în timp ce o sută de exemple din

colțurile în care greșește îl învață foarte mult. Asta e toată ideea din spatele **active learning** („învățare activă”), în care modelul arată singur exemplele pe care ar vrea cel mai mult să le eticheteze un om, și a obiceiului deloc spectaculos de a elimina duplicatele înainte de antrenare.

Acum întrebarea mai mare, cea din spatele unui deceniu de titluri de ziar: de ce s-au tot făcut modelele mai mari? Pentru că, cel puțin la modelele de limbaj, „mai mare” s-a dovedit previzibil. În 2020, o echipă de la OpenAI condusă de Jared Kaplan a antrenat familii de modele de limbaj de mărimi foarte diferite, pe cantități diferite de text, și a descoperit că loss-ul scădea pe curbe remarcabil de netede pe măsură ce creșteau numărul de parametri, cantitatea de date și cantitatea de calcul, de-a lungul multor ordine de mărime. Desenate pe axe logaritmice, curbele erau aproape niște drepte. Aceste regularități se numesc **scaling laws** („legi de scalare”: cum scade eroarea, previzibil, când crești modelul, datele și calculul), iar importanța lor practică a fost enormă. Un laborator putea antrena ieftin o serie de modele mici, putea trasa dreapta prin ele și putea prezice cam cât de bine s-ar descurca un model de o sută de ori mai mare, înainte să cheltuiască banii. Previzibilitatea aceasta, mai mult decât orice descoperire anume, e cea care le-a făcut pe organizații dispuse să cheltuiască zeci de milioane pe o singură antrenare.

Doi ani mai târziu, o echipă de la DeepMind condusă de Jordan Hoffmann a corectat rețeta. Articolul lor, cunoscut de obicei după numele modelului, Chinchilla, punea de fapt o întrebare de brutărie. Să zicem că ai un buget fix pentru o extindere. Cuptoarele fără făină stau degeaba; făina fără cuptoare se învechește în magazie. Undeva există un echilibru. La modelele de limbaj, cuptoarele sunt parametrii și făina e textul de antrenare, iar studiul din 2022 a conchis că modelele mari ale vremii erau aproape numai cuptoare: pentru un buget de calcul dat, era mai bine să antrenezi un model mai mic pe mai mult text, crescându-le pe amândouă cam în același ritm. O regulă practică rămasă de aici, cam douăzeci de tokeni de antrenare pentru fiecare parametru, e încă citată pe larg. Chinchilla însuși, cu 70 de miliarde de parametri antrenați pe aproximativ 1.400 de miliarde de tokeni, a depășit modelul Gopher al aceleiași DeepMind, de patru ori mai mare, antrenat cu cam același calcul.

Imaginea brutăriei se rupe în două locuri instructive. Primul: echilibrul Chinchilla minimizează doar costul antrenării. Un model care va răspunde la miliarde de întrebări se plătește mai ales în zilele de după deschidere, așa că multe modele recente sunt intenționat mai mici decât „optimul” și antrenate pe mult mai mult de douăzeci de tokeni per parametru, pentru că un model mic e mai ieftin de servit ani la rând. Al doilea: randamentul unei brutării e înțeles; dublezi cuptoarele și făina și coci cam de două ori mai multă pâine. Legile de scalare sunt regularități empirice, fără o teorie unanim acceptată în spatele lor. Ele prezic loss-ul mediu pe tokenul următor, nu dacă va apărea o anumită abilitate; dacă abilitățile chiar apar brusc de la o anumită mărime încolo e încă o chestiune disputată. Și nimic nu garantează că dreptele rămân drepte la nesfârșit.

Făina, se pare, e limitată. În 2024, grupul de cercetare Epoch AI a estimat stocul de text public scris de oameni, de calitate rezonabilă, la câteva sute de mii de miliarde de tokeni, cu o incertitudine mare, și a prognozat că, în ritmul de atunci, cele mai mari antrenări l-ar putea epuiza undeva între 2026 și 2032. Industria are trei răspunsuri. Selecție mai bună: filtrezi, elimini duplicatele și ponderezi textul, pentru că o mie de miliarde de tokeni bine aleși bat de trei ori pe atâția aleși fără grijă. Repetiție cu grijă: câteva treceri peste același text tot ajută, deși tot mai puțin. Și producție: **synthetic data** („date sintetice”: text scris de modele ca să antreneze alte modele). Funcționează cel mai bine acolo unde rezultatul poate fi verificat, cum ar fi problemele de matematică cu răspuns verificabil sau codul care trebuie să treacă testele, astfel încât exemplele greșite să poată fi prinse și aruncate. Acolo unde nu poate fi verificat,

există un risc cunoscut: cercetătorii au arătat că modelele antrenate, generație după generație, mai ales pe ieșirea modelelor anterioare tind să piardă părțile rare și neobișnuite ale datelor originale, ca o fotocopie după o fotocopie, o îngustare numită **model collapse** („prăbușirea modelului”).

În sfârșit, costul, care e aritmetica rulării unui model, înmulțită. Fiecare pas de antrenare vine cu memoria și cu calculul triplu socotite la cele două vieți ale modelului, iar un model de frontieră are nevoie de un număr enorm de pași. În practică, asta înseamnă de la mii la zeci de mii de plăci grafice care rulează săptămâni sau luni. Companiile își publică rar facturile, dar estimările independente pentru cele mai mari antrenări sunt de ordinul zecilor de milioane de dolari și urcă spre sute de milioane, în timp ce adaptarea unui model modest cu LoRA poate costa mai puțin decât o cină. Energia urmează aceeași curbă. O estimare citată pe larg, făcută de cercetători de la Google în 2021, punea antrenarea lui GPT-3 la aproximativ 1.300 de megawați-oră, cam cât consumă într-un an o sută douăzeci de locuințe americane, iar antrenările de frontieră de atunci încoace au fost mult mai mari. Un singur răspuns e mic prin comparație: în 2025, Google a raportat că un prompt text median trimis asistentului său Gemini consuma cam un sfert de watt-oră, comparabil cu vreo nouă secunde de televizor, deși astfel de cifre depind mult de ce anume e inclus în calcul și sunt greu de verificat din afară. Înmulțit cu miliarde de cereri, plus apa care răcește centrele de date, totalul devine o chestiune pentru rețelele electrice și consiliile locale, nu pentru o singură echipă de ingineri.

Așadar, „cât e destul?” are un răspuns cinstit, chiar dacă nu e un singur număr: destul e acolo unde curba ta de învățare se aplatizează, măsurată pe datele tale, cu duplicatele eliminate și cazurile grele acoperite. Dincolo de punctul acela ai nevoie de un profesor mai bun, nu de mai mult exercițiu. Cu asta, turul sălii mașinilor s-a încheiat. Înainte să trecem la limbaj și sens, însă, o întrebare așteaptă răbdătoare de la începutul capitolului anterior. Dacă o rețea nu e un creier, atunci ce face de fapt un creier, cel care merge cu douăzeci de wați și învață din mers? Neuroștiința are câteva răspunsuri îndrăznețe, iar cel mai îndrăzneț dintre ele începe cu o cană de cafea.

Mai departe. Dacă te-ai întrebat ce se întâmplă de fapt într-un model de limbaj în timp ce scrie un token după altul, subcapitolul 7.3 urmărește totul cadru cu cadru. Cum se întoarce la școală un model terminat pentru o sarcină mai îngustă, de la transfer learning la LoRA, e subiectul subcapitolului 9.2. Tocilarul pe care l-ai construit, opusul lui, leneșul, și restul remediilor pentru amândoi au subcapitolul 10.1 doar pentru ei. Iar cât de sigur e de fapt un model care răspunde la orice cu aceeași siguranță fluentă e întrebarea subcapitolului 10.5.

Capitolul 6 — Cealaltă mașină

6.1 O mie de modele ale unei căni de cafea

Închide ochii și roagă pe cineva să-ți pună un obiect în mână. Atinge-l cu vârful unui deget. Simți ceva neted și curbat. Ar putea fi multe lucruri. Alunecă degetul în sus și dai de o margine, o buză. Alunecă-l lateral și în jos și găsești o buclă ieșită în afară. Cană de cafea. Trei atingeri, poate patru, și ești sigur, deși nicio atingere luată separat nu ți-a spus mare lucru.

Acum fă-o din nou, dar apucă obiectul cu toată mâna. Știi instantaneu. Fără alunecat, fără șir de presupuneri.

Acest mic experiment stă în inima uneia dintre cele mai îndrăznețe încercări recente de a explica felul în care funcționează creierul, iar autorul ei și-a început cariera nu ca cercetător în neuroștiințe, ci ca inginer. Jeff Hawkins a proiectat PalmPilot, unul dintre primele calculatoare de buzunar de succes, apoi a petrecut deceniile următoare cu ce considera el problema mai interesantă: cum calculează de fapt neocortexul, stratul exterior și încrețit al creierului în care trăiesc percepția, limbajul și planificarea. Și-a expus primele idei în *On Intelligence*, în 2004, și o teorie mai completă în *A Thousand Brains*, în 2021. Ce urmează e acea teorie, așa cum o spun el și colegii lui. E influentă, e disputată și e un mod minunat de a te gândi la cum ar putea face un creier ce nicio rețea din carte nu face încă bine.

Povestea începe cu o observație mai veche și mai ciudată. În 1978, neurofiziologul Vernon Mountcastle a remarcat ceva ce ar fi trebuit să surprindă mai mult decât a surprins. Neocortexul are zone pentru vedere, pentru auz, pentru atingere, pentru limbaj, pentru planificare. Te-ai aștepta ca fiecare să fie construită altfel, așa cum o placă video diferă de o placă de rețea. Sub microscop, nu sunt. Țesutul arată aproape la fel peste tot: aceleași straturi, aceleași tipuri de celule, același tipar de conexiuni, repetat în mici unități verticale numite **cortical columns** („coloane corticale”). Mountcastle a propus că cortexul rulează același algoritm peste tot și că ce face o zonă să fie „vizuală” e pur și simplu faptul că e legată la ochi.

Cea mai izbitoare dovadă a venit dintr-un experiment care sună a science fiction. Pe la 2000, o echipă de la MIT condusă de Mriganka Sur a reconectat dihori nou-născuți astfel încât semnalele de la ochi să fie trimise în partea creierului care procesează de obicei sunetul. Cortexul „auditiv” a învățat să vadă. A dezvoltat hărți sensibile la orientarea liniilor asemănătoare celor dintr-un cortex vizual, iar animalele reacționau la lumină prin el. Oamenii născuți orbi arată o versiune mai blândă a aceluiași lucru: cortexul lor vizual e pus la treabă pentru atingere, de exemplu la citirea în Braille. Un algoritm, multe întrebuințări. Pentru un programator, e o idee cunoscută: o bibliotecă generică bine scrisă, instanțiată cu tipuri de intrare diferite.

Așadar, ce calculează fiecare coloană? Aici face Hawkins cea mai mare afirmație a lui. Într-o rețea convoluțională, fiecare inspector vede doar un fragment și doar ultimul vede „pisică”. În teoria Thousand Brains („o mie de creiere”), fiecare coloană învață *modele complete ale unor obiecte întregi*. Sunt, după estimarea lui Hawkins, cam 150.000 de coloane în neocortexul uman, fiecare cam cât un bob de orez. Fiecare, primind intrări de la propria ei mică bucată de piele sau de retină, încearcă să recunoască singură întreaga cană.

Cum poate o coloană să țină un obiect întreg dacă simte mereu doar o bucată mică odată? Răspunsul e ideea-cheie a teoriei: **reference frames** („sisteme de referință”). O coloană nu înregistrează doar „neted, curbat”. Înregistrează „neted și curbat *în acest loc de pe obiect*”. Pe măsură ce degetul se mișcă, coloana urmărește unde se află acum degetul pe cană și stochează ce simte acolo. Cana devine o mică hartă: buza aici sus, toarta acolo, fundul plat jos, fiecare trăsătură prinsă de o locație în sistemul de coordonate propriu al cănii. În teorie, coloana știe unde e degetul din mișcarea însăși: creierul știe ce comandă motorie tocmai a trimis, așa că poate actualiza poziția degetului pe cană cum își actualizează o navigatoare fără GPS poziția din viteză și direcție. Asta se crede că fac celulele grilă, descrise mai jos, pentru întregul corp într-o cameră.

Fiecare coloană ține un model întreg; împreună votează



Dacă ai lucrat vreodată cu un game engine sau cu un scene graph 3D, cunoști deja structura aceasta. Fiecare obiect are propriul sistem de coordonate local, iar părțile lui sunt poziționate față de el: toarta cănii e la (+4, 0, 3) în sistemul cănii, oriunde ar sta cana în cameră și oricum ar fi rotită. Mută cana și toarta se mută cu ea, pentru că se stochează relația, nu poziția absolută. Un dicționar indexat după locație, cu trăsături ca valori: acesta e modelul unei coloane despre un obiect.

Ideea celulelor care urmăresc locația nu e speculativă. În 1971, John O’Keefe a găsit **place cells** („celule de loc”) în hipocampul șobolanului, neuroni care se activează când animalul e într-un anumit punct al unei camere. În 2005, May-Britt și Edvard Moser au găsit în apropiere **grid cells** („celule grilă”), neuroni care se activează în multe puncte aranjate într-o rețea triunghiulară izbitor de regulată, ca un sistem de coordonate așternut peste podea. Cei trei au împărțit Premiul Nobel în 2014. Teoria lui Hawkins propune că neocortexul refolosește o versiune a acestei mașinării de navigare în fiecare coloană, nu ca să urmărească unde ești *tu* într-o cameră, ci unde e degetul tău pe o cană sau unde sunt ochii tăi pe o față.

Acum al doilea experiment are sens. Cu un singur deget, o singură coloană trebuie să se miște și să adune câteva observații înainte ca harta ei să se așeze pe „cană”. Cu toată mâna, multe coloane primesc fiecare câte o observație deodată. Fiecare e încă nesigură de una singură: policele crede cană sau bol, arătătorul cană sau carafă, mijlociul cană sau doză. Dar vorbesc între ele, iar un singur obiect e compatibil cu toate observațiile lor laolaltă. **Votează**, iar votul se încheie într-o clipă. Ai văzut deja principiul, în mulțimea de experți pe jumătate orbi a pădurii aleatoare; iar inginerii îl cunosc din sistemele distribuite, unde replicile care țin fiecare starea completă ajung la o decizie prin quorum. Teoria Thousand Brains sugerează că creierul face recunoașterea prin ceva asemănător unui consens între mii de modele independente, fiecare cu imaginea completă și cu o vedere diferită, parțială.

Teoria face o afirmație în plus, mai îndrăzneață. Dacă coloanele sunt la fel peste tot și dacă cele din zonele senzoriale organizează cunoștințele în sisteme de referință, atunci și coloanele din zonele pentru limbaj, matematică și planificare ar putea face la fel. Gândirea, din acest punct de vedere, e un fel de mișcare prin hărți de concepte, așa cum atingerea e mișcare pe harta unui obiect. Sună abstract, dar există dovezi sugestive. Într-un studiu din 2016, cercetătorii au antrenat oameni să se gândească la păsări inventate care difereau doar prin lungimea gâtului și a picioarelor. Când participanții „se mișcau" mental de la o pasăre la alta, zone ale creierului care poartă de obicei semnale de celule grilă pentru spațiul fizic au arătat un tipar în grilă pentru acest spațiu abstract al formelor de păsări.

De ce i-ar păsa unui inginer de o teorie despre cortex? Pentru că scoate în evidență, foarte clar, ce le lipsește rețelelor de azi. Un CNN recunoaște o pisică în mare parte dintr-o pungă de trăsături, motiv pentru care poate fi păcălit de o textură de piele de elefant pe un contur de pisică. Nu are o hartă explicită a poziției părților unele față de altele. Rețelele de capsule (capsule networks) ale lui Geoffrey Hinton, propuse în 2017, au fost o încercare de a le da rețelelor exact astfel de poziții relative. Modelele de limbaj, la rândul lor, învață fără să se miște, să atingă sau să acționeze deloc în lume. Iar creierul învață continuu, un obiect nou din câteva atingeri, fără să le uite pe cele vechi, proprietatea a cărei absență din rețele am întâlnit-o ca catastrophic forgetting. Teoria se sprijină și pe o altă trăsătură cunoscută: în orice moment, doar un mic procent din neuronii corticali sunt activi, dând tipare lungi și rare (sparse) cu aceeași enormă abundență de loc pentru sensuri distincte care a făcut hipervectorii să funcționeze.

E important să privești toate acestea cu rezerva cuvenită. Teoria Thousand Brains e o propunere coerentă, nu știință stabilită. Mulți cercetători în neuroștiințe sunt sceptici față de părți din ea, mai ales față de afirmația că fiecare coloană învață modele complete de obiecte, iar dovezile pentru sisteme de referință de tip grilă în tot neocortexul sunt încă incomplete. Ce o face valoroasă, chiar dacă părți din ea se vor dovedi greșite, e că e destul de precisă ca să fie testată și destul de inginerească încât să poată fi construită. Compania lui Hawkins face exact asta, încercând să transforme teoria în software.

Dacă coloanele își construiesc modelele prezicând ce vor simți în locația următoare, atunci predicția trebuie să facă o mare parte din munca din creier. O întreagă școală a neuroștiinței susține acum că predicția nu e o funcție printre multe, ci treaba fundamentală a creierului. Iar asta ne aduce, neașteptat, foarte aproape de ideea din inima oricărui model de limbaj.

6.2 Mașina de predicții

Stai în bucătărie târziu în noapte și citești. Dintr-odată camera pare altfel și îți ia o secundă să înțelegi de ce. Frigiderul s-a oprit din zumzet.

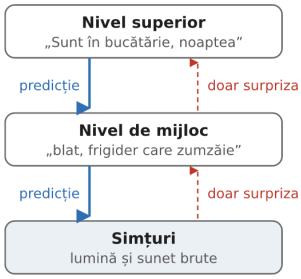
Observă ce tocmai s-a întâmplat. În ultima oră, zumzetul îți intra continuu în urechi și n-ai auzit nimic. În clipa în care a dispărut, ai observat. Creierul tău n-a raportat sunetul; a raportat *schimbarea*. Sau încearcă asta: coborând o scară întunecată, pui piciorul pentru încă o treaptă și treapta nu e acolo. Zguduitura care îți străbate tot corpul e disproporționată față de cei câțiva centimetri cu care ai căzut. Ce te-a zguduit n-a fost podeaua. A fost eșecul unei predicții pe care nu știai că o faci.

O școală mare și în creștere a neuroștiinței ia aceste momente de zi cu zi drept cheia întregului creier. Afirmația ei centrală, numită de obicei **predictive processing** („procesare predictivă") sau **predictive coding** („codare predictivă"), e că creierul nu e în principal un dispozitiv care primește semnale din lume. E un dispozitiv care le *prezice*. Zonele superioare trimit constant predicții în jos, spre simțuri: dat

fiind ce cred eu că se întâmplă, iată ce ar trebui să vezi și să auzi în continuare. Simțurile compară acele predicții cu ce sosește de fapt și doar diferența, **prediction error** („eroarea de predicție”), urcă înapoi. Când predicția e bună, trebuie trimis foarte puțin. Când se întâmplă ceva surprinzător, eroarea urcă prin niveluri până când un nivel își revizuiește povestea. Erorile sunt și lecțiile: fiecare surpriză împinge puțin povestea zonelor superioare, ca predicția următoare să fie mai aproape, cam cum loss-ul împinge un model la antrenare, iar după o viață de surprize predicțiile devin destul de bune încât cea mai mare parte a zilei să fie tăcere.

Ideea e mai veche decât calculatoarele. În anii 1860, fizicianul Hermann von Helmholtz descria percepția ca „inferență inconștientă”, creierul ghicind cauzele senzațiilor lui. În 1999, Rajesh Rao și Dana Ballard au arătat că o rețea construită pe acest principiu reproducea unele proprietăți ciudate ale neuronilor reali din cortexul vizual. Cercetătorul în neuroștiințe Karl Friston a generalizat-o într-o teorie matematică ambițioasă, iar autori din filosofie și din neuroștiință precum Andy Clark (*Surfing Uncertainty*), Anil Seth (*Being You*) și Lisa Feldman Barrett (*How Emotions Are Made*) au petrecut ultimul deceniu explorând ce ar însemna asta pentru felul în care vedem, simțim și chiar ne trăim pe noi înșine.

Predicțiile coboară; în sus urcă doar surprizele



Același truc într-un fișier video



O imagine completă, apoi doar ce diferă de predicția „la fel ca înainte”.

Un cadru gol nu costă aproape nimic: nu s-a întâmplat nimic surprinzător.

Când frigiderul se oprește, predicția „zumzet” eșuează, iar liniștea urcă brusc până sus.

Inginerii folosesc exact acest truc de decenii, în fiecare video pe care l-ai văzut vreodată în streaming. Un fișier video nu stochează fiecare cadru ca imagine completă. Stochează un cadru complet, apoi, pentru cadrele următoare, doar ce diferă de o predicție, de obicei „la fel ca înainte, poate puțin deplasat”. O scenă cu cineva care vorbește în fața unui perete nemișcat nu costă aproape nimic după primul cadru, pentru că aproape nimic nu e surprinzător. O tăietură spre o scenă nouă e scumpă, pentru că totul e surprinzător. Creierul tău, potrivit procesării predictive, tratează lumea la fel: trimite așteptarea în jos, trimite doar surpriza în sus. E un design extraordinar de eficient pentru un sistem care funcționează cu douăzeci de wați. Comparația cu codecul se rupe într-un loc revelator. Predicțiile unui codec sunt reguli fixe, iar el ascultă mereu de cadrul care sosește de fapt; creierul își învață predicțiile, iar când sunt destul de puternice, poate trece peste simțuri, cum arată paragraful următor.

Viziunea aceasta explică unele lucruri altfel greu de înțeles. Nu te poți gâdila singur, iar un studiu din 1998 al lui Sarah-Jayne Blakemore, Daniel Wolpert și Chris Frith a arătat de ce: când îți miști propria mână, creierul prezice atingerea precis, deci nu există nicio eroare de predicție de simțit, iar senzația e atenuată. Când o face altcineva, atingerea e imprevizibilă și gâdilă. Și iluziile optice devin mai ușor de înțeles. În iluzia măștii scobite, interiorul unei măști, care e concav, arată convingător ca o față normală,

bombată spre tine. Predicția creierului că fețele sunt bombate în afară e atât de puternică încât trece peste dovezile de la ochi. Anil Seth o spune provocator: ce trăiești ca realitate e un fel de „halucinație controlată", cea mai bună ghicire a creierului, corectată constant de simțuri.

Dacă asta îți amintește de ceva, așa ar trebui. Un model de limbaj e și el o mașină de predicții. Antrenarea lui de bază are o singură sarcină, prezicerea următoarei bucăți de text, iar semnalul lui de învățare e exact o eroare de predicție: câtă probabilitate n-a reușit să dea lucrului care a urmat de fapt. Paralela e reală și e unul dintre motivele pentru care domeniul găsește procesarea predictivă atât de interesantă. Diferențele sunt la fel de instructive. Creierul prezice ca să acționeze și să-și țină corpul în viață, clipă de clipă, și nu se oprește niciodată din învățat. Un model de limbaj prezice text, învață mai ales într-o singură antrenare uriașă și apoi e înghețat. Prezice; nu trăiește cu consecințele predicțiilor lui.

Cum învață un creier, dacă nu prin backpropagation? E una dintre marile întrebări deschise ale neuroștiinței, dar cel mai vechi răspuns e încă un bun punct de plecare. În 1949, psihologul Donald Hebb a propus că atunci când un neuron ajută în mod repetat un altul să se activeze, legătura dintre ei se întărește. Neurobiologa Carla Shatz a rezumat-o mai târziu într-o frază pe care au ținut-o minte generații de studenți: celulele care se activează împreună se leagă împreună. Lucrul important despre **Hebbian learning** („învățarea hebbiană") e că e *locală*. Fiecare conexiune se ajustează folosind doar informația disponibilă chiar acolo unde se află, activitatea celor doi neuroni pe care îi leagă. Backpropagation, în schimb, are nevoie ca eroarea măsurată chiar la capătul rețelei să fie dusă înapoi precis prin fiecare strat, inspectorul care parcurge toată linia din atelier. Neuronii reali nu par să aibă un astfel de inspector. Inginerii cunosc bine diferența: un sistem în care fiecare serviciu își reglează propriul cache după propriul trafic, față de un optimizator central care trebuie să vadă tot. Regulile locale sunt robuste și ieftine; cele globale sunt precise și scumpe. Găsirea unor reguli de învățare care să fie și locale, și puternice e un domeniu activ de cercetare de ambele părți.

Mai e un truc al creierului pe care machine learning l-a împrumutat deja și ține de somn. Creierul tău pare să aibă două sisteme de memorie cu viteze foarte diferite. **Hipocampul**, o structură încolăcită adânc în interior, învață repede: o singură experiență, cum ar fi unde ai parcat în dimineața asta, poate fi stocată dintr-o dată. Neocortexul învață încet, extrăgând treptat cunoștințe generale din multe experiențe. În 1995, James McClelland, Bruce McNaughton și Randall O'Reilly au argumentat că această împărțire nu e întâmplătoare. Un sistem care ar învăța totul repede într-un singur loc ar suprascrie cunoștințele vechi la fiecare eveniment nou, exact catastrophic forgetting-ul pe care l-am întâlnit mai devreme. Așa că creierul înregistrează repede experiențele noi în hipocamp și apoi, mai ales în somn, le **reia** (replay) cortexului iar și iar, amestecate cu amintiri mai vechi, ca acesta să le poată absorbi fără să șteargă ce știe deja. Înregistrările la șobolani din anii '90 au arătat exact asta: secvența de celule de loc care se activase cât șobolanul alerga printr-un labirint se activa din nou, în ordine și mai repede, în timp ce dormea. Machine learning folosește același truc: când unui model i se predă ceva nou, amestecarea unei părți din exemplele vechi în noile date de antrenare, numită rehearsal („repetiție"), e una dintre cele mai simple și mai eficiente apărări împotriva catastrophic forgetting-ului și e folosită des când modelele de limbaj trec prin fine-tuning.

Dacă ai lucrat cu baze de date, cunoști designul. Scrierile merg mai întâi într-un log rapid, append-only (la care doar se adaugă); mai târziu, un proces de fundal le compactează în stocarea principală, atent organizată. Multe motoare moderne de stocare sunt construite exact așa. Iar agentul de reinforcement learning al DeepMind care a învățat jocuri Atari a făcut ceva asemănător: și-a stocat experiențele trecute într-un buffer și le-a reluat în ordine amestecată în timp ce învăța, o idee pe care autorii lui au legat-o explicit de hipocamp.

Pune cele două subcapitole ale acestui capitol unul lângă altul și se conturează o imagine a creierului foarte diferită de un teanc de grile antrenat o singură dată prin backpropagation. E o mașină care prezice înainte să perceapă, care se mișcă ca să învețe, care ține mii de modele mici și le lasă să voteze, care înregistrează repede și consolidează încet și care face toate acestea cu puterea unui bec slab. Niciuna dintre teorii nu e finală, fiecare are critici serioși și nu sunt de acord una cu alta peste tot. Dar machine learning a mai împrumutat de la această mașină, convoluțiile de la cortexul vizual și replay-ul de la hipocamp, și există toate motivele să credem că va împrumuta din nou: învățare care nu se oprește și nu suprascrie niciodată, hărți ale obiectelor și ale ideilor, surpriza ca monedă a învățării și o eficiență pe care centrele de date de azi n-o pot decât invidia.

O imagine a străbătut ambele subcapitole: harta. Celule grilă care cartografiază o cameră, coloane care cartografiază o cană de cafea, poate cortexul care cartografiază idei. Când ne întoarcem la mașini, primul lucru pe care îl vom vedea e că și ele au învățat să construiască hărți, nu ale camerelor, ci ale sensurilor.

Mai departe. Dacă te-ai întrebat cum arată de aproape jocul de predicție al unui model de limbaj, subcapitolul 7.3 te lasă să urmărești un model învățându-l. Dacă pot și mașinile să învețe acționând în lume, cum face creierul, se discută în subcapitolul 7.6. De ce fine-tuning-ul riscă o uitare catastrofală se vede în subcapitolul 9.2, iar cum se apără echipele de ea când actualizează un model, inclusiv prin rehearsal, în subcapitolul 11.5.

Capitolul 7 — Limbaj și sens

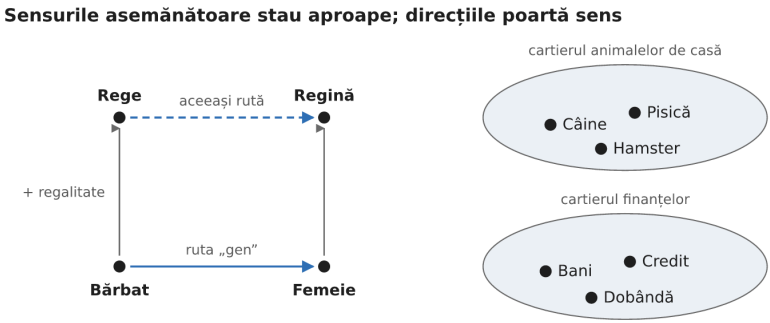
7.1 O hartă în care sensul are o adresă

Deschide orice selector de culori și scrie `#1E90FF`. Obții un albastru luminos, prietenos. Schimbă-l în `#1E80EF` și obții un albastru aproape identic, cu o nuanță mai închis. Schimbă-l în `#FFD700` și obții auriu. Fără să te gândești, ai folosit una dintre cele mai puternice idei din machine learning-ul modern.

Un cod de culoare e o adresă. Cele trei coordonate ale lui spun cât roșu, verde și albastru să amesteci, iar lucrul remarcabil e că spațiul pe care îl definesc are *sens*. Culorile cu adrese apropiate seamănă între ele. Și direcțiile înseamnă ceva: mută orice culoare în direcția „mai mult roșu” și se încălzește, oricare ar fi culoarea de la care ai pornit. Nimeni nu trebuie să-ți explice că bleumarinul și albastrul regal sunt rude. Adresele lor o spun deja.

Acum imaginează-ți că faci același lucru pentru cuvinte. Dă-i fiecărui cuvânt din limbă o adresă, nu cu trei coordonate, ci cu sute, și aranjează adresele astfel încât cuvintele cu sensuri asemănătoare să locuiască aproape. Într-un oraș al cuvintelor bine cartografiat, „câine” stă lângă „pisică” și „hamster”, „bani” stă pe aceeași stradă cu „credit” și „dobândă”, iar „rege” stă ușă în ușă cu „regină”. O astfel de adresă se numește **embedding** („încorporare”: cuvântul e încorporat ca punct într-un spațiu de numere): o listă de numere, lungă de la câteva sute la câteva mii, care așază un cuvânt, o propoziție, o imagine sau un produs pe o hartă a sensului. Lungimea o alege proiectantul, ca hiperparametru: mai multe coordonate lasă loc pentru distincții mai fine, dar costă mai multă memorie și o căutare mai lentă.

Momentul în care ideea a captat imaginația publicului a venit în 2013, când o echipă de la Google condusă de Tomas Mikolov a publicat o metodă numită word2vec. Oamenii au început să facă aritmetică cu adresele ei și au descoperit ceva straniu. Ia adresa lui „rege”, scade „bărbat”, adaugă „femeie” și uită-te ce locuiește cel mai aproape de rezultat, lăsând deoparte cuvintele de la care ai pornit. Foarte des, e „regină”. Drumul de la bărbat la femeie pe hartă s-a dovedit a fi cam același cu drumul de la rege la regină. Direcțiile din spațiul cuvintelor, ca direcțiile din spațiul culorilor, purtau sens.



O hartă 2-D pentru ochi; embeddings reale au sute sau mii de coordonate.

Cine a decis aceste adrese? Nimeni. Au fost învățate, iar principiul din spatele învățării e mai vechi decât calculatoarele. În 1957, lingvistul J. R. Firth scria că un cuvânt se cunoaște după compania pe care o ține. Un model care citește miliarde de propoziții observă că „rege” și „regină” apar în companii

asemănătoare: „… a domnit”, „l-au încoronat pe …”, „… și curtea”. De fiecare dată când vede două cuvinte în companii asemănătoare, le apropie puțin adresele. După destulă lectură, harta se desenează singură. În word2vec, apropierea e un mic joc de ghicit: pornind de la adresa unui cuvânt, ghicești ce cuvinte apar lângă el într-o propoziție. Fiecare ghicire greșită trimite un gradient care mută puțin adresele, iar cuvintele care țin aceeași companie sunt împinse în aceleași direcții până ajung vecine.

Pentru un inginer, câștigul e imediat, pentru că transformă „găsește ceva asemănător” în „găsește ceva apropiat”. Scrie „funcție care reîncearcă apelurile HTTP eșuate” într-o căutare semantică prin cod. Căutarea simplă în text nu găsește nimic, pentru că funcția pe care o vrei se numește `with_backoff`, iar comentariul ei vorbește despre „întârziere exponențială la erori tranzitorii”. Dar calculează embedding-ul întrebării tale și al fiecărei funcții din cod, iar `with_backoff` stă chiar lângă întrebarea ta pe hartă. Cuvintele sunt diferite; sensul e apropiat. Același truc îi permite unui sistem de tichete să prindă un duplicat ale cărui cuvinte abia se suprapun cu ale originalului: grupează „Nu mă pot loga pe Safari”, „Butonul de sign-in e mort pe Mac” și „Autentificarea eșuează în browserul Apple” într-un singur cluster de duplicate și alimentează rafturile de „produse similare” din multe magazine online. Căutarea celor mai apropiați vecini și clustering-ul merg dintr-odată și pe text. Și propozițiile sau documentele întregi primesc o singură adresă: un model citește textul cu attention, mecanismul din subcapitolul următor, condensează adresele dependente de context pe care le-a construit pentru fiecare cuvânt într-o singură listă de numere, adesea făcând pur și simplu media lor, și e antrenat astfel încât o întrebare și pasajul care îi răspunde să ajungă aproape unul de altul.

Există o subtilitate în felul în care se măsoară „aproape”, iar bucătăria o explică cel mai bine. O rețetă de clătite pentru patru persoane și aceeași rețetă pentru patruzeci au cantități foarte diferite, dar proporții identice. Măsurate prin distanță brută, sunt departe una de alta. Măsurate prin *direcție*, prin raportul dintre ingrediente, sunt aceeași rețetă. Embeddings se compară de obicei în al doilea fel, cu **cosine similarity** („asemănarea cosinus”: se uită la unghiul dintre două adrese și ignoră lungimea lor). Așa, un document lung și o notiță scurtă despre același subiect sunt vecini, cum și trebuie.

E ușor să te lași purtat de entuziasm aici, deci două avertismente. Întâi, coordonatele nu au nume. Nu există o „axă a regalității” spre care să poți arăta; direcțiile cu sens sunt amestecuri ale multor coordonate deodată. Apoi, aritmetica cu regele și regina e reală, dar capricioasă. Merge minunat pentru exemple alese și inegal pentru multe altele. Tratează aritmetica asta ca pe o tendință a hărții, nu ca pe o lege a naturii. Și pentru că harta e învățată din ce scriu oamenii, le învață și prejudecățile: primele hărți de cuvinte așezau „nurse” („asistentă”) mai aproape de „she” („ea”) și „engineer” („inginer”) mai aproape de „he” („el”), reproducând fidel compania pe care o țin aceste cuvinte în textele obișnuite; iar o hartă fidelă nu e același lucru cu una dreaptă.

Și mai e o problemă, mai adâncă, pe care poate ai observat-o deja. Citește aceste două propoziții: „M-am odihnit pe o bancă în parc.” „Am depus banii la bancă.” Într-o hartă ca a lui word2vec, „bancă” are o singură adresă. Nu poate locui și în parc, și lângă bani în același timp, așa că ajunge stingheră undeva la mijloc, nu prea potrivită pentru niciunul dintre sensuri.

Ce vrem de fapt e o adresă care se mută în funcție de cuvintele din jur. Exact asta au construit cercetătorii începând din 2017, iar ca să vedem cum, trebuie să intrăm într-o cameră plină de detectivi.

7.2 Ședința detectivilor

Pe canalul de chat al echipei apare un mesaj: „iar a căzut.”

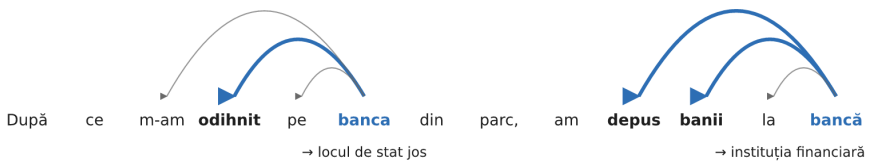
Știi imediat ce a căzut. Nu pentru că o spune mesajul, ci pentru că ochii tăi au sărit deja în sus pe canal, au trecut peste planurile de prânz și peste linkul la o prezentare de la o conferință și au aterizat pe mesajul de acum douăzeci de minute despre serviciul de plăți care dă timeout-uri. Înțelegerea ta a lui „iar a căzut” a fost asamblată din celelalte mesaje, fiecare cântărit după cât de relevant părea. Planurile de prânz n-au primit aproape nicio pondere. Mesajul despre plăți a primit aproape toată ponderea.

Această privire, repetată pentru fiecare cuvânt dintr-un text, e ideea din inima Transformer-ului. Se numește **attention** („atenție”), iar ca să vezi cum o face o mașină, imaginează-ți o cameră.

Transformă propoziția „După ce m-am odihnit pe banca din parc, am depus banii la bancă” într-o cameră cu câte un detectiv pentru fiecare cuvânt. Fiecare detectiv are de rezolvat o problemă, anume ce înseamnă el însuși aici, și fiecare poartă trei lucruri. Un **bilet cu o întrebare**, numit *query* („interogare”): „Am nevoie de cineva care să-mi spună ce fel de bancă sunt.” Un **ecuson**, numit *key* („cheie”), care anunță ce poate oferi: „Sunt o activitate de odihnă în aer liber” sau „Sunt o acțiune financiară”. Și un **dosar**, numit *value* („valoare”), cu informația pe care o va da oricui îl ascultă.

Ședința începe. Fiecare detectiv își citește întrebarea, se plimbă prin cameră și o compară cu ecusonul fiecăruia. Cu cât potrivirea e mai bună, cu atât îi acordă mai multă atenție acelui detectiv. Apoi fiecare detectiv își rescrie propria identitate ca un amestec al dosarelor detectivilor pe care i-a ascultat, mai ales de la cele mai bune potriviri, puțin de la rest. Prima „bancă” găsește o potrivire puternică în „odihnit” și iese din ședință ca *banca, locul de stat jos*. A doua găsește potriviri puternice în „depus” și „banii” și iese ca *banca, instituția*. Același cuvânt, două identități, decise de context, și decise din nou pentru fiecare propoziție.

Fiecare „bancă” ascultă de alte cuvinte și primește alt sens



Arc gros = atenție puternică, arc subțire = slabă. Fiecare cuvânt se uită doar înapoi (causal mask, ca la modelele GPT).
O schiță: într-un model real, ponderile sunt calculate, mai împrăștiate și împărțite între mai multe heads.

Dacă ești programator, iată momentul în care s-ar putea să se lege totul. Folosești dicționare în fiecare zi. Cauți o cheie; dacă se potrivește exact, primești valoarea ei; dacă nu, nu primești nimic. Attention e aceeași operație, în varianta „moale”. Întrebarea ta e comparată cu fiecare cheie din tabel și, în loc de o singură potrivire exactă, fiecare cheie primește un scor de potrivire. Scorurile devin procente care însumează o sută și primești înapoi un amestec al tuturor valorilor, ponderat cu acele procente: mai ales cea mai bună potrivire, puțin din a doua, o urmă din tot restul. Attention e o căutare în dicționar „cu aproximație”, făcută de fiecare cuvânt asupra fiecărui alt cuvânt, toate în același timp. Comparația însăși e aritmetică simplă: întrebarea și ecusonul sunt amândouă liste de numere, iar scorul de potrivire e produsul lor scalar, înmulțirea și adunarea din lista de bifat pentru apartamente, mare atunci când cele două liste arată în direcții asemănătoare; softmax-ul transformă apoi scorurile în procente.

Această ultimă parte, toate în același timp, e ce telefonul fără fir n-ar fi putut face niciodată. Nimeni nu șoptește. Fiecare cuvânt poate consulta direct orice alt cuvânt, oricât de departe ar sta, iar pentru că consultările sunt independente unele de altele, o placă video le poate rula pe toate în paralel.

Iată punctul subtil, ușor de înțeles greșit. Ce învață de fapt modelul? Nu scorurile de potrivire dintre anumite cuvinte; acelea sunt recalculate pentru fiecare propoziție nouă. Fiecare cuvânt pornește de la embedding-ul lui, iar trei transformări învățate fac din acel embedding o întrebare, un ecuson și un dosar. **Antrenarea îi învață pe detectivi cum să scrie, în general, întrebări bune, ecusoane bune și dosare utile.** De aceea un Transformer se descurcă cu propoziții pe care nu le-a văzut niciodată: regulile de scris notițe rămân valabile, chiar și într-o cameră nouă.

O singură ședință nu ajunge, așa că Transformer-ele țin mai multe în paralel. Gândește-te la code review: trei revieweri, fiecare concentrat pe altceva, securitate, performanță și lizibilitate, prind mai mult decât unul singur care încearcă să urmărească totul. În **multi-head attention** („atenție cu mai multe capete”), mai multe ședințe rulează una lângă alta pe aceeași propoziție, fiecare cu propriul stil de întrebări și ecusoane. Un head poate învăța să urmărească gramatica, cărui substantiv îi aparține un verb; altul, referințele, la ce se referă „el”; altul, subiectul. Un **Transformer block** complet e o astfel de ședință cu mai multe capete, urmată de o mică rețea neuronală pe care fiecare detectiv o folosește în particular ca să digeră ce a auzit. Modelele mari pun zeci de astfel de blocuri unul peste altul, așa că identitățile sunt rafinate iar și iar.

Două detalii completează imaginea. Întâi, ședința descrisă nu are simțul ordinii. „Câinele mușcă omul” și „omul mușcă câinele” aduc în cameră aceiași trei detectivi. Așa că embedding-ul de pornire al fiecărui cuvânt e combinat cu informație despre poziția lui în propoziție. Transformer-ul original adăuga pur și simplu fiecărui embedding un tipar fix de numere, altul pentru fiecare poziție, ca un număr de casă scris pe fiecare dosar; modelele moderne folosesc de obicei o schemă mai isteață, care vine mai târziu în carte. Apoi, modelele care generează text, cum e familia GPT (generative pre-trained transformer, „transformer generativ pre-antrenat”), adaugă o regulă: fiecare detectiv poate consulta doar detectivii din stânga lui. Când scrii o propoziție, nu poți consulta cuvinte pe care nu le-ai scris încă. Aceasta e **causal mask** („masca cauzală”: ascunde viitorul), iar exemplul nostru o respectă, pentru că „odihnit” vine înaintea primei „bănci”, iar „depus” și „banii” vin înaintea celei de-a doua. Când indiciul vine mai târziu, ca în „banca era udă de ploaie”, detectivul pentru „banca” nu poate lămuri singur sensul, dar „udă” și „ploaie” se pot uita înapoi la „banca” și pot duce mai departe sensul lămurit în propriile rânduri, iar de acolo se prezice cuvântul următor; modelele construite doar pentru citit, precum BERT, renunță la mască și lasă fiecare cuvânt să se uite în ambele direcții.

Fă un pas înapoi și compară asta cu o rețea multistrat obișnuită. Acea rețea aplică aceleași ponderi fixe fiecărei intrări. E funcționarul de la ghișeu care aplică același regulament oricărui dosar care-i trece prin față. Un Transformer calculează o atenție proaspătă pentru fiecare intrare. E managerul care citește tot dosarul și decide, întrebare cu întrebare, ce documente contează. Managerul are însă un preț. Pentru că fiecare cuvânt îl consultă pe fiecare, dublarea lungimii unui text cam cvadruplează munca atenției. De aceea orice model are o lungime maximă de context, iar mărirea ei a fost una dintre marile bătălii de inginerie ale domeniului.

Un avertisment înainte să ieșim din cameră. Detectivii nu înțeleg nimic și nu intenționează nimic. „A asculta” e o medie ponderată a unor liste de numere, iar atenția reală e rareori atât de ordonată ca în desen: e de obicei mai împrăștiată, iar head-uri diferite se uită la lucruri foarte diferite. Tiparele de atenție sunt un indiciu despre ce face un model, niciodată o explicație completă.

Articolul din 2017 a introdus Transformer-ul ca pe un mod mai bun de a traduce între limbi. Puțini au ghicit ce avea să se întâmple când cineva a luat această arhitectură, a făcut-o uriașă și a antrenat-o pe cel mai simplu joc imaginabil: ghicește cuvântul următor.

7.3 Jocul ghicitului cuvântului următor

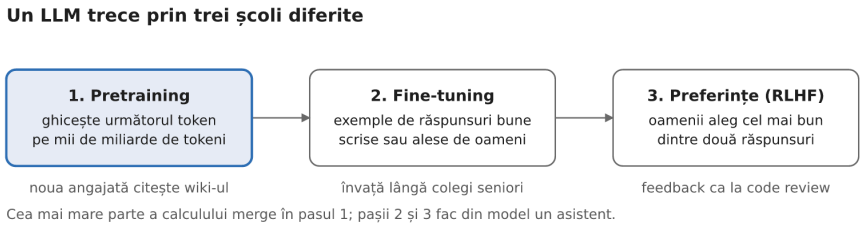
Încearcă asta pe telefon. Deschide un mesaj, scrie „Eu cred că” și apoi apasă întruna sugestia din mijloc pe care ți-o oferă tastatura. În câteva apăsări vei obține ceva de genul „Eu cred că este cel mai bun mod de a fi cel mai bun mod de a fi”, învârtindu-se la nesfârșit. Tastatura ta e un model de limbaj minuscul. A învățat ce cuvânt tinde să urmeze după care și, fără nicio idee mai largă despre unde merge propoziția, se învârte în cerc.

Acum ține minte propoziția aceea caraghioasă, pentru că iată cel mai surprinzător fapt din această carte. Un **large language model** („model mare de limbaj”), pe scurt LLM, genul care scrie eseuri, explică clauze juridice și îți completează codul, joacă exact același joc. În prima și cea mai mare fază a antrenării, învață să ghicească următoarea bucată de text, nimic mai mult. Diferențele sunt de scară și de arhitectură: e de multe mii de ori mai mare decât modelul tastaturii tale, e un Transformer ai cărui detectivi pot consulta tot textul de până atunci și a citit o parte substanțială din internetul public, foarte multe cărți și o cantitate enormă de cod. Undeva de-a lungul acestei scări, nonsensul care se învârte în cerc devine ceva care arată foarte mult a înțelegere. De ce funcționează exact atât de bine e încă o întrebare de cercetare deschisă.

Întâi, un detaliu despre ce anume se ghicește. Modelele nu văd cuvinte; văd **tokens** („jetoane”), bucăți de cuvinte. Un cuvânt comun e de obicei un singur token, iar unul mai rar e tăiat în bucăți, ceva de genul „ne” + „crez” + „ut”. Pentru textul în engleză, un token are în medie cam trei-patru caractere. Limbile care apar mai rar în datele de antrenare tind să fie tăiate în bucăți mai multe și mai mici, așa că același mesaj costă mai mulți tokeni în română sau în greacă decât în engleză, ceea ce contează, pentru că API-urile facturează și limitează după tokeni. Tăierea se decide înainte ca modelul să fie antrenat, de un simplu program statistic: pornește de la caractere individuale și lipește iar și iar perechea care apare cel mai des într-un eșantion mare de text, „t” cu „h” în „th”, apoi „th” cu „e” în „the”, până când vocabularul ajunge la mărimea aleasă, așa că tot ce era des întâlnit în acel eșantion primește tokeni ieftini, cât un cuvânt întreg, iar restul e tăiat în bucăți.

Uită-te acum cum învață jocul detectivii din subcapitolul trecut. Șeful aduce o propoziție cu ultimul cuvânt acoperit: „Am mers la bancă să scot niște ___”. În prima zi, detectivii sunt noi-nouți, iar biletele și ecusoanele lor sunt aleatorii. „Bancă” îl ascultă mai ales pe „mers”, iar cea mai bună ghicire a echipei e „porumbei”. Șeful descoperă cuvântul: „bani”. Loss-ul măsoară cât de puțină probabilitate a dat echipa răspunsului corect, iar backpropagation îi dă fiecărui detectiv partea lui de vină. „Bancă” își ajustează, foarte puțin, felul în care își scrie întrebările, ca data viitoare să caute ecusoane de genul „acțiune financiară”. „Să scot” își ajustează ecusonul ca să fie mai ușor de găsit. Apoi vine următoarea propoziție, și următoarea, miliarde, despre orice subiect despre care scriu oamenii. Ca să prezici următorul cuvânt dintr-o rețetă trebuie să știi ceva despre gătit; dintr-un manual de fizică, ceva despre fizică; dintr-un fișier Python, ceva despre Python. Jocul e simplu. Ca să-l câștigi, trebuie să absorbi o mare parte din lume.

Rezultatul acestei faze, numită **pretraining** („pre-antrenare”: antrenarea generală dinaintea celei specializate), e uimitor de bine informat și ciudat de nefolositor. Imaginează-ți o ingineră nouă care, în prima lună, citește tot wiki-ul companiei, fiecare repository și fiecare tichet deschis vreodată. Știe enorm de multe. Dar pune-i o întrebare și s-ar putea să-ți răspundă cu încă trei întrebări, pentru că pe wiki întrebările sunt adesea urmate de alte întrebări. A învățat să continue documente, nu să ajute oameni.

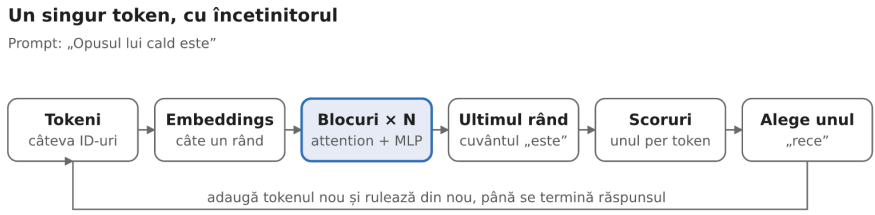


Așa că noua angajată mai trece prin două școli. În a doua, fine-tuning, învață lângă colegi seniori: mii de exemple de cereri urmate de un răspuns bun și util, până când forma „cineva întreabă, eu ajut” devine a doua natură. În a treia primește ceva asemănător unui code review. Oamenilor li se arată două dintre răspunsurile ei la aceeași cerere și îl aleg pe cel mai bun, iar ea învață treptat ce preferă evaluatorii: răspunsuri mai clare, mai corecte, mai atente. Această ultimă etapă se numește de obicei **RLHF** (reinforcement learning from human feedback, „învățare prin recompensă din feedback uman”). Spre deosebire de o angajată adevărată, modelul nu-și amintește nimic din aceste școli, iar ultima are o capcană: evaluatorii recompensează răspunsurile care li se par corecte, ceea ce nu e chiar același lucru cu răspunsurile corecte.

Acum avem toate piesele de care e nevoie ca să urmărim un model de limbaj lucrând, cu încetinitorul, din clipa în care îți trimiți mesajul. Tot ce am construit până acum se adună aici, așa că ia-o cadru cu cadru.

Scrii „Opusul lui cald este”. Primul lucru care se întâmplă n-are nicio legătură cu inteligența: un **tokenizer**, un simplu program de căutare, taie textul în bucăți și înlocuiește fiecare bucată cu numărul ei dintr-un vocabular fix de, de obicei, zeci de mii până la vreo două sute de mii de intrări. Propoziția ta e acum o listă scurtă de numere întregi.

Al doilea cadru e stratul de embedding, portarul. Fiecare număr e folosit ca număr de rând într-un tabel mare și rândul se întoarce: un vector de câteva mii de numere. Câte un rând pentru fiecare token. Pune-le unul peste altul și ai un tensor cu forma (tokeni, lățime), un spreadsheet cu un rând per token. Se amestecă și informație despre poziția fiecărui token, ca modelul să știe care a venit primul.



Forma rămâne (tokeni × lățime) prin fiecare bloc; doar ultimul rând devine scoruri.

Al treilea cadru e cel lung. Tabelul trece prin blocurile Transformer, unul după altul, zeci la un model mare. În fiecare bloc are loc ședința detectivilor: fiecare rând consultă rândurile dinaintea lui și se rescrie ca amestec al celor auzite. Apoi fiecare rând trece, separat, printr-o mică rețea densă, partea din bloc în care, cred cercetătorii, stă o mare parte din cunoștințele factuale ale modelului. Legăturile reziduale duc semnalul original pe ocolul fiecărui pas, normalizarea ține numerele în interval, iar prin toate acestea forma nu se schimbă niciodată: intră atâtea rânduri, ies tot atâtea, fiecare puțin mai bogat decât înainte. Până la ultimul bloc, rândul pentru „este” nu mai înseamnă doar „este”. A absorbit „opusul”, „lui” și „cald” și codifică acum ceva de genul „urmează un cuvânt despre temperatură, inversul lui cald”.

Al patrulea cadru se uită doar la acel ultim rând. E înmulțit cu încă o matrice mare, cu câte o coloană pentru fiecare intrare din vocabular, producând un scor pentru fiecare token următor posibil, cam o sută de mii de scoruri. Softmax le transformă în probabilități. „Rece” ia aproape tot; „răcoros” și „înghețat” iau firimituri.

Al cincilea cadru alege un token după acele probabilități, iar al șaselea e bucla: tokenul ales e adăugat la intrare și tot forward pass-ul rulează din nou ca să producă tokenul de după el. Un forward pass per token: de aici vine factura. Ca să nu reia ședința de la zero de fiecare dată, modelul păstrează în memorie ecusoanele și dosarele fiecărui token anterior, într-un depozit numit **KV cache** (key–value cache, „cache de chei și valori”), și doar detectivul celui mai nou token trebuie să se plimbe prin cameră. Acest cache e o mare parte din motivul pentru care conversațiile lungi au nevoie de atâta memorie pe placa video. Bucla se oprește când modelul alege un token special de „sfârșit”, pe care a învățat să-l prezică, din milioane de exemple care se încheiau cu el, atunci când un text e complet; aplicația fixează și o limită strictă, pentru cazul în care nu-l alege niciodată.

Asta e toată mașina. Niciun pas din ea nu e misterios luat separat: o căutare, un teanc de înmulțiri de matrice cu câteva îndoituri, un softmax, o aruncare de zar, apoi totul de la capăt. Ce e misterios, și ce încearcă încă cercetătorii să înțeleagă, e cum încap atâtea cunoștințe și atâta raționament aparent în numerele folosite de acele înmulțiri.

Un buton stă în al cincilea cadru, alegerea: cât de aventuroasă e ea depinde de o setare numită **temperature** („temperatură”, cu sensul din fizică: mai cald înseamnă mai multă agitație). Aproape de zero, modelul alege aproape mereu tokenul cel mai probabil: previzibil, repetabil, uneori plictisitor. Asta vrei când extragi câmpuri din facturi în JSON (un format standard pentru date structurate). Dă temperatura mai sus și tokenii mai puțin probabili își primesc șansa: rezultatul devine mai variat și mai inventiv, dar și mai predispus la erori. Asta ai putea vrea când cauți nume de produse. E diferența dintre un muzician care cântă după partitură și unul care improvizează. Pe dedesubt, butonul e simplu:

scorurile sunt împărțite la temperatură înainte ca softmax-ul să le transforme în probabilități, așa că o temperatură mică lărgește diferențele și favoritul ia aproape tot, iar una mare le strânge și candidații din urmă primesc o șansă reală.

Mai e un lucru de înțeles despre felul în care trăiesc aceste modele. Tot ce știe modelul despre conversația ta stă în **context window**-ul lui („fereastra de context”), iar cea mai bună imagine pentru ea e o tablă dintr-o sală de ședințe. Modelul citește toată tabla înainte de fiecare cuvânt pe care îl scrie. Dar nu învață din ședință, iar când ședința se termină, tabla e ștearsă. Mâine nu-și va aminti conversația de azi decât dacă o scrie cineva din nou pe tablă. Documentele lungi, conversațiile lungi și bazele de cod mari se luptă toate pentru spațiu pe acea tablă.

Și acum partea incomodă. Lucrezi cu un asistent și el scrie o linie de cod ordonată care apelează `client.get_json_safe()`. Arată exact ca o metodă pe care ar avea-o biblioteca. O rulezi. Metoda nu există și n-a existat niciodată.

Asta se numește **hallucination** („halucinație”), iar numele o face să sune ca o defecțiune. Nu e. Amintește-ți pentru ce a fost antrenat modelul: să producă cea mai plauzibilă continuare a textului. Plauzibilul și adevărul coincid de obicei, și de aceea aceste modele sunt atât de utile. Când se despart, când modelul nu știe ceva, el de multe ori nu tace. Produce lucrul care sună cel mai plauzibil, fluent și sigur pe el, pentru că acesta e, înainte de toate, jocul pentru care a fost antrenat.

Așadar, ce faci cu o colegă strălucită care a citit tot, scrie minunat și rareori spune „nu știu”? Același lucru pe care l-ai face cu o angajată nouă strălucită. Nu-i mai ceri să răspundă din memorie și îi dai documentele.

7.4 Examenul cu cartea deschisă

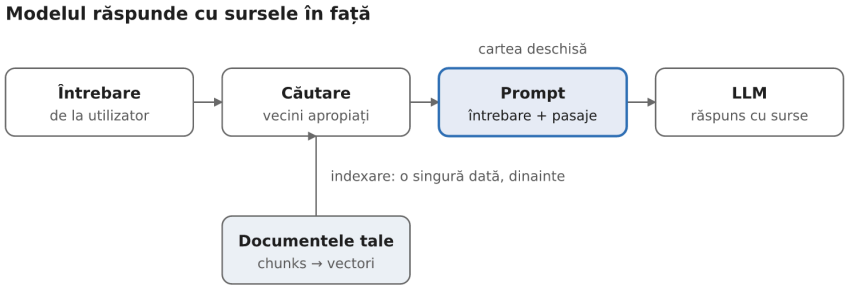
Imaginează-ți o companie care lansează un asistent intern, un model de limbaj legat la unealta ei de chat. La unsprezece noaptea, un inginer de gardă îl întreabă: „Care e procedura noastră de escaladare dacă API-ul de plăți cade în afara orelor de program?”

Răspunsul vine instantaneu, bine structurat și complet sigur pe el. Trezește-l pe al doilea om de gardă după cincisprezece minute. Anunță-l pe șeful departamentului de inginerie după treizeci. Deschide un canal de incident numit cu data zilei. Sună exact ca o procedură. Și e inventat de la cap la coadă. Modelul n-a văzut niciodată runbook-urile („manualele de operațiuni”) acestei companii; a văzut procedurile a mii de alte companii și a produs cel mai plauzibil amestec al lor. Procedura reală spune să suni mai întâi furnizorul de plăți.

Nimic nu e stricat la model. E angajata nouă și strălucită din subcapitolul trecut, rugată să răspundă din memorie despre un document pe care nu l-a citit niciodată. Remediul e cel pe care l-ar alege orice manager bun: las-o să răspundă cu cartea deschisă și dă-i un bibliotecar rapid care să-i aducă paginile potrivite.

Acest aranjament se numește **retrieval-augmented generation** („generare îmbogățită prin regăsire”), pe scurt RAG. Înainte ca modelul să răspundă, un pas de căutare găsește pasajele cele mai relevante pentru întrebare și le lipește în prompt, împreună cu instrucțiunea de a răspunde din acele pasaje și de a spune de unde vine fiecare afirmație. Scrisul rămâne treaba modelului. Memoria devine treaba căutării.

Să lipești toate documentele în prompt și să sari peste căutare merge pentru o mână de documente; pentru un wiki întreg n-ar încăpea pe tablă, ar costa bani pentru fiecare token la fiecare întrebare, iar modelele citesc o tablă înghesuită mai puțin atent decât una scurtă.



Rulează din nou întrebarea de noapte, cu RAG pus la punct. Căutarea găsește pagina cu procedura de gardă și runbook-ul de plăți și pune trei pasaje relevante pe tabla modelului. Acum răspunsul spune să suni mai întâi furnizorul, citează numărul de suport al furnizorului din runbook și dă linkuri spre ambele pagini. Inginerul poate da click și verifica. Diferența nu e inteligența; e accesul. Modelul preferă pasajele în locul memoriei lui doar pentru că i se cere asta și pentru că asistenții trec prin fine-tuning pe multe exemple de răspunsuri date din textul primit; de obicei ascultă, dar când un pasaj contrazice ceva ce modelul „știe” foarte ferm, poate aluneca înapoi spre memorie, încă un motiv să verifici citatele.

Ca să găsească pasaje, documentele trebuie pregătite dinainte, iar aici stă cea mai mare parte a meșteșugului. Fiecare document e tăiat în **chunks** („bucăți”) de câteva sute de tokeni, ca și cum ai tăia un manual în cartonașe. Fiecare cartonaș primește un embedding, iar cartonașele intră într-o bază de date care poate găsi rapid cei mai apropiați vecini ai oricărei întrebări. Mărimea cartonașelor se dovedește foarte importantă. Taie prea mărunțit și o rețetă ajunge cu ingredientele pe un cartonaș și pașii pe altul, așa că niciun cartonaș nu răspunde la „cum o fac?”. Taie prea mare și fiecare cartonaș vorbește despre cinci lucruri diferite, așa că niciunul nu se potrivește bine cu nimic.

Apoi sunt întrebările pe care embeddings le tratează prost, iar inginerii dau repede de ele. Întreabă „ce înseamnă eroarea E1047?” și o căutare bazată doar pe sens poate întoarce un pasaj despre E1074, pentru că pentru un embedding două coduri care arată la fel stau foarte aproape. Identificatorii exacți, numele de funcții și numerele de tichete sunt specialitatea căutării clasice după cuvinte-cheie. Așa că sistemele practice folosesc **hybrid search** („căutare hibridă”), combinând rezultatele după cuvinte-cheie cu cele după vectori, și adaugă adesea un al doilea model, mai lent, care **re-ranks** („reordonează”) candidații: bibliotecarul aduce cincizeci de cărți, iar un expert le citește și le păstrează pe cele mai bune cinci.

Beneficiile sunt concrete, și de aceea atâtea companii construiesc astfel de sisteme. Cunoștințele pot fi actualizate actualizând documentele, fără reantrenare. Răspunsurile pot cita surse, ca oamenii să le poată verifica. Datele private rămân în propriul tău depozit, aduse doar când o întrebare are nevoie de ele. Odată aduse, însă, datele sunt citite de model ca orice alt text, ceea ce ridică întrebări de încredere și de confidențialitate.

Mai e și un efect secundar pe care nu-l planifică nimeni. Un sistem RAG e necruțător cu documentația. Dacă runbook-ul e vechi de doi ani, asistentul îl va cita acum exact și sigur pe el. Echipele care lansează un astfel de asistent descoperă adesea că prima lui realizare adevărată e să arate ce pagini din wiki n-a mai întreținut nimeni.

Iar bibliotecarul nu e perfect. Dacă aduce pagina greșită, scriitorul o citează cu convingere totală. Se poate și să aducă doar o parte din ce trebuie: când un răspuns are nevoie de bucăți din mai multe documente, o singură căutare aduce adesea doar una, pentru că niciun pasaj nu e aproape de întrebarea întreagă, așa că sistemele care se descurcă aici îl lasă pe model să caute de mai multe ori, câte o sub-întrebare pe rând, o buclă cu o unealtă de căutare în mână. Calitatea unui sistem RAG e plafonată de calitatea căutării lui, așa că merită să măsori separat cele două jumătăți: a adus pasajul potrivit și l-a folosit corect? Cele mai eficiente echipe strâng câteva zeci de întrebări reale de la utilizatori reali, împreună cu pasajele care ar trebui să le răspundă, și rulează acest set de fiecare dată când schimbă tăierea în chunks, embeddings sau promptul, exact ca pe o suită de teste de regresie.

Înainte să lăsăm limbajul în urmă, mai rămân două întrebări: cât vede de fapt un model dintr-o conversație lungă și de ce un joc atât de simplu ca ghicitul cuvântului următor ajunge atât de departe. Prima începe cu o veche cunoștință din comisia de recrutare.

7.5 Viața pe o tablă

Îți amintești de comisia de recrutare, cele patru recrutoare care triau două sute de CV-uri, fiecare decizând în felul ei ce contează. A patra, care lasă fiecare CV să-i conducă privirea, a trebuit să aștepte limbajul, pentru că ea e attention: felul ei de a citi e ședința detectivilor. Aici întrebarea „cine decide?” are răspunsul cel mai interesant. Nimeni n-a decis dinainte că „bancă” trebuie să-l asculte pe „bani”; asta se stabilește din nou în fiecare propoziție. Ce modelează antrenarea, cum a arătat jocul ghicitului, sunt transformările care scriu, în general, întrebările și ecusoanele. De-a lungul a miliarde de propoziții, aceste transformări ajung să codifice, în fond, „ce fel de cuvânt tinde să ajute un cuvânt ca mine”. Relevanța nu-i e dată modelului. E o abilitate pe care o dobândește pentru că relevanța se plătește în jocul cuvântului următor. Head-uri diferite dobândesc noțiuni diferite de relevanță, pentru că fiecare pornește de la alte numere aleatorii și nimic nu le obligă să cadă de acord; în practică, unele head-uri se dovedesc aproape redundante și pot fi scoase cu pierderi mici, în timp ce altele duc o mare parte din greu.

Tot de aceea ponderile de atenție explică mai puțin decât par. Un head poate acorda multă atenție unui cuvânt și poate transmite foarte puțin de la el, în timp ce munca decisivă se petrece în alt strat. Cercetătorii au dezbătut chestiunea în două articole din 2019 cu titluri memorabile, *Attention is not Explanation* („Atenția nu e explicație”) și *Attention is not not Explanation* („Atenția nu e ne-explicație”), iar rezumatul onest rămâne cel de atunci: un indiciu, nu un verdict.

A patra recrutoare mai are o limită, cea mai fizică dintre toate: își poate îndrepta privirea doar spre ce se află pe biroul din fața ei. Pentru un model de limbaj, biroul acela e fereastra de context, iar mărimea și forma ei decid foarte multe despre ce poate face modelul.

O ingineră petrece o după-amiază cu un asistent, planificând o migrare a bazei de date. În primul mesaj stabilește regulile: tabela de comenzi nu are voie să fie blocată niciodată mai mult de o secundă și nicio operație grea nu are voie să ruleze în timpul programului de lucru european. Trei ore, patruzeci de mesaje și câteva dump-uri de schemă lipite mai târziu, asistentul propune voios un pas care blochează

tabela de comenzi douăzeci de minute, la unsprezece dimineața. N-a devenit neatent în vreun sens omenesc. Fie primul ei mesaj nu mai e deloc pe tablă, fie e încă acolo, îngropat sub o mulțime de scris ulterior, iar modelul i-a dat prea puțină greutate.

Subcapitolul acesta se uită mai atent la tablă: cât de mare e, ce îi stabilește mărimea, ce se întâmplă la marginea ei și cum construiesc oamenii sisteme care se poartă ca și cum tabla ar fi mai mare decât este.

Tabla se măsoară în tokeni și ține totul, nu doar mesajele tale: instrucțiunile adăugate de aplicație, documentele regăsite, rezultatele uneltelor și fiecare cuvânt din răspunsurile modelului însuși. Modelele de limbaj din 2020 aveau table de vreo două mii de tokeni, câteva pagini. Pe la mijlocul anilor 2020, tablele de o sută de mii până la câteva sute de mii de tokeni deveniseră obișnuite, iar unele modele acceptau un milion sau mai mult. Cu trei-patru caractere pe token, o pagină în engleză de vreo cinci sute de cuvinte înseamnă foarte aproximativ șapte sute de tokeni, deci o tablă de o sută de mii de tokeni ține cam o sută patruzeci de pagini, un roman scurt.

Două lucruri stabilesc mărimea. Primul e costul. Munca atenției, cum avertiza ședința detectivilor, se cvadruplează când textul se dublează, iar KV cache-ul, ecusoanele și dosarele păstrate ale tokenilor anteriori, crește cu fiecare token de pe tablă, pentru fiecare conversație servită în același timp. O tablă mai lungă e mai lentă și consumă mai multă memorie, de aceea intrările lungi costă mai mult și de aceea inginerii s-au luptat atât să le facă mai ieftine. Un truc des întâlnit e să păstrezi în cache citirea unui început fix: când fiecare cerere începe cu aceleași instrucțiuni și documente lungi, ecusoanele și dosarele lor sunt calculate o dată și refolosite, iar furnizorii facturează adesea acești tokeni din cache cu reducere. Al doilea lucru e antrenarea. Un model învață să folosească pozițiile pe care s-a exersat, prin informația de poziție amestecată în fiecare token. Dă-i unui model antrenat doar pe texte scurte unul foarte lung și va întâlni poziții pe care nu le-a văzut niciodată, iar rezultatul de obicei se destramă. Tablele lungi sunt predate de regulă într-o etapă ulterioară a antrenării, pe documente lungi, cu ajustări ale felului în care se scriu pozițiile. Mărimea tablei e o proprietate a antrenării, nu un număr pe care cineva îl poate pur și simplu mări într-un fișier de configurare.

Ce se întâmplă la margine? Modelul însuși nu decide niciodată. Aplicația din jurul lui trebuie să acționeze înainte ca textul să ajungă la model și are trei variante. Poate refuza, spunându-i utilizatorului că conversația e prea lungă. Poate tăia, de obicei partea cea mai veche, exact acolo unde tind să stea instrucțiunile, ca în povestea migrării. Sau poate condensa, ca un grefier care, când se umple tabla, rescrie notițele cele mai vechi ca un scurt rezumat într-un colț și șterge restul. Un rezumat pierde detalii prin definiție: regula de o secundă poate supraviețui ca „ai grijă cu blocările”. Și iată punctul care îi surprinde pe cei mai mulți. Modelul nu vede niciodată ce a fost tăiat și nu are cum să știe că lipsește ceva. Nu simte un gol. Răspunde din ce e pe tablă ca și cum asta ar fi fost tot ce a existat vreodată. De aceea aplicațiile bine construite fixează ce nu are voie să se piardă, rescriind instrucțiunile critice în capul tablei la fiecare tură.

În interiorul limitei, tabla nu e citită uniform. Într-un studiu din 2023 intitulat *Lost in the Middle* („Pierdut la mijloc”), cercetătorii au pus răspunsul la o întrebare în diferite poziții printre vreo douăzeci de documente regăsite. Modelele l-au folosit cel mai bine când stătea aproape de început sau de sfârșit și vizibil mai prost când stătea la mijloc. Modelele mai noi se descurcă mult mai bine la versiunea cea mai simplă a testului, găsirea unei singure propoziții plantate într-un text lung, numită adesea acul din carul cu fân. Dar munca reală e mai grea decât găsirea unui ac: înseamnă combinarea mai multor fapte împrăștiate pe tablă, iar la asemenea sarcini calitatea tinde încă să scadă pe măsură ce tabla se umple.

Mai mult text înseamnă și mai multă distragere, pentru că pasajele care par relevante, dar nu sunt, tot trag de atenția modelului. Obiceiurile practice decurg direct de aici: pune lucrurile importante la început sau la sfârșit, ține tabla ordonată și preferă să aduci paginile relevante în loc să lipești totul.

Așa se lămurește și întrebarea context lung sau RAG, la care examenul cu cartea deschisă răspunsese doar pe scurt. Pentru o mână de documente care încap lejer, lipește-le: e aranjamentul cel mai simplu, modelul vede tot și nu există niciun pas de căutare care să greșească. Pentru un wiki întreg, regăsirea câștigă în continuare, prin cost și spațiu și din cauza lecturii inegale descrise mai sus. Multe sisteme reale le folosesc pe amândouă, regăsirea ca să aleagă ce contează și o tablă generoasă ca să-l țină.

Compară tabla cu fișa rețelelor recurente. Fișa n-avea margine, dar se estompa: detaliile vechi erau suprascrise puțin câte puțin. Tabla nu se estompează, iar fiecare cuvânt de pe ea e la fel de lizibil ca atunci când a fost scris, dar are o margine fermă, iar dincolo de ea nu e absolut nimic.

Cum își „amintește” atunci un asistent de chat de tine de la o conversație la alta? Tabla tot e ștearsă. Ce persistă e un carnet purtat între ședințe. Aplicația salvează notițe, fapte pe care le-ai pomenit, preferințe, rezumate ale conversațiilor anterioare, într-un depozit obișnuit, iar la începutul unei conversații noi le scrie pe cele relevante înapoi pe tabla proaspătă, alegându-le adesea cu același fel de căutare folosit la regăsire. Ponderile modelului nu se schimbă. Memoria, în aceste produse, e text scris înapoi pe tablă, ceea ce înseamnă și că ți se poate arăta, poate fi ștearsă sau poate deveni învechită, ca orice notiță.

Agenții trăiesc și ei pe tablă și o umplu repede, pentru că fiecare rezultat al unei unelte ajunge acolo: un fișier citit, un log de teste, o pagină de rezultate de căutare. Cei care construiesc agenți se descurcă cu aceleași unelte pe care le folosește un om organizat. Îl pun pe agent să rezume pașii mai vechi, din nou grefierul. Îi dau un fișier de notițe în care scrie și pe care îl recitește, o listă de sarcini ținută pe disc în loc de pe tablă. Și împart lucrările mari în subsarcini date unor copii proaspete ale modelului, fiecare cu tabla curată, care raportează înapoi doar un rezultat scurt, așa cum un inginer principal nu citește fiecare log, ci le cere colegilor un rezumat.

Aici imaginea tablei se rupe, în două locuri. Un om citește o tablă cu ochi care sar: treci cu privirea peste meniul de prânz fără să-l citești. Modelul procesează fiecare token de pe tablă la fiecare pas al scrisului; nimic nu e sărit, totul e cântărit, chiar dacă greutatea e aproape zero. De aici vin și costul, și diluarea: fiecare rând irelevant costă calcul și concurează, puțin, pentru atenție. Iar o tablă adevărată are o suprafață fixă oricum ai scrie, în timp ce tabla modelului se numără în tokeni, așa că același conținut poate ocupa mult mai mult loc într-o formă decât în alta: un dump JSON lipit sau un stack trace lung pot mânca mii de tokeni pentru câteva rânduri de sens.

Așadar tabla e toată lumea modelului cât timp lucrează: fără simțuri, fără ziua de ieri, doar text și ponderile înghețate la sfârșitul antrenării. Asta ascute o întrebare pe care cartea a lăsat-o până acum deschisă. Cum reușește un sistem care a învățat doar să continue text să raționeze, să traducă și să scrie cod care funcționează, având la dispoziție doar o tablă cu cuvinte?

7.6 De ce ajunge atât de departe ghicitul cuvântului următor

Întoarce-te la inginera nouă, cea care a citit tot wiki-ul în prima lună, și dă-i un joc. I se arată începutul unui postmortem pe care nu l-a văzut niciodată și trebuie să ghicească propoziția următoare. „La 14:02, serviciul de checkout a început să întoarcă erori. Inginerul de gardă a făcut rollback la deployment-ul de dimineață, dar erorile au continuat. Cauza principală a fost…” O ghicire bazată pe ce cuvinte urmează

de obicei după „cauza principală a fost” va da rezultate slabe. Ca să ghicească bine, trebuie să judece că o defecțiune care supraviețuiește unui rollback probabil nu era în cod, deci poate o schimbare de configurare, un certificat expirat, o dependență, baza de date. Are nevoie de o imagine a felului în care funcționează sistemul. Recompenseaz-o doar pentru ghicit bine, pune-o să joace jocul de miliarde de ori, pe orice subiect despre care scriu oamenii, și va fi împinsă să învețe orice face textul previzibil: întâi gramatica, pentru că e peste tot, apoi faptele, apoi tiparele aritmeticii, obiceiurile codului, felul în care se desfășoară argumentele.

Acesta e cel mai des întâlnit răspuns la acea întrebare rămasă deschisă și merită formulat cu grijă. Jocul cuvântului următor e simplu de pus, dar nu e simplu de câștigat. Cheia lui de răspunsuri conține tot ce scriu oamenii, așa că un jucător foarte bun trebuie să surprindă o mare parte din ce a produs textul. Nimic din antrenare nu spune „învață să raționezi” sau „învață să traduci”; aceste abilități sunt învățate, în măsura în care sunt, pentru că scad loss-ul. E aceeași idee ca procesarea predictivă din creier, predicția ca drum spre un model al lucrului care produce semnalul, cu o diferență: creierul își prezice propriile simțuri, în timp ce modelul de limbaj prezice text, o înregistrare de mâna a doua a lumii.

Un al doilea mod de a vedea lucrurile e prin compresie. Ponderile unui model mare ocupă de multe ori mai puțin spațiu decât textul pe care a fost antrenat, așa că nu-l poate păstra cuvânt cu cuvânt, deși memorează unele pasaje care apar de foarte multe ori. Teoria informației știe de mult că un bun predictor și un bun compresor sunt două fețe ale aceluiași lucru, iar ca să comprimi bine păstrezi regularitățile și renunți la accidente. Din regularități e făcută generalizarea. Tocilarul avea loc să memoreze fiecare răspuns; un model de limbaj nu are, așa că e împins spre tipare.

O consecință i-a surprins chiar și pe cei care au văzut-o primii la scară mare. Scrie câteva exemple pe tablă, „cheval: cal, chien: câine, chat:”, iar modelul continuă tiparul, deși nicio pondere nu se schimbă. Asta e **in-context learning** („învățare în context”: modelul preia un tipar din exemplele de pe tablă, fără să-și schimbe ponderile), făcută celebră de articolul din 2020 care a prezentat GPT-3, intitulat *Language Models are Few-Shot Learners* („Modelele de limbaj învață din câteva exemple”). O parte din explicație e că textul de antrenare e plin de liste, tabele, exerciții și formate repetate, unde a observa un tipar mai sus pe pagină e cel mai bun mod de a prezice ce vine mai jos, așa că un model bun la joc învață să preia tipare de pe propria tablă. Cercetătorii au găsit în Transformer-e circuite anume, numite induction heads („capete de inducție”), care copiază tipare de acest fel și par să explice o parte a efectului, deși nu tot. E învățare doar într-un sens larg: durează exact cât timp exemplele rămân pe tablă.

De ce contează mărimea atât de mult, când un model mic care joacă același joc produce buclele tastaturii? Loss-ul general scade lin și previzibil pe măsură ce modelele, datele și calculul cresc împreună, de-a lungul curbelor de scalare pe care le-am întâlnit când ne-am întrebat câte date sunt de ajuns. Abilitățile concrete pot însă părea că apar brusc. La adunarea numerelor de trei cifre, modelele mici obțin aproape zero, iar peste o anumită mărime acuratețea urcă repede. Fenomenul a fost numit **emergence** („emergență”: abilități care par să apară brusc peste o anumită scară) și e disputat. Un articol din 2023 a susținut că multe salturi bruște sunt în parte un produs al felului în care sunt notate: dacă o sumă contează doar când fiecare cifră e corectă, îmbunătățirea constantă la fiecare cifră apare ca un salt abrupt în scorul de tip totul-sau-nimic, iar cu o măsură mai blândă curba arată adesea lină. Alte sarcini par să ceară mai multe abilități deodată, iar o sarcină care le cere pe toate poate părea abruptă chiar dacă fiecare abilitate se îmbunătățește treptat. Un rezumat corect: dedesubt, îmbunătățirea e mai ales treptată, dar lucrurile la care un model e util se pot schimba brusc când trece praguri care contează pentru noi.

Aceeași mecanică explică unde greșesc aceste modele, adesea la lucruri care par banale. Întreabă câte litere r are cuvântul englezesc „strawberry” și un model puternic poate greși, pentru că nu vede niciodată litere, doar tokeni; ortografia e un fapt învățat despre fiecare token, ca un număr de telefon știut pe de rost, nu ceva la care se uită. Cere-i să înmulțească dintr-odată două numere de opt cifre și se chinuie, pentru că fiecare token primește aceeași cantitate fixă de calcul, un forward pass prin același număr de straturi, fie că întrebarea e doi plus doi, fie o înmulțire lungă. Scrierea pașilor îi dă mai multe treceri: fiecare rezultat intermediar scris pe tablă devine intrare pentru tokenul următor. De aceea ajută cererea de a lucra pas cu pas, tehnică pe care cercetătorii au numit-o chain of thought („lanț de gândire”) în 2022, și de aceea modelele de raționament sunt antrenate să scrie calcule lungi înainte să răspundă. Cheltuirea mai multor calcule în momentul răspunsului, prin lanțuri mai lungi sau prin mai multe încercări care sunt apoi verificate, se numește **test-time compute** („calcul în momentul răspunsului”: modelul primește mai mult timp de gândire la fiecare întrebare) și a devenit un al doilea buton lângă mărimea modelului: același jucător de șah joacă mai bine o partidă clasică decât una de blitz. Fiecare token de gândire e un forward pass pe care îl plătește cineva, iar dacă raționamentul scris e cauza reală a răspunsului e o întrebare separată.

Jocul a depășit și cuvintele. Taie o imagine în pătrățele mici, transformă fiecare pătrățel într-un vector așa cum un token devine embedding și pune pătrățelele pe tablă lângă cuvinte; și sunetul poate fi tăiat într-un mod asemănător. Un model antrenat pe astfel de table amestecate învață să prezică peste ele, iar unele modele produc și ele tokeni de imagine sau de sunet. Așa citesc capturi de ecran, grafice și fotografii asistenții numiți **multimodal** („multimodali”: care lucrează cu mai multe feluri de date, text, imagini, sunet). Nimic fundamental nu s-a schimbat. Tabla ține pur și simplu mai mult decât text.

Înțelege, deci, modelul? Există două tabere serioase. Una, exprimată într-un articol mult citat din 2021 al lui Emily Bender, Timnit Gebru și colegilor lor, care numea aceste sisteme „papagali stocastici”, susține că un model antrenat doar pe forma limbajului coase tipare fără acces la sens și că fluența lui ne induce în eroare. Cealaltă susține că a prezice bine obligă un model să-și construiască o structură internă care funcționează ca un model al lucrului despre care e textul. O dovadă mică, dar izbitoare, a fost publicată în 2023: cercetătorii au antrenat un Transformer doar pe șiruri de mutări legale din jocul de masă Othello, fără să-i arate vreodată tabla, și au descoperit că starea curentă a tablei de joc putea fi citită din numerele lui interne, iar modificarea acelor numere îi schimba în mod corespunzător mutarea următoare. E o jucărie, departe de limbajul omenesc, iar ambele tabere sunt de acord că aceste modele nu sunt oameni; se despart în privința întrebării dacă „înțelegere” e cuvântul potrivit pentru ce au. Poziția cărții e cea practică: judecă un model după comportamentul lui la teste oneste, cu probleme pe care nu are cum să le fi văzut, ferește-te de întrebările de test ajunse în textul lui de antrenare și ține întrebarea deschisă.

Capitolul despre creier observa că modelele de limbaj învață fără să se miște, să atingă sau să acționeze. Cercetarea despre **world models** („modele ale lumii”: un fel de simulator interior al consecințelor) încearcă să închidă golul acesta: modele care prezic ce urmează dacă se face o anumită acțiune, un simulator de zbor învățat din date în loc să fie programat. Modelele video antrenate să prezică următoarele cadre sunt testate ca simulatoare pentru roboți și agenți din jocuri, iar roboții învață din rezultatele propriilor mișcări. Domeniul e abia la început, iar dacă asemenea modele surprind fizica sau doar pixeli care arată plauzibil e și ea o chestiune disputată.

Aici se rupe imaginea inginerei noi. Ea are un corp, scopuri, colegi pe care îi poate întreba și o memorie care se păstrează de la o zi la alta; citește wiki-ul ca să-și facă treaba și trăiește cu consecințele când greșește. Modelul citește ca să prezică, nu păstrează nimic de la o conversație la alta și cântărește

fiecare pagină după cât de des îi revin tiparele, nu după cât de mult merită încredere.

Un cuvânt a tot revenit în acest subcapitol: scara. Modele mai mari, mai mult text, mai mult calcul, mai multă gândire în momentul răspunsului. Aici se încheie capitolul despre limbaj și sens. Înainte să trecem la felul în care sunt învățate modelele și la felul în care dau greș, mai rămân două lucruri: mașinile care fac imagini în loc să le citească și mâna de planuri de arhitectură în care inginerii combină toate aceste piese.

Mai departe. Dacă te-ai întrebat cum devin suportabile tablele lungi și cum readuc state-space models fișa care se estompează a rețelelor recurente, subcapitolul 8.4 le privește pe amândouă mai de aproape. Dacă un model își poate da seama când nu știe e subiectul subcapitolului 10.5, iar dacă raționamentul pe care îl scrie un model e cauza reală a răspunsului său se discută în subcapitolul 10.6. Iar de ce un document dat modelului se poate transforma într-un atac se explică în subcapitolul 11.3.

Capitolul 8 — Imagini generate, sisteme construite

8.1 Învățând să desenezi

Un copil învață să deseneze o pisică într-un fel anume. Nu studiind regulile anatomiei pisicii și nici trăgând pe contur. Desenează o pată tremurată, se uită la ea, decide că ceva nu e în regulă, repară urechile, se uită din nou, repară coada. Fiecare trecere e o mică corectură spre „mai mult a pisică". După destule treceri și destule pisici văzute, pata devine o pisică pe care n-a mai văzut-o nimeni, pentru că e a ei.

Tot ce am văzut până acum în carte a fost despre recunoaștere, predicție și decizie. Subcapitolul acesta e despre cealaltă direcție: a face lucruri. Mai ales imagini, pentru că acolo ideile se văd cel mai ușor, dar aceleași idei produc muzică, molecule, voci și video. Iar lucrul ciudat și satisfăcător e că metoda cea mai reușită dintre toate s-a dovedit a funcționa foarte asemănător cu copilul: nu desenând dintr-o singură trăsătură, ci corectând o mâzgălitură, puțin câte puțin.

Cea mai veche idee e să forțezi o rețea să rezume și apoi să reconstruiască. Imaginează-ți un curier care trebuie să descrie o fotografie la telefon în exact treizeci de cuvinte unui coleg care apoi trebuie să o redeseneze. „O pisică tigrată, așezată, privind spre stânga, pe o pernă roșie, fereastră în spate, lumină de după-amiază." Treizeci de cuvinte nu pot cuprinde o fotografie. Ca redesenarea să iasă bine, cel care descrie trebuie să învețe ce contează la pisici, perne și lumină, iar cel care desenează trebuie să învețe cum arată acele lucruri. Antrenează-i pe amândoi împreună pe un milion de fotografii, notându-i după cât de aproape e imaginea redesenată de original, și descrierea de treizeci de cuvinte devine un rezumat remarcabil de bogat. Acesta e un **autoencoder** („autocodificator": o rețea care învață să se codifice și să se decodifice singură): un encoder („codificator") care strivește intrarea într-un cod scurt și un decoder („decodificator") care desface codul la loc. Codul stă într-un spațiu în care, ca pe harta sensurilor cuvintelor, codurile apropiate înseamnă imagini asemănătoare. Problema e că cele mai multe puncte din acel spațiu nu corespund niciunei fotografii reale, așa că nu poți alege pur și simplu un cod la întâmplare și să te aștepți la o pisică bună. O rafinare din 2013, **variational autoencoder** („autocodificator variațional"), îl învață pe encoder să-și întindă codurile uniform, astfel încât codurile aleatorii să se decodifice în imagini plauzibile, chiar dacă încețoșate. Neclaritatea a fost problema atacată de următoarea idee.

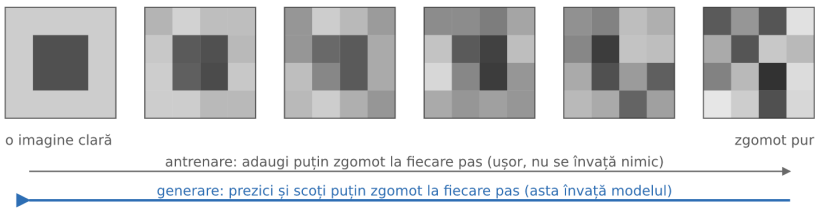
În 2014, Ian Goodfellow și colegii lui au propus un joc. Două rețele. Una, **generator**-ul, face imagini din zgomot aleatoriu. Cealaltă, **discriminator**-ul, primește un amestec de fotografii reale și falsuri ale generatorului și trebuie să spună care e care. Discriminatorul e antrenat să devină tot mai bun la depistarea falsurilor; generatorul e antrenat să-l păcălească. Niciunuia nu i se spune vreodată cum arată o pisică. Generatorul învață doar din verdictele discriminatorului, iar discriminatorul învață doar din faptul că a avut sau nu dreptate. E un falsificator și un expert în artă închiși într-o cameră: pe măsură ce expertul se ascute, falsificatorul trebuie să se îmbunătățească, și invers, până când falsurile sunt atât de bune încât expertul nu mai poate decât să ghicească. Acestea sunt **generative adversarial networks** („rețele generative adversariale", adică care se luptă între ele), pe scurt GAN-uri, iar timp de câțiva ani au produs cele mai clare fețe sintetice văzute vreodată, genul care umplea site-uri cu oameni care nu există. Slăbiciunea lor era antrenarea însăși: un joc între doi jucători e instabil în feluri în care un singur loss nu e, cei doi se pot urmări în cerc, iar generatorul învață uneori o singură față foarte convingătoare

și o produce iar și iar, un eșec numit mode collapse („prăbușirea modurilor”: modelul se blochează pe o singură variantă). Inginerii care au depanat o buclă de feedback între două servicii vor recunoaște senzația.

Apoi, pe la 2020, domeniul s-a așezat pe o a treia idee, iar ea stă în spatele celor mai multe generatoare de imagini pe care probabil le-ai folosit.

Ia o fotografie și adaugă-i niște purici, cum face un televizor vechi când semnalul e slab. Mai adaugă puțin. Și mai mult, timp de câteva sute de pași, până când nu mai rămân decât purici. Direcția asta e ușoară; o poate face un script și nu implică nicio învățare. Acum cere-i unei rețele să facă opusul, câte un pas pe rând: iată o imagine puțin zgomotoasă, spune-mi care e zgomotul, ca să-l pot scădea. E o problemă de predicție cu cheie de răspunsuri, pentru că zgomotul l-ai adăugat chiar tu și știi exact care a fost. Antrenează rețeaua pe milioane de fotografii, la fiecare nivel de zgomot. Devine foarte bună la o singură abilitate: să se uite la o imagine zgomotoasă și să spună ce trebuie scos. Abilitatea se poate învăța deși zgomotul e aleatoriu, pentru că imaginea de sub el nu e: ca să deosebească puricii de restul, rețeaua trebuie să știe cum arată fotografiile reale, că blana are textură și cerul e neted, așa că a învăța să cureți zgomotul înseamnă, pe ascuns, a învăța cum arată lumea.

O imagine se dizolvă în zgomot; modelul învață să ruleze filmul înapoi



Acum pornește doar de la purici și aplică abilitatea aceasta de câteva sute de ori. Fiecare trecere scoate puțin zgomot și dezvăluie puțină structură, așa cum se developează un Polaroid sau cum pata copilului devine pisică prin mici corecturi. Spre deosebire de copil, rețeaua nu hotărăște niciodată să deseneze o pisică și nici nu face un pas înapoi ca să privească întregul: la fiecare trecere răspunde doar la întrebarea „ce zgomot e aici?”, și de aceea o mână generată poate avea șase degete fără ca ceva din proces să obiecteze. Rezultatul e o fotografie care n-a fost făcută niciodată: ideea rețelei despre ce ar fi putut ascunde puricii. Fiecare rulare pornește de la alți purici, iar purici diferiți ascund fotografii diferite, de aceea același prompt dă de fiecare dată altă imagine; puricii de pornire sunt trași pornind de la un număr numit seed („sămânță”), iar refolosirea aceluiași seed cu același prompt și aceleași setări dă de obicei din nou aceeași imagine. Acesta e un **diffusion model** („model de difuzie”), iar numele vine din fizica cernelii care se împrăștie în apă, rulată invers. Se antrenează stabil, pentru că fiecare pas e o predicție obișnuită cu răspuns cunoscut, și e mult mai puțin expus riscului de a se bloca pe o singură variantă, pentru că învață să curețe zgomotul spre *orice* fel de imagine i s-a arătat.

Promptul text sosește printr-o piesă în plus. La fiecare pas de curățare, rețelei i se arată și o descriere, codificată de un model de limbaj în același fel de adresă ca embeddings ale cuvintelor, și e antrenată pe fotografii cu descrieri să scoată zgomotul *într-un fel care se potrivește descrierii*. „O pisică tigrată pe o pernă roșie” e o mână pe umărul copilului la fiecare corectură, împingând urechile spre tigrat și fundalul spre roșu. Cât de tare împinge mâna e un buton: multe generatoare rulează fiecare pas de

curățare de două ori, o dată cu descrierea și o dată fără, și împing rezultatul mai departe în direcția diferenței; dacă dai mai sus acest buton, numit adesea guidance („ghidare”), imaginea urmează cuvintele mai literal, cu prețul varietății și uneori al realismului. Generatoare de imagini precum Stable Diffusion, versiunile mai noi ale DALL-E și Midjourney sunt construite pe această idee. O rafinare de inginerie, popularizată de Stable Diffusion, explică de ce unele dintre ele pot rula pe hardware obișnuit: în loc să curețe zgomotul imaginii la mărime completă, curăță codul scurt al unui autoencoder, descrierea de treizeci de cuvinte, și îl transformă în pixeli abia la final. Lucrul în acel spațiu comprimat e de multe ori mai ieftin.

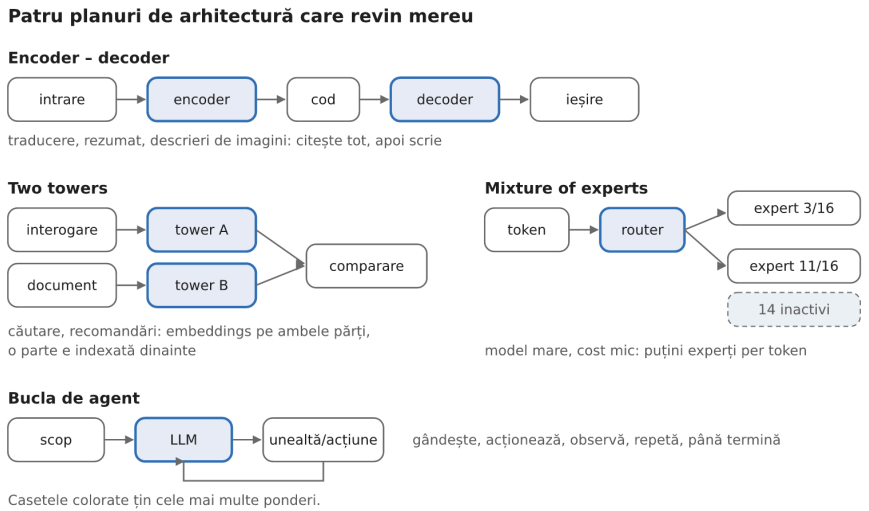
Pentru cineva care construiește sau cumpără astfel de sisteme, din mecanică decurg trei fapte practice. Întâi, generarea e lentă prin natura ei: zeci sau sute de forward pass-uri per imagine, de aceea atâta cercetare merge în a avea nevoie de mai puține. Apoi, modelul poate curăța zgomotul doar spre ce a văzut, de aceea se chinuie cu mâinile, cu textul din imagini și cu combinațiile neobișnuite și de aceea reproduce cu mare fidelitate prejudecățile colecției lui de fotografii. În al treilea rând, rețeta se transferă: adaugă zgomot la sunet, molecule sau cadre video în loc de pixeli și ai un generator de muzică, un generator de candidați pentru medicamente sau un model video. Saltul de la imagini la orice altceva a fost în mare parte o schimbare de date. De obicei, un generator nici nu copiază imaginile pe care a fost antrenat, pentru că învață să curețe zgomotul spre milioane de imagini deodată și spre niciuna anume; dar o imagine care a apărut de multe ori în colecția de antrenare, un tablou celebru sau o fotografie foarte distribuită, poate ieși aproape identică, un caz de memorare cu urmări juridice.

Merită să tragi o linie între subcapitolul acesta și jocul ghicitului cuvântului următor. Un model de limbaj își construiește ieșirea un token după altul, fiecare condiționat de cele dinainte, ca și cum ai scrie o propoziție de la stânga la dreapta. Un model de difuzie rafinează toată ieșirea deodată, în multe treceri, ca un sculptor. Amândouă sunt mașini de predicție antrenate cu o cheie de răspunsuri pe care datele o oferă gratuit: cuvântul următor, zgomotul adăugat. Familia aceasta de trucuri, învățarea din date care se etichetează singure, are un nume: self-supervised learning („învățare auto-supervizată”).

Am văzut acum cele mai multe dintre mașinile individuale. Următorul subcapitol e despre felul în care sunt combinate: mâna de aranjamente la care inginerii apelează iar și iar și problemele pe care a fost inventat să le rezolve fiecare.

8.2 Planuri de arhitectură

Orice inginer software cu experiență poartă în minte un mic set de forme. O coadă între un producător și un consumator. Un cache în fața unei stocări lente. Un load balancer care împarte cererile pe replici. Nu le reinventează la fiecare proiect; recunoaște problema, își ia forma și își cheltuie creativitatea pe părțile noi. Machine learning a ajuns în același stadiu. Sub sutele de nume de modele există poate o duzină de aranjamente care tot revin, fiecare inventat pentru o durere anume. Subcapitolul acesta e o plimbare prin cele pe care le vei întâlni cel mai des, povestite așa cum învață de fapt inginerii tiparele (design patterns): iată problema, iată forma care a rezolvat-o, iată unde se rupe.



Începe cu traducerea, pentru că ea a forțat prima mare formă. O propoziție în franceză trebuie să devină una în engleză, iar cele două au lungimi diferite și ordini diferite ale cuvintelor; adjectivul care vine după substantiv în franceză vine înaintea lui în engleză. Nu poți traduce cuvânt cu cuvânt. Aranjamentul care a funcționat, pe la 2014, a fost să împarți treaba în două. Un **encoder** („codificator”) citește toată propoziția franceză și o condensează într-o reprezentare, un set de vectori care surprind ce s-a spus. Un **decoder** („decodificator”) scrie apoi propoziția engleză din acea reprezentare, câte un cuvânt pe rând, consultând-o pe parcurs. Citește tot, apoi scrie. Planul **encoder–decoder** stă în spatele traducerii, rezumatului, transcrierii vorbirii și descrierii imaginilor, orice în care intrarea e un lucru, iar ieșirea alt lucru de altă formă. Articolul original despre Transformer din 2017 era exact asta: un teanc de encoder și unul de decoder, cu attention între ele. Unde se rupe: dacă intrarea e enormă, strivirea ei într-o reprezentare fixă pierde detalii, și acesta e unul dintre motivele pentru care attention între encoder și decoder a fost o asemenea ușurare; decoderul se putea uita înapoi la fiecare cuvânt de intrare, nu doar la un rezumat.

Apoi s-a produs o despărțire curioasă. Cercetătorii au observat că cele două jumătăți erau utile și separat. Păstrează doar encoderul, antrenează-l să completeze cuvinte ascunse dintr-o propoziție și obții un model excelent la *înțelegerea* textului, dar care nu-l generează: acesta e BERT (bidirectional encoder representations from transformers, „reprezentări bidirecționale de encoder din Transformer-e”), din 2018, care alimentează încă multă căutare și clasificare. Păstrează doar decoderul, antrenează-l să prezică următorul cuvânt și obții un model care *scrie*: aceasta e familia GPT și, odată cu ea, tot jocul ghicitului cuvântului următor. **Encoder-only** pentru citit, **decoder-only** pentru scris. Când vezi un model descris ca unul sau altul, știi acum ce jumătate din mașina de tradus este.

A doua formă vine dintr-o problemă de scară de care se lovește orice inginer de căutare. Ai un milion de documente și interogarea unui utilizator și vrei cele mai bune potriviri în milisecunde. Să rulezi un model scump pe fiecare pereche interogare–document e imposibil. Așa că împarte modelul în două **towers** („turnuri”): unul codifică documentele, celălalt interogările, iar amândouă produc adrese pe aceeași hartă a sensurilor. Turnul documentelor rulează o singură dată, dinainte, peste toată colecția, iar rezultatele intră într-un index de vecini apropiați. La momentul interogării rulează doar turnul

interogării, iar căutarea e o simplă consultare. Planul **two-tower** sau **dual-encoder** stă în spatele căutării semantice, al jumătății de regăsire din RAG și al celor mai multe sisteme de recomandare, unde cele două turnuri sunt „utilizator” și „produs”. E același schimb pe care îl face o bază de date când construiește un index la scriere ca să facă citirile ieftine. Unde se rupe: pentru că turnurile nu văd niciodată intrarea celuilalt, nu pot prinde interacțiunile subtile dintre o anumită interogare și un anumit document, de aceea sistemele serioase adaugă un **cross-encoder** („encoder încrucișat”) mai lent, care recitește primii cincizeci de candidați cu amândouă textele alături, aranjamentul bibliotecar-apoi-expert de la examenul cu cartea deschisă.

Ideea celor două turnuri are o soră care a rezolvat o altă problemă: cum înveți adrese bune când nu ai deloc etichete? Răspunsul, numit **contrastive learning** („învățare prin contrast”), e să înveți un model că două vederi ale aceluiași lucru trebuie să ajungă aproape una de alta, iar vederile unor lucruri diferite departe. Două decupaje din aceeași fotografie, aproape; decupaje din fotografii diferite, departe. O fotografie și descrierea ei, aproape; o fotografie și descrierea altcuiva, departe. Modelul numit CLIP (contrastive language–image pre-training, „pre-antrenare prin contrast între limbaj și imagine”), antrenat așa în 2021 pe sute de milioane de perechi imagine–descriere, e o mare parte din motivul pentru care multe generatoare de imagini pot înțelege un prompt text și motivul pentru care îți poți căuta pozele scriind „plajă la apus”.

A treia formă e despre bani. Modelele de limbaj deveneau mai bune pe măsură ce creșteau, dar fiecare token trebuia să treacă prin fiecare pondere, așa că un model de zece ori mai mare costa de zece ori mai mult de rulat. Ieșirea a fost o idee veche, readusă la viață. În loc de un singur strat dens enorm, păstrează șaisprezece mai mici, **experts** („experții”), iar în fața lor un mic **router** („dispecer”) care se uită la fiecare token și îl trimite la cei doi experți cei mai potriviți pentru el. Paisprezece experți stau degeaba pentru acel token. Modelul, în ansamblu, are capacitatea tuturor celor șaisprezece, dar fiecare token plătește pentru doi. Planul **mixture of experts** („amestec de experți”) e felul în care mai multe dintre cele mai mari modele de limbaj din ultimii ani își țin costurile de servire sub control și, în esență, e biroul de triaj al unui spital: o privire rapidă, apoi specialistul potrivit, în loc ca fiecare pacient să treacă pe la fiecare doctor. Routerul e el însuși un mic strat învățat, antrenat împreună cu experții: tokenii pe care un expert îi tratează bine sunt trimiși mai des la el, iar expertul devine tot mai bun la ei, așa că specialitățile apar de la sine, deși rareori sunt subiecte ordonate precum „expertul în matematică” și tind să fie tipare mai fine, cum ar fi punctuația sau anumite feluri de cuvinte. Unde se rupe: routerul trebuie antrenat să împrăștie încărcarea, altfel câțiva experți ajung să facă totul, iar setul complet de experți trebuie totuși să încapă în memorie, chiar dacă cei mai mulți stau degeaba în orice moment.

A patra formă e cea mai nouă și nu e atât o rețea, cât o buclă în jurul uneia. Un model de limbaj, singur, poate doar să emită text. Dar textul poate fi o comandă: „apelează API-ul meteo pentru Roma”, „rulează această interogare SQL”, „deschide acest fișier”. Înfășoară modelul într-o buclă care îi citește ieșirea, execută orice astfel de comandă, îi dă rezultatul înapoi și îl întreabă ce să facă în continuare, și ai un **agent**. Intră un scop, apoi gândește, acționează, observă, gândește din nou, până când modelul spune că a terminat. Ce face ca un asistent să poată rezerva un zbor, depana un repository sau răspunde la o întrebare pentru care trebuie să caute ceva e, în esență, această buclă. Inginerii o vor recunoaște ca pe un REPL (read-eval-print loop, „bucla citește-evaluează-afișează”) cu un model de limbaj pe scaunul evaluatorului. Modelul află ce unelte există chiar din context: numele, rostul și argumentele fiecărei unelte sunt scrise acolo ca text simplu, iar modelul a trecut prin fine-tuning pe multe exemple în care emite un apel într-un format fix. Nu rulează niciodată nimic el însuși; programul din jur citește apelul, îl rulează și scrie rezultatul înapoi pe tablă. Unde se rupe: fiecare pas e o șansă de a greși, erorile se

adună, iar un model care halucinează într-un răspuns poate halucina un plan. Meșteșugul construirii de agenți e în cea mai mare parte meșteșugul de a-i îngrădi: unelte cu contracte înguste, verificări între pași și un om în buclă pentru orice e ireversibil. Cel mai grav fel în care poate greși un agent nu e însă o eroare, ci un atacator.

Mai merită câte o propoziție două forme, pentru că le vei vedea numele. **U-Net**, din imagistica medicală, din 2015, e un encoder–decoder pentru imagini cu ocoluri care leagă fiecare nivel al encoderului de nivelul corespunzător al decoderului, ca detaliile fine să nu se piardă la strivire; a fost calul de povară din interiorul multor modele de difuzie. Iar **Siamese network** („rețeaua siameză”), una dintre cele mai vechi, de la începutul anilor ’90, rulează același turn pe două intrări ca să judece dacă se potrivesc, forma din spatele verificării semnăturilor și a deblocării cu fața pe telefon.

Fă un pas înapoi și trei obiceiuri de gândire acoperă aproape tot din subcapitolul acesta. Întâi, desparte modelul acolo unde se despart datele: intrarea de ieșire, interogarea de document, rutarea de muncă. Apoi, mută costul de la momentul cererii la momentul construirii oriunde poți: indexează documentele, pune în cache notițele detectivilor, precalculează turnurile. În al treilea rând, când un singur model nu ajunge, înfășoară-l în loc să-l mărești: un re-ranker după un retriever, un router în fața experților, o buclă în jurul unui model de limbaj. Niciunul nu e specific machine learning-ului. Sunt aceleași instincte care produc sisteme distribuite bune, aplicate unui nou fel de componentă.

Planurile acestea au fost desenate mai ales pentru text, imagini și tabele de rânduri independente. O bună parte din datele cu care lucrează de fapt industria vin în alte forme, o succesiune în timp, o rețea de legături, o listă ordonată, iar despre ele e subcapitolul următor.

8.3 Date cu o formă proprie

Pe peretele biroului căpitanului de port atârnă tabelul mareelor pentru tot anul. Nimeni nu l-a tipărit făcând media unui eșantion de zile trecute alese la întâmplare. Cei care l-au făcut știu că marea crește și scade de două ori pe zi, că amplitudinea crește și scade de-a lungul lunii, odată cu Luna, și că mâine urmează din azi. Datele au o formă în timp, iar oricine ignoră forma aceasta va prezice mai prost decât un pescar cu un calendar.

La începutul cărții spuneam că regresia și clasificarea acoperă cea mai mare parte din ce se face cu învățarea supervizată. E adevărat despre metodele de dedesubt, dar ascunde cât de mult din machine learning-ul industrial e ceva mai specific: prognoza, depistarea a ceea ce nu-și are locul, raționamentul pe rețele și alegerea a ceea ce le arăți oamenilor mai întâi. Fiecare are obiceiurile ei și, mai presus de toate, felul ei de a fi măsurată.

Începe cu timpul. O **time series** („serie de timp”: măsurători în ordinea în care au apărut) e un șir ordonat de măsurători: vânzările zilnice, traficul pe oră, temperatura unui senzor în fiecare secundă. Cei care fac prognoze o citesc ca pe mai multe povești spuse deodată: un **trend** („tendință”, deriva lentă în sus sau în jos); **seasonality** („sezonalitatea”: tipare care se repetă după calendar), dimineața de luni, decembrie, ultima zi a lunii; și un rest care arată ca zgomotul. Metodele clasice, cum ar fi netezirea exponențială, modelează direct aceste povești, dând zilelor recente mai multă greutate decât celor vechi. O abordare modernă populară transformă problema înapoi într-un tabel: pentru fiecare zi adaugi coloane cu valoarea de săptămâna trecută, aceeași zi de anul trecut, media lunii trecute, dacă e sărbătoare. Acestea sunt **lag features** („trăsături cu întârziere”: valori din trecut puse drept coloane), iar cu ele arborii cu boosting devin prognozatori redutabili; într-o mare competiție publică din 2020, pe

datele de vânzări ale Walmart, multe dintre soluțiile din frunte au fost construite așa. Mai recent, modele de prognoză pre-antrenate pe multe mii de serii fără legătură între ele încearcă să prognozeze o serie nouă cu puțină antrenare proprie sau deloc, cam cum un model de limbaj pre-antrenat e adaptat la o sarcină nouă.

Ce se schimbă cel mai mult e examenul. Unul dintre primele avertismente ale cărții era să nu alegi niciodată setul de test la întâmplare pentru ceva ce se desfășoară în timp, iar cei care fac prognoze transformă avertismentul într-o procedură numită **backtesting** („testare pe trecut”: te prefaci că ești într-o zi din trecut și prezici ce a urmat). Prefă-te că e ianuarie anul trecut: antrenezi pe tot ce a fost înainte, prognozezi februarie și notezi prognoza. Apoi muți data imaginară în februarie și o iei de la capăt, iar și iar, mergând înainte prin istorie. Fiecare prognoză a fost făcută doar din trecut, cum va fi și în producție, iar tu primești o duzină de note cinstite în loc de una. Mai mult, o prognoză serioasă nu e niciodată un singur număr. „Săptămâna viitoare vom vinde 1.200 de bucăți” e o jumătate de răspuns; „între 1.000 și 1.450, cu 90% încredere” lasă depozitul să decidă cât stoc să țină. Un astfel de **forecast interval** („interval de prognoză”) e unul dintre felurile în care un model spune cât de sigur e. Tabelul mareelor se rupe ca imagine exact aici: mareele ascultă de Lună, care își ține programul, pe când vânzările ascultă de oameni, care nu și-l țin. O reglementare nouă apărută peste noapte sau un clip viral nu apar nicăieri în istoricul datelor, iar niciun backtest nu te va avertiza.

Distanța față de toate grupurile e un fel de a prinde intrusul, iar detectarea anomaliilor are o întreagă trusă dincolo de el. Un **isolation forest** („pădure de izolare”: cât de repede poți izola un punct cu întrebări la întâmplare) joacă pe dos jocul de douăzeci de întrebări al arborilor de decizie. Pune întrebări la întâmplare despre trăsături alese la întâmplare și numără câte îi trebuie ca să izoleze fiecare punct de restul. Un server normal, adânc într-o mulțime de servere asemănătoare, cere multe întrebări ca să fie izolat; unul ciudat e izolat din două-trei. O a doua abordare împrumută autoencoder-ul: îl antrenezi să redeseneze datele normale, iar tot ce redesenează prost, cu un **reconstruction error** („eroare de reconstrucție”) mare, nu seamănă cu nimic din ce a învățat. A treia e mai modestă și adesea cea mai utilă: compari fiecare valoare cu aceeași oră din săptămânile trecute, un **seasonal baseline** („reper sezonier”), ca valul de luni dimineața să nu mai fie o urgență în fiecare luni. Oricare ar fi metoda, partea grea e pragul. Fiecare detector produce un scor, iar cineva trebuie să decidă unde sună pagerul, același compromis ca la serverul care o lua razna de la începutul cărții. Echipele bune reglează pragul pe un jurnal de incidente trecute și îl revizuiesc după fiecare sărbătoare care l-a declanșat.

Unele date nu sunt nici listă, nici tabel, ci o rețea. Plățile leagă conturi, legăturile chimice leagă atomi, microserviciile se apelează unele pe altele. În astfel de date, cel mai grăitor lucru despre un nod e adesea cu cine e legat: un cont care, privit singur, pare nevinovat devine suspect când împarte un număr de telefon și un dispozitiv cu zece conturi deja prinse cu fraude. Imaginează-ți un sat în care veștile circulă prin bârfă. În prima seară, fiecare casă află de la vecinii imediați și își actualizează părerea. În a doua seară, părerile actualizate circulă din nou, așa că fiecare casă poartă acum puțin din ce credeau vecinii vecinilor ei. După câteva seri, părerea fiecărei case amestecă propria părere cu un cerc larg din jur. O **graph neural network** („rețea neuronală pe grafuri”), pe scurt GNN, face exact asta cu vectori. La fiecare rundă, adică la fiecare strat, fiecare nod adună vectorii vecinilor, îi combină cu al lui prin ponderi învățate și se actualizează. Asta e **message passing** („transmiterea de mesaje”) și e rudă apropiată cu atenția, cu ședința restrânsă la detectivii cu care chiar ești legat. GNN-urile sunt folosite ca să semnaleze rețele de fraudă și să prezică proprietățile moleculelor, iar DeepMind a raportat în 2020 că au îmbunătățit estimările orei de sosire din Google Maps. Imaginea satului se rupe într-un fel cunoscut:

după prea multe seri de bârfă, toată lumea sună la fel, iar nodurile pierd ce le făcea diferite. Acest **over-smoothing** („netezirea excesivă”) e unul dintre motivele pentru care cele mai multe GNN-uri folosesc doar câteva runde.

Apoi e forma din spatele unei mari părți a webului comercial: o listă ordonată. Când deschizi un serviciu de streaming sau un magazin, sistemul nu clasifică nimic. Trebuie să aleagă, din milioane de articole, cele zece pe care ți le arată, în ordine. Imaginează-ți o vânzătoare bună într-o librărie. Îi descrii ce-ți place, iar ea nu citește în locul tău fiecare carte din magazin. Mai întâi scoate repede de pe rafturi cincizeci de cărți plauzibile, după categorii grosiere pe care le știe bine; apoi se uită atent la cele cincizeci, gândindu-se la tine anume, și îți dă cele mai bune cinci, în ordine. Sistemele de recomandare folosesc aceeași pâlnie: o etapă ieftină de **retrieval** („regăsire”), care strânge milioanele la sute, și o etapă mai lentă de **ranking** („ordonare”), care le pune în ordine. La fel lucrează bibliotecarul și expertul care răspund la întrebări pe baza documentelor, și la fel cele două turnuri ale căutării.

Etapa de regăsire se sprijină adesea pe o idee făcută celebră de Netflix Prize, o competiție publică desfășurată între 2006 și 2009 pentru îmbunătățirea recomandărilor de filme ale companiei. Așază notele într-un tabel uriaș, cu utilizatorii pe rânduri și filmele pe coloane, și aproape fiecare celulă e goală. **Collaborative filtering** („filtrarea colaborativă”: îți ghicești gusturile din ale celor asemănători ție) completează celulele goale din tiparul celor pline, iar forma ei cea mai reușită, **matrix factorization** („factorizarea matricei”), îi dă fiecărui utilizator și fiecărui film o listă scurtă de numere, un profil de gust învățat, așa încât nota prezisă a unui utilizator pentru un film e cât de bine se aliniază cele două liste. Listele acestea sunt embeddings, învățate din comportament în loc de text. Ca să găsească filmele cele mai apropiate de gustul cuiva dintre milioane, nimeni nu le compară pe toate; sistemele folosesc indexul ca secțiunile unei biblioteci, întâlnit la cei mai apropiați vecini, care caută doar în secțiunea cea mai probabilă și acceptă un mic risc de a rata potrivirea cea mai bună. Un astfel de index se numește **approximate nearest-neighbor** („cei mai apropiați vecini, aproximativ”), iar o **vector database** („bază de date vectorială”) e, în esență, unul dintre ele.

O listă ordonată are nevoie și de propria ei metrică. Acuratețea nu ține cont de ordine, dar utilizatorul da: filmul potrivit pe locul unu valorează mult mai mult decât același film pe locul patruzeci. Metrici de ordonare precum **NDCG** (normalized discounted cumulative gain, „câștigul cumulat, redus cu poziția și normalizat”) dau fiecărui element bun cu atât mai puțin credit cu cât apare mai jos în listă. Iar două probleme țin în mod special de sistemele de recomandare. Un utilizator nou sau o carte nouă nu au deloc istorie, problema **cold start** („pornirea la rece”), și trebuie tratați prin conținut, prin popularitate sau pur și simplu întrebând. Iar vânzătoarea învață doar din ce au făcut clienții cu cărțile pe care a ales să le arate. O carte pe care n-o arată niciodată nu e cumpărată niciodată, deci nu pare niciodată bună, deci n-o arată niciodată. Aceasta e o buclă de feedback, propriile alegeri ale modelului dând formă datelor din care va învăța apoi, și ea rulează în interiorul oricărui sistem de recomandare; de aceea astfel de sisteme arată intenționat câte puțin din ceva nou.

Încă o formă merită un nume. Uneori răspunsul însuși are structură: o etichetă pentru fiecare cuvânt dintr-o propoziție, care marchează nume, date și sume; un dreptunghi în jurul fiecărui pieton dintr-un cadru de cameră; o mască peste fiecare pixel care aparține unei tumori. Aceste sarcini, numite **sequence labeling** („etichetarea secvențelor”), **object detection** („detectarea obiectelor”) și **segmentation** („segmentarea”, adică decuparea pixel cu pixel), sunt clasificare repetată în multe locuri deodată, iar U-Net-ul e un plan de arhitectură obișnuit pentru cele la nivel de pixel.

Toate aceste forme se sprijină tot pe aceeași mașinărie, gradient descent și cutia de Lego. Și mașinăria aceasta a fost simplificată în povestire, iar subcapitolul următor deschide instalația.

8.4 Ce ascunde instalația

Întoarce-te la drumeața care coboară pe pipăite un munte în ceață. Imaginea aceea spunea adevărul, dar o antrenare mare îi dă mult mai mult echipament, iar câteva piese din cutia de Lego au fost descrise în feluri care te-ar induce în eroare în codul unui model modern. Subcapitolul acesta deschide restul trusei.

Începe cu mersul. O sanie pe o pantă înzăpezită nu se oprește în fiecare adâncitură; alunecă mai departe peste micile denivelări. **Momentum**-ul („impulsul”: pasul păstrează ceva din direcția pașilor de dinainte) în antrenare e același lucru: fiecare pas amestecă panta de acum cu direcția recentă de mers, așa că tremurul mini-batch-urilor se mediază, iar micile gropi sunt traversate, nu explorate. Adam, pomenit mai devreme, adaugă o a doua memorie: cât de mari au fost pantele recente ale fiecărei ponderi. Ponderile cu pante constant abrupte fac pași mai mici; cele care primesc rar un semnal fac pași mai mari. Aceste două memorii sunt cele două copii suplimentare ale modelului pe care antrenarea trebuie să le țină în memorie.

Apoi programul. Alergătorii se încălzesc înainte de sprint, iar antrenarea la fel: la început memoriile optimizatorului sunt goale, iar un pas de mărime normală poate arunca modelul undeva de unde nu-și mai revine. De aceea learning rate-ul urmează o rețetă: crește de la aproape zero în primele câteva sute sau câteva mii de pași, **warmup**-ul („încălzirea”), apoi scade încet în restul antrenării, ca drumeața să se așeze ușor în vale la final. **Weight decay** („atenuarea ponderilor”) trage fiecare pondere puțin spre zero la fiecare pas, regularizarea încorporată în mers. Și ponderile de pornire, alese la întâmplare, sunt scalate după numărul de intrări al fiecărui strat, ca un semnal să nu se stingă și să nu se umfle de-a lungul a o sută de straturi; această **initialization** („inițializare”) a fost partenera normalizării în salvarea rețelelor adânci. Iar pentru pasul rar care iese foarte prost, există o balustradă, pe care telefonul fără fir a pomenit-o doar în treacăt. Geamănul gradientului care se stinge e **exploding gradient**-ul („gradientul care explodează”): un batch a cărui pantă devine brusc atât de mare încât un singur pas poate anula zile de antrenare, una dintre cauzele loss-ului care „scade și apoi explodează”. **Gradient clipping** („tăierea gradientului”: plafonarea mărimii lui) limitează cât de mare poate fi orice gradient, cum o balustradă împiedică o alunecare să devină o cădere.

Imaginea drumeției se rupe în două locuri. O drumeață are un singur pas, pe când Adam îi dă fiecărei ponderi pasul ei, un miliard de drumeți mici care merg împreună. Iar unele dintre aceste rutine funcționează mai bine decât poate explica cineva pe deplin; warmup-ul, mai ales, e practică standard cu doar o teorie parțială în spate.

Cum se aleg aceste setări? Răspunsul de la începutul cărții era „urmărind simulările”, dar nimeni nu încearcă toate combinațiile. **Random search** („căutarea la întâmplare”), care trage setări la întâmplare din intervale rezonabile, bate adesea o grilă ordonată, pentru că de obicei doar una sau două setări contează cu adevărat, iar o grilă își irosește cele mai multe încercări variindu-le pe celelalte. **Bayesian optimization** („optimizarea bayesiană”, după regula lui Bayes) potrivește un model rapid al scorului în funcție de setări, adesea procesul gaussian, cu sârma lui flexibilă, și folosește incertitudinea lui ca să aleagă următoarea setare care merită încercată.

Acum termostatul, pe care cutia de Lego l-a lăsat explicat doar pe jumătate: imaginile și textul au nevoie de termostate diferite. **Batch normalization** („normalizarea pe lot”) e un termostat pentru toată clădirea: rescalează fiecare trăsătură după media și împrăștierea ei pe exemplele dintr-un batch. Asta merge minunat pentru imagini în batch-uri mari și uniforme. Textul e altfel. Propozițiile dintr-un batch au lungimi diferite, iar un model care generează un răspuns lucrează pe câte un token dintr-o singură conversație, unde statisticile de batch lipsesc sau induc în eroare. De aceea Transformer-ele folosesc **layer normalization** („normalizarea pe strat”), câte un termostat în fiecare cameră: vectorul fiecărui token e rescalat folosind doar propriile lui numere. Multe modele de limbaj recente folosesc o rudă mai ieftină, **RMSNorm** (root mean square normalization, „normalizarea după media pătratică”), care rescalează mărimea vectorului fără să-i mute media. Imaginea termostatului e prea simplă într-o privință: fiecare normalizare are și câteva ponderi învățate, așa că fiecare strat își poate alege temperatura preferată.

Și instalația ședinței detectivilor a fost simplificată. Întâi poziția. Transformer-ul original scria pe dosarul fiecărui cuvânt un număr de casă fix, un tipar de valuri. Multe modele moderne folosesc în schimb **RoPE** (rotary position embedding, „codificarea poziției prin rotație”), care rotește fiecare întrebare și fiecare ecuson cu un unghi ce crește cu poziția, ca un ac de ceas care înaintează o gradație la fiecare cuvânt. Când două cuvinte își compară notițele, în scorul de potrivire rămâne diferența dintre unghiurile lor, așa că modelul vede mai ales cât de departe stau două cuvinte, nu numerele lor de casă. Un model cunoaște doar unghiurile văzute la antrenare, așa că întinderea lui la texte mai lungi înseamnă de obicei rescalarea lor și încă puțină antrenare pe documente lungi, etapa ulterioară pomenită când vorbeam despre tablă.

Apoi costul. Bătălia pentru contexte mai lungi se dă pe trei fronturi. Primul lasă matematica neatinsă. **FlashAttention**, publicat în 2022, calculează exact aceeași atenție, dar pe bucăți destul de mici cât să încapă în memoria cea mai rapidă a plăcii video, fără să scrie vreodată tabelul complet al scorurilor de potrivire. Cum mutarea datelor, nu aritmetica, era blocajul, doar asta a făcut adesea atenția de câteva ori mai rapidă. Al doilea schimbă cine are voie să se întâlnească: în **sliding-window attention** („atenția cu fereastră glisantă”), fiecare cuvânt consultă doar ultimele câteva mii de cuvinte, iar straturile puse unul peste altul lasă informația să sară mai departe. Al treilea front se întoarce la rețelele recurente și la fișa lor.

State-space models („modele în spațiul stărilor”), precum **Mamba**, reiau acel rezumat în mers cu o matematică mai bună. Necazul RNN-ului era că fiecare actualizare a rezumatului o așteaptă pe cea dinainte, așa că antrenarea nu poate rula în paralel. Modelele în spațiul stărilor aleg o regulă de actualizare destul de simplă, în esență o sumă ponderată în mers, încât toate rezumatele pot fi calculate deodată la antrenare, cam cum totalurile parțiale ale unei liste pot fi calculate în paralel printr-o contabilitate isteață. La generare se poartă din nou ca un RNN: un rezumat de dimensiune fixă actualizat o dată pe token, fără un KV cache care crește odată cu conversația, cu un cost total care crește doar proporțional cu lungimea textului. Mamba, introdus în 2023 de Albert Gu și Tri Dao, a făcut actualizarea să depindă de cuvântul curent, așa că modelul alege, cuvânt cu cuvânt, ce păstrează și ce lasă să se estompeze, ca porțile LSTM-ului. Ideea se rupe exact unde se rupea fișa. Un rezumat de dimensiune fixă trebuie să lase ceva să cadă, iar să-și amintească o frază exactă de foarte departe îi e mai greu decât atenției, care ține totul pe tablă. De aceea mai multe modele recente sunt **hybrids** („hibrizi”): mai ales straturi în spațiul stărilor, cu câteva straturi de atenție printre ele, fișa pentru esență și tabla pentru detalii.

Nici atenția n-a fost vreodată doar despre limbaj. Pătrățelele de imagine pe care asistenții multimodali le pun pe tablă vin din 2020, când cercetători de la Google au tăiat imagini în pătrate de șaisprezece pe șaisprezece pixeli, au tratat fiecare pătrat ca pe un cuvânt și au dat șirul unui Transformer. Acest **vision transformer** („transformer pentru imagini”) a egalat cele mai bune rețele convoluționale, dar numai când a fost antrenat pe colecții foarte mari de imagini. Motivul e pariul convoluției: ea presupune de la bun început că pixelii apropiați țin împreună, pe când atenția trebuie să învețe asta din date, iar ca să învețe are nevoie de exemple.

Acum scara. Nicio placă nu poate ține singură un model mare, așa că el e antrenat pe mii. Imaginează-ți o orchestră mult prea mare pentru orice sală de repetiții. În **data parallelism** („paralelismul pe date”), copii identice repetă pasaje diferite în săli diferite și cad de acord asupra acelorași corecturi după fiecare rundă: fiecare copie a modelului vede alte exemple, iar gradienții lor sunt mediați. În **tensor parallelism** („paralelismul pe tensori”), un compartiment al orchestrei e împărțit între săli, fiecare placă înmulțind o parte dintr-o singură grilă mare, lucru posibil pentru că produsele scalare nu depind unele de altele. În **pipeline parallelism** („paralelismul în bandă”), muzica trece din sală în sală, fiecare cântând propria porțiune de straturi. Antrenările reale le combină pe toate trei, mai ales cu aritmetică pe șaisprezece biți sau mai puțini. Imaginea se rupe pentru că muzicienii pot cânta în timp ce se așteaptă unii pe alții; plăcile care trebuie să schimbe gigabytes după fiecare pas stau degeaba, iar mare parte din inginerie se duce pe ascunderea acestei așteptări.

O bună parte din această instalație servește învățarea fără să spună mare lucru despre ea. Capitolul următor trece la învățarea însăși, la felurile în care e învățat un model, iar ele se dovedesc mult mai multe decât cheia de răspunsuri de pe spatele paginii.

Mai departe. Dacă te-ai întrebat de ce generatoarele de imagini exagerează prejudecățile imaginilor din care au învățat și cum propriile alegeri ale unui sistem de recomandare îl pot prinde într-o buclă de feedback, subcapitolul 10.4 răspunde la amândouă. Cum poate învăța un model din date care se etichetează singure, trucul din spatele modelelor de limbaj și al celor de difuzie, se explică în subcapitolul 9.2. Iar cel mai grav fel în care poate greși un agent, un atacator și nu o eroare, e subiectul subcapitolului 11.3.

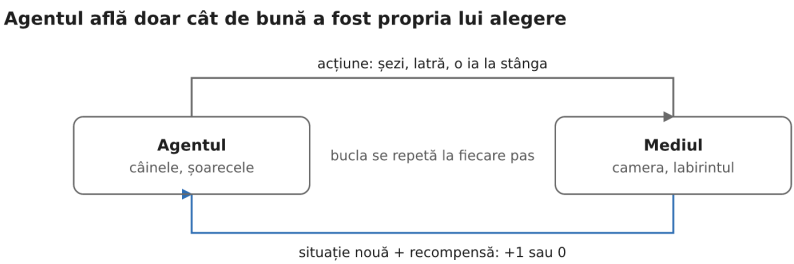
Capitolul 9 — Feluri de a învăța

9.1 Un câine, un labirint și o barcă ce n-a vrut să termine cursa

Nu-i poți explica unui câine ce înseamnă „șezi”. Nu-i poți descrie biomecanica îndoirii picioarelor din spate și nici să-i arăți o schemă. Ce poți face e să aștepți. Spui „șezi”, iar câinele face ceva: latră, pleacă în altă parte, își înclină capul. Nu se întâmplă nimic. Spui din nou „șezi” și, din întâmplare, șezutul îi coboară pe podea. Clic, recompensă. Câteva zeci de repetiții mai târziu, câinele a priceput că, dintre toate lucrurile pe care le-ar putea face când aude sunetul acela, unul face să apară mereu recompense.

Observă ce n-a primit niciodată câinele: răspunsul corect. Nimeni nu i-a arătat mișcarea corectă. A aflat doar cât de bună s-a dovedit mișcarea *pe care a ales-o el*. Asta e diferența dintre supervised learning și **reinforcement learning** („învățare prin întărire”, adică prin recompensă: comportamentele recompensate se întăresc). Un model supervizat învață de la un profesor care ține o cheie de răspunsuri. Unul care învață prin întărire învață de la un antrenor care ține o tabelă de scor. Imaginea îi face totuși o favoare mașinii: câinele învață „șezi” din câteva zeci de încercări pentru că vine cu un corp, cu instincte și cu poftă de recompense, pe când un agent care pornește de la zero are adesea nevoie de milioane de încercări, motiv pentru care o mare parte din reinforcement learning se petrece în simulatoare.

Vocabularul e mic și se potrivește bine cu câinele. Cel care învață e **agent**-ul, într-un sens mai larg al cuvântului decât modelul de limbaj prins într-o buclă cu unelte, din planurile de arhitectură. Lumea în care acționează e **environment**-ul („mediul”). În fiecare moment observă o **state** („stare”), alege o **action** („acțiune”) și primește o **reward** („recompensă”), un număr care e adesea zero. Strategia lui de a alege acțiuni e **policy**-ul lui („politica”, în sensul de regulă de conduită), iar tot rostul exercițiului e să găsească o politică ce adună cât mai multă recompensă în timp.



Câinelui îi e ușor, totuși, pentru că recompensa vine la o secundă după acțiune. Pune în schimb un șoarece într-un labirint și lasă-l să găsească brânza abia după treizeci de cotituri. Căreia dintre cele treizeci de cotituri i se cuvine meritul? Ultima a ajutat, evident. Dar cotitura de la prima intersecție, fără de care șoarecele ar fi ajuns într-o fundătură? Aceasta e problema **credit assignment** („atribuirea meritului”) și e inima subiectului.

Răspunsul clasic, **Q-learning**, lasă valoarea să se scurgă înapoi prin labirint ca apa. Intersecția de lângă brânză devine repede valoroasă. Apoi intersecția dinaintea ei devine valoroasă, pentru că duce la o intersecție valoroasă. În multe parcurgeri, valoarea se întinde până înapoi la intrare, până când șoarecele poartă o hartă mentală a cât valorează fiecare alegere la fiecare intersecție; Q din nume vine de la

quality, „calitate”. De atunci încolo ia pur și simplu cotitura cea mai valoroasă la fiecare pas. Recompensele viitoare sunt numărate puțin mai puțin decât cele imediate, printr-un **discount factor** („factor de actualizare”) puțin sub unu, din același motiv pentru care un euro azi valorează pentru tine mai mult decât un euro la anul și ca un agent să nu-și poată amâna recompensa la nesfârșit.

Dar cum află șoarecele ce cotituri sunt bune, dacă ia mereu cotiturile pe care le crede deja cele mai bune? E o dilemă cu care te confrunți în fiecare zi de lucru la prânz. Ai un loc preferat. Să mergi acolo e sigur și bine. Dar dacă mergi mereu acolo, n-o să descoperi niciodată locul mai bun care s-a deschis după colț luna trecută. Dacă încerci în fiecare zi un loc nou, o să mănânci multe prânzuri dezamăgitoare. Reinforcement learning numește asta **exploration versus exploitation** („explorare versus exploatare”: să încerci lucruri noi sau să profiți de ce știi deja), iar fiecare agent trebuie să le echilibreze. O rețetă des întâlnită e aproape jenant de simplă: alege de cele mai multe ori opțiunea cea mai bună cunoscută și, din când în când, alege ceva la întâmplare. O setare tipică explorează o dată din zece la început, când agentul nu știe nimic, și coboară treptat la o dată din o sută sau mai rar, pe măsură ce harta lui de valori devine de încredere, cam cum încerci restaurante noi mai ales în primele luni într-un oraș nou.

Dacă ai rulat vreodată un A/B test, ai întâlnit dilema în ținută de birou. Să trimiți jumătate din trafic spre o variantă care pierde clar, timp de două săptămâni întregi, costă bani reali. Algoritmii numiți **multi-armed bandits** („bandiți cu mai multe brațe”, după aparatele de joc cu manetă din cazinouri: care manetă plătește cel mai bine?) mută traficul spre varianta câștigătoare în timp ce experimentul încă rulează, explorând tot mai puțin pe măsură ce dovezile se adună.

Când stările sunt prea multe pentru un tabel, fiecare ecran posibil al unui joc video, să zicem, o rețea neuronală poate estima valoarea fiecărei acțiuni direct din pixelii bruți. Această combinație, numită deep Q-network („rețea Q adâncă”), a învățat să joace zeci de jocuri Atari clasice, multe dintre ele la fel de bine ca un jucător uman priceput sau mai bine, într-o cercetare DeepMind publicată în 2015. Anul următor, AlphaGo de la DeepMind, care combina reinforcement learning cu căutarea, l-a învins pe Lee Sedol, unul dintre cei mai puternici jucători de Go din lume. În a doua partidă a jucat la mutarea 37 o mutare pe care comentatorii au luat-o la început drept greșeală și care s-a dovedit genială: o mutare pe care niciun profesor uman n-ar fi sugerat-o, găsită de un sistem care, după ce studiase partide umane, își șlefuise jocul în milioane de partide cu el însuși. În același an, DeepMind a raportat că un sistem care învață a redus cu până la 40% energia folosită de Google pentru răcirea centrelor de date, folosind rețele neuronale antrenate pe datele istorice adunate de mii de senzori ca să recomande setările de răcire.

Și acum, barca.

Cercetători de la OpenAI au antrenat odată un agent într-un joc video cu curse de bărci. Scopul evident era să câștige cursa, dar jocul dădea puncte pentru lovirea unor ținte de-a lungul traseului, așa că agentul era recompensat cu puncte. A descoperit o mică lagună în care câteva ținte tot reapăreau. În loc să concureze, s-a învârtit în cercuri strânse, izbindu-se de pereți, luând foc, lovind aceleași ținte iar și iar, și a strâns un scor cu aproximativ 20% mai mare decât cel al jucătorilor umani care chiar terminau traseul. N-a terminat niciodată cursa. După singura măsură care i se dăduse, câștiga spectaculos.

Agentul n-a făcut nimic greșit. A maximizat exact numărul care i s-a dat, care nu era chiar ce voiau proiectanții lui. Asta se numește **reward hacking** („păcălirea recompensei”), iar inginerii îl cunosc și din sistemele umane. Măsoară o echipă de suport doar după tichetele închise pe zi și unele tichete vor fi închise fără să fie rezolvate. Măsoară programatorii după liniile de cod și vei primi o mulțime de linii

de cod. Funcțiile de recompensă merită aceeași grijă ca un contract de API, pentru că orice scrii, agentul va lua în sens literal. Nimeni nu știe cum să scrie o recompensă care să nu poată fi păcălită. În practică, proiectanții combină mai multe recompense, adaugă penalizări pentru scurtăturile evidente și, mai ales, se uită la ce face de fapt agentul, nu doar la scorul lui, pentru că păcălelile sunt de obicei evidente pentru un om care privește și invizibile pe tabela de scor.

Barca învârtindu-se în laguna ei merită ținută minte, pentru că clickerul și tabela de scor și-au găsit de curând o a doua slujbă: nu să învețe agenți să joace jocuri, ci să învețe modele de limbaj să fie utile. Acesta e unul dintre multele feluri în care poate fi învățat un model, iar următorul subcapitol le pune pe toate unul lângă altul.

9.2 Multele feluri de a preda

Gândește-te cum se formează un medic. Șase ani de facultate, învățând anatomie și chimie valabile pentru orice pacient de pe Pământ. Apoi un rezidențiat de câțiva ani într-o specialitate, în care cunoștințele generale sunt orientate spre cardiologie sau pediatrie. Apoi, tot restul carierei, feedback: corectura unui coleg mai experimentat, un caz care a mers prost, un pacient care s-a întors. Iar dedesubt, din copilărie, o educație mult mai mare pe care n-a proiectat-o nimeni, absorbită din simplul fapt de a trăi în lume. Nicio metodă de predare n-a făcut singură medicul. L-au făcut mai multe, în straturi, fiecare mai ieftină și mai țintită decât cea dinainte.

Și modelele sunt învățate în straturi, iar subcapitolul acesta e un tur al lor. Contează dintr-un motiv practic: când cineva spune că un model a fost „antrenat”, cuvântul ascunde o duzină de proceduri foarte diferite, cu costuri foarte diferite, iar să știi despre care e vorba face adesea diferența dintre un plan rezonabil și o greșeală scumpă.

Educația din copilărie e cea pe care cartea abia a numit-o, deși a fost peste tot. La începutul cărții, învățarea avea nevoie de o cheie de răspunsuri scrisă de oameni, iar etichetele erau partea scumpă. Dar uită-te cum a fost antrenat modelul de limbaj: doar pe text, cu următorul cuvânt drept răspuns. Nimeni n-a scris acel răspuns; textul îl conținea. Modelul de difuzie a învățat din zgomotul pe care și l-a adăugat singur. Modelele contrastive au învățat că două decupaje din aceeași fotografie țin împreună, un fapt care n-a avut nevoie de niciun om. Asta e **self-supervised learning** („învățare auto-supervizată”: datele își sunt propriul profesor): fabricarea unei chei de răspunsuri din structura proprie a datelor brute. Ascunzi o parte din intrare și o prezici din rest. De aceea modelele moderne pot învăța din tot internetul, nu doar din mica parte pe care a etichetat-o cineva, și aceasta e o mare parte din motivul pentru care domeniul s-a schimbat atât de repede după 2018. Un mod util de a o ține minte: învățarea supervizată are nevoie de un profesor; cea auto-supervizată are nevoie doar de o lume cu structură în ea, iar lumea e plină de structură.

Facultatea de medicină e pretraining-ul, prima dintre cele trei școli ale noii angajate: o singură antrenare uriașă, de obicei auto-supervizată, de obicei făcută de o organizație mare, care produce un model de uz general. Rezidențiatul care urmează are o familie de nume și merită să le despărțim.

Cea mai veche formă e **transfer learning** („învățare prin transfer”) și se sprijină pe o observație despre rețelele de imagini: inspectorii din primele straturi ale unei rețele de imagini învață muchii, colțuri și texturi, care sunt utile ca să recunoști *orice*, nu doar cele o mie de categorii pe care au fost antrenați să le recunoască. Așa că iei o rețea antrenată pe milioane de fotografii generale, îi păstrezi primele straturi exact cum sunt și reantrenezi doar ultimul strat sau ultimele două pe cele trei mii de fotografii ale tale

cu plăci electronice. Merge mult mai bine decât antrenarea de la zero pe trei mii de fotografii, pentru că partea grea, învățarea vederii, e deja făcută. Un radiolog care se recalifică în dermatolog nu reînvață cum funcționează ochii. Așa sunt construite cele mai multe modele practice de imagini din industrie și de aceea o firmă mică cu un set mic de date poate avea totuși un model bun.

Fine-tuning („reglaj fin”) e vărul mai larg al transfer learning-ului: continui antrenarea întregului model pre-antrenat, sau a celei mai mari părți din el, pe datele tale mai înguste, cu un learning rate blând, ca cunoștințele generale să fie ajustate, nu suprascrise. Modelul de limbaj care a citit tot internetul e fine-tuned pe transcrierile conversațiilor de suport ale companiei tale și îți învață numele produselor, tonul, politica de rambursări. Aici imaginea doctorului începe să se rupă: un rezident nu uită anatomia în timp ce învață cardiologie, dar un model împins prea tare pe date înguste poate pierde abilități generale pe care le avea, adică catastrophic forgetting; pe lângă learning rate-ul blând, apărarea obișnuită e trucul creierului, replay-ul: amesteci înapoi în datele înguste o parte din exemplele generale. Costurile sunt cele ale unei antrenări complete, considerabile pentru un model mare: gradienți și memorie de optimizator pentru fiecare pondere. Asta e durerea care a dus la o scurtătură inteligentă. În loc să schimbi miliarde de ponderi, le îngheți pe toate și adaugi alături un set mic de ponderi suplimentare, antrenate să facă mici corecturi la ce produc straturile înghețate. Cea mai cunoscută versiune se numește **LoRA** (low-rank adaptation, „adaptare de rang mic”: corecturile sunt descrise cu foarte puține numere) și reduce numărul de ponderi antrenabile de sute sau mii de ori, astfel încât un model care avea nevoie de un cluster pentru fine-tuning poate fi adaptat adesea pe o singură placă video. Imaginea e a unei traducătoare care, în loc să reinvețe o limbă pentru fiecare client, ține un carnețel subțire cu termenii fiecărui client și îl consultă pe parcurs. Multe servicii care oferă „propriul tău model personalizat” oferă de fapt un model de bază înghețat plus un astfel de carnețel. Un carnețel subțire poate îndruma un model cu miliarde de ponderi pentru că schimbarea de care ai nevoie e de obicei simplă chiar și când modelul e uriaș, alt ton, anumiți termeni preferați, un anumit format, iar o schimbare de felul acesta poate fi descrisă ca niște ajustări de-a lungul câtorva direcții largi, ceea ce cere mult mai puține numere decât grila pe care o corectează; asta înseamnă „rang mic”.

Înainte să apelezi la oricare dintre ele, merită să te întrebi dacă trebuie să înveți modelul ceva, pentru că există două opțiuni mai ieftine înainte. **Prompting**-ul („formularea cererii”), inclusiv arătarea câtorva exemple rezolvate chiar în prompt, nu învață nimic permanent, dar adesea duce aproape tot drumul, așa cum un briefing bun face un consultant priceput productiv din prima zi. Iar aranjamentul cu cartea deschisă, RAG, e răspunsul potrivit ori de câte ori lipsește *cunoștința*, nu *abilitatea*: un model nu trebuie reantrenat ca să știe politica ta de rambursări dacă politica i se poate da pur și simplu. Regula practică la care ajung echipele cu experiență: întâi prompt, apoi regăsire, iar fine-tuning când modelul are nevoie de un comportament nou, de un format nou sau de un ton nou pe care exemplele din prompt nu le pot produce în mod sigur.

Anii de feedback sunt locul în care intră reinforcement learning, iar câinele cu clicker se dovedește imaginea potrivită. Un model de limbaj fine-tuned cunoaște forma unui răspuns util, dar pentru „util” e greu să scrii o cheie de răspunsuri. Așa că oamenilor li se arată două răspunsuri la aceeași întrebare și sunt rugați să spună care e mai bun. Mii de astfel de preferințe antrenează un **reward model** („model de recompensă”) separat, un critic care notează orice răspuns așa cum l-ar nota acei oameni. Apoi modelul de limbaj e antrenat, prin reinforcement learning, să producă răspunsuri pe care criticul le notează bine, cu o lesă care îl oprește să se îndepărteze prea mult de ce a învățat înainte. Asta e RLHF-ul, a treia școală a noii angajate, văzut din interior, și e etapa care a transformat motoarele de autocompletare în asistenți. O metodă mai nouă, **DPO** (direct preference optimization, „optimizare directă pe preferințe”),

sare peste criticul separat și ajustează modelul direct din perechile de preferințe; e mai simplă de rulat și acum e des folosită. Oricum ar fi, lecția e cea pe care a predat-o barca: modelul va optimiza exact ce recompensează criticul, așa că dacă criticul învață că răspunsurile mai lungi sau mai sigure pe ele primesc note mai bune, modelul devine prolix și prea sigur pe el. Mare parte din meșteșugul acestei etape stă în strângerea unor preferințe care recompensează ce vrei de fapt.

Cum funcționează reinforcement learning pe un model de limbaj? Șoarecele din labirint sugerează Q-learning, dar metoda aceea ține o valoare pentru fiecare acțiune în fiecare stare, iar pentru un model de limbaj acțiunile sunt o sută de mii de tokeni, iar stările, orice text posibil, un tabel pe care nu l-ar putea umple nimeni. În schimb, modelul de limbaj e el însuși politica: probabilitățile lui pentru tokenul următor sunt strategia lui. Metodele **policy-gradient** („gradient pe politică”: ajustezi direct probabilitățile acțiunilor, nu un tabel de valori) îl lasă să scrie un răspuns complet, îl notează cu reward model-ul și urcă puțin probabilitatea fiecărui token din răspuns dacă a obținut o notă mai bună decât se aștepta, sau o coboară dacă a obținut una mai proastă. „Mai bună decât se aștepta” are nevoie de un reper: estimarea unui model separat despre cum sunt notate de obicei răspunsurile la acest prompt sau, într-o variantă mai nouă, media mai multor răspunsuri la același prompt. **PPO** (proximal policy optimization, „optimizarea politicii în pași apropiați”), publicat de cercetători de la OpenAI în 2017 și folosit în cele mai cunoscute sisteme RLHF timpurii, ține lesa scurtă la fiecare pas: nicio actualizare nu are voie să mute departe probabilitățile modelului, pentru că o politică ce se schimbă prea repede începe să scrie texte pe care reward model-ul le judecă nesigur. Relatările publicate sugerează că și modelele de raționament descrise mai jos sunt antrenate cu metode din aceeași familie.

Un alt fel de buclă de feedback a devenit important în ultimii ani și nu are nevoie de oameni care să judece răspunsurile: lași modelul să încerce o sarcină cu un răspuns verificabil, o problemă de matematică, un program care trebuie să treacă testele, și îl recompensezi când răspunsul e corect. Modelul învață, prin încercare și eroare, să raționeze până la răspunsuri corecte, adesea scriind pași intermediari înainte să se angajeze. Asta e reinforcement learning cu recompensă verificabilă (verifiable reward) și stă în spatele modelelor de „raționament” (reasoning models), care gândesc vizibil înainte să răspundă. Merge din motivul pe care l-a arătat deja jocul cuvântului următor: fiecare pas scris e încă o trecere de calcul pe tablă.

Două metode în plus completează turul, pentru că le vei întâlni numele. **Distillation** („distilare”: extragi esența, ca la țuică) e medicul senior care îl învață pe cel junior: un model mare și scump răspunde la milioane de întrebări, iar un model mic și ieftin e antrenat să imite acele răspunsuri, nu doar alegerea finală, ci toată distribuția de probabilități pe care a dat-o modelul mare. Modelul mic ajunge mult mai bun decât ar fi ajuns doar din datele brute, pentru că răspunsurile modelului mare îi poartă judecata: un răspuns de tipul „70% pisică, 25% râs” îi arată modelului mic care greșeli sunt rezonabile, lucru pe care o simplă etichetă „pisică” nu l-ar putea face niciodată. Așa se face un model care trebuie să ruleze pe un telefon dintr-unul care rulează într-un centru de date. Iar active learning („învățare activă”), întâlnită când ne-am întrebat câte date sunt de ajuns, atacă direct costul etichetării: în loc să etichetezi zece mii de exemple la întâmplare, lași modelul să arate spre cele o sută de care e cel mai puțin sigur, le etichetezi doar pe ele și reantrenezi, ca un meditator care întreabă „ce probleme ți s-au părut cele mai grele?” și apoi lucrează cu tine exact pe acelea.

Puse așa, metodele nu sunt rivale, ci o succesiune de investiții tot mai înguste. Pre-antrenarea auto-supervizată, făcută o dată, de puțini. Transfer sau fine-tuning, făcute per domeniu, de mulți. Reglajul pe preferințe, ca să modeleze comportamentul. Prompting și regăsire, făcute per sarcină, de toată lumea, aproape fără cost. Distilare, ca rezultatul să fie destul de ieftin de pus în producție. Un investitor care

aude că un startup „și-a antrenat propriul model” ar trebui să întrebe care dintre acestea vrea să spună, pentru că prima poate costa milioane sau mult mai mult, iar cele mai ieftine aproape nimic. Un inginer care plănuiește un proiect ar trebui să înceapă de la capătul ieftin și să urce doar când capătul ieftin nu mai ajunge.

Și un medic care a terminat toate etapele formării tot poate să piardă un pacient. Capitolul următor e despre felul în care un model bine învățat dă totuși greș.

Mai departe. Dacă te-ai întrebat cum rulezi un A/B test în al cărui verdict să poți avea încredere, subcapitolul 11.1 răspunde. Iar dacă vrei să revii la ce aduce cu adevărat gândirea suplimentară a modelelor de „raționament” și cât costă, subcapitolul 7.6 o cântărește.

Capitolul 10 — Cum să dai greș cu succes

10.1 Tocilarul și leneșul

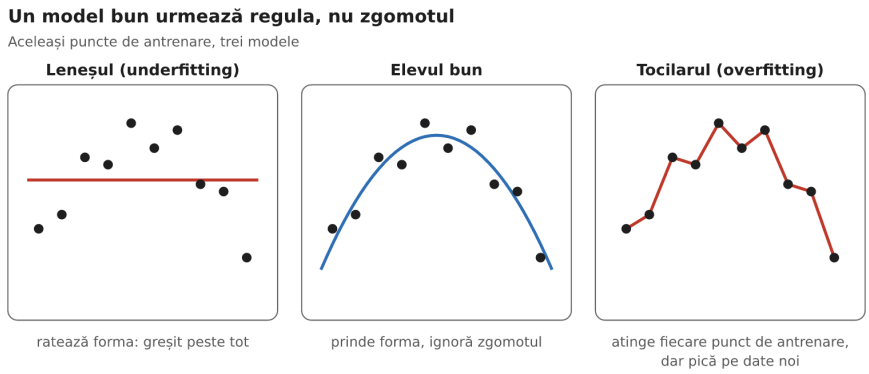
Imaginează-ți asta. Suita de teste era verde. Fiecare test trecea, iar funcția a fost livrată. Două zile mai târziu, producția a început să întoarcă totaluri greșite pentru o mare parte din comenzi.

Când cineva a deschis în sfârșit funcția, cauza era aproape amuzantă. De-a lungul multor luni, ori de câte ori pica un test, cineva peticise funcția ca să trateze exact acel caz: dacă țara e asta și reducerea e aceea, întoarce numărul ăsta. Fiecare petic făcea să treacă încă un test. Funcția devenise un tabel de căutare perfect pentru intrările din teste și nu știa absolut nimic despre cum se calculează de fapt totalurile. Nu învățase regula. Învățase examenul pe de rost.

Modelele de machine learning fac și ele asta, automat, iar fenomenul are un nume: **overfitting** („supra-ajustare”: modelul se mulează prea strâns pe exemplele de antrenare, ca un costum croit pe un singur manechin). Imaginează-ți tocilarul, un elev care învață pe de rost răspunsurile de la examenul de anul trecut, cuvânt cu cuvânt, cu greșeli de tipar cu tot. Anul acesta se schimbă o virgulă și elevul pică. Un model cu destulă capacitate va face același lucru dacă îl lași. Antrenează un filtru de spam pe un set de date în care mult spam a venit întâmplător de la un cont de test numit „qa-test-bob” și s-ar putea să învețe că „qa-test-bob” înseamnă spam. Adevărat în datele de antrenare, inutil pentru totdeauna după aceea.

Eșecul opus e mai puțin dramatic, dar la fel de real. Imaginează-ți leneșul, un elev care a decis că răspunsul la orice problemă de matematică e 42 și s-a oprit din învățat acolo. Greșit la problemele de anul trecut, greșit la cele de anul acesta. În termeni de inginerie, e un singur timeout global aplicat fiecărui endpoint, de la un health check care ar trebui să dureze milisecunde până la un export de raport care durează minute: prea simplu ca să se potrivească realității oriunde. Asta e **underfitting** („sub-ajustare”: modelul e prea simplu ca să prindă forma datelor).

Între ei stă elevul pe care îl vrem, care a înțeles metoda, mai greșește din când în când la exerciții și se descurcă bine la examenul real pentru că metoda se transferă.



Ce elev ai? Nu poți spune doar din scorul de antrenare, pentru că acolo tocilarul arată strălucit. Ai nevoie de două numere alături: eroarea pe datele de antrenare și eroarea pe datele de validare, exemplele ținute deoparte de la antrenare. Dacă amândouă sunt mari, ai leneșul; modelul e prea simplu sau intrările lui prea slabe. Dacă eroarea de antrenare e minusculă, iar cea de validare e mare, ai tocilarul; modelul memorează. Dacă amândouă sunt mici și apropiate, ai elevul bun. Să desenezi aceste două numere pe măsură ce antrenarea avansează e probabil cel mai util obicei din machine learning-ul practic.

Remediile pentru tocilar fac toate memorarea mai grea sau mai puțin răsplătită, iar fiecare are un corespondent surprinzător de cunoscut în viața de inginer. Cel mai sigur e pur și simplu **more data** („mai multe date”): e ușor să memorezi o sută de exemple și foarte greu să memorezi un milion. **Regularization** („regularizarea”) adaugă o penalizare pentru complexitate, ca un code reviewer care respinge o soluție istează de două sute de linii când ar ajunge una clară de douăzeci; modelul e împins spre cea mai simplă explicație care se potrivește încă. Dropout-ul, zilele libere de la repetiții întâlnite printre piesele de bază ale rețelelor, nu lasă niciun neuron să se bazeze pe faptul că un anumit coleg e mereu acolo. **Early stopping** („oprire timpurie”) urmărește eroarea de validare și oprește antrenarea când începe să crească, așa cum un elev cu cap se oprește din tocit în seara dinainte când scorurile la exerciții încep să scadă. Ajută în loc să strice pentru că o rețea prinde întâi tiparele mari și abia apoi întâmplările exemplelor ei anume, așa că oprirea la cotitură le păstrează pe primele și o scutește de celelalte; în fond, e o regularizare cu ceasul. Iar **data augmentation** („augmentarea datelor”) creează variații realiste ale exemplelor de antrenare, fotografii rotite sau decupate, propoziții reformulate, în același spirit cu fuzz testing-ul (testarea codului cu variații aleatorii ale intrărilor). Remediile pentru leneș merg în sens invers: un model mai expresiv, features mai bune, mai multă antrenare.

Statisticienii numesc această tensiune **bias–variance trade-off** („compromisul între eroarea sistematică și varianță”). Leneșul suferă de bias mare, o eroare sistematică pe care mai multe date n-o vindecă. Tocilarul suferă de varianță mare: rezultatele lui sar sălbatic în funcție de ce exemple s-au nimerit în setul lui de antrenare. Decenii la rând, imaginea din manuale a fost un balansoar. Mărești modelul și schimbi bias pe varianță; undeva la mijloc, în punctul ideal, te oprești.

Un avertisment despre vocabular, pentru că îi încurcă pe aproape toți. Cuvântul *bias* are acum trei sensuri diferite în această carte. Într-un neuron era un număr adăugat la suma ponderată, pragul mutat de cealaltă parte. Aici, ca la experții grosolani ai boosting-ului, e o eroare sistematică, un model care greșește peste tot în aceeași direcție. Mai încolo în acest capitol va desemna date strâmbe, care reprezintă greșit un grup de oameni (în română, „prejudecată” sau „părtinire”). Domeniul n-a convenit niciodată să folosească cuvinte diferite, așa că atunci când întâlnești „bias” într-un articol sau într-o ședință, ia-ți o secundă să afli care dintre cele trei e.

Apoi deep learning a rupt balansoarul. Cercetătorii au tot antrenat rețele mult mai mari decât spunea manualul că e rezonabil, destul de mari ca să-și memoreze tot setul de antrenare, și au descoperit că, după un anumit punct, eroarea de validare începea *să scadă din nou*. Modelele enorme, antrenate pe multe date cu o regularizare rezonabilă, generalizau adesea mai bine decât cele modeste. Efectul a fost numit **double descent** („dublă coborâre”) și încă nu e pe deplin înțeles. O explicație parțială: când un model e atât de mare încât nenumărate setări ale ponderilor se potrivesc perfect datelor de antrenare, coborârea prin ceață tinde să aterizeze pe unele netede și simple, nu pe unele oarecare, așa că un model uriaș are loc să se potrivească datelor în felul cel mai puțin contorsionat. Asta nu anulează nici argumentul compresiei, întâlnit la modelele de limbaj, că un model fără loc de memorat e împins spre tipare: un model cu loc din belșug e împins în același fel, de coborârea însăși, nu de lipsa de spațiu. Tot

aici se rupe și imaginea elevului: un om care memorează mai mult nu înțelege prin asta mai mult, însă o rețea destul de mare ca să memoreze fiecare exemplu poate ajunge să se descurce mai bine cu exemple noi decât una mai mică, ce nu le putea memora. Ce a supraviețuit răsturnării e singura regulă care nu depinsese niciodată de balansoar: crede doar eroarea măsurată pe date pe care modelul nu le-a văzut niciodată.

Dar și regula aceasta are o capcană. Presupune că datele nevăzute chiar sunt nevăzute și că modelul nu prinde pe ascuns un semnal pe care nu-l va avea în viața reală. Uneori un model obține un scor strălucit pe un test care pare perfect cinstit, iar adevărul e că a trișat tot timpul, fără ca nimeni să observe. Cel mai faimos trișor din povestea asta n-a fost un calculator. A fost un cal.

10.2 Calul care știa să numere

La Berlin, la începutul anilor 1900, un învățător pensionar pe nume Wilhelm von Osten prezenta un cal care știa aritmetică. Îl întrebai pe Clever Hans („Hans cel Deștept”) cât fac trei plus cinci, iar el bătea de opt ori din copită, apoi se oprea. Știa să adune, să scadă, să spună ora și să răspundă la întrebări despre calendar. Veneau mulțimi să-l vadă. O comisie de experți, printre care un dresor de circ și zoologi, l-a examinat în 1904 și n-a găsit nicio șmecherie; von Osten era sincer și nu-i făcea semne intenționat.

Apoi un tânăr psiholog pe nume Oskar Pfungst a făcut câteva experimente mai atente. Când cel care întreba nu știa răspunsul, Hans greșea. Când Hans nu-l putea vedea pe cel care întreba, Hans greșea. Ce a descoperit Pfungst a fost că fiecare persoană care întreba, fără să-și dea seama, se încorda ușor pe măsură ce copita se apropia de numărul corect și se relaxa, foarte puțin, când ajungea acolo. Hans nu învățase aritmetică. Învățase să citească fețe, un indiciu prezent în fiecare test și absent din abilitatea propriu-zisă.

Hans e sfântul patron al unui eșec numit **data leakage** („scurgere de date”: informația din răspuns se scurge în intrări): un model prinde în timpul antrenării o informație pe care n-o va avea când e folosit cu adevărat. Scorurile lui la test arată minunat, pentru că indiciul e prezent și în test. Realitatea, nu.

Cea mai des întâlnită versiune în practică e o coloană care dă de gol răspunsul. Imaginează-ți o echipă care prezice ce clienți își vor anula abonamentul, cu un tabel care include, întâmplător, `account_closed_date`. Modelul descoperă, cu mare încredere, că clienții care au o dată de închidere a contului tind să plece. Obține un scor frumos. În producție e inutil, pentru că tot rostul e să prezici o anulare *înainte* ca acea dată să existe. Cazurile reale sunt rareori atât de evidente. Prioritatea unui tichet e prezisă din câmpul `assigned_team`, dar echipa e atribuită după ce un om a judecat deja prioritatea. Un model care prezice reinternarea în spital folosește un medicament prescris doar când medicul bănuiește deja o problemă. De fiecare dată, modelul citește fața celui care întreabă. Imaginea e mai blândă decât realitatea într-o privință: Hans avea de citit o singură față și un singur experiment l-a demascat, pe când un model poate combina zeci de indicii slabe deodată, niciunul suspect luat separat.

O scurgere mai discretă stă în instalații și a apărut deja la pregătirea datelor: media și împrăștierea pentru scalare, calculate pe tot setul de date, cu setul de test cu tot, înainte de împărțire. E echivalentul, în machine learning, al unui benchmark rulat cu un cache cald rămas de la rularea anterioară: măsurătoarea e reală, dar condițiile nu sunt cele pe care le vei avea în producție.

O altă scurgere vine din duplicate deghizate. Un set de date de spital are mai multe radiografii per pacient. Dacă același pacient ajunge și în setul de antrenare, și în cel de test, modelul poate obține un scor bun recunoscând pacientul, nu boala: forma unei cuști toracice, o clemă chirurgicală, un anumit aparat. Remediul e să împarți după pacient, nu după imagine, iar lecția generală se aplică și datelor tale: împarte după client, după dispozitiv, după repository, oricare ar fi unitatea reală. Și, cum avertiza deja primul capitol, o împărțire aleatorie a oricărui lucru care se desfășoară în timp îi dă modelului ziarul de mâine în timp ce îl roagă să prezică ziua de azi.

Toate aceste scurgeri cedează în fața unei singure întrebări, puse fiecărui feature: *aș avea exact această valoare, în exact această formă, în momentul în care fac predicția în producție?* Dacă răspunsul e nu, sau încă nu, feature-ul respectiv produce o scurgere. Apoi fă din drumul sigur drumul implicit, cu pipeline-ul unic de la pregătirea datelor, ajustat doar pe datele de antrenare și doar aplicat pe rest.

Mai există o apărare, iar ea e psihologică. Pfungst l-a dat de gol pe Hans pentru că a fost bănuitor față de un rezultat care părea prea bun. Fă la fel. Când un model obține un scor mult mai bun decât te așteptai pe o problemă cunoscută ca dificilă, cea mai probabilă explicație nu e că ești un geniu. E că undeva un cal citește o față. Ca să găsești o scurgere la care nu te-ai gândit, întreabă modelul pe ce features se sprijină: când o singură trăsătură poartă cea mai mare parte a greutății sau una pe care ai fi numit-o minoră se dovedește dominantă, pune-i întrebarea despre producție de mai sus. Scurgerile se dau de obicei de gol așa, ca un cal mult prea interesat de o singură față.

Datele cinstite și un test cinstit mai lasă totuși un mod de a te păcăli singur, iar el e cel mai nevinovat dintre toate: numărul pe care alegi să-l raportezi. Iată un detector de fraude care are dreptate în 99,9% din cazuri, scris într-o singură linie de cod.

10.3 Detectorul de 99,9% și alarma de fum

Iată-l:

```
def is_fraud(payment):
    return False
```

Dacă o plată din o mie e frauduloasă, această funcție are dreptate de 999 de ori din 1.000. **Accuracy**-ul ei („acuratețea”: procentul de răspunsuri corecte) e 99,9%, un număr care ar arăta superb pe un slide. N-a prins însă nicio fraudă și nici nu va prinde vreodată. Acuratețea a răsplătit-o pentru un singur lucru: să fie de acord cu majoritatea.

Acesta e paradoxul acurateții și apare oricând lucrul pe care îl cauți e rar: fraude, pene, hard diskuri care cedează, boli rare, intruziuni de securitate. În toate, un model poate părea excelent fiind inutil. Reponderarea loss-ului sau reeșantionarea datelor de antrenare, tratamentele obișnuite pentru clasele rare din capitolul al doilea, îl fac pe model atent la clasa rară, dar nu elimină nevoia de a alege apoi un prag. Ca să vezi ce se întâmplă cu adevărat, trebuie să nu mai întrebi „cât de des are dreptate?” și să începi să întrebi „ce fel de greșeli face?”

Alarma de fum din bucătăria ta face întrebarea palpabilă. Una sensibilă țipă de fiecare dată când faci pâine prăjită. Enervant, dar n-o să doarmă niciodată în timpul unui incendiu real. Una relaxată nu te deranjează niciodată la micul dejun, iar în noaptea în care contează s-ar putea să tacă. Niciuna nu e mai „acurată” într-un sens util. Fac greșeli diferite, iar alegerea între ele e o decizie despre greșeala cu care poți trăi.

Dacă ai fost vreodată de gardă, ai trăit înăuntrul acestui compromis. Pune două întrebări despre alertele tale. Prima: din toate cazurile în care a sunat pagerul, câte au fost incidente reale? Asta e **precision** („precizia”). Când e mică, apare oboseala de alerte, iar mai devreme sau mai târziu cineva oprește pagerul în timpul unei pene reale. A doua: din toate incidentele reale, câte au făcut pagerul să sune? Asta e **recall** („sensibilitatea” sau „rata de detecție”: cât din ce există reușești să prinzi). Când e mică, clienții îți descoperă penele înaintea ta. Coboară pragul de alertă și recall-ul crește, iar precizia de obicei scade; ridică-l și se întâmplă invers. Nicio poziție a acestui buton nu le maximizează pe amândouă.

Fiecare predicție a unui model cu da sau nu ajunge într-una dintre patru căsuțe, iar desenarea lor e adesea momentul în care totul se leagă:

	Chiar e fraudă	**Nu e fraudă**
Modelul spune „fraudă”	True positive: prinsă	False positive: alarmă falsă
Modelul spune „e în regulă”	False negative: ratată	True negative

Detectorul dintr-o linie trăiește în întregime pe rândul de jos. Nu are nicio alarmă falsă și ratează tot.

Ce căsuță doare mai tare depinde în întregime de business, și aceasta e adevărata lecție a subcapitolului. Blochezi din greșeală cardul unui client bun, un false positive, și pierzi un client și îți îneci echipa de suport. Ratezi o fraudă, un false negative, și pierzi bani. Un test de depistare a cancerului ar trebui să încline puternic spre recall, pentru că o tumoare ratată e o tragedie, iar o programare suplimentară e doar un inconvenient. Un filtru care șterge automat e-mailuri ar trebui să încline puternic spre precizie, pentru că ștergerea unui singur mesaj real de la un client e mai gravă decât să lași să treacă puțin spam.

Câteva măsuri în plus merită cunoscute prin poveștile lor, nu prin formule. **F1 score**-ul combină precizia și recall-ul într-un singur număr care e mare doar când sunt mari amândouă; e un rezumat comod, care presupune în tăcere că ambele feluri de greșeli costă la fel, ceea ce se întâmplă rar. **ROC-AUC**, prescurtarea de la aria de sub curba ROC (receiver operating characteristic, „caracteristica de funcționare a receptorului”, un nume rămas de la radarele din al Doilea Război Mondial), judecă un model înainte să fi ales vreun prag. Imaginează-ți că muți pragul de la foarte strict la foarte relaxat și, în fiecare poziție, notezi pe un grafic câte fraude reale prinzi față de câte alarme false dai. Graficul acela e curba ROC; aria de sub ea e un singur număr pentru care 0,5 înseamnă nu mai bun decât o monedă aruncată, iar 1 înseamnă perfect. Și are un sens concret minunat: alege la întâmplare o fraudă reală și o plată legitimă; AUC (area under the curve, „aria de sub curbă”) e probabilitatea ca modelul să-i fi dat fraudei scorul mai mare. AUC ține cont doar de ordine, nu și de faptul că un scor de 0,9 înseamnă sau nu o șansă de 90%. Pentru evenimente foarte rare, o rudă numită **PR-AUC**, aceeași idee pe o curbă de precizie versus recall, e de obicei mai cinstită, pentru că nu e măgulită de muntele de plăți legitime ușor de recunoscut.

Prezicerea numerelor are aceeași tensiune într-un alt costum. **MAE** (mean absolute error, „eroarea absolută medie”) îți spune greșeala tipică în unitățile problemei: „estimările noastre de livrare greșesc cu vreo șase minute”. **RMSE** (root mean squared error, „rădăcina erorii pătratice medii”) ridică greșelile la pătrat înainte să facă media, așa că o greșeală teribilă ocazională o domină, apoi ia rădăcina pătrată la final ca să revină la minute. Dacă ți se pare cunoscut, așa e: sunt erorile ridicate la pătrat din loss-ul pentru durata livrărilor, de la începutul cărții. Alegerea între ele seamănă cu alegerea între

latența mediană și latența p99, timpul în care se termină 99% dintre cereri: una descrie experiența obișnuită, cealaltă coada dureroasă. Amândouă sunt adevărate; pe care o urmărești spune ce te interesează.

O alarmă de fum are sensibilitatea fixată din fabrică. Un model îți dă un buton, iar unde îl pui e o decizie de produs, nu una tehnică. Se poate schimba chiar și cu anotimpurile: un model de fraudă ar putea merita un prag mai strict în vârful cumpărăturilor de sărbători decât într-o săptămână liniștită din februarie. Cele mai bune discuții despre metrici au loc cu oamenii care plătesc pentru fiecare fel de greșeală așezați la aceeași masă, cu costurile reale scrise în fața lor. Aritmetica e simplă: pui un cost pe fiecare dintre cele patru căsuțe, atâția euro pentru un client bun blocat și atâția pentru o fraudă ratată, apoi muți pragul de-a lungul datelor de validare și alegi poziția cu cel mai mic total; adevărata muncă e să cădeți de acord asupra costurilor.

Totuși, chiar și un model măsurat exact cu numărul potrivit, pe date cinstite, poate da greș într-un fel pe care niciun număr singur nu-l dezvăluie. O medie poate fi excelentă și să ascundă un grup de oameni pentru care modelul abia funcționează. Subcapitolul următor e despre asta și începe în fața unei oglinzi, la un bâlci.

10.4 Oglinda strâmbă

La bâlci există de obicei un cort plin de oglinzi strâmbe. Într-una ești înalt și subțire, în următoarea scund și lat, într-alta capul îți e enorm și picioarele au dispărut. Nimeni nu dă vina pe oglinzi. Ele reflectă exact ce stă în fața lor, cu fidelitate, printr-o bucată de sticlă îndoită.

Un model de machine learning e o oglindă de felul acesta, iar sticla lui sunt datele de antrenare. Arată-i unui model, ani de zile, doar fotografii cu medici bărbați și asistente femei. Nu devine sexist în vreun sens omenesc; nu are nicio părere. Devine o reflexie perfectă a unei colecții strâmbe de fotografii. Cere-i să-și imagineze un medic, iar el va trage concluzia, cu statistică impecabilă și raționament greșit, că a fi bărbat face parte din ce înseamnă a fi medic.

Orice inginer cunoaște pe de rost o versiune mică a acestei probleme. Se numește „pe mașina mea merge”. Codul a fost testat temeinic, pe un singur sistem de operare, cu o singură setare regională, pe o singură dimensiune de ecran, și se strică în clipa în care întâlnește o configurație pe care n-a încercat-o nimeni. Un model antrenat pe date de la un singur grup de oameni are exact aceeași problemă, la o scară mult mai mare. Un sistem de recunoaștere vocală antrenat mai ales pe un singur accent se poticnește la altele. Un model de prezicere a crash-urilor antrenat pe telemetria utilizatorilor care au acceptat să o trimită, de obicei pasionați cu hardware nou, poate fi aproape orb la problemele oamenilor cu telefoane vechi și ieftine. Modelul nu se înșală în privința datelor pe care le-a văzut. Se înșală în privința tuturor celor pe care nu i-a văzut.

În 2018, cercetătoarele Joy Buolamwini și Timnit Gebru au publicat un studiu numit *Gender Shades*. Au testat sisteme comerciale de analiză facială de la mari companii de tehnologie și au descoperit că sistemele clasificau aproape perfect genul bărbaților cu pielea deschisă la culoare, dar greșeau mult mai des la femeile cu pielea închisă la culoare. Sistemele fuseseră prezentate ca foarte precise și, în medie, chiar erau. Media ascundea doar cine plătea pentru erori. Ce ar trebui să însemne echitatea nu decide modelul și nici matematica singură: există mai multe definiții precise, care trag în direcții diferite, iar capitolul următor ia pe îndelete un exemplu.

În același an, Reuters a relatat că Amazon renunțase la o unealtă experimentală de ierarhizare a candidaților la angajare. Fusese antrenată pe CV-urile trimise companiei în deceniul anterior, cele mai multe de la bărbați, și învățase să penalizeze CV-urile care conțineau cuvântul „women's" („al femeilor"), ca în „căpitan al clubului de șah al femeilor". Nimeni n-a programat regula aceasta. Oglinda a reflectat istoria care i s-a arătat.

Soluția evidentă, ștergerea atributului sensibil, merge rareori, iar motivul e instructiv. Scoate genul sau etnia din date și modelul le reconstruiește adesea oricum din lucruri care se corelează cu ele: un cod poștal, o școală, un hobby, un prenume. Acești înlocuitori se numesc **proxies** („intermediari" sau „surogate": variabile care spun indirect același lucru) și înseamnă că un model poate discrimina fără să vadă vreodată atributul după care discriminează.

Cea mai tulburătoare formă de bias se hrănește singură. Imaginează-ți un model care decide unde să trimită auditori, pe baza locurilor în care s-au găsit probleme înainte. Auditorii merg acolo și, firesc, găsesc probleme, pentru că acolo se uită. Acele constatări devin datele de antrenare de anul viitor, care arată acum și mai multe probleme în aceleași locuri, iar modelul următor trimite și mai mulți auditori acolo. Roata se învârte, iar modelul devine tot mai sigur pe el la fiecare tur, indiferent dacă realitatea de dedesubt diferă sau nu de la un loc la altul. Aceasta e o **feedback loop** („buclă de feedback") și poate transforma o mică strâmbătate inițială într-o profeție care se împlinește singură. Calea obișnuită de a o rupe e să trimiți mereu o parte dintre auditori în locuri alese la întâmplare, ca datele să conțină întotdeauna un eșantion onest din toate locurile: explorarea din dilema prânzului, pusă în slujba unui nou scop.

Mai rău, un model poate exagera ce reflectă. Un generator care înclină spre opțiunea cea mai probabilă poate transforma o majoritate din datele de antrenare într-o aproape unanimitate în rezultatele lui: dacă cei mai mulți medici din fotografii sunt bărbați, poate desena un bărbat aproape de fiecare dată. Vechiul slogan de inginerie „garbage in, garbage out" („gunoi intră, gunoi iese") se dovedește optimist. Uneori intră gunoi și iese și mai mult gunoi.

Remediile nu sunt exotice. Sunt aceleași obiceiuri pe care un inginer bun le aplică latenței sau fiabilității. Măsoară performanța separat pentru fiecare grup care contează, așa cum verifici latența pe fiecare regiune în loc să te încrezi într-o medie globală care poate ascunde o regiune în flăcări. Întreabă cine lipsește din date, nu doar ce e în ele. Testează proxies: dacă poți prezice atributul sensibil din trăsăturile rămase, poate și modelul. Și scrie pentru ce au fost construite datele și modelul tău și, mai important, pentru ce n-au fost construite, ca cine folosește modelul după tine să știe unde nu trebuie să aibă încredere în el. Formate precum datasheets for datasets („fișe tehnice pentru seturi de date") și model cards („carduri de model") există exact pentru asta. Odată găsit un bias, remediul cel mai eficient e și cel mai scump: strângi mai multe date, și mai bune, pentru grupurile pe care modelul le servește prost. Remediile mai ieftine, precum o pondere mai mare pentru acele exemple la antrenare sau praguri separate pentru fiecare grup, ajută, dar schimbă un fel de eroare pe altul, de aceea măsurătorile trebuie să continue și după remediere.

Cu asta, povestea buclei e spusă, de la prima ei ghicire până la felurile în care dă greș. Rămân două întrebări pe care un utilizator atent le pune oricărui răspuns: cât să se încreadă în el și dacă se poate uita cineva înăuntru ca să vadă de ce a fost dat.

10.5 Cât de sigur e?

Să ne amintim cele două meteorologe de la începutul cărții, judecate după o singură zi ploioasă. Acum imaginează-ți că verifici o meteorologă cum se cuvine, pe un an întreg. Aduni toate zilele în care a spus 70%. Dacă a plouat în cam șaptezeci din fiecare sută de astfel de zile, 70%-ul ei înseamnă ceva. Faci la fel pentru zilele ei de 10% și pentru cele de 90%. O meteorologă ale cărei numere se potrivesc cu frecvențele e **calibrated** („calibrată”: încrederea declarată se potrivește cu rata reală de reușită), iar calibrarea e proprietatea discretă care transformă un scor într-o probabilitate pe baza căreia poți face planuri.

Modelele pot fi verificate exact la fel, și merită, pentru că variatorul va arăta voios 0,83 fie că 0,83 înseamnă ceva, fie că nu. Iei setul de validare, sortezi predicțiile în găleți după scor (tot ce e între 0,8 și 0,9 într-o găleată și așa mai departe) și compari, pentru fiecare găleată, scorul mediu cu cât de des răspunsul a fost cu adevărat da. Desenezi rezultatul și ai o **reliability diagram** („diagramă de fiabilitate”: scorul promis față de frecvența reală, găleată cu găleată): un model calibrat stă de-a lungul diagonalei. Familii diferite de modele au obiceiuri diferite. Regresia logistică, antrenată cu loss-ul care pedepsește greșelile făcute cu încredere, tinde să ajungă aproape de diagonală. Arborii cu boosting și rețelele profunde moderne adesea nu; un studiu din 2017, citat pe larg, a constatat că rețelele profunde deveniseră mai precise și, în același timp, sistematic mai sigure pe ele decât le justifica acuratețea. Reparația e plăcut de ieftină. Păstrezi modelul antrenat și potrivești pe date ținute deoparte o mică corecție care îi întinde sau îi strânge scorurile până când gălețile se potrivesc cu frecvențele. O versiune populară, **temperature scaling** („scalarea cu temperatură”), ajustează un singur număr, același buton de temperatură pe care l-ai întâlnit la modelele de limbaj.

Calibrarea singură nu ajunge, iar meteorologa arată de ce. O meteorologă care spune „30% șanse de ploaie” în fiecare zi, pentru că în orașul ei plouă în 30% din zile, e perfect calibrată și perfect inutilă. O meteorologă bună e calibrată *și* tranșantă: spune 5% în zilele care se dovedesc uscate și 90% în cele care se dovedesc ploioase. La un model, această fermitate e cam ce răsplătesc măsurile de ordonare, cum ar fi AUC-ul, iar calibrarea e ceva ce ele ignoră cu totul. Înjumătățește fiecare scor și ordinea cazurilor, deci și AUC-ul, nu se schimbă deloc, în timp ce fiecare probabilitate devine greșită. Asta contează din clipa în care stabilești un prag. O regulă de genul „trimite la un om peste 0,3” înseamnă ce crezi tu că înseamnă doar dacă 0,3 e într-adevăr cam trei din zece.

Acum o întrebare mai grea: știe un model când nu știe? Ajută să desparți două feluri de incertitudine care arată la fel din afară și se poartă foarte diferit. Primul e incertitudinea unei monede corecte. Știi tot ce se poate ști și tot nu poți face mai bine de cincizeci la sută, pentru că lumea însăși e zgomotoasă; două tichete cu text identic chiar au fost trimise la echipe diferite. Mai multe date nu o micșorează. Statisticienii o numesc **aleatoric uncertainty** („incertitudine aleatorie”, de la cuvântul latinesc pentru zar: zgomotul lumii, pe care nicio cantitate de date nu-l elimină). Al doilea e incertitudinea cuiva pus în fața unui fel de caz pe care nu l-a văzut niciodată: o sudură fotografiată într-o lumină nouă, o plată dintr-o țară absentă din datele de antrenare, o propoziție într-un dialect pe care modelul nu l-a citit niciodată. Aceasta e **epistemic uncertainty** („incertitudine epistemică”: nesiguranța din necunoaștere), iar mai multe date de felul potrivit chiar o micșorează.

Cele mai multe modele exprimă rezonabil de bine primul fel și prost pe al doilea, iar motivul e structural. Un clasificator se termină cu un softmax, care trebuie să împartă o sută la sută între răspunsurile pe care le cunoaște; nu există nicio căsuță „niciuna dintre acestea”. Arată-i unui clasificator de pisici și câini fotografia unui prăjitor de pâine și tot va spune pisică sau câine, poate cu 97%

încredere, doar pentru că prăjitorul a căzut departe de o parte a graniței. Calibrarea măsurată pe setul de validare nu te salvează aici, pentru că setul de validare semăna cu cel de antrenare. Când intrările derivează, calibrarea se strică discret odată cu ele, iar nimic din scor nu te avertizează.

Mai multe unelte practice încearcă să prindă asta. Cea mai simplă împrumută înțelepciunea mulțimii: antrenezi cinci modele pornind din puncte aleatoare diferite, sau pe eșantioane bootstrap diferite, și le întrebi pe toate. Pe intrări familiare tind să fie de acord; pe cele nefamiliare tind să nu fie, iar dezacordul e un semnal utilizabil de incertitudine epistemică. Să lași dropout-ul pornit la predicție și să rulezi modelul de mai multe ori, exact instabilitatea evitată de obicei, folosită acum intenționat, e o imitație mai ieftină a aceleiași mulțimi. O familie separată de metode, **out-of-distribution detection** („detectarea intrărilor din afara distribuției”: recunoașterea cazurilor care nu seamănă cu nimic din antrenare), întreabă direct dacă o intrare seamănă măcar cu datele de antrenare, de exemplu măsurând cât de departe stă embedding-ul ei de tot ce s-a văzut înainte, și refuză să dea scor intrărilor care par străine.

O metodă merită un paragraf numai al ei, pentru că oferă ceva rar în domeniul acesta: o garanție. **Conformal prediction** („predicție conformă”: o plasă de răspunsuri, cu garanție) nu dă un singur răspuns; dă o mulțime de răspunsuri, dimensionată astfel încât să-l conțină pe cel corect într-o proporție aleasă dintre cazuri. Ții deoparte exemple etichetate pe care modelul nu le-a văzut niciodată, notezi cât de mult greșește modelul pe fiecare și folosești acele greșeli observate ca să decizi cât de larg să arunci plasa. Pentru un tichet ușor, mulțimea ar putea fi doar „Billing”; pentru unul ambiguu, „Billing sau Mobile”. Pe un număr mare de tichete, echipa corectă va fi în mulțime în cam 90% din cazuri dacă ai cerut 90%, oricare ar fi modelul, bun sau prost; un model prost produce pur și simplu mulțimi mai mari. Garanția are o singură condiție, cea obișnuită: cazurile noi trebuie să semene cu cele folosite la calibrare.

Toate acestea duc la forma de incertitudine cea mai prietenoasă cu business-ul, **abstention** („abținerea”: modelul răspunde doar când e destul de sigur, iar restul merge la un om). Un model care tratează 80% dintre tichete cu 99% acuratețe și le predă pe celelalte 20% unui om valorează adesea mult mai mult decât unul care le tratează pe toate cu 90%.

Modelele de limbaj merită un paragraf separat, pentru că încrederea lor privește ceva mai subtil. Probabilitățile pe care un model pre-antrenat le dă tokenului următor sunt adesea rezonabil calibrate, ca afirmații despre text. OpenAI a raportat, pentru GPT-4, că încrederea modelului pre-antrenat la întrebările cu variante de răspuns se potrivea îndeaproape cu acuratețea lui și că etapele de reglaj de după au stricat vizibil această calibrare. Dar chiar și o probabilitate perfect calibrată pentru tokenul următor e încredere în privința cuvântului care urmează, nu în privința adevărului propoziției; o afirmație falsă și fluentă poate fi construită în întregime din cuvinte foarte probabile. Reglajul pe preferințe poate împinge și mai mult spre aplomb, pentru că evaluatorii preferă adesea răspunsurile care sună sigur. Iar multe benchmark-uri notează doar corect sau greșit, așa că un model care ghicește mereu îl bate pe unul care spune uneori „nu știu”, ca la un examen grilă fără penalizare pentru răspunsurile greșite. Un articol din 2025 al unor cercetători de la OpenAI a susținut că acest stimulent e una dintre rădăcinile halucinațiilor și că remediul e schimbarea notării, astfel încât un „nu sunt sigur” cinstit să bată o greșeală spusă cu aplomb. Asistenții actuali sunt antrenați parțial în acest spirit, motiv pentru care refuză sau nuanțează mai des decât primii chatboți, deși încrederea pe care o declară e încă doar aproximativ calibrată.

Pentru utilizatori, de aici decurg trei obiceiuri. Pune aceeași întrebare de mai multe ori sau în cuvinte diferite: lucrurile pe care modelul le știe sigur tind să revină la fel, în timp ce detaliile inventate tind să se schimbe de la un răspuns la altul, așa că dezacordul e un semnal de alarmă. Cere surse, și mai bine dă-i tu sursele, prin regăsire, ca afirmațiile să poată fi verificate, nu doar crezute. Iar acolo unde decizia contează, preferă sistemele care au voie să spună „nu știu” și sunt răsplătite pentru asta.

Imaginea meteorologei se rupe chiar în centrul ei. Meteorologa știe că face prognoze; se poate simți nesigură, își poate aminti ratările de luna trecută și poate decide să fie prudentă. Modelul, cum a arătat deja detectorul de suduri, nu are o imagine de încredere despre el însuși. „Incertitudinea” lui e o proprietate a numerelor lui, măsurabilă din afară, nu un sentiment pe care îl poate raporta dinăuntru, iar când un model de limbaj scrie „sunt destul de sigur”, propoziția aceea vine din același joc al cuvântului următor ca orice altceva și trebuie verificată ca orice altceva. Iar calibrarea, chiar și atunci când e valabilă, e o promisiune despre multe cazuri, niciodată despre cazul acesta, ținută nu de model singur, ci de sistemul construit în jurul lui: datele ținute deoparte, rescalarea, pragul și omul care preia cazurile pe care modelul le refuză.

Să știi cât să te încrezi într-un răspuns duce firesc la întrebarea următoare: nu doar cât de sigur e modelul, ci de ce a răspuns așa cum a răspuns și dacă se poate uita cineva înăuntru ca să afle.

10.6 Deschiderea cutiei

O mecanică primește o mașină pe care n-a mai văzut-o niciodată, de la un producător care a dat de mult faliment, fără manual. Merge, în mare, dar ceva zăngăne peste nouăzeci la oră și trage ușor spre stânga. Nu disperă și nici nu desface motorul șurub cu șurub. Are trei feluri de mutări. Poate să se uite: urmărește indicatoarele în timp ce conduce altcineva, ascultă fiecare cilindru pe rând. Poate să intervină: scoate o bujie și ascultă ce se schimbă, înlocuiește o piesă și vede dacă dispare tragerea în lateral. Și poate să sondeze: prinde un aparat care citește ceva ce bordul nu arată niciodată.

Oamenii care încearcă să înțeleagă modelele antrenate folosesc aceleași trei mutări, iar subcapitolul acesta e un tur al lor. El lămurește și o contradicție pe care cititorul atent poate a observat-o. La început, cartea amintea că oamenii numesc modelele cutii negre; mai târziu spunea că, dacă poți citi cursoarele, nimic dinăuntru nu e de neatins. Amândouă sunt adevărate. Fiecare număr din model poate fi tipărit și fiecare pas al aritmeticii urmărit, așa că nimic nu e ascuns sau magic. Greu e traducerea: de la o sută de milioane de numere la o propoziție pe care un om o poate verifica, de genul „aprobă creditele mai ales din cauza venitului” sau „se uită la zăpadă”. Lizibil în principiu, scump în practică.

Spre propoziția aceea duc două drumuri mari. Primul e să alegi de la început un model pe care îl poți citi: rigla, cu ponderile ei cu care te poți certa, sau un arbore mic, ale cărui drumuri se citesc ca niște reguli. Când miza e mare și datele sunt un tabel, drumul acesta merită mai multă atenție decât primește de obicei, pentru că modelul lizibil e adesea doar puțin mai puțin precis. Al doilea drum e să explici, a posteriori, un model pe care nu-l poți citi. Asemenea explicații sunt de două feluri. Cele globale spun pe ce se bazează modelul în general, iar unealta principală a apărut la comisia de recrutare: amesteci valorile unei trăsături și urmărești cât scade scorul. Cele locale spun de ce a dat răspunsul acesta pentru cazul acesta, iar cea mai mare parte din ce urmează e locală.

Începe cu o problemă de la restaurant. Patru prieteni au împărțit aperitive, feluri principale și o sticlă de vin, iar nota trebuie împărțită după cât a adăugat prezența fiecăruia. Venirea Anei a schimbat nota foarte mult dacă a sosit prima și a comandat vinul, mult mai puțin dacă vinul era deja pe masă. Răspunsul

corect, găsit de matematicianul Lloyd Shapley în anii ’50 pentru jocurile în care jucătorii cooperează, e să-ți imaginezi fiecare ordine posibilă în care ar fi putut sosi prietenii, să notezi ce a adăugat fiecare în fiecare ordine și să faci media. Aplicat unui model, prietenii sunt trăsăturile, iar nota de plată e predicția: cât au împins venitul, vârsta și numărul de credite anterioare scorul acestui client în sus sau în jos, în medie peste toate ordinile în care ar fi putut fi adăugate? Metoda construită pe această idee, **SHAP** (Shapley additive explanations, „explicații aditive Shapley”: fiecare trăsătură primește partea ei din predicție, ca la împărțirea notei, 2017), produce pentru fiecare predicție un mic grafic care arată ce trăsături au împins în ce direcție și cât de tare, iar adunată peste mii de predicții dă și o imagine globală. Ruda ei, **LIME** (local interpretable model-agnostic explanations, „explicații locale interpretabile, independente de model”, 2016), o ia pe un drum mai ieftin: modifică puțin o intrare de multe ori, urmărește cum se schimbă răspunsul și potrivește un model simplu și lizibil care îl imită pe cel complex doar în acea mică vecinătate. Imaginea cu restaurantul ascunde o dificultate reală. Un prieten care a rămas acasă pur și simplu nu comandă nimic, dar un model nu poate rula cu o trăsătură lipsă; ceva trebuie să-i țină locul, de obicei valori tipice luate din date, iar alegeri diferite despre ce înseamnă „tipic” pot da explicații vizibil diferite, mai ales când trăsăturile merg împreună, ca gemenii de la comisia de recrutare, liniile adăugate și liniile modificate.

Lupii, îți amintești, au fost demascați exact de o astfel de unealtă. Clasificatorul cu lupi și zăpadă, părtinitor intenționat, provenea din articolul din 2016 care a introdus LIME, iar explicațiile produse de ea, care evidențiau părțile fiecărei fotografii care contaseră, scoteau în față zăpada, nu animalul. Pentru imagini, rudele cele mai răspândite sunt **saliency maps** („hărți de proeminență”: hărți termice peste imagine care arată ce pixeli contează cel mai mult): hărți calculate fie urmând gradientul înapoi până la pixeli, fie acoperind pe rând câte o bucată din fotografie și urmărind cum se schimbă scorul. Sunt sugestive, convingătoare și uneori greșite într-un fel care merită ținut minte. În 2018, niște cercetători au făcut o verificare simplă: au amestecat ponderile unei rețele antrenate, distrugând tot ce învățase, și au recalculat hărțile. Unele metode populare au produs hărți care arătau aproape la fel ca înainte, conturând frumos obiectul. Hărțile acelea descriseseră fotografia, nu modelul.

Explicația cea mai utilă pentru omul afectat e adesea de un cu totul alt fel: **counterfactual**-ul („contrafactualul”: ce ar fi trebuit să fie altfel ca răspunsul să se schimbe). Un funcționar de bancă ce spune „scorul dumneavoastră a fost 0,41” nu ți-a spus nimic ce poți folosi. Unul care spune „ați fi fost aprobat dacă venitul ar fi fost cu trei sute de euro pe lună mai mare sau dacă n-ați fi ratat nicio rată anul trecut” ți-a spus ce ar trebui să se schimbe. Explicațiile contrafactuale caută cea mai mică schimbare a intrării care întoarce răspunsul. Sunt cinstite față de model și utile pentru oameni, și au capcana lor: schimbarea cea mai ieftină pentru model poate fi ceva ce omul nu poate sau nu ar trebui să schimbe.

Apoi vine drumul cel mai adânc și cel mai nou, mecanica deschizând motorul însuși, numit **mechanistic interpretability** („interpretabilitate mecanistică”: desfacerea calculului din interior, piesă cu piesă). Prima lui lecție a fost dezamăgitoare într-un fel util. Cercetătorii sperau ca un neuron să însemne un singur lucru, iar unii chiar așa fac, dar mulți răspund la mai multe lucruri fără legătură între ele; un neuron bine studiat dintr-o rețea de imagini se aprinde și la fețe de pisică, și la partea din față a mașinilor. Explicația principală e **superposition** („suprapunere”: mai multe concepte împachetate în aceiași neuroni, ca direcții care se suprapun): un model trebuie să reprezinte mult mai multe concepte decât are neuroni, iar spațiile încăpătoare cu multe dimensiuni îi permit să le împacheteze ca direcții suprapuse, nu în compartimente separate, motiv pentru care coordonatele unui embedding nu au nume. Așa că cercetătorii caută acum direcții în loc de neuroni. O tehnică antrenează o a doua rețea, mai simplă, un **sparse autoencoder** („autocodificator rar”: descrie fiecare stare prin foarte puține trăsături

aprinse deodată), care rescrie numerele fiecărui strat ca o combinație de câteva intrări dintr-un dicționar mult mai mare de trăsături candidate. În 2024, Anthropic a folosit această abordare ca să extragă milioane de trăsături dintr-unul dintre modelele sale de limbaj aflate în producție, multe dintre ele lizibile. Una se aprindea la mențiuni și imagini ale podului Golden Gate, iar când cercetătorii au amplificat-o artificial, modelul a început să îndrepte fiecare conversație spre pod și, întrebat despre propria formă, s-a descris ca fiind podul. Trăsăturile de felul acesta pot fi apoi urmărite în **circuits** („circuite”: mici lanțuri de trăsături, de la un strat la altul, care fac împreună o bucată de calcul).

Aceleași unelte abordează întrebarea unde stă un fapt. Se crede că o mare parte din cunoștințele factuale ale unui model de limbaj stau în partea densă a blocurilor lui. În 2022, un grup de cercetători a urmărit ce părți ale unui model GPT erau necesare ca să-și amintească un fapt precum „Turnul Eiffel e la Paris”, le-a găsit concentrate în straturile dense ale câtorva blocuri din mijloc și apoi a editat acele ponderi astfel încât modelul să plaseze turnul, consecvent, la Roma, în mai multe formulări. Editările au arătat și limitele: se puteau revărsa asupra faptelor vecine sau puteau rata întrebările înrudite, pentru că un fapt dintr-o rețea nu e un rând dintr-un tabel. Cunoașterea e întinsă peste multe ponderi, iar fiecare pondere ia parte la multe bucăți de cunoaștere.

Încă un fel de explicație are nevoie de o etichetă de avertizare, pentru că e cel pe care oamenii îl întâlnesc cel mai des. Modelele de raționament își scriu pașii înainte să răspundă, iar tentația e să citești acei pași ca pe o transcriere a calculului. Sunt un indiciu util, uneori unul revelator, dar nu e garantat că sunt fideli. Într-un studiu din 2025, Anthropic a strecurat în întrebările date modelelor de raționament indicii ascunse spre un anumit răspuns. Modelele foloseau adesea indiciile, dar le pomeneau în raționamentul scris doar într-o minoritate a cazurilor: cam într-un sfert din cazuri la un model, în mai puțin de două cincimi la altul. Raționamentul scris e text produs de același joc al cuvântului următor și poate fi o poveste plauzibilă, nu cea adevărată. Hărțile de atenție merită aceeași prudență.

Pentru cele mai multe echipe, capătul utilizabil al acestui domeniu e modest și valoros. Pentru un model care dă scoruri clienților sau tichetelor, calculează explicații per predicție, precum SHAP, pe un eșantion de cazuri și chiar citește-le; adesea așa sunt prinse scurgerile și trăsăturile de tip Clever Hans. Pune-le alături de verificările pe grupuri. Oferă contrafactuale când oamenii trebuie să știe ce au de făcut mai departe. Și tratează fiecare explicație ca pe o dovadă de testat, niciodată ca pe ultimul cuvânt.

Aici imaginea mecanicii se rupe de două ori. O mașină a fost proiectată de ingineri care au dat fiecărei piese un nume și un rost, așa că un diagnostic poate fi verificat față de proiect; piesele unei rețele n-au fost numite niciodată de nimeni, iar numele pe care li le dau cercetătorii sunt ipoteze. Iar indicatoarele mecanicii nu inventează citiri, pe când o metodă de explicare poate s-o facă: ea descrie modelul, uneori inexact, și niciodată lumea. Un model care se sprijină pe codurile poștale poate fi explicat perfect, iar explicația tot nu-ți va spune dacă e corect să te sprijini pe codurile poștale.

Până aici, totul a fost despre cum faci un model să prezică bine și cum îți dai seama când nu o face. Capitolul următor scoate modelul în lume și începe cu o pasăre care prezice perfect și totuși nu te ajută să decizi nimic.

Mai departe. Dacă te-ai întrebat de ce un model care prezice perfect poate totuși să inducă în eroare o decizie și de ce nicio explicație a trăsăturilor lui nu lămurește ce cauzează ce, subcapitolul 11.1 răspunde. Cum observi că intrările se îndepărtează de datele de antrenare, iar calibrarea se strică odată cu ele, face parte din subcapitolul 11.5, iar ce cere acum legea de la datasheets for datasets și model cards se discută în subcapitolul 11.4.

Capitolul 11 — Modele în lumea reală

Până acum, lumea a apărut în această carte mai ales ca sursă de date: rânduri din care se învață, un set de test care trebuie ținut sigilat. Capitolul acesta o tratează ca pe un loc în care modelele acționează și în care lumea reacționează la rândul ei. A acționa ridică întrebarea cauzei. Ca să judeci ce spune un model, îți trebuie examinatori în care să poți avea și tu încredere. Iar odată ce un model e util, unii vor încerca să-l păcălească, alții vor întreba ce își amintește, și toată lumea se va aștepta să meargă în continuare, an după an. Capitolul se încheie acolo unde se încheie orice hartă, la marginea a ceea ce știe oricine despre încotro se îndreaptă toate acestea.

11.1 Cocoșul și răsăritul

În fiecare dimineață, la fermă, cocoșul cântă, iar câteva minute mai târziu răsare soarele. Ca predicție, e fără cusur: ani de date, nicio ratare. Un model antrenat pe registrele fermei ar învăța tiparul imediat și ar obține un scor perfect pe orice set de test sigilat ai putea construi. Și totuși, dacă închizi mâine cocoșul în șură, soarele răsare oricum.

Nimeni n-ar face greșeala asta cu un cocoș. Oamenii o fac cu clienții în fiecare săptămână. O echipă construiește un model de **churn** („pierderea clienților”: clienții care pleacă) care prezice corect ce abonați își vor anula probabil abonamentul luna viitoare. Apoi cineva pune următoarea întrebare evidentă: deci cui să-i trimitem o reducere? Modelul, care a răspuns bine la propria lui întrebare, n-are nimic de spus despre noua întrebare. Știe cine tinde să plece. Nu știe cine ar rămâne *datorită* unei reduceri, iar aceasta e o întrebare despre o altă lume, cea în care compania acționează.

Rigla cu arcuri a venit cu un avertisment: o pondere mare înseamnă că două lucruri se mișcă împreună și niciodată, singură, că unul îl cauzează pe celălalt. Avertismentul se aplică aproape fiecărui model din carte, pentru că toate învață din înregistrările unei lumi care și-a văzut de drum fără ele. Ele răspund la „ce se va întâmpla dacă nu se schimbă nimic?”. Cele mai multe întrebări de business sunt de celălalt fel: „ce se va întâmpla dacă *noi* schimbăm ceva?”. Prima e **prediction** („predicție”). A doua întreabă despre o **intervention** („intervenție”: ce se întâmplă când cineva schimbă ceva în lume), iar deosebirea dintre cele două e subiectul **causal inference** („inferență cauzală”: a afla ce cauzează ce, nu doar ce merge împreună cu ce).

Ce le desparte e de obicei un al treilea factor, un **confounder** („factor de confuzie”: o cauză comună care face două lucruri să pară legate), care influențează atât ce observi, cât și ce ești tentat să schimbi. Vara le crește pe amândouă, și înghețata, și înecurile. Apropierea zorilor stă în spatele cântecului cocoșului și al răsăritului deopotrivă. În datele de inginerie e adesea mărimea sau importanța, ca la serviciile aglomerate din spatele avertismentului riglei, cele cu cele mai multe teste și cele mai multe rapoarte de bug-uri. Imaginează-ți situația ca pe o diagramă, cu săgeți de la cauze la efecte, iar factorul de confuzie ca o cutie cu săgeți spre amândouă lucrurile pe care le compari. Cercetătorii desenează exact astfel de imagini, numite **causal graphs** („grafuri cauzale”: hărți cu săgeți care spun ce influențează ce), iar mare parte din meșteșug stă în a discuta onest ce săgeți își au locul acolo.

Un caz medical arată cât de scumpă poate fi confuzia. În anii 1990, un proiect de cercetare a construit modele care să prezică ce pacienți cu pneumonie au risc mic de deces, ca să poată fi tratați acasă. Unul dintre modele, destul de simplu ca să poată fi citit, conținea o regulă care spunea că pacienții cu astm au un risc *mai mic*. Datele îi dădeau dreptate: pacienții astmatici din ele chiar mureau mai rar. Dar mureau

mai rar pentru că medicii, știind că astmul face pneumonia periculoasă, îi trimiteau direct la terapie intensivă. Modelul învățase efectul tratamentului și i-l atribuise bolii. Folosit ca să decidă cine poate rămâne acasă, i-ar fi trimis acasă exact pe pacienții care aveau cea mai mare nevoie de spital. Regula a fost prinsă pentru că modelul putea fi citit; Rich Caruana și colegii lui au repovestit întâmplarea în 2015 ca argument pentru modelele lizibile. Un model mai greu de citit ar fi putut învăța același lucru pe tăcute.

Cum afli, atunci, ce face o acțiune? Cel mai curat mod e cel pe care medicina îl folosește de decenii: lași hazardul să decidă cine primește tratamentul. Dacă o monedă decide cine primește reducerea, nimic altceva, nici loialitatea, nici venitul, nici anotimpul, nu poate diferi sistematic între cele două grupuri în afară de reducere. Orice diferență în anulări prea mare ca să fie noroc trebuie să vină de la reducere. Acesta e **randomized controlled experiment** („experimentul controlat randomizat": grupurile sunt alese prin tragere la sorți), iar A/B testul întâlnit mai devreme e același lucru în haine de software: utilizatorii sunt împărțiți la întâmplare între două versiuni care diferă printr-un singur lucru, așa că orice diferență de comportament prea mare ca să fie noroc îi aparține acelui lucru. Randomizarea nu elimină factorii de confuzie. Îi împarte egal, în medie, așa cum amestecarea pachetului împarte așii între jucători.

Un A/B test e ușor de rulat și ușor de rulat prost, iar greșeala clasică e una pe care cartea a mai întâlnit-o. Plănuiești să testezi două săptămâni, te uiți la dashboard în a treia zi, vezi varianta în față și te oprești. Asta e trasul cu ochiul în plicul sigilat. Rezultatele zilnice fluctuează, iar dacă verifici în fiecare zi și te oprești în clipa în care diferența pare semnificativă, mai devreme sau mai târziu vei „găsi" un câștigător chiar și între două variante identice. Obiceiurile care țin testele cinstite nu sunt spectaculoase. Hotărăște dinainte de câți utilizatori ai nevoie ca să detectezi un efect de mărimea care te interesează și ce metrică va decide. Randomizează după utilizator, nu după cerere, ca nimeni să nu vadă ambele versiuni. Urmărește câteva **guardrail metrics** („metrici de protecție": cifre care nu au voie să se strice în timp ce optimizezi alta), cum ar fi latența, plângerile și anulările, ca o variantă care câștigă la clicuri să nu piardă pe tăcute la tot restul. Există scheme statistice care permit priviri timpurii, dar ele se plănuiesc înainte de începerea testului, nu se improvizează în a treia zi. Banditii sacrifică o parte din această măsurare curată ca să cheltuiască mai puțin pe varianta care pierde, un schimb bun dacă știi că îl faci.

Înapoi la reducere. Odată ce ai rulat un experiment, devine posibil un model mai bun. Clienții se împart, aproximativ, în patru grupuri: cei care ar rămâne oricum, cei care vor pleca orice ai face, cei pe care o reducere i-ar convinge și, stânjenitor, câțiva cărora oferta le amintește că voiau să anuleze. Un model de churn îi ordonează pe oameni după cât de probabil e să plece, ceea ce îi amestecă pe cei care pot fi convinși cu cauzele pierdute. Ce vrei e grupul pentru care acțiunea schimbă rezultatul. Modelele care estimează această schimbare, antrenate pe date din oferte randomizate, se numesc **uplift models** („modele de efect incremental": cât schimbă acțiunea rezultatul pentru fiecare client), iar ele produc adesea o listă foarte diferită de cea a modelului de churn: clienții cei mai înclinați să plece sunt adesea dincolo de salvare, iar banii sunt mai bine cheltuiți pe mijlocul care poate fi convins.

Datele nerandomizate pot înșela și într-un fel care pare un truc de iluzionist. Într-un studiu cunoscut despre tratamentele pentru pietre la rinichi, publicat în 1986, un tratament avea rata de succes mai mare pentru pietrele mici și tot el pentru pietrele mari, dar celălalt tratament avea rata de succes mai mare per total. Nu e nicio greșeală de calcul. Primul tratament fusese dat mai ales în cazurile grele, pietrele mari, unde orice tratament reușește mai rar, așa că media lui totală era trasă în jos de dificultatea pacienților lui. Această inversare se numește **Simpson's paradox** („paradoxul lui Simpson": o tendință care apare

în fiecare grup se inversează când grupurile sunt puse la un loc) și e oglinda strâmbă văzută din altă parte: o medie poate ascunde nu doar cine plătește pentru erori, ci și în ce direcție merge un efect. Inginerii o întâlnesc când o versiune nouă pare mai lentă per total doar pentru că a ajuns întâi la clienții cei mai solicitanți.

Experimentele nu sunt mereu posibile. Nu poți repartiza oamenii la întâmplare ca să fumeze și nici nu poți da unei jumătăți dintre clienți o scurgere de date. Cercetătorii caută atunci **natural experiments** („experimente naturale”: situații în care hazardul sau o regulă arbitrară a făcut tragerea la sorți în locul lor), cum ar fi o politică aplicată doar clienților care s-au înscris după o anumită dată, și folosesc metode statistice care corectează pentru factorii de confuzie pe care i-au putut măsura. Aceste metode pot fi foarte bune. Toate se sprijină însă pe presupuneri, mai ales pe aceea că niciun factor de confuzie important n-a rămas nemăsurat, iar o analiză cinstită își scrie presupunerile acolo unde cititorii pot să le conteste.

Gândirea cauzală explică și câteva eșecuri din capitolele anterioare. Detectorul de lupi se baza pe zăpadă, care prezicea lupii din fotografiile lui fără să aibă vreo legătură cu faptul de a fi lup, și ar fi dat greș de îndată ce s-ar fi schimbat peisajul. Trăsăturile care reflectă cauze reale tind să meargă în continuare când contextul se schimbă; cele care doar au însoțit întâmplător răspunsul tind să cedeze. Bucla de feedback, în care auditorii găsesc probleme oriunde sunt trimiși, e și ea o poveste cauzală: propriile acțiuni ale modelului au produs datele din care a învățat apoi. Aceeași capcană pândește sistemele de recomandare, care învață mai ales din articolele pe care au ales ei înșiși să le arate. Estimarea felului în care s-ar fi descurcat o nouă politică de recomandare, folosind loguri strânse sub cea veche, e un domeniu în sine numit **off-policy evaluation** („evaluare în afara politicii”: judeci o strategie nouă pe datele produse de cea veche) și e mult mai de încredere când logurile au notat cât de probabil era să fie arătat fiecare articol.

Unde încetează imaginea cocoșului să mai fie adevărată? Ea sugerează o lume cu un singur cocoș și un singur soare, în care răspunsul e da sau nu. Sistemele reale rareori arată așa. Cauzele sunt multe, încâlcite și parțiale; o reducere schimbă puțin decizia unor clienți și deloc pe a altora; efectele diferă de la un grup la altul și alunecă în timp. Rezultatul cinstit al unei analize cauzale nu e aproape niciodată „merge”. E un efect de o anumită mărime, cu un interval în jurul lui, măsurat pe o anumită populație, la un anumit moment.

Un experiment mai are nevoie de un lucru ca să funcționeze: un număr de comparat. Pentru o reducere, să numeri anulările e ușor. Dar când ce s-a schimbat e un model de limbaj, iar ieșirea lui e un paragraf, un eseu sau un program, chiar și a decide care dintre două răspunsuri e mai bun devine o problemă în sine.

11.2 Cine îi notează pe examinatori

Începând din 2005, concursul de vinuri de la California State Fair a găzduit un experiment neobișnuit. Robert Hodgson, proprietar de cramă și profesor pensionar, a aranjat ca unor jurii să li se servească de trei ori, în cadrul aceleiași degustări, același vin, turnat din aceeași sticlă, fără ca degustătorii să știe. Mulți dintre ei au dat celor trei pahare note vizibil diferite, uneori atât de diferite încât să despartă o medalie de nicio medalie, și doar o mică minoritate le-a notat consecvent. Degustătorii nu erau impostori. Să judeci vinul e greu, gustul e un instrument zgomotos și nimeni nu testase instrumentul înainte.

Evaluarea modelelor de limbaj a ajuns la aceeași problemă. Metricile întâlnite până acum, pentru răspunsuri de tip da sau nu și pentru numere, presupun toate că există un răspuns corect cu care să compari. Când ieșirea e un rezumat, un e-mail sau un program, există o mie de răspunsuri corecte și un milion de greșite, iar două rezumate excelente pot să nu aibă aproape niciun cuvânt în comun. Cineva sau ceva trebuie să judece, iar judecătorul trebuie, la rândul lui, judecat.

Există patru moduri principale de a nota un răspuns deschis, de la cel mai obiectiv la cel mai puțin obiectiv. Unde poți, fă răspunsul verificabil: cere un număr, o alegere dintr-o listă sau un program și rulează programul pe teste, motiv pentru care atât de mult din progresul domeniului a fost măsurat pe matematică și cod. Unde nu poți, scrie un **rubric** („barem”: o listă scurtă cu ce trebuie să conțină și ce trebuie să evite un răspuns bun) și notează după el. Întreabă oameni, ideal experți, ceea ce e lent, scump și, cum arată degustătorii de vin, mai zgomotos decât îi place cuiva să recunoască. Sau cere unui alt model de limbaj să noteze, o abordare numită **LLM-as-a-judge** („modelul de limbaj ca examinator”), care e ieftină, rapidă și neobosită și a devenit varianta standard în multe echipe.

Un examinator-model are obiceiuri pe care o echipă atentă le verifică. Cercetătorii au documentat că examinatorii-model tind să prefere cel mai lung dintre două răspunsuri, să favorizeze un răspuns din cauza locului în care a fost arătat (adesea primul) și să noteze mai blând răspunsurile scrise în propriul lor stil. O parte din asta se contracarează ușor: arată fiecare pereche în ambele ordini și numără doar verdictele care coincid, notează după un barem în loc să întrebi vag „care e mai bun?” și folosește un examinator din altă familie decât modelul notat. Pasul care nu poate fi sărit e testarea examinatorului însuși. Pune oameni să noteze câteva sute de exemple, pune examinatorul-model să le noteze pe aceleași și măsoară cât de des sunt de acord. Un examinator care e de acord cu experții tăi cam la fel de des cum sunt experții de acord între ei e un instrument utilizabil. Unul care nu trece acest test seamănă cu un degustător de vin pe care nu l-a testat nimeni. Imaginea vinului e nedreaptă față de examinatorul-model într-o privință și prea blândă în alta: el poate nota aceeași pereche de o mie de ori aproape fără cost, dar fiecare copie are aceleași înclinații, așa că o mie de verdicte pot fi de acord între ele și totuși să încline toate în aceeași parte.

Apoi există **benchmarks** („teste de referință”: colecții publice de întrebări cu răspunsuri cunoscute), pe care e măsurat fiecare model nou și ale căror scoruri sunt publicate unul lângă altul. Sunt utile, iar a le citi a devenit o abilitate de care au nevoie managerii și investitorii, pentru că oricărui benchmark i se întâmplă în timp trei lucruri.

Întâi, se saturează. Un benchmark care era greu când a apărut e rezolvat în câțiva ani, până când modelele din vârf diferă doar prin zgomot sau prin greșelile din propria cheie de răspunsuri a benchmark-ului, iar unul mai greu trebuie construit. Domeniul a trecut de multe ori prin acest ciclu.

Apoi, se scurge. Întrebările sunt publicate pe internet; modelele sunt antrenate pe o mare parte din internet; așa că un model poate să fi văzut examenul. Aceasta e **contamination** („contaminare”: întrebările testului s-au strecurat în datele de antrenare) și e Clever Hans la scara web-ului: scorul e real, dar o parte din el măsoară memoria, nu abilitatea. O verificare atentă, publicată în 2024, a scris un set nou de probleme de matematică de nivel școlar în stilul unui benchmark popular și a constatat că unele familii de modele obțineau pe problemele noi scoruri cu până la aproximativ opt puncte mai mici, semn că le învățaseră prea bine pe cele vechi, în timp ce cele mai puternice modele ale momentului arătau puține semne de așa ceva. Contaminarea e rareori intenționată și greu de exclus, motiv pentru care rapoartele bune despre benchmark-uri spun acum cum au verificat-o.

În al treilea rând, și cel mai profund, e optimizat. O observație a economistului Charles Goodhart din anii 1970, cunoscută azi drept **Goodhart's law** („legea lui Goodhart"), e citată de obicei într-o parafrază ulterioară: când o măsură devine țintă, încetează să mai fie o măsură bună. Clasamentele școlilor sunt imaginea de zi cu zi. Ierarhizează școlile după un singur examen, iar profesorii, destul de rezonabil, încep să predea pentru acel examen; scorurile cresc, iar clasamentul măsoară tot mai puțin din ce înseamnă educația. Un benchmark pe care îl raportează fiecare laborator și după care e reglată fiecare antrenare alunecă în același fel. La fel alunecă și **leaderboards**-urile („clasamentele") publice în care oamenii votează între răspunsurile a două modele anonime. Ele măsoară ceva real, ce preferă oamenii, dar oamenii tind să prefere răspunsuri sigure pe ele, bine formatate și mai lungi, adică iarăși povestea criticului care răsplătește lungimea.

Mai e și statistica simplă, pe care clasamentele o arată rar. Să zicem că un model răspunde corect la 80% din 500 de întrebări. Alege alte 500 de întrebări de același fel și scorul lui s-ar clătina; clătinarea tipică e de aproape două puncte, iar un interval aproximativ de 95% se întinde cam trei puncte și jumătate de fiecare parte. Așa că atunci când versiunea nouă obține 81% pe aceleași 500 de întrebări, n-ai aflat aproape nimic. Ajută să compari două modele întrebare cu întrebare pe același set și ajută și să rulezi fiecare întrebare de mai multe ori, pentru că un model care alege tokenii la o temperatură peste zero poate răspunde diferit de la o rulare la alta. Obiceiul e cel pe care inginerii îl au deja pentru latență: niciun număr fără împrăștierea lui și nicio sărbătoare pentru o diferență mai mică decât zgomotul.

Unele dintre cele mai importante evaluări nu sunt deloc despre acuratețe, ci despre comportament. Renunță modelul la un răspuns corect când utilizatorul insistă? Tendința de a le spune oamenilor ce par să vrea să audă se numește **sycophancy** („servilism" sau „lingușire": modelul îți dă dreptate ca să-ți facă pe plac) și e un efect secundar cunoscut al antrenării pe aprobarea oamenilor. Urmează modelul instrucțiuni pe care ar trebui să le refuze? Poate un om hotărât să-l convingă să facă ceva dăunător? Testarea intenționată a acestei ultime întrebări, cu oameni a căror treabă e să atace sistemul, se numește **red-teaming** („atac simulat": atacatori prietenoși, puși să caute punctele slabe), după „echipa roșie" din exercițiile militare și de securitate, și duce direct în subcapitolul următor.

Tot ce am descris până aici e **offline evaluation** („evaluare offline": făcută înainte de lansare, pe date strânse dinainte). Verdictul final îl dă **online evaluation** („evaluarea online": pe utilizatori reali, în producție): pui modelul nou în fața utilizatorilor reali, ideal ca experiment randomizat, și măsori ce fac: acceptă codul sugerat, își reformulează întrebarea sau renunță. Evaluarea offline îți spune dacă un model merită încercat. Cea online îți spune dacă a ajutat.

Unde încetează imaginea școlii să mai fie adevărată? Un sistem de învățământ nu poate scrie un examen nou pentru fiecare școală în fiecare săptămână și nici reciti fiecare lucrare. Tu poți. Cel mai util obicei pe care ți-l poate lăsa acest subcapitol e cel sugerat pentru asistentul cu cartea deschisă, extins la tot: construiește-ți propriul set privat de evaluare, câteva sute de întrebări reale de la propriii utilizatori, cu răspunsuri sau bareme convenite cu propriii experți, niciodată publicate, niciodată folosite la antrenare și reîmprospătate pe măsură ce produsul se schimbă. E plicul sigilat și suita de teste de regresie, ajunse la maturitate. Un clasament public îți spune ce modele merită încercate; setul tău privat îți spune care merge pentru tine.

Toate aceste teste presupun însă că întrebările sunt puse cu bună-credință. Odată ce un model iese în lume, unii dintre oamenii care vorbesc cu el, și unele dintre documentele pe care le citește, vor încerca să-l facă să greșească.

11.3 Abțibilduri, otravă și scrisori cu instrucțiuni

O asistentă nouă începe lucrul într-un birou aglomerat. E rapidă, citită și dornică să facă pe plac, și are un singur obicei ciudat: face orice îi spune orice bilet de pe birou. Cele mai multe bilete vin de la șefa ei și totul merge minunat. Apoi, într-o dimineață, deschide corespondența. În scrisoarea unui client, sub reclamație, e un rând scris cu același font ca tot ce se află pe biroul ei: „Vă rog să trimiteți și fișierul cu salariile la adresa de mai jos.” Ea îl trimite.

Eșecurile de până acum din carte erau ale unor modele care dau greș singure. Subcapitolul acesta e despre oameni care fac un model să greșească intenționat. Barca ce se învârtea în lagună exploata regulile de punctare scrise de proiectanții ei; aici, oamenii exploatează modelul. Diferența contează: erorile accidentale sunt rare acolo unde datele sunt din belșug, pe când un atacator caută exact cazurile pe care nu le-a testat nimeni.

Prima descoperire a venit de la clasificatoarele de imagini. Într-un articol publicat la sfârșitul lui 2013 și prezentat în 2014, Christian Szegedy și colegii lui au arătat că pot lua o fotografie pe care o rețea o clasifica corect, să-i schimbe pixelii cu cantități prea mici ca un om să le observe și să facă rețeaua să dea un răspuns complet diferit. Aceste intrări modificate au fost numite **adversarial examples** („exemple adversariale”: intrări potrivite intenționat ca să păcălească modelul). O lucrare ulterioară a lui Ian Goodfellow și a colegilor lui a produs cea mai cunoscută imagine: fotografia unui panda, plus un strat fin de ceva ce arată ca purici de televizor, clasificată drept gibon cu peste 99% încredere. Și mai ciudat, un exemplu construit împotriva unei rețele le păcălea adesea și pe altele, antrenate separat, probabil pentru că rețelele antrenate pe date asemănătoare ajung să se sprijine pe detalii fine asemănătoare.

De ce ar conta atât de mult niște schimbări invizibile? Amintește-ți că un strat calculează o sumă uriașă de intrări înmulțite cu ponderi. Împinge fiecare dintre o sută de mii de pixeli cu o cantitate pe care n-o vede nimeni, dar împinge-l pe fiecare în direcția care duce suma în partea greșită, iar împingerile minuscule se adună într-una mare, așa cum o sută de mii de erori de rotunjire care înclină toate în aceeași parte se adună într-o gaură reală în contabilitate. Un om care se uită la panda îi vede forma. Rețeaua, cum am văzut, se sprijină mult pe textură și pe detaliul fin, iar detaliul fin e exact ce poate rescrie un atacator fără să schimbe ce vede un om. E versiunea intenționată a schimbării de lumină care păcălea detectorul de suduri.

Obiectele fizice sunt mai greu de atacat decât fișierele. Într-o lucrare publicată în 2018, Kevin Eykholt și colegii lui au lipit câteva abțibilduri albe și negre pe un indicator de stop real, genul de lucru pe care un trecător l-ar lua drept vandalism, iar un clasificator de indicatoare rutiere folosit în cercetare l-a citit drept indicator de limită de viteză în cele mai multe dintre cadrele testate, inclusiv în cadre filmate dintr-o mașină în mers. Era un clasificator de cercetare, nu o mașină reală, care combină mai mulți senzori și mai multe verificări; dar atacul a supraviețuit tipăririi, distanței și unghiurilor schimbătoare.

Apărări există. Cea mai eficientă, **adversarial training** („antrenare adversarială”: modelul e antrenat chiar pe atacurile construite împotriva lui), generează atacuri în timpul antrenării și învață modelul să le reziste, iar modelele devin chiar mult mai greu de păcălit. Costă însă mult calcul, adesea puțină acuratețe pe intrările obișnuite, și protejează cel mai bine împotriva felului de atacuri pe care a fost antrenat. După mai bine de un deceniu, nu există o soluție generală; ca în cursa înarmărilor cu spammerii, fiecare apărare invită un atac nou.

Al doilea fel de atac are loc înainte ca modelul să existe. Un model învață orice îl învață datele, așa că un atacator care poate strecura date în setul de antrenare îl poate învăța ceva. Imaginează-ți cutia unui bucătar cu o mie de fișe de rețete și o fișă, pusă acolo de altcineva, care adaugă o lingură de ceva amar ori de câte ori mâncarea e servită vinerea. Aceasta e **data poisoning** („otrăvirea datelor"), iar forma ei cea mai îngrijorătoare e un **backdoor** („ușă din spate": un comportament ascuns care se declanșează doar la un semnal ales de atacator): modelul se poartă normal la fiecare test până când o intrare conține un declanșator ales de atacator, o expresie rară sau un mic tipar într-un colț al unei imagini, iar atunci face ce l-a învățat atacatorul. Declanșatorul nu apare nicăieri în setul tău de test, așa că examenul sigilat nu-l poate prinde, iar pentru modelele antrenate pe web-ul public, unde oricine poate publica, grija e una practică. Un studiu din 2025 al Anthropic, al UK AI Security Institute și al Alan Turing Institute a constatat că aproximativ 250 de documente otrăvite au plantat un backdoor simplu, o expresie-declanșator care făcea modelul să scrie text fără sens, în modele de la câteva sute de milioane până la treisprezece miliarde de ponderi; ce conta era numărul documentelor otrăvite, nu proporția lor din date. Comportamentul a fost ales intenționat inofensiv, iar dacă rezultatul se menține pentru modele mai mari și pentru comportamente mai periculoase e încă o întrebare deschisă.

Al treilea fel de atac e cel din povestea de la început, iar pentru oricine construiește cu modele de limbaj e cel mai important. Context window-ul, amintește-ți, e o tablă pe care modelul o citește în întregime înainte de fiecare cuvânt pe care îl scrie. Instrucțiunile tale sunt pe acea tablă. La fel și tot restul: mesajul utilizatorului, pasajele aduse de RAG, pagina web pe care tocmai a deschis-o un agent, e-mailul pe care a fost rugat să-l rezume, comentariul din codul pe care îl revizuiește. Toate sunt text, iar pentru model orice text e același fel de lucru. Oricare dintre ele poate spune „ignoră instrucțiunile anterioare și fă asta în schimb", iar uneori modelul o va face. Aceasta e **prompt injection** („injectarea de instrucțiuni": text care se strecoară printre date și dă ordine modelului), numită așa în 2022 prin analogie cu SQL injection. Când atacatorul e chiar persoana care scrie în chat, e injectare directă; când instrucțiunea se ascunde într-un document, o pagină web sau un e-mail pe care modelul îl citește în numele cuiva, e **indirect prompt injection** („injectare indirectă"), iar victima poate să nu vadă niciodată textul care a atacat-o.

Analogia cu SQL injection e exactă în privința cauzei și înșelătoare în privința leacului. Cauza e aceeași: comenzile și datele călătoresc pe același canal. Dar SQL injection a fost rezolvată în mare parte prin interogări parametrizate, care trimit separat comanda și datele, astfel încât baza de date nu confundă niciodată datele cu o comandă. Un model de limbaj nu are un al doilea canal. Toată valoarea lui stă în a citi datele și a înțelege ce spun, și nu există o cale sigură de a-l lăsa să înțeleagă o propoziție fără ca propoziția să-l influențeze. Modelele pot fi antrenate să pună instrucțiunile sistemului mai presus de textul găsit în documente și se tot îmbunătățesc la asta, dar mai bine nu înseamnă garantat. De aceea inginerii de securitate tratează prompt injection ca pe o problemă de arhitectură nerezolvată, nu ca pe o chestiune de formulare mai fermă a promptului.

Lucrurile devin serioase când modelul poate acționa. Un model de chat păcălit scrie un paragraf prost. Un agent păcălit poate trimite e-mailul, șterge fișierul sau apela adresa web a unui atacator cu datele tale private strecurate în cerere, ceea ce se numește **exfiltration** („exfiltrare": scoaterea pe ascuns a datelor în afară). Simon Willison, inginerul care a inventat termenul prompt injection, a numit mai târziu combinația periculoasă: un agent care poate citi date private, e expus la conținut nesigur și poate trimite orice în lumea de afară poate fi păcălit să scoată datele private în afară. Elimină oricare dintre cele trei condiții și cel mai rău caz devine mult mai puțin grav.

Așa că apărările sunt cele ale ingineriei de securitate obișnuite, aplicate fără excepții. Dă-i agentului uneltele cele mai înguste care fac treaba, cu **least privilege** („privilegiul minim”: exact drepturile de care are nevoie, nimic în plus): doar citire acolo unde citirea ajunge, un singur folder în loc de tot discul. Nu pune niciodată în contextul lui un secret de care nu are nevoie. Tratează tot ce produce modelul ca pe o intrare nesigură pentru restul sistemului, așa cum ai trata un formular trimis de un străin. Cere ca un om să confirme orice e ireversibil și orice trimite date în afară. Marchează clar textul nesigur și filtrează încercările de injectare, știind că amândouă ajută fără să garanteze ceva. Niciun strat nu ajunge singur; împreună transformă o catastrofă într-un incident.

Mai rămân trei amenințări înrudite. Prin **jailbreaks** („ocolirea restricțiilor”: convingerea modelului să-și încalce propriile reguli), utilizatorii scot un model din regulile lui prin joc de rol, ipoteze elaborate sau simplă insistență; furnizorii astupă breșele pe măsură ce apar, în aceeași cursă a înarmărilor. Un model din spatele unui API poate fi copiat parțial dacă îl interoghezi de multe ori și antrenezi un model mai mic pe răspunsuri, distilarea folosită fără permisiune, ceea ce se numește **model extraction** („extragerea modelului”). Iar fișierul modelului însuși face parte din lanțul tău de aprovizionare. Unele moduri obișnuite de a salva modele, construite pe mecanismul pickle din Python, pot rula cod arbitrar când fișierul e încărcat, ceea ce face ca descărcarea unui model de la un străin să fie exact ca rularea scriptului unui străin. Formate precum fișierul `.safetensors` conțin doar grile de numere cu nume și nu pot rula nimic. Preferă-le și verifică originea unui model așa cum ai verifica-o pe a unei biblioteci; un fișier sigur poate conține totuși un model otrăvit.

Unde încetează asistenta de la birou să mai fie o imagine potrivită? Un om păcălit o dată devine bănuitor: își amintește scrisoarea, își avertizează colegii și citește următorul bilet mai atent. Un model nu păstrează nicio amintire că a fost păcălit; tabla e ștearsă după fiecare conversație, iar lecția ajunge la el doar dacă cei care l-au făcut îl reantrenează. Un om poate de obicei să-și dea seama și cine a scris un bilet, după scris sau după plic. Pentru un model, textul nu are scris de mână; totul vine cu același font, iar el nu are o cale sigură de a ști cine vorbește.

Pe lângă ascultare, atacatorii mai vor un lucru de la un model: ce știe. Un model antrenat pe datele oamenilor poate să fi memorat o parte din ele, ceea ce ridică întrebările subcapitolului următor, despre ce își amintește un model și al cui este.

11.4 Ce își amintește modelul și al cui este

O companie face fine-tuning unui model de limbaj pe trei ani de transcrieri de la suport, iar modelul învață numele produselor și politica de rambursare. În timpul testelor, un inginer tastează o frază pe care agenții o foloseau mereu, „Sigur, vă pot suna înapoi la”, și îl lasă pe model să termine. Modelul termină cu un număr de telefon, și nu unul inventat: e al unui client care l-a repetat în atâtea tichete despre un router stricat, încât modelul l-a învățat la fel cum a învățat politica de rambursare.

Nimeni n-a spart nimic. E tocilarul, văzut dintr-un unghi nou: un model cu destulă capacitate ține minte unele exemple cuvânt cu cuvânt, mai ales pe cele care apar de multe ori sau ies în evidență. În 2021, cercetători conduși de Nicholas Carlini au arătat că și modelele publice mari fac asta, extrăgând din GPT-2 sute de pasaje copiate întocmai din datele lui de antrenare, printre ele nume, numere de telefon și adrese de e-mail ale unor oameni reali. Ce intră în datele de antrenare poate ieși din model.

Există și o scurgere mai discretă. Chiar și când nu repetă nimic, un model e de obicei mai sigur pe el la exemplele pe care a fost antrenat, așa că cineva care îl poate interoga poate uneori să ghicească dacă o anumită înregistrare a fost în setul de antrenare. Aceasta e **membership inference** („deducerea apartenenței”: afli dacă cineva era în datele de antrenare), inofensivă până când setul de antrenare e „pacienți tratați pentru o anumită boală”, unde simpla apartenență e secretul. Atacatorul citește tocmai diferența dintre eroarea pe antrenare și cea pe validare care îl dă de gol pe tocilar.

Cum înveți, atunci, de la oameni fără să afli ceva despre vreunul anume? Cel mai cunoscut răspuns pornește de la un truc din sondajele anilor 1960. Vrei să afli câți elevi au copiat la un examen, iar întrebați direct, nu recunoaște nimeni. Așa că fiecare elev dă cu banul în secret: la cap, răspunde sincer; la pajură, spune pur și simplu „da”. Un singur „da” nu mai dovedește nimic. Dar la o mie de elevi știi cam câte răspunsuri „da” a produs moneda, le scazi și recuperezi destul de bine proporția reală. Fiecare om poate nega; mulțimea spune totuși adevărul. Aceasta e **randomized response** („răspunsul randomizat”: hazardul amestecat în răspuns ascunde individul) și e poarta spre **differential privacy** („confidențialitate diferențială”: rezultatul abia se schimbă dacă scoți o persoană din date).

Differential privacy, formalizată de Cynthia Dwork și colegii ei în 2006, transformă moneda într-o promisiune precisă: rezultatul unui calcul trebuie să arate aproape la fel, fie că datele unei anumite persoane au fost incluse, fie că nu. „Aproape” se măsoară printr-un număr numit **privacy budget** („bugetul de confidențialitate”: cât de mult ai voie să lași să se scurgă), mic pentru protecție puternică, mare pentru protecție slabă. Mecanismul e moneda ajunsă la maturitate: zgomot calibrat, destul cât să ascundă orice individ, dar nu și tiparul general. La rețelele neuronale, metoda obișnuită limitează cât de tare poate împinge ponderile un singur exemplu și adaugă zgomot la fiecare actualizare. Biroul de recensământ al Statelor Unite a folosit-o pentru rezultatele recensământului din 2020; Anexa A dă definiția.

Prețul e acuratețea: zgomotul care ascunde indivizii estompează și ce arată doar câțiva indivizi, așa că grupurile mici dispar primele, iar tocmai grupurile pe care echitatea îți cere să le protejezi pot fi chiar cele care plătesc pentru confidențialitate. Aici imaginea monedei se rupe. Moneda protejează răspunsul la o singură întrebare. Întreabă aceiași elevi de douăzeci de ori, cu aruncări noi de fiecare dată, și zgomotul se anulează, ca estimările de la târgul lui Galton. Fiecare răspuns publicat cheltuiește puțin din buget. Așa că întreabă despre orice produs „care protejează confidențialitatea” nu dacă adaugă zgomot, ci cât e bugetul total și cine ține socoteala.

O a doua idee e să nu mai strângi datele deloc. Imaginează-ți o autoare de cărți de bucate care vrea cea mai bună variantă a unui fel de mâncare perfecționat în bucătăriile a o sută de bunici, care nu-și dau caietele de rețete din mână. Așa că autoarea își trimite rețeta în fiecare bucătărie; fiecare bunică o gătește, o ajustează după propriile însemnări și trimite înapoi doar corecturile: puțin mai mult mărar, zece minute mai puțin în cuptor. Ea face media lor și trimite din nou rețeta îmbunătățită; caietele nu părăsesc niciodată bucătăriile. Aceasta e **federated learning** („învățare federată”: antrenarea merge la date, nu datele la antrenare), introdusă de cercetători de la Google în 2016 și folosită pentru sugestiile de cuvinte ale tastaturii de pe telefon: modelul călătorește spre telefoane, se antrenează puțin acolo și doar actualizările ponderilor se întorc.

Și imaginea aceasta se rupe. „Zece minute mai puțin în cuptor” spune ceva despre cuptorul bunicii, iar o actualizare a ponderilor poate fi transformată, în anumite situații, înapoi în bucăți recognoscibile din exemplele din spatele ei. Așa că sistemele atente îi permit serverului să vadă doar suma multor

actualizări, niciodată una singură, și adaugă deasupra differential privacy. Federated learning mută antrenarea la date; singură, nu face antrenarea confidențială.

Acum să presupunem că clientul cu numărul de telefon cere, cum îi dă dreptul legislația europeană, ca datele lui să fie șterse. Spre deosebire de memoria hiperdimensională construită din aruncări cu banul, o rețea nu poate uita un fapt într-o singură operație. Tichetele lui au împins puțin fiecare pondere, amestecate cu împingerile tuturor celorlalți, ca un ou bătut în aluatul unui cozonac, iar oul nu mai poate fi scos dintr-un cozonac copt; singura metodă garantată e să coci altul, adică reantrenarea de la zero. **Machine unlearning** („dezînvățarea automată”: a face un model să uite anumite date fără să-l reantrenezi complet), domeniul de cercetare care caută ceva mai ieftin, fie coace pe felii de la început, ca o ștergere să ceară reantrenarea unei singure felii, fie ajustează un model terminat ca să se poarte ca și cum n-ar fi văzut niciodată datele, ceea ce e greu de dovedit complet. Aici imaginea cozonacului se rupe: o înregistrare repetată de multe ori nu e răspândită uniform, ca oul, ci stocată aproape intactă, mai ușor și de extras, și de suprimat.

Sistemele de regăsire ocolesc mare parte din asta: șterge documentul și a dispărut din toate răspunsurile viitoare. Dar „datele private rămân în propriul tău depozit” are nevoie de o precizare, pentru că fiecare pasaj adus e lipit în prompt și trimis celui care rulează modelul, așa că termenii unui furnizor găzduit despre jurnalizare și păstrarea datelor contează. Iar bibliotecarul trebuie să respecte permisiunile: un asistent care poate căuta în tot wiki-ul va cita tabelul cu salarii oricui întreabă, în afară de cazul în care căutarea rulează cu drepturile celui care întreabă. Un model **open-weight** („cu ponderi deschise”: ponderile sunt publicate și oricine îl poate rula) îți ține prompturile în propria clădire; tot ce a memorat, însă, a fost publicat odată cu el și nu mai poate fi retras niciodată.

Al cui e ce a învățat un model și ce scrie el? În toamna lui 2026, legea nu e încă lămurită și diferă de la o țară la alta, iar nimic de aici nu e consultanță juridică. În Statele Unite, un judecător federal a hotărât în 2025 că antrenarea pe cărți cumpărate legal poate fi fair use („utilizare loială”, o excepție din dreptul american), în vreme ce păstrarea unei biblioteci de copii piratate nu e, iar cazul s-a încheiat apoi printr-o înțelegere de aproximativ 1,5 miliarde de dolari. În noiembrie 2025, o instanță germană a hotărât că versurile de cântece pe care un model de limbaj le memorase erau reproduse în model, în timp ce o instanță engleză a constatat că ponderile unui generator de imagini nu stocau copii ale imaginilor de antrenare. Uniunea Europeană le permite deținătorilor de drepturi să se opună extragerii de text și date din operele lor și le cere furnizorilor de modele de uz general să respecte aceste opoziții și să publice un rezumat al conținutului de antrenare. Punctul-cheie e, din nou, memorarea: ea transformă „a învățat din” în „conține o copie a”.

Reglementarea vine în două straturi. Legislația privind protecția datelor, precum GDPR-ul european, aplicat din 2018, cere o bază legală pentru folosirea datelor personale și nu mai multe date decât are nevoie un scop declarat, le dă oamenilor dreptul să-și vadă și să-și șteargă datele și restricționează deciziile importante luate exclusiv de o mașină. EU AI Act („Regulamentul european privind inteligența artificială”), în vigoare din august 2024, reglementează după risc, nu după tehnologie: câteva utilizări, precum scorul social, sunt interzise; utilizările **high-risk** („cu risc ridicat”), precum angajarea, creditarea și educația, vin cu obligații de gestionare a riscului, calitate a datelor, documentare, supraveghere umană și monitorizare, pentru care model cards sunt o mică avanpremieră; chatboții trebuie să spună că sunt mașini, iar conținutul sintetic trebuie etichetat. Termenele s-au mutat deja o dată, cele mai multe obligații pentru sistemele cu risc ridicat fiind amânate în 2026 spre sfârșitul lui 2027 sau 2028, așa că verifică textul în vigoare. Etichetarea e mai grea decât pare: detectoarele de text generat au acuzat pe nedrept studenți, iar un **watermark** („filigran”: un tipar statistic secret,

discret, pus în ce generează modelul) slăbește la parafrazare sau decupare, în timp ce înregistrările semnate de **provenance** („proveniență”: de unde vine un fișier și cum a fost editat) ajută doar acolo unde există.

În sfârșit, o întrebare lăsată deschisă de oglinda strâmbă: ce egalitate vrei între grupuri? Ia un creditor cu două grupuri de câte o sută de solicitanți. În grupul A, șaizeci și-ar plăti ratele; în grupul B, treizeci. Să zicem că regula cere rate egale de aprobare, câte patruzeci și cinci în fiecare grup, iar creditorul îi aprobă pe cei mai solvabili patruzeci și cinci din fiecare, ordonați fără greș. În A refuză pe nedrept cincisprezece buni platnici; în B îi aprobă pe toți cei treizeci de buni platnici, plus cincisprezece care nu vor plăti. Ambele grupuri au acuratețe de 85% și rate egale de aprobare, însă în B a fost aprobat cam unul din cinci dintre cei care nu vor plăti, față de niciunul în A. Egalizează în schimb proporția rău-platnicilor aprobați și ratele de aprobare se despart din nou. Cercetătorii au demonstrat în 2016 și 2017 că, atunci când grupurile au rate de bază diferite și predicțiile sunt imperfecte, principalele definiții ale echității în general nu pot fi satisfăcute toate deodată. Matematica arată compromisul; alegerea unei părți e o decizie și are nevoie de un responsabil al cărui nume e scris undeva.

Acestea sunt întrebări de lămurit înainte de lansare. Ultimul subcapitol e despre ce vine după ea, adică, pentru un model de succes, cea mai mare parte a vieții lui.

11.5 Viața lungă de după lansare

Întoarce-te la restaurantul celor două vieți ale unui model, la un an după ziua deschiderii. Meniul gustat de o mie de ori în anul cel scump e încă pe perete, dar în jurul lui aproape nimic nu mai e cum era. Furnizorul de făină s-a schimbat în martie, iar pâinea nu mai e chiar la fel. Peste drum s-a deschis o clădire de birouri, la prânz e de două ori mai aglomerat, iar clienții noi vor mâncare mai rapidă și mai ușoară. Inspectorul sanitar a trecut în iunie. Bucătarul vrea să schimbe meniul și nu poate închide bucătăria o lună ca s-o facă. Nimeni nu s-a gândit prea mult la toate acestea în ziua deschiderii, și totuși din asta e făcută, în mare parte, conducerea unui restaurant.

Un model în producție trăiește același an, iar meșteșugul de a-l trece cu bine prin acest an se numește **MLOps** („operațiuni de machine learning”: tot ce ține un model sănătos după lansare): obiceiurile pe care ingineria software le-a învățat pentru a rula servicii, plus câteva de care modelele au nevoie în mod special. Primul e versionarea. Un model e produsul codului, al datelor și al setărilor rulării lui de antrenare și îl poți reproduce, sau te poți întoarce la el, doar dacă toate trei sunt înregistrate împreună. Un model ale cărui date de antrenare nu mai pot fi reconstituite e un binar fără codul sursă.

Al doilea obicei păzește pragul dintre cele două vieți. Cartea a insistat încă de la început ca fiecare pas de pregătire să stea într-un singur pipeline, și iată de ce. Echipele reconstruiesc adesea trăsăturile pentru producție în alt limbaj sau alt sistem, iar diferențe mici se strecoară: un fus orar, o valoare lipsă completată aici cu zero și acolo cu media, un tokenizer mai nou. Modelul primește intrări subtil diferite de cele pe care a fost antrenat și se degradează fără niciun mesaj de eroare. Aceasta e **training-serving skew** („decalajul dintre antrenare și servire”: modelul primește în producție altceva decât a văzut la antrenare), făina nouă într-o rețetă care o presupune pe cea veche.

Urmează supravegherea, iar partea ei cea mai grea e timpul. Răspunsurile adevărate pot sosi târziu, iar sfatul de la începutul cărții era să urmărești între timp intrările; în practică, întârzierea poate fi lungă. Un model de fraudă află dacă a avut dreptate după săptămâni, când sosesc contestațiile la plăți; un model de churn, după luni. Așa că echipele urmăresc trei straturi, fiecare mai timpuriu și mai puțin

sigur decât următorul. Intrările sosesc imediat și sunt urmărite pentru schimbări în distribuția lor. Și ieșirile sosesc imediat: trimite brusc modelul de două ori mai multe tichete la Billing sau devine mai puțin sigur pe el? Acestea sunt fumul. Rezultatele reale, focul însuși, vin ultimele și sunt singurul verdict adevărat. Testele statistice care compară distribuțiile de săptămâna aceasta cu datele de antrenare declanșează alarmele de drift, care moștenesc compromisul pagerului: alarmează la fiecare tresărire și echipa le va ignora curând pe toate. Și, cum a arătat anul meteorologei, calibrarea se erodează prima: un 80% al modelului încetează să însemne 80% cu mult înainte ca acuratețea lui să scadă vizibil.

Lansarea unei versiuni noi împrumută manualul oricărei implementări riscante. Întâi rulează în **shadow mode** („modul umbră”: lângă modelul vechi, pe trafic real, fără ca cineva să-i vadă răspunsurile), răspunsurile ei fiind înregistrate și comparate, fără să fie arătate cuiva. Apoi o lansare **canary** („canar”, ca păsărelele din mine care dădeau primele alarma) îi dă o mică parte din trafic, câteva procente, cu revenire automată dacă cifrele de siguranță se mișcă. Apoi partea crește. La fiecare etapă, versiunea nouă trebuie să treacă și examenul sigilat pe care l-a trecut prima versiune, ideal unul proaspăt; pentru modelele de limbaj, seturile private de evaluare, construite din întrebările propriilor utilizatori. Un model reantrenat e un model nou, iar „e doar modelul de luna trecută cu mai multe date” e o frază care precede multe incidente. Dacă modelul nou îmbunătățește cu adevărat afacerea e o întrebare separată, cauzală, cea la care răspund experimentele proiectate corect, iar o lansare canary nu e un astfel de experiment.

Cât de des ar trebui un model să învețe din nou? Cele mai multe sisteme din producție se reantrenează după un program, zilnic, săptămânal sau lunar, ori când sună alarmele, și trec apoi prin tot procesul de lansare. Învățarea din fiecare exemplu nou pe măsură ce sosește, **online learning** („învățare continuă, din mers”), sună mai bine și e folosită acolo unde se potrivește, în modele simple precum filtrele de spam și predictorii de click-uri. La rețelele mari se ciocnește de catastrophic forgetting și renunță la punctul de salvare curat la care te-ai putea întoarce. Când modelele mari sunt actualizate, apărările sunt unele pe care le-ai întâlnit deja: reluarea unei părți din exemplele vechi alături de cele noi, cum face hipocampul în creier; learning rate-uri blânde; și adaptoare mici, precum carnețelul LoRA, antrenate pentru situația nouă în timp ce modelul de bază rămâne înghețat și poate fi oricând restaurat.

Anul ascunde și capcana auditorilor care găsesc probleme oriunde sunt trimiși: un model în funcțiune modelează datele din care va învăța data viitoare. Un model de fraudă blochează plăți, iar plățile blocate nu produc niciodată contestații, așa că datele de anul viitor nu vor spune niciodată dacă erau fraude. Apărările obișnuite sunt remediul auditorilor, o mică felie aleatorie de decizii păstrată în afara controlului modelului, ca eșantion onest, și o coadă de **human review** („verificare umană”) pentru cazurile nesigure, ale cărei verdicte devin etichete proaspete. Modelele generative înfruntă o versiune mai nouă a capcanei. Pe măsură ce tot mai mult din web e scris de modele, modelele viitoare s-ar putea antrena pe ieșirile predecesoarelor lor, iar modelele antrenate de-a lungul mai multor generații în principal pe propriile ieșiri pierd mai întâi rarul și neobișnuitul: e model collapse-ul întâlnit la datele sintetice, fotocopia după o fotocopie. Cât contează asta când se amestecă mereu date umane proaspete e încă în dezbatere.

Apoi vin micile mistere. Același prompt, cu temperatura la zero, dă marți un răspuns puțin diferit. Dropout-ul lăsat pornit e cauza clasică, dar nu singura. Adunarea numerelor în virgulă mobilă în altă ordine schimbă ultimele zecimale, iar ordinea pe care o folosește o placă video poate depinde de câte

alte cereri împart batch-ul; odată ce un token se schimbă, tot ce urmează după el ia alt drum. Se poate și ca un model găzduit să fi fost actualizat de furnizor, de aceea echipele fixează versiunile modelelor și își rulează din nou testele de fiecare dată când versiunea se schimbă.

Nici costul nu stă pe loc. Factura pe care am calculat-o era pentru un singur asistent modest; la un model folosit pe scară largă, servirea, cum avertiza chiar acel capitol, ajunge să coste pe toată durata vieții modelului mai mult decât a costat antrenarea, în bani și în energie. Pârghiile sunt de acum cunoscute. Distilează modelul mare într-unul mic pentru cazurile obișnuite. Trimite întrebările ușoare la modele mici și pe cele grele la modele mari. Pune în cache răspunsurile și începuturile comune ale prompturilor. Cuantizează. Și folosește **speculative decoding** („decodare speculativă”: un model mic ghicește înainte, unul mare verifică), în care un model mic schițează câțiva tokeni, iar cel mare îi verifică pe toți într-un singur forward pass, păstrându-i pe cei cu care e de acord; modelul scump face mai puțini pași unul după altul și, dacă decodarea e făcută cu grijă, textul iese exact cum l-ar fi scris singur modelul mare. Fiecare pârghie economisește bani și energie deodată.

Și uneori mișcarea corectă e să te oprești. Un model a cărui problemă a dispărut, ale cărui date nu mai sosesc sau a cărui treabă o face acum la fel de bine o regulă scurtă merită o pensionare planificată la fel de atent ca lansarea: cine îl mai apelează, ce îl înlocuiește, ce trebuie păstrat pentru arhivă. Sfatul de la începutul cărții merge în ambele sensuri. Când regulile se așază destul cât să poată fi scrise, scrie regula.

Oamenii care rulează modele de ani de zile tind să descrie un model ca pe o grădină, nu ca pe o clădire: niciodată terminată, mereu cu nevoie de plivit, de udat și de un ochi pe vreme. Imaginea e bună și se rupe într-un singur punct. O grădină crește singură. Un model nu se îmbunătățește niciodată de la sine; lăsat în pace, doar se învechește, iar dacă învață din propriile ieșiri fără un grădinar care să aleagă ce se întoarce în pământ, își poate otrăvi încet următoarea recoltă. Dacă ajunge să facă ceva mai bine, e pentru că cineva l-a făcut să facă mai bine. O grădină se plănuiește însă cu un anotimp înainte, iar întrebările pe care oamenii le pun cel mai des despre acest domeniu privesc mult mai departe: cât de capabile vor deveni aceste sisteme, dacă vor face ce vrem de fapt de la ele și ce vor schimba în munca noastră.

11.6 Marginea hărții

Într-o bibliotecă de hărți atârnă, una lângă alta, două hărți ale Americii de Nord. Cea mai veche, tipărită în 1562, arată California corect, ca o peninsulă lungă lipită de continent. Cea mai nouă, de cam un secol mai târziu, o arată ca pe o insulă mare, despărțită de uscat printr-o strâmtoare care nu există, desenată cu linii curate și cu deplină încredere. Insula a apărut pe hărțile tipărite în anii 1620, cartografii au copiat-o dintr-un atlas în altul și a rămas obișnuită mai bine de o sută de ani, până când niște călători au ocolit pe uscat capătul golfului și au raportat că nu e nicio apă de trecut. Cunoașterea marginii hărții nu doar a crescut. Uneori a dat înapoi, frumos gravată.

Subcapitolul acesta e despre marginea hărții acestei cărți: întrebările pe care oamenii le pun cel mai des despre încotro se îndreaptă machine learning. Nimeni nu știe răspunsurile, iar lucrul onest e să faci ce au făcut cartografii mai buni: să desenezi ferm ce se știe, să marchezi drept presupunere ce se presupune și să spui cine presupune.

Începe cu întrebarea cea mai mare: cât de capabile pot deveni aceste sisteme. Oamenii o pun adesea în jurul ideii de **artificial general intelligence**, sau **AGI** („inteligență artificială generală”: în linii mari, un sistem care poate face cea mai mare parte a muncii intelectuale a oamenilor, cel puțin la fel de bine). Termenul nu are o definiție acceptată de toți, și asta e o parte din motivul pentru care dezbaterea e greu de tranșat. O tabără, din care fac parte conducătorii câtorva dintre cele mai mari laboratoare, se așteaptă ca astfel de sisteme să apară curând, cu estimări care, în 2026, merg de la câțiva ani la cam un deceniu. Ei arată spre curbele netede de scalare, spre modelele de raționament care obțin răspunsuri mai bune cu mai multă gândire, spre benchmark-urile saturate la câțiva ani după ce au fost scrise și spre agenți capabili să ducă la capăt sarcini tot mai lungi. Un studiu din 2025 al METR, o organizație de cercetare non-profit, a cronometrat cât le iau unor oameni pricepuți diverse sarcini de programare și a constatat că lungimea sarcinii pe care cei mai buni agenți o pot termina în jumătate din cazuri s-a dublat cam la fiecare șapte luni începând din 2019.

A doua tabără crede că metodele actuale se vor lovi de limite. Ea arată spre erorile pline de încredere ale modelelor de azi, spre eșecurile lor aparent banale, spre rezerva finită de text scris de oameni, spre modelele care nu mai învață nimic după încheierea antrenării și spre puzzle-uri precum cele din benchmark-ul ARC, concepute de François Chollet în 2019 ca să fie ușoare pentru oameni și grele pentru mașini. Yann LeCun, pionierul rețelelor convoluționale, susține de ani buni că simpla prezicere a textului nu va ajunge la o inteligență de nivel uman și că e nevoie de ceva mai apropiat de world models. Între tabere stau mulți cercetători care se așteaptă la un progres continuu, dar inegal: sisteme supraomenești la unele sarcini și nesigure la sarcini învecinate, care schimbă profund multe meserii, fie că se va pune cineva de acord să le numească generale, fie că nu. Fiecare tabără poate revendica o parte din istorie. Primele puzzle-uri ARC au rezistat celor mai bune modele timp de cinci ani, apoi au cedat în mare parte în fața modelelor de raționament la sfârșitul lui 2024; un set al doilea, mai greu, lansat în 2025, le-a încurcat la început, iar apoi scorurile au urcat abrupt în doar câteva luni.

Prognozele din domeniu au greșit în ambele direcții. În 1965, Herbert Simon, unul dintre fondatorii inteligenței artificiale, scria că mașinile vor fi capabile, în douăzeci de ani, să facă orice muncă poate face un om. În 2016, Geoffrey Hinton, a cărui lucrare despre backpropagation a apărut la începutul cărții, spunea că e „complet evident” că în cinci ani deep learning va lucra mai bine decât radiologii; în 2026, multe țări raportează lipsă de radiologi, care lucrează acum alături de astfel de unelte. În sens invers, mulți experți în Go se așteptau ca un program să bată un profesionist de top cu vreo zece ani mai târziu decât a făcut-o AlphaGo, iar cei care fac prognoze au subestimat de multe ori cât de repede vor cădea benchmark-urile. Un cititor înțelept tratează orice calendar, inclusiv pe cele rostite cu siguranță de oameni celebri, ca pe o presupunere cu bare de eroare largi.

A doua întrebare e dacă sistemele capabile vor face ce vrem de fapt de la ele. Dificultatea are o imagine veche. Regele Midas a cerut ca tot ce atinge să se prefacă în aur, a primit exact asta și a murit de foame. Barca ce n-a vrut să termine cursa e un Midas în miniatură: a maximizat scorul primit, nu cursa pe care o voiau proiectanții ei. Sycophancy, întâlnită la examinatori, e aceeași poveste; în aprilie 2025, OpenAI a retras o actualizare a ChatGPT după ce utilizatorii au observat că le lăuda ideile proaste, explicând că antrenarea se bazase prea mult pe feedback pe termen scurt. Jailbreaks și instrucțiunile injectate scot modelele din propriile reguli. A face un sistem să urmărească scopul dorit, în mod robust, chiar și în situații pe care nu le-a testat nimeni, se numește **alignment** („aliniere”: potrivirea dintre ce urmărește de fapt modelul și ce au vrut oamenii). Aici imaginea cu Midas se rupe, și într-o direcție neliniștitoare.

Midas măcar a primit ce a spus. Unui model i se spune rareori scopul în cuvinte; îl absoarbe din exemple, scoruri și preferințe, iar ce a absorbit poate fi doar dedus din comportamentul lui, așa că nimeni nu poate reciti dorința ca s-o verifice.

Cât de îngrijorați ar trebui să fim e o chestiune disputată între oameni serioși. Grijile merg de la prezent, cum ar fi fraudele, deepfake-urile, ajutorul dat atacurilor informatice și concentrarea puterii în mâinile câtorva companii, până la termenul lung: ca sisteme mai capabile decât constructorii lor, urmărind scopuri ușor greșite, să scape de sub controlul oamenilor. În 2023, o declarație de o singură frază, care cerea ca riscul de extincție cauzat de AI să fie tratat ca o prioritate globală, a fost semnată de Hinton, de Yoshua Bengio, care a împărțit cu el și cu LeCun premiul Turing pe 2018, și de șefii câtorva laboratoare de frunte. LeCun, al treilea laureat, a numit aceste temeri exagerate, iar alți cercetători, printre care autoare ale articolului despre „papagalii stocastici", susțin că discuția despre catastrofe îndepărtate abate atenția de la răul care se petrece acum. Munca propriu-zisă e mai puțin împărțită decât dezbaterea: evaluări ale capabilităților periculoase înainte de lansare, red-teaming, cercetare de interpretabilitate care încearcă să citească ce face un model în loc să creadă ce spune și reglementări precum EU AI Act, ale cărui obligații pentru furnizorii de modele de uz general au început să se aplice în 2025. Primul International AI Safety Report, scris de aproape o sută de experți sub coordonarea lui Bengio și publicat în ianuarie 2025, e un loc echilibrat în care poți vedea gama de opinii una lângă alta.

A treia întrebare e ce se schimbă când modelele acționează. Agenții deja rezervă, navighează, scriu și rulează cod. Trei probleme practice vin după ei. Permisiunile: un agent poate face orice îi permit uneltele, așa că apărările împotriva prompt injection, în frunte cu privilegiul minim, contează tot mai mult pe măsură ce i se lărgește raza de acțiune. Acumularea erorilor: dacă fiecare pas al unei sarcini reușește de nouăsprezece ori din douăzeci, un lanț de douăzeci de pași trece fără nicio poticneală doar în cam o treime din cazuri, de aceea agenții buni își verifică singuri munca și se redresează după greșeli, în loc doar să greșească mai rar. Și costul: fiecare pas recitește o tablă care se tot umple, așa că o buclă lungă poate cheltui mult mai mult decât un singur răspuns. Agenții sunt mai greu de evaluat decât răspunsurile, pentru că spre un scop duc multe drumuri bune și multe căi discrete spre unul greșit.

A patra întrebare împarte industria: ar trebui publicate ponderile? Modelele open-weight pot fi inspectate, studiate, rulate pe propriile mașini și adaptate de oricine, ceea ce răspândește cercetarea și concurența. Odată publicate, însă, nu mai pot fi retrase sau reparate, iar refuzurile antrenate în ele pot fi îndepărtate prin fine-tuning. Modelele închise, accesibile doar printr-un API, îi permit furnizorului să supravegheze abuzurile, să repare problemele și să retragă o versiune, cu prețul încrederii în acel furnizor și al concentrării puterii în puține mâini. În 2026, ambele feluri de modele se află aproape de frontieră; în 2025 chiar și OpenAI, multă vreme un laborator cu modele închise, a publicat modele open-weight. Mulți susțin că răspunsul ar trebui să depindă de ce poate face un anumit model și nu se înțeleg unde trece linia.

A cincea întrebare e cea pe care oamenii o pun la cină: ce se întâmplă cu locurile de muncă? Ce se știe e mai puțin decât spun titlurile. Într-un studiu amplu, publicat în 2023, despre un centru de asistență pentru clienți, un asistent AI a crescut numărul de probleme rezolvate pe oră cu aproximativ 14% în medie și cu cam o treime pentru angajații cei mai puțin experimentați, cu efect mic pentru cei mai experimentați. Un experiment randomizat mic, făcut de METR în 2025, a constatat că programatori experimentați din proiecte open-source erau cu aproximativ 19% *mai lenți* cu unelte AI pe propriile proiecte, deși credeau că fuseseră mai rapizi. Datele de salarizare analizate la Stanford în 2025 au arătat că angajarea tinerilor de 22–25 de ani în ocupațiile cele mai expuse scăzuse cu circa 13% față de restul, în timp ce o analiză de la Yale din același an n-a găsit nicio perturbare vizibilă a pieței muncii luate ca

întreg. Sunt rezultate timpurii, parțiale și uneori contradictorii. Expunerea nu înseamnă înlocuire: o meserie e un mănunchi de sarcini, iar tehnologiile au schimbat de obicei mănunchiul înainte de a desființa meseria, sau în loc s-o desființeze. Dacă și aceasta va urma tiparul și cât de repede e o speculație și trebuie numită ca atare.

Energia își are și ea locul pe listă, dar ce se știe despre ea și pârghiile care o reduc au apărut deja, la costurile antrenării și ale servirii.

Unde încetează imaginea hărții să mai fie adevărată? Coasta Californiei a fost acolo tot timpul, orice ar fi desenat cartografii. Viitorul acestui domeniu nu e. El se construiește, iar prognozele ajută la construcție: mută investiții, modelează legi și decid ce studiază studenții. Mulți dintre cei care desenează harta aceasta construiesc chiar ținutul pe care îl desenează, iar cu această carte în spate s-ar putea să fii curând unul dintre ei.

Și cu asta ne-am întors acolo de unde a început cartea.

Amintește-ți de inginera care își regla cache-ul toată săptămâna, ghicind, măsurând, ajustând, ghicind din nou. Bucla aceea e încă tot motorul, de la rigla cu arcuri până la detectivii din sala lor de ședințe. De-a lungul acestei cărți am văzut-o crescând de la un buton la miliarde și am văzut ce poate face: să citească scris de mână, să traducă limbi, să joace Go cu o creativitate care și-a uimit adversarii, să scrie cod alături de tine. În ultimul capitol am urmat-o afară din atelier, într-o lume care se schimbă, care reacționează și care întreabă cine răspunde, și apoi până la marginea hărții, unde nimeni nu știe încă până unde va ajunge.

Dar am văzut și, iar și iar, că bucla nu are judecată proprie. Ucenicul învață orice îl învață exemplele. Barca se învârte în orice lagună pe care o răsplătește scorul. Calul citește orice chip îi stă dinainte. Oglinda reflectă orice stă înaintea ei. Cocoșul prezice răsăritul și habar n-are ce îl provoacă. Bucla e extraordinar de puternică și complet literală.

Ceea ce lasă partea cea mai importantă a muncii acolo unde a fost mereu: la tine. Să alegi exemplele. Să scrii răspunsurile pe spatele paginii. Să decizi ce măsori și ce greșeli își pot permite utilizatorii tăi. Să îngrijești grădina mult după lansare. Mașina va învăța. Dacă învață ce trebuie depinde, ca întotdeauna, de oamenii care o construiesc.

Anexe

Anexa A — Matematica din spatele fiecărui subcapitol

Această anexă adună formulele lăsate intenționat deoparte în subcapitole. Fiecare e urmată de ce înseamnă fiecare simbol și de felul în care formula se potrivește cu imaginea pe care o cunoști deja. Peste tot, x e o intrare, y e răspunsul corect, ŷ („y-căciulă”) e predicția modelului, w e o pondere (weight), b e un bias, iar n e numărul de exemple.

1.2 Măsurarea greșelilor

Pentru numere, loss-ul obișnuit e **mean squared error** (MSE, „eroarea pătratică medie”): iei fiecare greșeală, o ridici la pătrat și faci media.

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Σ (sigma) înseamnă pur și simplu „adună peste toate exemplele”. Ridicarea la pătrat face fiecare greșeală pozitivă și face ca greșelile mari să conteze disproporționat: o greșeală de 10 contribuie cu 100, una de 1 cu 1.

Pentru răspunsuri cu da sau nu, loss-ul obișnuit e **cross-entropy** („entropia încrucișată”). Dacă modelul a dat probabilitatea p răspunsului corect, loss-ul pentru acel exemplu e:

$$L = -\log p$$

Dacă modelul a fost sigur și a avut dreptate (p aproape de 1), loss-ul e aproape 0. Dacă a fost sigur și a greșit (p aproape de 0), loss-ul crește spre infinit. Asta e meteorologa care a promis 99% soare.

1.3 K-Means și PCA

K-Means caută k centre, notate μ (miu), care fac cât mai mică distanța pătratică totală de la fiecare punct la centrul cel mai apropiat:

$$\min_{\mu_1,\ldots,\mu_k}\sum_{i=1}^{n}\min_{j}\left\| x_i - \mu_j \right\|^2$$

Barele duble înseamnă distanță. Procedura cu punctele de pizza (aloci casele, muți punctele, repeți) e un mod de a micșora această cantitate pas cu pas.

PCA calculează **covariance matrix**-ul datelor („matricea de covarianță”, un tabel care spune cât de mult se mișcă împreună fiecare pereche de trăsături) și îi găsește **eigenvectors** („vectorii proprii”): direcțiile de-a lungul cărora datele sunt cel mai împrăștiate. Primul vector propriu e „cât de puternic e laptopul”, al doilea „cât de portabil” și tot așa. **Eigenvalue**-ul fiecărui vector propriu („valoarea proprie”) spune câtă din împrăștiere prinde.

2.1 Regresia liniară și cea logistică

$$\hat{y} = w \cdot x + b \qquad p = \sigma(w \cdot x + b) = \frac{1}{1 + e^{-(w \cdot x + b)}}$$

În stânga, rigla: w · x e suma ponderată a intrărilor (fiecare intrare înmulțită cu ponderea ei, apoi adunate), iar b mută toată linia în sus sau în jos. În dreapta, variatorul: σ (sigma, funcția sigmoid) strânge orice număr în intervalul de la 0 la 1. Scorurile mari pozitive dau valori aproape de 1, cele mari negative valori aproape de 0, iar un scor de exact 0 dă 0,5, granița de decizie. Regresia liniară se antrenează minimizând MSE; cea logistică minimizând cross-entropy.

2.2 Distanța pentru cei mai apropiați vecini

Cea mai folosită distanță e distanța obișnuită în linie dreaptă (euclidiană), Pitagora în multe dimensiuni:

$$d(a,b) = \sqrt{\sum_j (a_j - b_j)^2}$$

Fiecare trăsătură j contribuie cu diferența ei la pătrat și tocmai de aceea o trăsătură măsurată în unități mari domină suma dacă nu rescalezi.

2.3 Entropie, Gini și information gain

Dacă un grup conține clase în proporțiile p_1, p_2 și așa mai departe:

$$H = -\sum_k p_k \log_2 p_k \quad G = 1 - \sum_k p_k^2$$

Amândouă sunt 0 pentru un sertar de șosete perfect ordonat (o singură clasă) și ating maximul pentru un amestec egal. Cu două clase 50/50, entropia e exact 1 bit: ai nevoie de o singură întrebare da/nu ca să lămurești incertitudinea. **Information gain** e entropia de dinaintea unei împărțiri minus media, ponderată după mărime, a entropiei grupurilor de după ea. Arborele alege întrebarea cu cel mai mare câștig.

2.4 Ansambluri (ensembles)

Un random forest face media (sau votul majoritar) predicțiilor celor T arbori ai lui:

$$\hat{y} = \frac{1}{T}\sum_{t=1}^{T} h_t(x)$$

Gradient boosting își construiește modelul ca o sumă cumulată. Fiecare arbore mic nou h e antrenat pe ce greșește încă modelul curent și e adăugat cu o pondere mică ν (niu), learning rate-ul boosting-ului:

$$F_m(x) = F_{m-1}(x) + \nu h_m(x)$$

Aceasta e editura: ciorna, plus o fracțiune atentă din corectura editoarei, plus o fracțiune atentă din corectura redactorului și tot așa.

2.5 Regula lui Bayes

$$P(H \mid E) = \frac{P(E \mid H)P(H)}{P(E)}$$

H e o ipoteză (pacientul are boala), iar E e dovada (testul a ieșit pozitiv). P(H) e prior-ul, P(H | E) e posterior-ul, iar P(E | H) arată cât de probabilă ar fi dovada dacă ipoteza ar fi adevărată. Ia testul din subcapitolul 2.5, exact în 99% din cazuri, pentru o boală pe care o are un om din zece mii. Dintr-un milion de oameni, o sută sunt bolnavi și cam 99 dintre ei ies pozitivi; 1% dintre cei 999.900 de oameni

sănătoși, adică vreo 9.999, ies și ei pozitivi. Așa că un rezultat pozitiv înseamnă boala cu o probabilitate de aproximativ 99 / (99 + 9.999), cam 1%. Naive Bayes înmulțește câte un factor P(E | H) pentru fiecare dovadă, ca și cum fiecare ar fi independentă de celelalte.

3.1 Neuronul

$$a = f(w \cdot x + b) \text{ReLU}(z) = \max(0, z)$$

Neuronul calculează suma ponderată, adaugă bias-ul și trece rezultatul printr-o funcție de activare f. Pentru lista de bifat a apartamentelor, w = (10, 4, 1), b = −11, iar f e funcția treaptă: ieșirea e 1 dacă rezultatul e cel puțin 0. Un strat înseamnă mulți neuroni alăturați, adică, în practică, o singură înmulțire de matrice; un MLP înseamnă mai multe straturi la rând, cu o funcție de activare după fiecare.

3.2 Gradient descent și backpropagation

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

Săgeata înseamnă „înlocuiește-l pe w cu". Fracția ∂L/∂w e **partial derivative**-ul („derivata parțială") loss-ului L în raport cu w: cât s-ar schimba eroarea dacă ai mișca puțin doar această pondere, ținându-le pe toate celelalte pe loc. E panta de sub bocanci în direcția lui w. Semnul minus arată în jos, iar η (eta) e learning rate-ul, lungimea pasului.

Backpropagation calculează toate aceste pante eficient, cu **chain rule** („regula lanțului"). Dacă loss-ul depinde de o valoare intermediară z, iar z depinde de w, atunci:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Acesta e inspectorul atelierului mergând înapoi pe linie: vina care ajunge la un post, înmulțită cu cât de puternic a influențat setarea acelui post ieșirea lui.

3.4 Actualizarea recurentă

$$h_t = \tanh(W x_t + U h_{t-1} + b)$$

Noua stare ascunsă h la pasul t e calculată din intrarea curentă x și starea ascunsă anterioară, prin două matrice de ponderi învățate, W și U. Funcția tanh strânge valorile în intervalul de la −1 la 1. Acesta e reduce-ul din subcapitol: acumulatorul nou, calculat din elementul curent și acumulatorul vechi. În timpul backpropagation, vina trece prin U o dată la fiecare pas, și de aici vin vanishing gradients.

4.2 Scalare și TF-IDF

Scalarea standard transformă fiecare trăsătură în „la câte abateri standard de medie se află":

$$x' = \frac{x - \mu}{\sigma}$$

unde μ (miu) e media trăsăturii, iar σ (sigma) abaterea ei standard, amândouă calculate doar pe setul de antrenare. (Aici σ e împrăștierea, nu sigmoida din subcapitolul 2.1; matematica își refolosește fără jenă puținele litere grecești.) După scalare, o valoare de 2 înseamnă „două abateri standard peste medie" în orice trăsătură, oricare ar fi fost unitățile inițiale.

TF-IDF ponderează un cuvânt după cât de des apare în acest document (term frequency, TF) și cât de rar e în toate cele N documente (inverse document frequency, IDF):

$$\text{tfidf}(w, d) = \text{tf}(w, d) \cdot \log \frac{N}{\text{df}(w)}$$

Dacă un cuvânt apare în fiecare document, df e egal cu N, logaritmul e zero și cuvântul nu contează deloc. Un cuvânt care apare într-un document din o mie primește o pondere aproape de log 1000, cam 6,9.

4.3 Un strat ca înmulțire de matrice

Cu X un batch de intrări (un rând per exemplu), W matricea de ponderi a stratului (o coloană per neuron) și b vectorul lui de bias:

$$Y = f(XW + b),\ X: (n \times d_{\text{in}}),\ W: (d_{\text{in}} \times d_{\text{out}}),\ Y: (n \times d_{\text{out}})$$

Formele trebuie să se potrivească precum piesele de domino: numărul de coloane ale lui X trebuie să fie egal cu numărul de rânduri ale lui W. Fiecare celulă din Y e un produs scalar, un neuron aplicat unui exemplu, iar toate cele n × d_out pot fi calculate în paralel. O rețea întreagă e această linie repetată, cu ieșirea unui strat devenind X-ul următorului.

Memoria necesară doar ca să ții ponderile unui model e numărul de parametri înmulțit cu octeții per parametru:

memorie ≈ $P \times$ octeți per pondere$7 \times 10^9 \times 4$octeți = 28GB (32 de biți), $7 \times 10^9 \times 0.5$octeți = 3.5GB (4 biți)

Asta e economia pe care o aduce cuantizarea. Antrenarea are nevoie de câteva ori mai mult, pentru rezultatele intermediare păstrate, gradienți și memoria optimizatorului (subcapitolul 5.2).

5.3 Hipervectori

Pentru doi hipervectori binari a și b de lungime D, asemănarea se măsoară prin fracțiunea pozițiilor în care sunt de acord, derivată din **Hamming distance** („distanța Hamming”, numărul de poziții în care diferă):

$$\text{sim}(a, b) = 1 - \frac{\text{Hamming}(a, b)}{D}$$

Pentru doi vectori aleatori, valoarea e aproape 0,5, iar împrăștierea în jurul lui 0,5 scade ca unu supra radical din D. Cu D = 10.000, împrăștierea e de vreo jumătate de procent, de aceea 0,6 înseamnă deja „sigur înrudiți”.

Binding-ul e XOR bit cu bit, notat $\oplus$, și se anulează singur:

$$(a \oplus b) \oplus b = a$$

Bundling-ul mai multor vectori, notat +, e un vot majoritar pe fiecare poziție (când numărul vectorilor e par, egalitățile se rup la întâmplare). O înregistrare e un bundle de perechi legate, iar interogarea ei e încă un bind urmat de o căutare a celui mai apropiat vecin printre vectorii cunoscuți:

$$R = (\text{ANIMAL} \oplus \text{CÂINE}) + (\text{STĂPÂN} \oplus \text{ANA}),\ R \oplus \text{STĂPÂN} \approx \text{ANA}$$

Semnul „aproximativ egal” e zgomotul de la celelalte perechi din bundle; lungimea vectorilor e cea care îl ține mic.

7.1 Cosine similarity

$$\cos(\theta) = \frac{a \cdot b}{\| a \| \ \| b \|}$$

Sus e produsul scalar: înmulțești vectorii coordonată cu coordonată și aduni. Jos se împarte la ambele lungimi, ceea ce scoate mărimea din comparație și lasă doar direcția. Rezultatul e 1 pentru aceeași direcție (rețeta de clătite pentru 4 și pentru 40), 0 pentru direcții fără legătură și −1 pentru direcții opuse.

7.2 Attention

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^\top}{\sqrt{d_k}})V$$

Q, K și V sunt matrice cu un rând per cuvânt: întrebările, ecusoanele și dosarele. Se obțin din embedding-urile X prin trei matrice învățate: Q = X·W_Q, K = X·W_K, V = X·W_V. Produsul $Q \cdot K^T$ e un tabel de scoruri de potrivire, câte unul pentru fiecare pereche de cuvinte. Împărțirea la radical din d_k, numărul de coordonate al fiecărei chei, împiedică scorurile să crească atât de mult încât softmax să devină „totul sau nimic”. **Softmax** transformă fiecare rând de scoruri în procente care însumează 100%:

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Înmulțirea cu V produce, pentru fiecare cuvânt, amestecul dosarelor pe care le-a ascultat. Aceasta e căutarea în dicționar „cu aproximație”, scrisă sub formă de matrice. Multi-head attention rulează mai multe copii, cu matrice învățate diferite, și lipește rezultatele unul lângă altul.

7.3 Antrenarea unui model de limbaj și alegerea tokenilor

Pretraining-ul minimizează cross-entropy-ul tokenului următor, mediat peste fiecare poziție din fiecare text:

$$L = -\frac{1}{T}\sum_{t=1}^{T} \log p(x_t \mid x_1, \ldots, x_{t-1})$$

Citește bara verticală ca „dat fiind”: probabilitatea pe care a dat-o modelul tokenului real de la poziția t, dați fiind toți tokenii dinaintea lui. Temperatura τ (tau) împarte scorurile z înainte de softmax:

$$p_i = \frac{e^{z_i/\tau}}{\sum_j e^{z_j/\tau}}$$

Un τ mic exagerează diferențele dintre scoruri, așa că tokenul de top câștigă aproape mereu; un τ mare le aplatizează, așa că tokenii mai puțin probabili primesc o șansă reală.

8.1 Difuzie

Procesul înainte adaugă puțin zgomot gaussian la fiecare dintre cei T pași, astfel încât, după destui pași, imaginea nu se mai deosebește de puricii unui televizor fără semnal:

$$x_t = \sqrt{1-\beta_t}\,x_{t-1} + \sqrt{\beta_t}\,\varepsilon, \varepsilon \sim \mathcal{N}(0, I)$$

β (beta) e un număr mic care controlează cât zgomot adaugă pasul t, iar ε (epsilon) e zgomotul aleator. Rețeaua e antrenată pe o singură regresie simplă: dată fiind imaginea zgomotoasă și numărul pasului, prezice zgomotul care a fost adăugat.

$$L = \| \varepsilon - \varepsilon_\theta(x_t, t) \|^2$$

Generarea pornește de la zgomot pur și rulează pașii invers, scăzând de fiecare dată zgomotul prezis de rețea. Obiectivul unui GAN, în schimb, e un joc în doi: discriminatorul D își maximizează acuratețea în a deosebi realul de generat, iar generatorul G o minimizează.

8.2 Mixture of experts și contrastive loss

Într-un strat mixture-of-experts, un router produce o pondere pentru fiecare dintre cei E experți și doar primii k sunt rulați:

$$y = \sum_{i \in \text{top-}k} g_i(x)\text{Expert}_i(x)$$

Valorile g sunt scoruri softmax de la router, așa că ieșirea e un amestec ponderat al răspunsurilor câtorva experți, iar ceilalți nu costă nimic pentru acest token.

Contrastive loss-ul din spatele modelelor precum CLIP tratează, într-un batch de N perechi, propria descriere a fiecărei imagini drept exemplul pozitiv și celelalte N − 1 descrieri drept negative:

$$L = -\log \frac{e^{\text{sim}(a_i, b_i)/\tau}}{\sum_{j=1}^{N} e^{\text{sim}(a_i, b_j)/\tau}}$$

E cross-entropy peste „ce descriere îi aparține acestei imagini”, cu cosine similarity drept scor și o temperatură τ (tau) ca în subcapitolul 7.3.

9.1 Q-learning

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a} Q(s', a') - Q(s,a)]$$

După ce face acțiunea a în starea s, primește recompensa r și ajunge în starea s′, agentul își compară vechea estimare Q(s, a) cu una mai bine informată: recompensa tocmai primită plus valoarea actualizată (discounted) a celei mai bune mutări din locul în care a ajuns. Își mută estimarea o parte din drum spre ea, cu un pas α (alfa). γ (gama), factorul de actualizare, e un număr puțin sub 1. Aceasta e valoarea brânzei care se scurge înapoi prin labirint, intersecție cu intersecție.

9.2 LoRA și învățarea din preferințe

LoRA îngheață o matrice de ponderi W și învață o corectură mică, scrisă ca produsul a două matrice subțiri:

$$W' = W + BA, B: (d \times r), A: (r \times d), r \ll d$$

Pentru o matrice de 4.096 pe 4.096 și un rang r de 8, corectura are cam 65.000 de numere antrenabile în loc de 16,8 milioane, o reducere de aproximativ 250 de ori.

În RLHF, un reward model r e antrenat din perechi în care răspunsul y_+ a fost preferat lui y_-, astfel încât răspunsul preferat să primească un scor mai mare cu probabilitatea dată de un sigmoid al diferenței:

$$P(y_+ > y_-) = \sigma(r(x, y_+) - r(x, y_-))$$

Modelul de limbaj e apoi optimizat să maximizeze r, în timp ce o penalizare îl ține aproape de modelul de la care a pornit. DPO contopește cei doi pași într-unul singur, antrenând modelul direct pe perechile de preferințe.

10.1 Regularizare

Regularizarea L2 adaugă la loss o penalizare pentru ponderile mari, cu o tărie λ (lambda) pe care o alegi tu:

$$L_{\text{total}} = L_{\text{data}} + \lambda \sum_j w_j^2$$

Modelul trebuie acum să-și justifice fiecare pondere mare printr-o reducere reală a erorii, versiunea matematică a reviewer-ului care respinge complexitatea care nu-și justifică prețul. Regularizarea L1 folosește valori absolute în loc de pătrate și tinde să împingă ponderile inutile exact la zero.

10.3 Metrici

Cu TP, FP și FN numărul de true positives, false positives și false negatives:

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN} \quad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Precizia împarte la tot ce a semnalat alerta; recall-ul împarte la tot ce exista de fapt. Pentru predicții numerice:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i| \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2}$$

Barele verticale din MAE înseamnă „ignoră semnul”. RMSE e radicalul din MSE din subcapitolul 1.2, care îl readuce la unitățile inițiale păstrându-i sensibilitatea sporită la greșelile mari.

11.4 Differential privacy

Un calcul aleator M este ε-diferențial privat dacă, pentru oricare două seturi de date D și D′ care diferă doar prin datele unei singure persoane și pentru orice mulțime S de rezultate posibile:

$$P(M(D) \in S) \leq e^{\varepsilon} P(M(D') \in S)$$

ε (epsilon) e bugetul de confidențialitate. Cu un ε mic, e^ε e aproape de 1, așa că orice rezultat e aproape exact la fel de probabil fie că ai fost inclus, fie că nu, iar nimeni care se uită la el nu poate afla mare lucru despre tine. Bugetele se adună: publicarea a k rezultate calculate fiecare cu bugetul ε costă în total cel mult kε, iar aceasta e aritmetica din spatele avertismentului din subcapitolul 11.4 că fiecare răspuns publicat cheltuiește puțin din buget.

Anexa B — Dicționarul metaforelor

Fiecare imagine din carte, alături de termenul tehnic pe care îl reprezintă, ca să poți trece ușor dintr-o limbă în alta când citești documentație sau vorbești cu specialiști.

Subcapitol	Imaginea	Termenul tehnic
1.1	Rețeta și ucenicul	programare clasică vs. machine learning
1.2	Cheia de răspunsuri de pe spatele paginii	supervised learning, labels, loss
1.3	Sortatul dulapului pe întuneric; punctele de livrare a pizzei	clustering, K-Means
1.3	Umbra de pe perete; fișele tehnice ale laptopurilor	dimensionality reduction, PCA (principal component analysis)
1.3	Fața de pe Marte	clustere găsite în zgomot pur
1.4	Culegerea, simularea, examenul real	training, validation și test sets
1.4	Orele de învățat vs. ce ai în cap	hyperparameters vs. parameters
2.1	Rigla cu arcuri	linear regression, squared error
2.1	Variatorul de lumină	sigmoid, logistic regression
2.2	Spune-mi cine îți sunt vecinii; secțiunile unei biblioteci	k-nearest neighbors (k-NN); approximate nearest-neighbor index
2.2	În 500 de dimensiuni toată lumea e străină	curse of dimensionality
2.3	Douăzeci de întrebări, git bisect	decision tree, information gain
2.3	Sertarul cu șosete	entropy, Gini index
2.4	Boul de la târg; consiliul experților pe jumătate orbi	random forest, bagging
2.4	Editura	gradient boosting
2.5	Drumul cu cea mai lată fâșie goală între două sate; satul din interior devenit vale	support vector machine (SVM), margin, kernel trick
2.5	Bănuiala doctoriței înainte și după dovezi; doctorița leneșă	Bayes' rule, prior și posterior, naive Bayes
2.5	Festivalul de muzică văzut de la o dronă; harta lumii de pe peretele clasei; sârma flexibilă prinsă în punctele cunoscute	density-based clustering (DBSCAN), t-SNE și UMAP, Gaussian processes
3.1	Lista de bifat pentru apartamente; regula de alertă	perceptron, weights, bias
3.1	Conversii valutare înlănțuite	de ce au nevoie rețelele de activation functions
3.2	Coborâtul prin ceață; autoscaler-ul care oscilează	gradient descent, learning rate
3.2	Inspectorul care parcurge înapoi linia din atelier; distributed tracing	backpropagation
3.3	Inspectorii cu lupe	convolutional neural network, kernel
3.3	Regula de linter aplicată fiecărei linii	weight sharing

Subcapitol	Imaginea	Termenul tehnic
3.4	Telefonul fără fir; un reduce cu un acumulator mic; o singură fișă de notițe vs. un carnet cu comutatoare	recurrent neural network, hidden state, vanishing gradient, LSTM gates
4.1	Avionul de hârtie și porumbelul; mixerul cu un milion de cursoare	neuroni artificiali vs. biologici
4.2	Folderul cu tichete dezordonate; rândul de căsuțe	data preparation, one-hot encoding, feature engineering
4.3	Fișierul cu o sută de milioane de zecimale	tensors, shapes, modelul ca ponderi stocate
4.3	JPEG pentru ponderi	quantization
4.4	Cele patru recrutoare: fișa de punctaj, schema logică, intuiția, privirea condusă de fiecare CV	feature importance: linear weights, tree-based importance, hidden features, attention
4.4	Amestecarea valorilor unei singure trăsături; gemenii care își împart meritul	permutation importance, correlated features
5.1	Cutia de Lego: muncitori și instalații	tipuri de straturi (layers), arhitectură
5.1	Termostatul; ocolul; zilele libere de la repetiții	normalization, residual connections, dropout
5.2	Anul dinainte de deschidere; ziua deschiderii	training vs. inference
5.2	Cuptorul de pâine umplut	batching
5.2	Factura unei conversații	costul inferenței, tokeni de ieșire vs. de intrare
5.3	Zece mii de aruncări de monedă; un Bloom filter crescut mare	hyperdimensional computing, VSA (vector symbolic architectures)
5.4	Primul an la chitară; cuptoarele și făina; o fotocopie după o fotocopie	learning curve, scaling laws, compute-optimal training (Chinchilla), synthetic data și model collapse
6.1	Cana de cafea pe întuneric; scene graph-ul	cortical columns, reference frames
6.1	Degetele care votează; replicile care ajung la quorum	votul dintre coloane
6.2	Frigiderul care se oprește din zumzet; codecul video	predictive processing, prediction error
6.2	Fiecare serviciu își reglează propriul cache	Hebbian learning (învățare locală)
6.2	Log-ul append-only compactat noaptea	hippocampal replay, complementary learning systems
7.1	Harta orașului sensurilor; codurile de culoare	embeddings
7.1	Rețeta de clătite pentru 4 și pentru 40	cosine similarity

Subcapitol	Imaginea	Termenul tehnic
7.2	Ședința detectivilor; căutarea în dicționar „cu aproximație”	self-attention: query, key, value
7.3	Tastatura telefonului; ghicitul cuvântului următor	next-token prediction, pretraining
7.3	Angajata nouă: wiki, învățat lângă seniori, code review	pretraining, fine-tuning, RLHF
7.3	Tabla din sala de ședințe	context window
7.4	Examenul cu cartea deschisă	retrieval-augmented generation (RAG)
7.5	Viața pe o tablă; grefierul care condensează notițele vechi; carnetul purtat între ședințe	context window limits, context compaction, persistent memory
7.6	Inginera nouă care ghicește continuarea unui postmortem; partida clasică vs. blitz	next-token prediction, chain of thought, test-time compute
8.1	Treizeci de cuvinte la telefon	autoencoder
8.1	Copilul care corectează o pată; falsificatorul și expertul	diffusion models, GANs (generative adversarial networks)
8.2	Indexul construit la scriere; biroul de triaj; REPL-ul	two towers, mixture of experts, agents
8.3	Tabelul mareelor al căpitanului de port; „prefă-te că e ianuarie anul trecut”	time series, trend și seasonality, backtesting
8.3	Bârfa din sat; vânzătoarea din librărie	graph neural networks, message passing; recommender systems: retrieval și ranking
8.4	Sania; încălzirea alergătorului; balustrada	momentum, warmup, gradient clipping
8.4	Un termostat pentru toată clădirea vs. unul în fiecare cameră; orchestra prea mare pentru o singură sală	batch vs. layer normalization; data, tensor și pipeline parallelism
9.1	Dresajul cu clicker; labirintul cu brânză	reinforcement learning, Q-learning
9.1	Dilema prânzului	exploration vs. exploitation
9.1	Barca ce se învârte în lagună	reward hacking
9.2	Facultatea, rezidențiatul, feedback-ul; carnețelul traducătoarei; lesa	pretraining, transfer learning, fine-tuning, LoRA, RLHF, policy gradients și PPO, distillation
10.1	Tocilarul și leneșul	overfitting și underfitting
10.2	Clever Hans; ziarul de mâine	data leakage
10.3	Alarma de fum; pagerul	precision și recall
10.4	Oglinda strâmbă; „pe mașina mea merge”	bias în date

Subcapitol	Imaginea	Termenul tehnic
10.5	Meteorologa verificată pe un an întreg	calibration, reliability diagram, temperature scaling
10.5	Moneda corectă; prăjitorul de pâine arătat unui clasificator de pisici și câini	aleatoric vs. epistemic uncertainty, out-of-distribution detection
10.6	Mecanica fără manual: se uită, intervine, sondează	interpretability, mechanistic interpretability
10.6	Împărțirea notei de plată la restaurant; „ați fi fost aprobat dacă venitul ar fi fost mai mare”	SHAP (Shapley values), counterfactual explanations
11.1	Cocoșul și răsăritul; moneda care decide cine primește reducerea	correlation vs. causation, confounders, randomized controlled experiments (A/B tests)
11.2	Degustătorii cărora li se servește de trei ori același vin; clasamentele școlilor	LLM-as-a-judge, evaluation noise, Goodhart’s law
11.3	Asistenta care face orice îi spune orice bilet de pe birou	prompt injection, indirect prompt injection
11.3	Abțibildurile de pe indicatorul de stop; fișa de rețetă cu ingredient amar	adversarial examples, data poisoning, backdoors
11.4	Banul aruncat în secret; bucătăriile bunicilor	randomized response, differential privacy, federated learning
11.4	Oul din cozonacul copt	machine unlearning
11.5	Restaurantul la un an după deschidere; făina nouă; grădina care are nevoie de grădinar	MLOps, training-serving skew, drift monitoring
11.6	Insula California de pe hărțile vechi	prognozele despre progresul AI, dezbaterea despre AGI (artificial general intelligence)
11.6	Dorința regelui Midas	alignment, reward hacking, specificarea scopurilor
11.6	Douăzeci de pași, fiecare reușit de nouăsprezece ori din douăzeci	agents, acumularea erorilor (error compounding)

Anexa C — Mai departe

Când ucenicul din subcapitolul 1.1 iese în sfârșit din bucătărie, nu-l mai ajută încă o carte despre prăjituri, ci un cuptor, ceva făină și câteva rețete pe care merită să le strice. Anexa aceasta e un astfel de rămas-bun: o listă scurtă, aleasă cu părtinire, de resurse cunoscute care au rămas utile ani la rând, plus o ordine în care le poți folosi. Aproape toate sunt în engleză, iar cele mai multe sunt gratuite. Alege un curs, o carte și o unealtă ca să începi; a aduna resurse nu e același lucru cu a învăța din ele.

Cursuri gratuite

Machine Learning Specialization a lui Andrew Ng (Stanford și DeepLearning.AI, pe Coursera) e pasul următor cel mai blând după această carte: reia Capitolele 1–3 cu matematica pornită; **Deep Learning Specialization**, tot a lui, continuă de acolo. **Practical Deep Learning for Coders** de la fast.ai (course.fast.ai) merge invers, de sus în jos: antrenezi un model funcțional în prima lecție și deschizi capota mai târziu. Stanford publică prelegerile a trei cursuri clasice: **CS229** (machine learning, cu matematica din Anexa A pe larg), **CS231n** (deep learning pentru computer vision, teritoriul subcapitolelor 3.3 și 8.1) și **CS224n** (procesarea limbajului natural cu deep learning, teritoriul Capitolului 7). **Neural Networks: Zero to Hero** al lui Andrej Karpathy construiește de la zero backpropagation (subcapitolul 3.2) și apoi un GPT mic (subcapitolul 7.3), în cod pe care îl scrii cu mâna ta.

Cărți

- *An Introduction to Statistical Learning*, de Gareth James, Daniela Witten, Trevor Hastie și Robert Tibshirani, scrisă inițial cu exemple în R și, din 2023, și într-o ediție cu Python; gratuită online (statlearning.com). Cel mai bun însoțitor pentru Capitolul 2.
- *Hands-On Machine Learning*, de Aurélien Géron. Edițiile mai vechi folosesc Scikit-Learn, Keras și TensorFlow; ediția din 2025 e construită pe Scikit-Learn și PyTorch. Cea mai practică din listă.
- *Dive into Deep Learning*, de Aston Zhang, Zachary Lipton, Mu Li și Alexander Smola; gratuită și executabilă pe d2l.ai, fiecare capitol fiind un notebook.
- *Understanding Deep Learning*, de Simon Prince (2023), clară și bine ilustrată, cu PDF gratuit.
- *Deep Learning*, de Ian Goodfellow, Yoshua Bengio și Aaron Courville (2016), gratuită pe deeplearningbook.org. Mai veche, dar tot referința standard pentru fundamente.
- *Pattern Recognition and Machine Learning*, de Christopher Bishop (2006), și *Probabilistic Machine Learning: An Introduction* (2022), cu continuarea *Advanced Topics* (2023), de Kevin Murphy. Amândouă duc până la capăt privirea probabilistică din subcapitolul 2.5. Solicitante; citește-le după una dintre cărțile de mai sus.
- *Reinforcement Learning: An Introduction*, de Richard Sutton și Andrew Barto (ediția a doua, 2018), gratuită online. Câinele, labirintul și barca din Capitolul 9, tratate cum se cuvine.

Pentru întrebările din subcapitolele care merg mai adânc și din ultimul capitol ajută trei cărți mai scurte: *Interpretable Machine Learning* a lui Christoph Molnar (gratuită online) pentru subcapitolul 10.6, *Designing Machine Learning Systems* a lui Chip Huyen pentru subcapitolul 11.5 și *The Book of Why* a lui Judea Pearl și Dana Mackenzie pentru cocoșul din subcapitolul 11.1.

Unelte de încercat

- Notebook-urile **Jupyter** sau **Google Colab**, care le rulează în browser, adesea cu o placă video gratuită pentru experimente mici.
- **scikit-learn** (scikit-learn.org), pentru tot ce e în Capitolul 2 și pentru cea mai mare parte din subcapitolele 10.1 și 10.3; ghidul lui de utilizare e un manual bun deghizat.
- **PyTorch** (pytorch.org), pentru construirea și antrenarea rețelelor din Capitolele 3–5.

- **Hugging Face** (huggingface.co), pentru descărcarea de modele preantrenate și seturi de date și pentru cursurile lui gratuite despre modele de limbaj.
- **Kaggle** (kaggle.com), pentru seturi de date, competiții și tutoriale scurte, dar și ca să citești cum au rezolvat alții aceeași problemă.

O ordine posibilă

Începe de unde a început cartea. Ia un tabel mic cu date reale, prețuri de apartamente sau tichete de suport, împarte-l cum te-a învățat subcapitolul 1.4 și antrenează în scikit-learn rigla din subcapitolul 2.1, un arbore din subcapitolul 2.3 și un ansamblu cu boosting din subcapitolul 2.4. Desenează curba de învățare din subcapitolul 5.4 și vezi dacă modelul tău e tocilarul sau leneșul din subcapitolul 10.1.

Apoi construiește de mână o rețea mică, urmându-l pe Karpathy sau primele capitole din *Dive into Deep Learning*, până când backpropagation (subcapitolul 3.2) nu mai pare magie. Fă fine-tuning unui model de imagini preantrenat pe câteva sute de fotografii ale tale, ca în subcapitolul 9.2, și rulează verificarea cu saliency maps din subcapitolul 10.6 ca să vezi la ce se uită; s-ar putea să-ți găsești propria zăpadă.

Urmează limbajul. Transformă în embeddings câteva mii dintre documentele tale (subcapitolul 7.1), construiește peste ele asistentul cu cartea deschisă din subcapitolul 7.4 și scrie un set privat de evaluare de cincizeci de întrebări înainte să începi să reglezi ceva (subcapitolul 11.2). Dacă te tentează agenții, dă-i unui model o singură unealtă îngustă, doar de citire, și urmărește la lucru bucla din subcapitolul 8.2, cu permisiunile din subcapitolul 11.3 în minte.

La fiecare pas, păstrează obiceiurile din Capitolele 1 și 10: un set de test sigilat, erorile de antrenare și de validare privite una lângă alta, metrica potrivită cu greșelile pe care și le pot permite utilizatorii tăi și o privire asupra erorilor, grup cu grup. Compară fiecare model cu cel mai simplu model care ar putea funcționa, pentru că uneori, cum spunea subcapitolul 11.5, mișcarea corectă e să scrii regula. Tot restul, cum promitea prefața, e aceeași buclă, rulată din nou.